Programmation en *Java*

Janvier 2026

Farida Ait-Ammar
Nicolas Leduc

Table des matières

1 Introduction à la programmation . . . 7
1.1 Application informatique . . . 7
1.2 Qu'est-ce qu'un ordinateur . . . 9

2 Algorithmie . . . 14
2.1 Notions de base . . . 14
2.2 Analyse du problème . . . 20
2.3 Pseudo-code . . . 21
2.4 Ordinogramme . . . 31
2.5 Trace d'exécution . . . 33
2.6 Ordinogramme : référence . . . 36

3 Introduction à *Java* . . . 38
3.1 Structure d'un programme . . . 38
3.2 Constante et variable . . . 40
3.2.1 Types de données . . . 41
3.2.2 Déclaration et nomenclature . . . 45
3.2.3 Initialisation et affectation . . . 47
3.2.4 Conversion entre types numériques . . . 49
3.3 Commentaire de programmation . . . 51
3.4 Entrée et sortie au terminal . . . 52
3.5 Bibliothèque `FenetreDessin` . . . 56

4 Sous-programme . . . 58
4.1 Définition . . . 58
4.2 Procédure, fonction et paramètre . . . 60
4.3 Trace d'exécution . . . 63
4.4 Éléments d'une méthode en *Java* . . . 65
4.5 Ajout d'une méthode dans `Facture` . . . 69
4.6 Visibilité des variables . . . 71
4.7 Généralisation et polymorphisme . . . 73
4.8 Exemple de résolution incluant des méthodes . . . 75

5 Décision ... 79
5.1 Variable booléenne et opérateurs ... 79
5.2 SI ... ALORS ... (`if _`) ... 81
5.3 SI ... ALORS ... SINON ... (`if _ else _`) ... 84
5.4 Décisions imbriquées ... 87
5.5 SELON ... CAS ... DEFAUT ... (`switch _ case _ default _`) ... 89
5.6 SI ... ALORS ... SINON SI ..... SINON ... (`if _ else if __ else _`) ... 92

6 Répétition ... 95
6.1 Introduction ... 95
6.2 TANTQUE ... FAIRE ... (`while _`) ... 96
6.3 FAIRE ... TANTQUE ... (`do _ while _`) ... 99
6.4 Opérateur : complément ... 102
6.5 POUR ... FAIRE ... (`for _`) ... 103
6.6 Imbrication, équivalence et choix de boucle ... 107

7 Tableau unidimensionnel ... 110
7.1 Déclaration et initialisation ... 110
7.2 Accès aux éléments ... 113
7.3 Copie et calcul ... 115
7.4 Algorithmes : tableau non ordonné ... 116
7.4.1 Somme et moyenne ... 116
7.4.2 Recherche d'occurrence ... 117
7.4.3 Minimum et maximum ... 118
7.4.4 Éléments distincts, ajout et retrait ... 119
7.5 Tri par échange ... 120
7.6 Algorithmes : tableau ordonné ... 122
7.6.1 Recherche d'occurrence ... 122
7.6.2 Ajout et retrait ... 123
7.7 Fusion de tableaux ... 124
7.8 Recherche binaire ... 125

8 Tableau multidimensionnel ... 127
8.1 Déclaration et initialisation ... 127
8.2 Accès aux éléments ... 130
8.3 Parcours par ligne ou colonne ... 131
8.4 Lecture et affichage ... 132
8.5 Paramètre de type tableau ... 134
8.6 Somme et moyenne ... 135
8.7 Minimum et maximum ... 138
8.8 Recherche de toutes les occurrences ... 140

8.9 Recherche de la première occurrence . . . 141
8.10 Tri par échange . . . 143

9 Chaîne de caractères . . . 144
9.1 Classe `String` . . . 144
9.1.1 Introduction . . . 144
9.1.2 Constructeur . . . 146
9.1.3 Comparaison . . . 147
9.1.4 Recherche de caractère . . . 150
9.1.5 Transformation de chaîne . . . 153
9.2 Classe `StringBuilder` . . . 155
9.2.1 `StringBuffer` vs `StringBuilder` . . . 155
9.2.2 Constructeur . . . 157
9.2.3 Transformation de chaîne . . . 158
9.2.4 Comparaison . . . 160
9.2.5 Exemple . . . 161

10 Expression régulière . . . 163
10.1 Principe de base . . . 163
10.2 Début et fin . . . 165
10.3 Classe de caractères . . . 166
10.4 Quantificateur . . . 167
10.5 Groupe . . . 168
10.6 Expression régulière en *Java* . . . 170

11 Exception . . . 173
11.1 Capturer une exception (`try/catch`) . . . 173
11.2 Capturer plusieurs exceptions . . . 176
11.3 Bloc `finally` . . . 179
11.4 Exception vérifiée (`throws`) . . . 180
11.5 Lancer une exception (`throw`) . . . 181
11.6 Créer une exception . . . 184
11.7 Conseil d'utilisation . . . 186

12 Flux et fichier . . . 188
12.1 Type de fichier . . . 188
12.2 Manipulation de flux . . . 192
12.3 Fichier texte . . . 196
12.4 Fichier binaire . . . 198
12.5 Fichier objet *Java* . . . 201
12.6 Fichier compressé . . . 204

A Guide d'installation 207

A.1 *Java JDK* 25 207

A.1.1 Validation de l'installation 207

A.1.2 Installation du *JDK* 208

A.2 *Visual Studio Code* 210

A.3 *Visual Studio Code* : Extensions 214

A.3.1 *French Language Pack* 214

A.3.2 *Extension Pack for Java* 214

A.3.3 *Checkstyle for Java* 215

A.3.4 *Javadoc-Generator* 215

A.3.5 *Javadoc Tools* 216

A.3.6 *Draw.io Integration* 216

A.3.7 *Edit CSV* 216

A.3.8 *Rainbow CSV* 217

A.3.9 *Hex Editor* 217

A.3.10 *Git Graph* 217

A.3.11 *Live Share* 218

A.4 *Git* 218

A.4.1 Validation de l'installation 218

A.4.2 Installation de *Git* 219

B *Visual Studio Code* 225

B.1 Environnement de développement *Java* 225

B.2 Démarrage de *Visual Studio Code* 227

B.3 Interface 228

B.4 *Package* et classe *Java* 229

B.5 Exécution d'un programme 232

B.6 Édition du code 233

B.7 Bibliothèque `FenetreDessin` 235

B.8 Interpréteur interactif 236

B.9 Débogueur 236

B.10 Édition d'un ordinogramme (*draw.io*) 243

B.11 Édition d'une trace d'exécution (`.csv`) 244

C Gestion de versions (*Git*) 245

C.1 Introduction à la gestion de versions 245

C.2 Introduction à *Git* 247

C.3 Configuration de *Git* et *Gitea* 249

C.4 Ignorer certains fichiers (`.gitignore`) 257

C.5 Sauvegarder les modifications 258

C.6 Naviguer dans l'historique 259

C.7 Annuler les modifications 261

C.8 Interagir avec un serveur *Git* . . . 262
C.9 Gérer une branche . . . 265
C.10 Ligne de commande . . . 268

D Test unitaire . . . 269
D.1 Stratégie de test . . . 269
D.2 Plan de test . . . 271
D.3 Tester sans outil . . . 273
D.4 Bibliothèque *JUnit* . . . 274
D.5 Création d'un test . . . 275
D.6 Exécution d'un test . . . 276
D.7 Méthode de test . . . 278
D.8 Test de procédure ou de fonction . . . 280
D.9 Cycle de vie d'un test . . . 282
D.10 Tester un tableau . . . 286
D.11 Tester une classe . . . 288
D.12 Exercice : Test de classe . . . 291
D.13 Complément : Comparaison de nombres en virgule flottante . . . 294

E Documentation *Javadoc* . . . 296
E.1 Introduction . . . 296
E.2 Commentaire *Javadoc* . . . 298
E.3 Étiquette *Javadoc* . . . 300
E.4 Exemple de documentation . . . 303
E.5 Génération de la documentation . . . 305

F Normes de codage . . . 307

1. Introduction à la programmation

1.1 Application informatique

2

L'informatique

- L'informatique permet de **traiter l'information** pour analyser, gérer, prévoir ou concevoir des évènements dans divers domaines tels que la médecine, la météo, la bureautique, l'économie...
- Grâce à internet, l'informatique donne accès à une information mondiale.
- Cette communication et ces traitements sont possibles grâce aux applications qui s'exécutent sur des ordinateurs.

Source: [redacted]

Farida Ait-Ammar et Nicolas Leduc

3

Application informatique

- Une **application** (ou logiciel) est **composée d'un ensemble de programmes** destinés à effectuer un traitement particulier sur un ordinateur. Elle apporte à l'ordinateur des fonctionnalités supplémentaires inexistantes à l'origine.
- Pour créer une application, il est important de savoir programmer et de connaitre les langages de programmation.
- Éléments d'un système informatique

[1] Un système d'exploitation (*Operating System* ou *OS*) est un ensemble de programmes spécialisés qui gèrent le matériel et assurent l'exécution des logiciels applicatifs

Farida Ait-Ammar et Nicolas Leduc

4

Le processus de développement d'une application

- Le processus de développement d'une application (ou logiciel) est long et minutieux. Il comprend un ensemble d'activités qui dépendent de la méthodologie de développement choisie, comme la méthodologie *Agile*.
- Les grandes étapes du processus sont:
  - Définir les besoins et les exigences du client et des utilisateurs
  - Analyser le système *(validation de la faisabilité du projet)*
  - Concevoir le système *(rédaction du cahier des charges et des principaux algorithmes)*
  - **Programmer le logiciel** *(écriture du code dans les langages de programmation choisis)*
  - **Tester le logiciel** *(tester le bon fonctionnement et validation avec le client)*
  - Déployer *(installation du logiciel dans l'environnement de production, chez le client)*
  - Maintenir le système *(réparation des failles et ajout de nouvelles fonctionnalités)*

Farida Ait-Ammar et Nicolas Leduc

5

Programme informatique

- Programme informatique: « Suite d'instructions écrites sous une forme que l'ordinateur peut comprendre pour traiter un problème ou pour effectuer une tâche »[1]
- Les étapes de création d'un programme, qui doit résoudre un problème ponctuel, sont similaires au processus de développement d'une application (ensemble de programmes).
- À partir des spécifications du problème, on **analyse** le problème dans le but de trouver une solution, qui doit être expliquée dans le dossier d'analyse de manière à préparer le travail du programmeur. **L'algorithme** fait partie de la solution.

1 Office québécois de la langue française, 2002

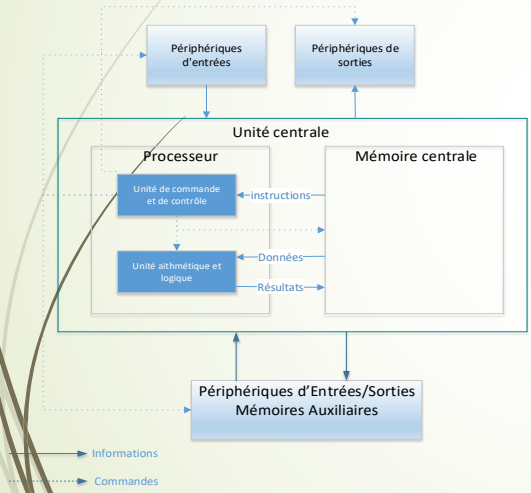
Farida Ait-Ammar et Nicolas Leduc

1.2 Qu'est-ce qu'un ordinateur

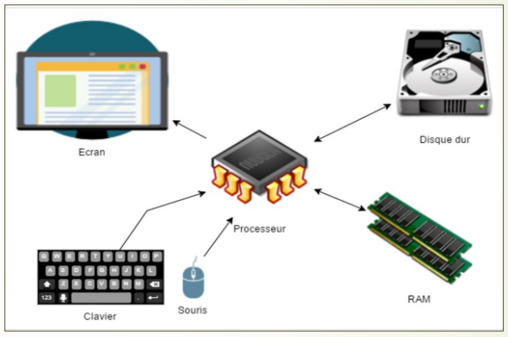
7

Fonctionnement d'un ordinateur

Principe de fonctionnement



Exemple



- **La mémoire centrale** mémorise toutes les informations nécessaires à l'exécution d'un programme: les données et les instructions(ordres).
- **Le processeur** exécute les instructions placées en mémoire centrale.

Farida Ait-Ammar et Nicolas Leduc

8

Actions d'un ordinateur (1/6)

- Les actions d'un ordinateur sont très limitées en nombre et se résument comme suit :

1. Communiquer une information élémentaire
2. Coder l'information
3. Déposer ou lire une information dans une case mémoire
4. Exécuter des opérations simples telles que l'addition et la soustraction
5. Comparer des valeurs

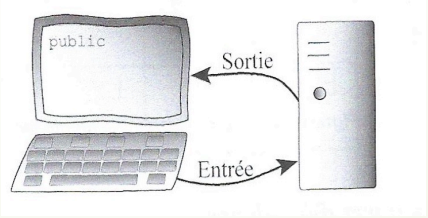
Farida Ait-Ammar et Nicolas Leduc

9

Actions d'un ordinateur (2/6)

1. Communiquer une information élémentaire

- Un programme traite d'information.
- Cette information est transmise par l'utilisateur par l'intermédiaire du clavier ou de la souris. On parle de **saisie** ou de **lecture** de données (*input* en anglais).
- Après traitement, le programme fournit des résultats à l'utilisateur par l'intermédiaire de l'écran ou sous forme de fichier. On parle **d'affichage** ou **d'écriture** de données (*output* en anglais).



source: livre de Java premier langage

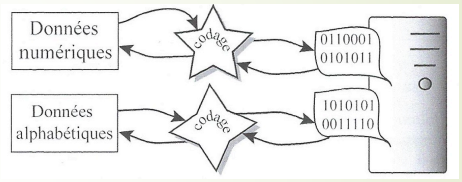
Farida Ait-Ammar et Nicolas Leduc

10

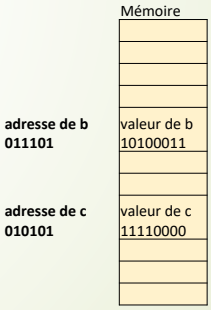
Actions d'un ordinateur (3/6)

2. Coder l'information

- L'ordinateur est constitué de composants électroniques qui ne perçoivent que **deux états** : composant allumé (1) ou composant éteint (0).
- Toute information fournie à l'ordinateur est codée en **binaire : un nombre, un caractère alphabétique, une instruction...**
- Exemple
  - L'instruction b + c peut être codé 0101011101010101
  - 0101 Le code de l'addition
  - 011101 L'adresse de la case mémoire où est stocké le premier nombre
  - 010101 L'adresse de la case mémoire où est stocké le deuxième nombre



source: livre de Java premier langage



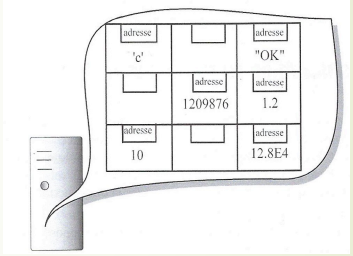
Farida Ait-Ammar et Nicolas Leduc

11

Actions d'un ordinateur (4/6)

3. Déposer ou lire une information dans une case mémoire

- La mémoire est composée de cases qui possèdent chacune une adresse (un numéro).
- Chaque case peut contenir une seule information.
- Lorsque l'ordinateur place une information dans une case mémoire, il doit mémoriser son adresse pour la retrouver ultérieurement.
- Notes:
  - Lorsque l'ordinateur prend l'information dans une case mémoire, il ne fait que la lire sans la modifier.
  - Pour effacer une information dans une case mémoire, il faut placer une nouvelle information dans cette même case.



source: livre de Java premier langage

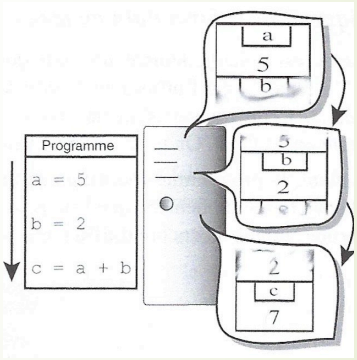
Farida Ait-Ammar et Nicolas Leduc

12

Actions d'un ordinateur (5/6)

4. Exécuter des opérations simples telles que l'addition et la soustraction

- Les opérations disponibles sont très limitées
- Le processeur **exécute les opérations dans l'ordre** dans laquelle elles sont écrites.



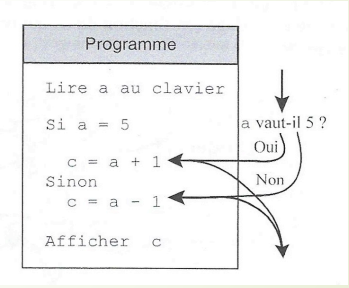
source: livre de Java premier langage

13

Actions d'un ordinateur (6/6)

5. Comparer des valeurs

- L'ordinateur est capable de comparer deux valeurs entre elles : plus grande, égale, différente...
- Grâce à la comparaison, le processeur est capable de tester une condition et d'exécuter une instruction plutôt qu'une autre en fonction du résultat du test et peut ainsi modifier l'ordre d'exécution des instructions.



source: livre de Java premier langage

14

L'ordinateur n'est qu'un exécutant

- L'ordinateur exécute des tâches beaucoup plus rapidement qu'un être humain, en revanche il n'est pas intelligent. Par exemple il n'est pas capable de choisir une action plutôt qu'une autre.
- L'ordinateur ne comprend pas le sens des instructions qu'il exécute, mais seulement leur forme. Ainsi, il ne peut qu'exécuter aveuglément une suite d'instructions sans prendre de décisions.
- Chaque instruction doit s'écrire avec des mots particuliers et une forme ou **syntaxe préétablie**.
- L'ensemble des mots constitue le **langage informatique** (ou de programmation) comme : *Java*, *C++*, *C#*, *Python*, *Perl*…

Farida Ait-Ammar et Nicolas Leduc

15

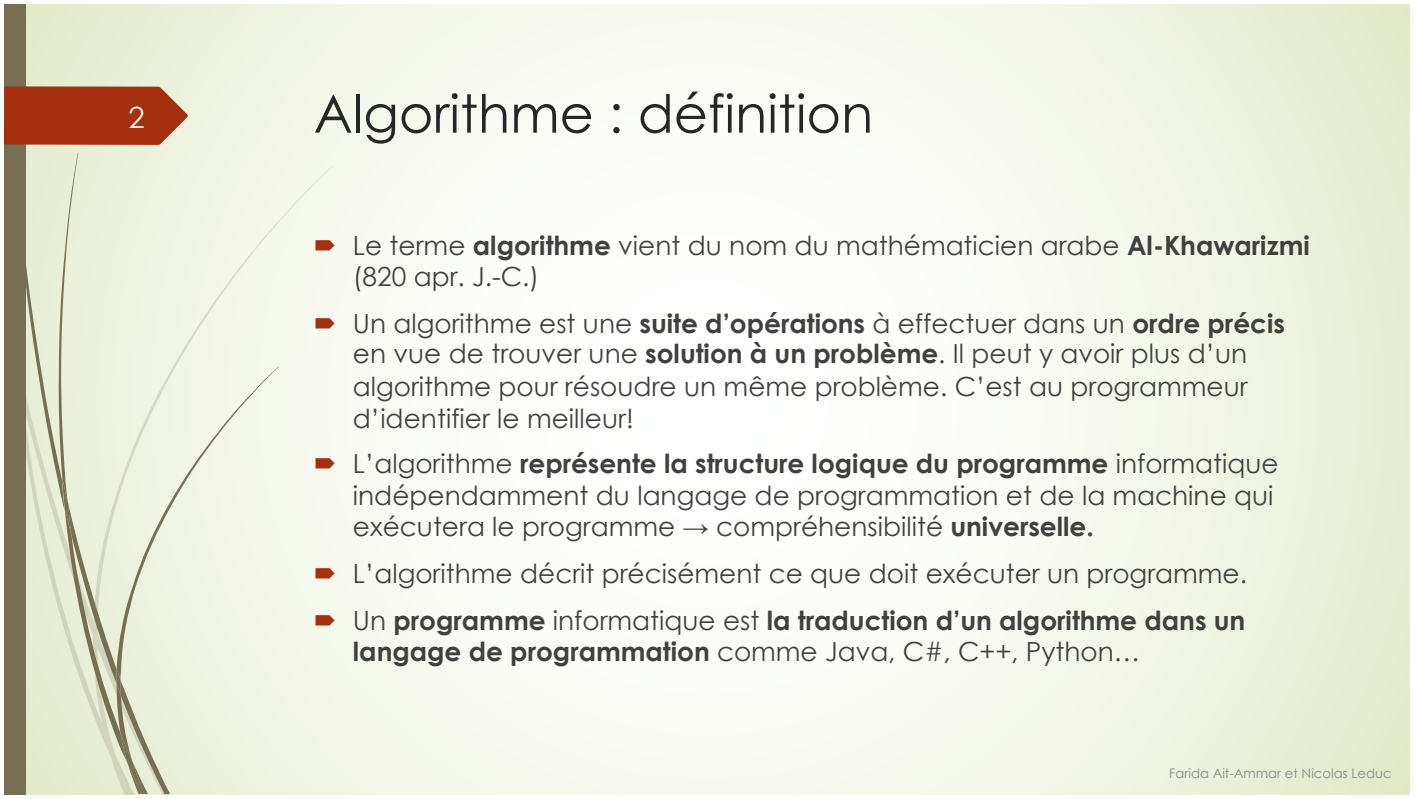
Sources

- *Débutez l'analyse logicielle avec UML*. (s.d.). Récupéré sur OpenClassrooms: [redacted]
- Tasso, A. (s.d.). *Le livre de JAVA premier langage avec 109 exercices corrigés.* Éditions EYROLLES.

Farida Ait-Ammar et Nicolas Leduc

2. Algorithmie

2.1 Notions de base



3

Algorithme vs programme : exemple

Algorithme qui additionne deux nombres.

```
ALGORITHME Somme

VARIABLES
  nombre1, nombre2, somme  TYPE NUMÉRIQUE

DÉBUT
  AFFICHER("Entrez le premier nombre: ")
  LIRE(nombre1)
  AFFICHER("Entrez le deuxième nombre: ")
  LIRE(nombre2)
  somme ← nombre1 + nombre2
  AFFICHER("Le résultat est ", somme)
FIN
```

Programmation équivalent en Java.

Somme.java

```java
package sandbox;

import java.util.Scanner;

public class Somme {
    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);
        int nombre1;
        int nombre2;
        int somme;

        System.out.print("Entrez le premier nombre: ");
        nombre1 = clavier.nextInt();
        System.out.print("Entrez le deuxième nombre: ");
        nombre2 = clavier.nextInt();

        somme = nombre1 + nombre2;

        System.out.println("Le résultat est " + somme);
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

4

Exemples d'algorithmes

De manière plus générale, un algorithme explique comment faire un travail sans la moindre ambiguïté. Par exemple:

- recette de cuisine
- notice de montage d'un meuble
- instructions pour préparer du café chaud non sucré
- méthode pour calculer la somme des nombre de 1 à 100 : 1 + 2 + … + 100

Farida Ait-Ammar et Nicolas Leduc

5

Traitement des données

- Un algorithme est une séquence d'instructions opérant sur des données.
- Ces séquences d'instructions sont appelées **traitements**.
- Les traitements fournissent de nouvelles données appelées **résultats**.

Farida Ait-Ammar et Nicolas Leduc

6

Construction d'un algorithme (1/4)

Construire un algorithme c'est analyser l'énoncé du problème dans le but de:

- définir les objets à manipuler : les variables
- identifier la sortie : le résultat
- lister les opérations élémentaires à faire : les instructions
- ordonner les opérations

Farida Ait-Ammar et Nicolas Leduc

7

Construction d'un algorithme (2/4)

Exemple de l'algorithme du café chaud.

- Énoncé : Comment préparer un café chaud non sucré?

Dans ce cas, l'exécutant est un humain et non un ordinateur.

1. **Définition des objets à manipuler**

   C'est la liste des ingrédients et ustensiles à manipuler:

   - café moulu
   - filtre
   - eau
   - cafetière électrique
   - pichet de la cafetière
   - tasse
   - électricité
   - table

Farida Ait-Ammar et Nicolas Leduc

8

Construction d'un algorithme (3/4)

2. **Liste des opérations**

- Verser l'eau dans la cafetière, le café dans la tasse, le café moulu dans le filtre.
- Remplir le pichet d'eau.
- Prendre du café moulu, une tasse, une cafetière électrique, un filtre, le pichet de la cafetière.
- Brancher, allumer ou éteindre la cafetière électrique.
- Attendre que le café remplisse le pichet.
- Poser la tasse sur la table, la cafetière sur la table, le filtre dans la cafetière, le pichet dans la cafetière.

Note : Ces opération sont élémentaires : elles ne peuvent pas être décomposées par rapport à l'environnement.

Farida Ait-Ammar et Nicolas Leduc

Construction d'un algorithme (3/4)

9

2. Liste des opérations

1. Verser l'eau dans la cafetière
2. Verser le café dans la tasse
3. Verser le café moulu dans le filtre
4. Remplir le pichet d'eau
5. Prendre du café moulu
6. Prendre une tasse
7. Prendre une cafetière électrique
8. Prendre un filtre
9. Prendre le pichet de la cafetière
10. Brancher la cafetière électrique
11. Attendre que le café remplisse le pichet
12. Allumer la cafetière électrique
13. Éteindre la cafetière électrique
14. Prendre le pichet de la cafetière
15. Poser la tasse sur la table
16. Poser la cafetière sur la table
17. Poser le filtre dans la cafetière
18. Poser le pichet dans la cafetière

Farida Ait-Ammar et Nicolas Leduc

Construction d'un algorithme: corrigé (4/4)

10

3. Liste des opérations ordonnées

7. Prendre une cafetière électrique
16. Poser la cafetière sur la table
8. Prendre un filtre
17. Poser le filtre dans la cafetière
5. Prendre du café moulu
3. Verser le café moulu dans le filtre
9. Prendre le pichet de la cafetière
4. Remplir le pichet d'eau
1. Verser l'eau dans la cafetière
18. Poser le pichet dans la cafetière
10. Brancher la cafetière électrique
12. Allumer la cafetière électrique
11. Attendre que le café remplisse le pichet
6. Prendre une tasse
15. Poser la tasse sur la table
13. Éteindre la cafetière électrique
14. Prendre le pichet de la cafetière
2. Verser le café dans la tasse

Farida Ait-Ammar et Nicolas Leduc

11

Précisions

- Un algorithme doit absolument respecter les règles suivantes:
  - Respect de **l'ordre des opérations** (exécution séquentielle)
  - Conduire à l'exécution d'un **nombre fini d'opérations**
  - Définir chaque opération avec **précision** (sans aucune ambiguïté)
  - Produire le **résultat prévu**
- L'algorithme n'est pas nécessairement unique, un problème peut avoir plusieurs solutions.
- Exemples:
  - Le café
  - La réussite de ce cours
- Ainsi, un bon programmeur doit être capable non seulement de trouver une solution à un problème, mais il doit, de plus, déterminer si c'est la meilleure!

Farida Ait-Ammar et Nicolas Leduc

12

Représentation d'un algorithme

- Un algorithme peut être formulé dans un « pseudo langage » qui est plus restreint et plus précis que le langage naturel. Toutefois, il **ne peut être exécuté directement par un ordinateur.**
- Deux types de notation :
  - **Pseudo-code**: utilise un langage presque naturel. Il est composé de quelques mots clés appartenant à une langue naturelle qui servent à exprimer des actions ou des indications.
  - **Ordinogramme:** représentation graphique qui utilise des symboles spéciaux comme le losange, le rectangle, etc. afin de représenter les traitements et les structures de contrôle.

Farida Ait-Ammar et Nicolas Leduc

2.2 Analyse du problème

14

Analyse du problème: définition

- Avant de se lancer dans l'écriture d'un algorithme, il est important de s'assurer que nous avons toutes les informations nécessaires pour résoudre notre problème. En effet, il peut arriver que les données que l'usager désire entrer à la console soient insuffisantes pour calculer le résultat recherché.
- **La phase d'analyse consiste à identifier les entrées, les sorties ainsi que les équations qui permettent de les relier**. Il n'y a pas de contraintes spécifiques sur la présentation de l'analyse comme c'est le cas pour le pseudo-code. On doit simplement s'assurer que toutes les données pour réaliser un calcul sont soit inscrites par l'usager ou calculées par une autre formule mathématique.
- Une stratégie d'analyse consiste à partir de la sortie et à identifier l'équation qui permet de l'obtenir. Pour chacune des données faisant partie de l'équation, soit on trouve une nouvelle équation qui permet de l'obtenir ou on l'ajoute à la liste des entrées, ce qui signifie que c'est l'usager qui devra indiquer sa valeur. On refait systématiquement la même procédure pour chaque équation que l'on ajoute.

Farida Ait-Ammar et Nicolas Leduc

15

Analyse: exemple problématique

- Calculez la quantité de peinture nécessaire pour peindre une pièce. L'usager veut entrer le nombre de mètres carrés couvert par un litre de peinture ainsi que la longueur et la largeur de la pièce.
- Analyse:
  - Entrées: longueur, largeur, mètres carrés par litre
  - Traitements:
    - surface = 2 * longueur * hauteur + 2 * largeur * hauteur
    - nb litre = surface à couvrir / mètres carrés par litre
  - Sortie: nombre de litres
- Dans cet exemple, on voit que la hauteur de la pièce n'a pas été mentionné dans l'énoncé du problème, mais qu'elle est nécessaire pour obtenir la sortie voulue.

Farida Ait-Ammar et Nicolas Leduc

16

Analyse: exemple

- Écrivez un algorithme qui permet de calculer le coût total d'un produit, incluant les taxes. L'usager désire inscrire la quantité achetée ainsi que le prix unitaire. Les valeurs de la TPS et de la TVQ sont de 5% et 9,975% respectivement.
- Analyse:
  - Entrées: prix unitaire, quantité achetée
  - Traitements:
    - total produit = prix * quantité
    - montant TPS = total produit * 5%
    - montant TVQ = total produit * 9,975%
    - total = total produit + montant TPS + montant TVQ
  - Sortie: coût total

Farida Ait-Ammar et Nicolas Leduc

2.3 Pseudo-code

18

Exemple : algorithme en pseudo-code

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
    montantTPS, montantTVQ  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    montantTPS ← totalProduit * TPS
    montantTVQ ← totalProduit * TVQ
    total ← totalProduit + montantTPS + montantTVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

19

Exemple : algorithme en pseudo-code

- Algorithme qui calcule le coût, incluant les taxes, d'un produit
  solution alternative

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

20

Mots clés

- Les mots clés sont des mots ayant une **signification particulière** lorsque l'on écrit du pseudo-code. Ils permettent de délimiter les différentes sections de l'algorithme ou d'indiquer des structures de contrôle.
- Dans un algorithme en pseudo-code, on écrit les **mots clés en majuscules**.
- Dans le cadre de ce cours, nous utiliserons les mots clés suivants:
  - ALGORITHME
    - Définit le nom de l'algorithme
  - CONSTANTES, VARIABLES
    - En-tête de la section qui contient des constantes ou des variables respectivement
  - TYPE NUMÉRIQUE, CARACTÈRE, CHAÎNE, BOOLÉEN
    - Permet de définir les types de variables utilisés dans l'algorithme
  - DÉBUT, FIN
    - Délimite la liste des instructions contenues dans l'algorithme
  - LIRE, AFFICHER
    - Permet respectivement de lire une information provenant de l'usager ou d'afficher un résultat

Farida Ait-Ammar et Nicolas Leduc

21

Exemple : mots clés

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

22

Constante

- Une constante est une **donnée** qui reste **invariable** durant toute l'exécution du programme. Elle ne change donc pas après son initialisation.
- Une constante est **caractérisée** par un **nom** et une **valeur.**
- L'identificateur de la constante est écrit en **majuscule**. Si le nom est composé de plusieurs mots, on utilise le symbole « _ » pour séparer les mots.
- **Syntaxe**

```
CONSTANTES
    <ID_CONSTANTE> = <expression>
```

- **Exemples**

```
CONSTANTES
    NB_HEURES = 24       // Nombre d'heures par jour
    NB_LETTRES = 26      // Nombre de lettres dans l'alphabet latin
    SALUTATIONS = "Allo" // Constante de type chaîne de caractères
```

Farida Ait-Ammar et Nicolas Leduc

23

Exemple : constante

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

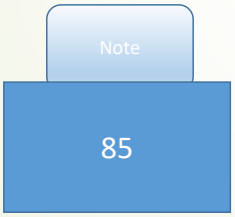
```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

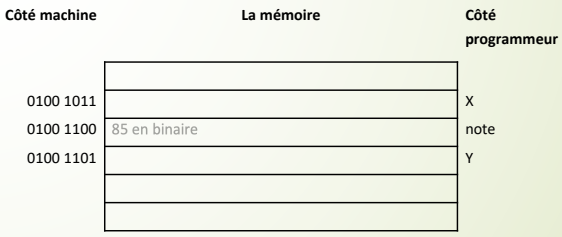
Farida Ait-Ammar et Nicolas Leduc

24

Variable

- Une variable permet de stocker provisoirement les valeurs des données en entrée ainsi que les résultats des traitements.
- Une variable est une **zone en mémoire** qui possède trois caractéristiques :
  - un **identificateur**: le nom attribué à la donnée;
  - un **type**: définit le domaine de la variable (nombre, chaîne de caractère, etc.);
  - une **valeur**: le contenu de la variable.





Farida Ait-Ammar et Nicolas Leduc

25

Nom de la variable (identificateur)

- Chaque **langage** de programmation a ses **particularités** en ce qui concerne le choix du nom des variables. Les règles ci-dessous concernent le langage Java. Vous devez aussi les respecter lorsque vous écrivez votre pseudo-code.
- Le nom d'une variable doit être en lettres minuscules, sauf s'il est composé de plusieurs mots. Dans ce cas, le premier mot commence par une lettre minuscule et tous les mots suivants débutent par une majuscule. (*camel case*)
  - Exemples: nomEtudiant, salaire, prixVenteMinimum.
- Le choix du nom doit être un court mnémonique et indiquer au lecteur l'utilité de la variable. **Il ne doit pas être un mot réservé du langage**.
- Le nom d'une variable avec un caractère unique comme `i`, `j`, `k` doit être évité; sauf pour une variable temporaire à usage local.
- Le nom d'une variable ne doit pas commencer par un blanc souligné «_», un symbole dollar «$» ou un nombre.
- N'utilisez que les caractères `[a-z]`, `[A-Z]` et `[0-9]`.
- **Vous ne devez pas utiliser les lettres accentuées.**

Farida Ait-Ammar et Nicolas Leduc

26

Type de la variable

- Le type de la variable détermine le **genre de données** qu'elle va enregistrer.
- Dans un langage de programmation, le choix du type de variable **influence la taille de son emplacement mémoire** et le type de codage utilisé.

Type	Utilité	Notation	Exemples de valeur
Numérique	Représente un nombre quelconque entier ou réel.	On utilisera la forme d'écriture 1234.5678 pour les nombres réels.	12 0.5
Caractère	Représente un seul caractère.	Un caractère sera noté entre deux apostrophes simples	'a' '!'
Alphanumérique ou chaîne de caractères	Représente un texte. Peut comporter des lettres, des chiffres, des signes de ponctuation, ou des espaces.	Une chaîne sera notée entre guillemets doubles.	"Entrez un nombre" "Option#1: Addition"
Booléen	Représente les valeurs logiques binaires vrai et faux.		VRAI FAUX

Farida Ait-Ammar et Nicolas Leduc

27

Déclaration de variables

- On doit **déclarer** une variable **avant de l'utiliser** pour mémoriser une donnée.
- Spécifier son nom (ou identificateur) et son type selon une syntaxe prédéfinie dans la section prévue à cet effet.
- **Syntaxe**

```
VARIABLES
    <id_variable> TYPE <TYPE_VARIABLE>
    <id_variable1>, <id_variable2>, … TYPE <TYPE_VARIABLE>
```

- Exemple de déclaration d'une variable, **reponse**, de type **caractère** et deux variables, **salaire** et **taux**, de type **numérique**.

```
VARIABLES
    reponse TYPE CARACTÈRE
    salaire, taux TYPE NUMÉRIQUE
```

Farida Ait-Ammar et Nicolas Leduc

28

Exemple : variables

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

29

Instruction de sortie: écriture

- L'instruction d'écriture permet au programme de **transférer une valeur vers un équipement externe**. Généralement, elle consiste à afficher des résultats à l'écran, que l'on appelle également sortie standard.
- **Syntaxe**

```
AFFICHER(<expression>)
AFFICHER(<expression1>, <expression2>, …, <expressionN>)
```

- Exemples:

```
AFFICHER(6)
AFFICHER(nom)
AFFICHER("La somme est: ", n + m)
    Si n = 10 et m = 90, l'affichage obtenu sera:
```

La somme est : 100

- Il est fortement recommandé d'utiliser une instruction d'écriture avant une instruction de lecture pour préciser à l'utilisateur quelle information il doit entrer.
  - Exemple:

```
AFFICHER("Entrez le nom de l'étudiant")
LIRE(nom)
```

Farida Ait-Ammar et Nicolas Leduc

30

Instruction d'entrée: lecture

- L'instruction de lecture est une action réciproque de celle d'écriture.
- Elle permet à l'utilisateur **d'entrer une donnée** à partir du clavier, appelé également entrée standard, pour qu'elle soit utilisée par le programme.
- **Syntaxe**

```
LIRE(<id_variable>)
```

- Exemple

```
LIRE(nom)
```

- Effet de l'exécution de l'instruction de lecture:
  - l'ordinateur **interrompt l'exécution** du programme et attend que l'on tape une valeur à l'aide du clavier.
  - l'utilisateur **termine** la saisie en appuyant sur la touche **Entrée** (Enter en anglais).
  - la **valeur saisie** est **affectée à la variable** indiquée (ici *nom*).

Il est par conséquent important que cette valeur soit du **même type** que la variable spécifiée dans l'instruction de lecture.

Farida Ait-Ammar et Nicolas Leduc

31

Exemple : écriture et lecture

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

32

Instruction d'affectation

- L'affectation consiste à **attribuer une valeur** à une **variable**.
- En pseudo-code, on utilise le symbole **flèche gauche** « ← » pour noter une affectation.
- **Syntaxe**

```
variable ← <expression>
```

- Exemples
  - On peut attribuer une constante intrinsèque à une variable.
    `nombre ← 12.5`
  - On peut attribuer (copier) la valeur d'une variable à une autre variable.
    `nombre1 ← nombre2`
    La valeur de nombre1 est maintenant celle de nombre2, sans modifier cette dernière.
  - On peut attribuer la valeur (résultat) d'une expression à une variable.
    `y ← x + 9`
    Si x contenait 10, la valeur de y devient 19. La variable x n'est pas modifiée dans ce cas.
  - On peut attribuer à une variable la valeur d'une expression dans laquelle elle figure.
    `i ← i + 1`
    Si *i* valait 7 avant l'exécution de l'affectation, *i* vaut 8 à la suite de l'affectation. La **variable *i* est modifiée**, car elle se trouve à **gauche de l'affectation**.

Farida Ait-Ammar et Nicolas Leduc

33

Expression numérique

- Une **expression numérique** combine des **opérandes** et des **opérateurs** pour **calculer une valeur** qui doit être du **même type** que la **variable** à laquelle elle est affectée.
- Un **opérande** est soit une **valeur**, soit un **nom** qui donne accès à une valeur comme des identificateurs de variables, de constantes ou des appels de fonctions.
- Un **opérateur** correspond à une **opération** sur un ou plusieurs opérandes. Il est identifié par un symbole («+», «–», «/», …) ou une chaîne de caractères («`div`», «`mod`», …) spécifiant une opération.
- Exemple

```
i ← n * 2
```

  - **Les opérandes** sont la variable **`n`** et la constante **`2`**
  - **L'opérateur** utilisé est **`*`** qui représente **l'opération de multiplication**.

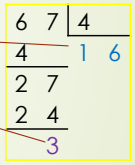
- Note: une **expression** doit toujours **se trouver à droite de la flèche** de l'affectation, jamais à gauche.

Farida Ait-Ammar et Nicolas Leduc

34

Opérateurs arithmétiques

Opération	Symbole	Type d'opérandes	Type du résultat	Exemple
Addition	+	numérique	numérique	3 + 4 = 7
Soustraction	-			4.2 – 5 = -0.8
Multiplication	*			3 * 2.5 = 7.5
Division	/			67 / 4 = 16.75
Division entière	÷	numérique (entier)	numérique (entier)	67 ÷ 4 = 16
Modulo: reste de la division entière *a % b = a – (a ÷ b) * b*	%			67 % 4 = 3 *67 - (67 ÷ 4) * 4 = 3*
Opposé	- (unaire)	numérique	numérique	-3.5



Farida Ait-Ammar et Nicolas Leduc

35

Exemple : affectations et expressions

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

36

Exemple : deux produits?

- Algorithme qui calcule le coût, incluant les taxes, de deux produits

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du premier produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Entrez le prix du second produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    totalProduit ← prixProduit * quantiteProduit
    total ← total + totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

37

Syntaxe d'un algorithme simple incluant des constantes

- Les constantes sont déclarées avant les variables

```
ALGORITHME <id_algorithme>
CONSTANTES
   // Déclaration des constantes
VARIABLES
   // Déclaration des variables
DÉBUT
   // Instructions
FIN
```

Farida Ait-Ammar et Nicolas Leduc

2.4 Ordinogramme

39

Ordinogramme: définition

- Un **ordinogramme** est une **représentation graphique de l'algorithme** d'un programme à l'aide de **symboles normalisés**. Il permet de visualiser la structure du programme et les différentes étapes du traitement.
- Autres appellations: organigramme de programmation, algorigramme, logigramme
- La représentation graphique permet une lecture plus facile des algorithmes. Cependant, elle n'est pas adaptée pour les algorithmes longs, car elle **consomme beaucoup d'espace**.
- La norme *ISO 5807* décrit en détail les différents symboles à utiliser pour construire un ordinogramme de manière normalisée.

Farida Ait-Ammar et Nicolas Leduc

40

Ordinogramme: symboles

Symbole	Utilisation	Exemple
Début et fin	Début, fin ou interruption d'un ordinogramme	Début / Fin
Traitement	Opération ou groupe d'opérations sur des données.	i ← i + 1
Entrée / sortie	Représente les données qui sont disponibles en entrée ou en sortie.	Lire: rayon / Afficher: "La somme est : ", somme
Renvoi	Lorsqu'un algorithme est scindé, ce symbole est utilisé deux fois pour assurer sa continuité.	A / A
Liaison	Le sens général des liaisons est de haut en bas et de gauche à droite.	

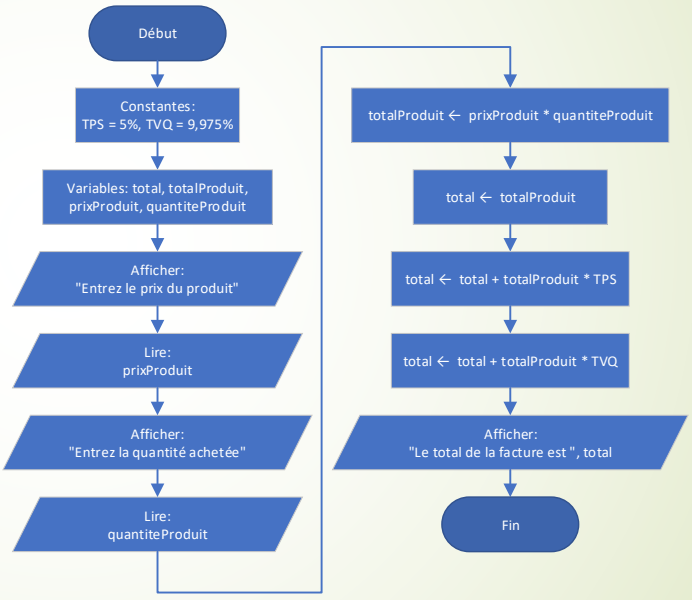
Farida Ait-Ammar et Nicolas Leduc

41

Exemple : pseudo-code vs ordinogramme

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
      quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
      AFFICHER("Entrez le prix du produit")
      LIRE(prixProduit)
      AFFICHER("Entrez la quantité achetée")
      LIRE(quantiteProduit)
      totalProduit ← prixProduit * quantiteProduit
      total ← totalProduit
      total ← total + totalProduit * TPS
      total ← total + totalProduit * TVQ
      AFFICHER("Le total de la facture est ", total)
FIN
```



Farida Ait-Ammar et Nicolas Leduc

2.5 Trace d'exécution

43

Traces d'exécution: définition

- La **trace d'exécution** est un outil, sous forme d'un **tableau**, qui permet de vérifier le bon fonctionnement d'un algorithme. Elle montre **l'évolution des valeurs de chacune des variables** tout au long du déroulement de l'algorithme (**exécution pas-à-pas**).
- Construction du tableau de la trace
  - Les **colonnes** du tableau correspondent aux **variables** déclarées dans l'algorithme. Le titre des colonnes est le nom des variables.
  - Le titre des **lignes** est le numéro des **lignes de l'algorithme**.
  - Dans une **cellule**, on inscrira la **valeur de la variable** correspondante seulement si elle a changé après l'exécution de l'instruction.
  - On peut ajouter une colonne « Console » pour simuler l'affichage à la console.
- Vidéo explicative: [redacted]

Farida Ait-Ammar et Nicolas Leduc

44

Exemple : trace d'exécution

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
      quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
1     AFFICHER("Entrez le prix du produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     totalProduit ← prixProduit * quantiteProduit
6     total ← totalProduit
7     total ← total + totalProduit * TPS
8     total ← total + totalProduit * TVQ
9     AFFICHER("Le total de la facture est ", total)
FIN
```

Pour cette trace on suppose que l'usager entre un prix de 10$ et une quantité de 5.

#	total	total Produit	prix Produit	quantite Produit	Console
1					
2					
3					
4					
5					
6					
7					
8					
9					

Farida Ait-Ammar et Nicolas Leduc

45

Exemple : trace d'exécution

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME CalculFacture
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
      quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
1     AFFICHER("Entrez le prix du produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     totalProduit ← prixProduit * quantiteProduit
6     total ← totalProduit
7     total ← total + totalProduit * TPS
8     total ← total + totalProduit * TVQ
9     AFFICHER("Le total de la facture est ", total)
FIN
```

Pour cette trace on suppose que l'usager entre un prix de 10$ et une quantité de 5.

#	total	total Produit	prix Produit	quantite Produit	Console
1					Entrez le prix du produit
2			10		
3					Entrez la quantité achetée
4				5	
5		50			
6	50				
7	52,50				
8	57,49				
9					Le total de la facture est 57,49

Exemple : deux produits?

46

- Algorithme qui calcule le coût, incluant les taxes, de deux produits

```
ALGORITHME CalculFacture
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
      quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
1     AFFICHER("Entrez le prix du premier produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     totalProduit ← prixProduit * quantiteProduit
6     total ← totalProduit
7     total ← total + totalProduit * TPS
8     total ← total + totalProduit * TVQ
9     AFFICHER("Entrez le prix du second produit")
10    LIRE(prixProduit)
11    AFFICHER("Entrez la quantité achetée")
12    LIRE(quantiteProduit)
13    totalProduit ← prixProduit * quantiteProduit
14    total ← total + totalProduit
15    total ← total + totalProduit * TPS
16    total ← total + totalProduit * TVQ
17    AFFICHER("Le total de la facture est ", total)
FIN
```

Pour cette trace on suppose que l'usager entre un premier prix de **5**$ avec une quantité de **4** et un deuxième prix de **15**$ avec une quantité de **2**.

#	total	total Produit	prix Produit	quantite Produit	Console
1					Entrez le prix du premier produit
2			5		
3					Entrez la quantité achetée
4				4	
5		20			
6	20				
7	21				
8	23				
9					Entrez le prix du second produit
10			15		
11					Entrez la quantité achetée
12				2	
13		30			
14	53				
15	54,50				
16	57,49				
17					Le total de la facture est 57,49

47

Trace d'exécution
Liste de vérification

- Numérotez les lignes de l'algorithme se trouvant entre DÉBUT et FIN.
- Si vous avez *n* variables, votre tableau contiendra *n* + 2 colonnes, soit une pour chaque variable, une pour le numéro de ligne et une pour la console.
- Inscrivez le nom de chacune de vos variables sur la première ligne du tableau.
- Créez un nombre de lignes égal au nombre de lignes contenues dans votre algorithme et inscrivez les numéros de ligne dans la colonne prévue à cet effet.
- Le résultat d'une instruction AFFICHER doit apparaître dans la colonne « Console » de la ligne correspondante. Attention, vous devez remplacer le nom des variables contenues dans l'instruction d'affichage par leurs valeurs. Par exemple, si *note* vaut 85, le résultat de l'instruction AFFICHER("Votre note finale est : ", note) doit apparaître de la manière suivante dans la colonne « Console » :
  - Votre note finale est : 85
- Pour les instructions de lecture ou d'affectation, la valeur résultant de l'opération doit apparaître dans la colonne de la variable se trouvant à gauche de l'affectation ou de celle inscrite entre parenthèses dans le cas d'une instruction LIRE.

Farida Ait-Ammar et Nicolas Leduc

48

Sources

- Smith, J. A. (2007). Java structuré. Groupe Modulo.
- Tasso, A. (s.d.). Le livre de JAVA premier langage avec 109 exercices corrigés. Éditions EYROLLES.

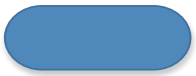
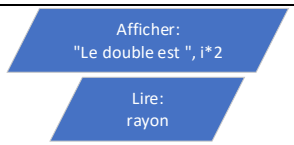
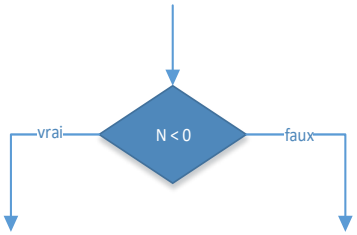
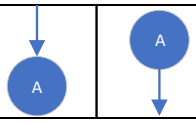
Farida Ait-Ammar et Nicolas Leduc

2.6 Ordinogramme : référence

COLLÈGE MONTMORENCY

Ordinogrammes : référence

Les principaux symboles utilisés pour construire les ordinogrammes

Symbole	Utilisation	Exemples
Début et Fin 	Début, fin ou interruption d'un ordinogramme.	
Traitement	Opération ou groupe d'opérations sur des données.	
Entrée / Sortie	Représente les données qui sont disponibles en entrée ou en sortie.	
Sous-programme	Appel d'une portion de programme considérée comme une simple opération.	
Test ou branchement conditionnel	Pour représenter une décision ou un aiguillage après un test. L'entrée du test est l'angle du haut. Il y a uniquement deux sorties : vrai et faux.	
Renvoi	Lorsqu'un algorithme est scindé, ce symbole est utilisé deux fois pour assurer sa continuité.	
Liaison	Le sens général des liaisons est de haut en bas et de gauche à droite.	

Exemple

Un algorithme qui calcule la somme de deux nombres en pseudo-code et en ordinogramme.

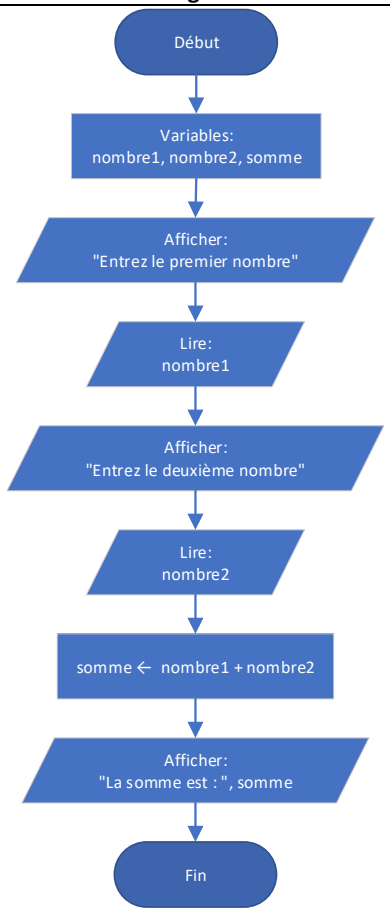
Pseudo-code

```
ALGORITHME CalculSomme

VARIABLES
    nombre1, nombre2, somme TYPE NUMÉRIQUE

DÉBUT
    AFFICHER("Entrez le premier nombre")
    LIRE(nombre1)
    AFFICHER("Entrez le deuxième nombre")
    LIRE(nombre2)
    somme ← nombre1 + nombre2
    AFFICHER("La somme est : ", somme)
FIN
```

Ordinogramme



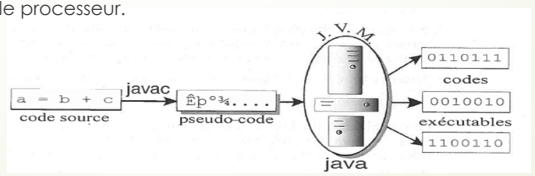
3. Introduction à *Java*

3.1 Structure d'un programme

2

Application *Java*

- Pour exécuter une application *Java*, vous devez réaliser les étapes suivantes:
  - Écriture du code source *Java*
    - Le code source est le programme que vous écrivez en *Java*. Il peut être contenu dans un ou plusieurs fichiers `.java` .
    - **Attention**: Le nom du fichier contenant le code source (`.java`) doit être le même que celui de la classe publique qu'il contient, en respectant les majuscules/minuscules.
  - Compilation du programme *Java*
    - Si votre code source ne comporte pas d'erreurs, le compilateur (`javac`) crée le fichier binaire `.class` avec le même nom que votre fichier `.java` d'origine. Le pseudo-code binaire obtenu est un langage intermédiaire qui se situe entre le code source et le code machine.
    - *Visual Studio Code* exécute automatiquement le compilateur après chaque enregistrement de votre fichier source.
  - Exécution du programme *Java*
    - Le pseudo-code obtenu lors de la compilation est indépendant d'une plateforme particulière et peut être exécuté sur divers systèmes par l'interpréteur de la *Machine Virtuelle Java* (*JVM*) (`java`). Cette dernière convertit le pseudo-code binaire en code machine qui peut être exécuté sur le processeur.



Farida Ait-Ammar et Nicolas Leduc

3

Structure d'un programme *Java*

```
// Package;

// Imports;

public class PremierExemple {
    // Constantes;

    public static void main(String[] args) {
        // Variables;

        // Instructions;
    }

    // Autres méthodes
}
```

Farida Ait-Ammar et Nicolas Leduc

4

Structure d'un programme *Java*

- `public class PremierExemple { … }`
  - `public` : mot clé qui indique que la classe est accessible au moment d'exécuter le programme. Elle pourra éventuellement être utilisée par d'autres classes.
  - `class` : mot clé qui permet de définir un conteneur (classe) pour les méthodes, variables et constantes que vous allez créer. Tout programme Java doit se trouver à l'intérieur d'une classe.
  - `PremierExemple` : nom de la classe qui sera créée. Par convention, le nom d'une classe doit commencer par une lettre majuscule. En *Java*, le nom de la classe et le nom du fichier `.java` qui la contient doivent être identique.
  - `{ }` : les accolades indiquent le début et la fin de la classe.

Farida Ait-Ammar et Nicolas Leduc

5

Structure d'un programme *Java*

- `public static void main(String[] args) { ... }`
  - Cette ligne indique à la *Machine Virtuelle Java* le début du programme. La méthode appelée `main()` est la première à s'exécuter quand un programme est lancé. C'est la méthode principale du programme.
  - `public` : mot clé qui indique que la méthode sera accessible au moment de l'exécution.
  - `static` : mot clé qui indique que la méthode peut être appelée sans avoir à créer une instance de la classe.
  - `void` : mot clé qui indique que la méthode ne retourne aucune valeur.
  - `main()` : nom de la méthode. La méthode nommée `main()` est réservée pour le programme principal. Les paramètres de la méthode sont indiqués entre parenthèses.
  - `String[] args` : la méthode *main* reçoit les arguments inscrits sur la ligne de commande par ce paramètre.
  - `{ }` : les accolades indiquent le début et la fin de la méthode `main()` .

Farida Ait-Ammar et Nicolas Leduc

3.2 Constante et variable

7

Constantes et variables Définitions

- Une **constante** est une donnée qui reste **invariable** durant toute l'exécution du programme. Elle ne change donc pas après son initialisation.
- Une **variable** permet de **stocker provisoirement** les valeurs des données en entrée ainsi que les résultats des traitements.
- En ***Java***, pour déclarer **une constante ou une variable**, il faut indiquer minimalement **son type et son nom**.
  - **type**: permet de spécifier l'espace mémoire à réserver et le genre de données qui y seront stockées
  - **nom**: identificateur qui permet d'accéder à la variable. Il doit respecter les règles de nomenclature définis dans les **normes de codage**

Farida Ait-Ammar et Nicolas Leduc

3.2.1 Types de données

8

Types en *Java*

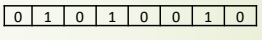
- Types simples (primitifs)
  - En *Java*, il existe 8 types primitifs
    - 6 types numériques
      - 4 types d'entiers : `byte`, `short`, `int`, `long`
      - 2 types de réels à virgule flottante : `float`, `double`
    - le type caractère : `char`
    - le type booléen : `boolean`
  - Tous les types primitifs sont écrits uniquement en lettres minuscules
- Types composés (classes)
  - Les classes regroupent différents types de données ainsi que des méthodes et peuvent être utilisées comme type de variable
  - Le nom d'une classe débute toujours par une lettre majuscule.
    - Exemple : `String`, `FenetreDessin`

Farida Ait-Ammar et Nicolas Leduc

9

Types simples

- Une variable de type **simple** ne peut contenir qu'**une seule valeur** à un instant donné.
- Le **type** d'une variable détermine sa **taille**, soit le nombre d'octets qu'elle occupe en mémoire, ainsi que son **encodage** en binaire.
  - Un **octet** est un regroupement de **8 bits**.
  - Un **bit** ne peut prendre comme valeur que **0** ou **1**.



  - Pour un octet, les valeurs admissibles sont comprises entre 00000000 et 11111111.
    - Exemple : 00000001, 00000010, 00100011
  - En général, avec *n* bits on peut encoder 2^n valeurs différentes qui seront comprises dans l'intervalle $[-2^{n-1} ; 2^{n-1} -1]$
    - Pour un octet, on peut coder 256 valeurs comprises dans l'intervalle $[-2^7 ; 2^7 -1]$ (-128 à 127).
    - Exemples : 85 et -4 peuvent être encodés sur un octet, mais pas 148 ou -129.

Farida Ait-Ammar et Nicolas Leduc

10

Types simples numériques

Catégorie	Type *Java*	Taille (octets)	Intervalle de valeurs
Entier (Ex: 25)	`byte`	1	-128 à 127 (-2^7 à 2^7-1)
	`short`	2	-32 768 à 32 767 (-2^{15} à $2^{15}-1$)
	`int`	4	-2 147 483 648 à 2 147 483 647 (-2^{31} à $2^{31}-1$)
	`long`	8	-9 223 372 036 854 775 808 à 9 223 372 036 854 775 807 (-2^{63} à $2^{63}-1$)
Réel (Ex: 3.25)	`float`	4	±1.40239846E-45 à ±3.40282347E38
	`double`	8	±4.9406564584124654E-324 à ±1.797693134862316E308

Farida Ait-Ammar et Nicolas Leduc

11

Types simples numériques
Entiers

- En *Java*, un **littéral**, par exemple 2345, est par défaut de type **`int`**
- Pour spécifier un nombre de type **`long`**, on lui ajoute le **suffixe `l` ou `L`**
  - Exemples : `2345` est de type `int` alors que `2345l` ou `2345L` est de type `long`
- Considérons les déclarations suivantes

```
long g1 = 456975;           // Valide
long g2 = 45697512546987;   // Invalide
long g3 = 45697512546987L; // Valide
```

  - Dans le premier cas, *Java* est en mesure de faire la conversion de `int` à `long`
  - Dans le second cas, *Java* tente de transformer le littéral en `int` avant de l'affecter à la variable de type `long`, ce qui provoque une erreur de compilation: *The literal 45697512546987 of type int is out of range*
  - En ajoutant le caractère `l` ou `L`, comme dans le troisième cas, on spécifie que ce littéral est de type `long` et non de type `int`
- Le type *`int`* se révèle le plus pratique dans la majorité des cas

Farida Ait-Ammar et Nicolas Leduc

12

Types simples numériques
Réels / virgule flottante

- Une valeur de la catégorie des réels peut s'écrire de deux façons:
  - Notation décimale: on utilise le **point** comme séparateur décimal
    - Exemples: `2.0`, `67.1`, `-5.4`
  - Notation exponentielle: on utilise la lettre `E` (ou `e`) pour indiquer la valeur de l'exposant en puissances de 10
    - Exemple: `8.67E3` (8,67*10^{3}), `-8.0E10` (-8,0*10^{10}), `3.02E-15` (3,02*10^{-15})
- La précision des nombres réels est approximative. Elle sera de:
  - 6 chiffres significatifs (après la virgule) pour le type `float`
  - 15 chiffres significatifs (après la virgule) pour le type `double`
- En *Java*, toute **valeur réelle littérale** est définie par **défaut en double précision**. Par conséquent, dès qu'on utilise une variable de type **`float`**, le suffixe **`f` ou `F`** est nécessaire pour éviter les erreurs de compilation: *Type mismatch: cannot convert from double to float*
  - Exemples:

```
float y = 12.5;  // Invalide
float y = 12.5f; // Valide
```

Farida Ait-Ammar et Nicolas Leduc

13

Types simples
Booléen

- En *Java*, le type booléen est indiqué par le mot clé **`boolean`**
- Il ne peut prendre que deux valeurs: **`true`** et **`false`**
- Exemple:

```
boolean estPositif = true;
boolean estMarie = false;
```

Farida Aït-Ammar et Nicolas Leduc

14

Types simples
Caractères

- Le type caractère, **`char`** en *Java*, permet de coder **l'ensemble des caractères *Unicode*** de `'\u0000'` à `'\uffff'` sur deux octets (16 bits).
- Les 128 premiers caractères sont les codes *ASCII* et se notent entre apostrophes
  - Exemple: `'a'`, `'1'`, `'/'`, `'w'`
- Les caractères non disponibles sur le clavier peuvent être saisis en passant par l'identifiant numérique *Unicode* en hexadécimal du caractère selon la syntaxe:
  - `\uxxxx` où `xxxx` représente l'identifiant hexadécimal *Unicode*
  - Exemple: `'~'` a pour code `007E`, on peut donc l'écrire `'\u007E'`
- Les caractères *Unicode* peuvent aussi s'utiliser directement dans les chaînes de caractères:
  - `"\u007E/bin"` affiche `"~/bin"`

15

Type composé
Chaîne de caractères

- En *Java*, il n'existe pas de type simple pour stocker une chaîne de caractères. Toutefois, le type composé (classe) **`String`** nous permet de déclarer des variables de type **chaîne de caractères** et d'interagir avec celles-ci.
- La classe `String` ne contient pas une donnée unique, car elle regroupe l'ensemble des caractères faisant partie de la chaîne.
- La classe `String` nous offre aussi des méthodes pour interagir avec la chaîne de caractère qui y est stockée.
  - Exemple: Extraire un caractère spécifique, mettre la chaîne en majuscule, etc.
- En *Java*, une chaîne de caractères est indiqué entre guillemets anglais (`" "`)
  - Exemple: `"Bonjour à tous!"`, `"Le prix est de \"25\u20AC\""`
  - Pour inclure un guillemet à l'intérieur de la chaîne, on le précède d'un `\`
- Comme le type `String` est une classe, il s'écrit avec un `S` majuscule.

3.2.2 Déclaration et nomenclature

16

Identificateurs Nomenclature

- Certaines règles de nommage ne sont pas imposées par le langage, mais sont traditionnellement respectées. Ce sont des conventions qui ont pour objectif de rendre les programmes plus aisément compréhensibles, car plus lisibles. Elles donnent des informations sur la fonction de l'identificateur. Ainsi, selon la nomenclature utilisée, on peut facilement reconnaître s'il s'agit d'une variable, d'une constante, d'une méthode ou d'une classe.
- L'identificateur (nom) d'une constante ou d'une variable doit respecter les règles suivantes:
  - Général
    - Choisir des noms significatifs qui ne sont pas des mots réservés
    - Les identificateurs ne doivent pas débuter par un chiffre
    - N'utiliser que l'alphabet non-accentué (`A-Z`, `a-z`), les chiffres (`0-9`) et le caractère '_'
    - **Attention**: *Java* est sensible à la casse. L'identificateur `nbNotes` est différent de `nbnotes` .
  - Constante
    - Son nom doit être entièrement en majuscules. Les mots composés doivent être séparés par un caractère de soulignement (_).
  - Variable
    - Le nom d'une variable, incluant les paramètres de méthodes, doit être en lettres minuscules, sauf s'il est composé de plusieurs mots. Dans ce cas, le premier mot commence par une lettre minuscule et tous les mots suivants débutent par une majuscule.
    - Le nom d'une variable avec un caractère unique, comme `i`, `j`, `k`, doit être évité; sauf pour une variable à usage local.

Farida Ait-Ammar et Nicolas Leduc

17

Déclaration des constantes

- En *Java* le mot clé **`final`** sert à désigner une **constante**.
- Afin d'éviter des problèmes au cours des prochaines semaines, on vous recommande de déclarer vos constantes directement en dessous de la déclaration de la classe, soit entre `public class` et `public static void main()`.
- La syntaxe suivante doit être utilisée:
  - `// Pseudo-code: <identificateur> = <valeur>`
  - `public static final <type> <identificateur> = <valeur>;`
  - `public static final <type> <identificateur1> = <valeur1>, <identificateur2> = <valeur2>, …;`
- Exemples

```
// Pseudo-code: TAILLE = 1.84
public static final float TAILLE = 1.84f;
public static final int BONUS = 2;
public static final float TAUX_TPS = 0.05f, TAUX_TVQ = 0.09975f;
```

Farida Ait-Ammar et Nicolas Leduc

18

Déclaration des variables

- En *Java*, il faut d'abord **déclarer** une variable **avant** de pouvoir l'**utiliser**.
- Dans le cadre de ce cours, toutes les variables pertinentes doivent être déclarées au début de chacune de vos méthodes, sauf dans des cas exceptionnels, qui seront mentionnés par votre enseignant.
- On déclare une variable en spécifiant d'abord son type, puis son nom.
- Syntaxe
  - `// Pseudo-code: <identificateur> TYPE <type>`
  - `<type> <identificateur>;`
  - `<type> <identificateur1>, <identificateur2>, … ;`
- Exemples

```
// Pseudo-code: salaire TYPE NUMERIQUE
float salaire;
int superficie, nbRouleaux;
```

Farida Ait-Ammar et Nicolas Leduc

19

Déclaration: constantes et variables Exemple

```
package sandbox;

import java.util.Scanner;

public class Facture {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner (System.in);

        float total, totalProduit, prixProduit;
        int quantiteProduit;

        System.out.print("Entrez le prix du produit : ");
        prixProduit = clavier.nextFloat();
        System.out.print("Entrez la quantite achetee : ");
        quantiteProduit = clavier.nextInt();
        totalProduit = prixProduit * quantiteProduit;
        total = totalProduit;
        total = total + totalProduit * TPS;
        total = total + totalProduit * TVQ;
        System.out.println("Le total de la facture est " + total + "$.");
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

3.2.3 Initialisation et affectation

20

Variables
Initialisation et affectation

- Initialisation
  - Il est possible d'initialiser une variable lors de sa déclaration.
  - Syntaxe
    - `<type> <identificateur> = <valeur>;`
    - `<type> <identificateur1> = <valeur1>, <identificateur2> = <valeur2>, … ;`
  - Exemples

```
int nombre = 200;
String reponse = "oui", prenom = "Nicolas";
```

  - On peut omettre d'initialiser une variable lors de sa déclaration, mais il faut lui affecter une valeur avant de l'utiliser.
- Affectation
  - Après avoir déclaré une variable, il est possible de stocker une valeur à l'emplacement mémoire désigné.
  - Le caractère **=** est utilisé (équivalent à ← en pseudo-code) et signifie : **reçoit la valeur de**
  - Exemples

```
int p, n;       // Déclaration
n = 4;          // Affecte de la valeur 4 à la variable n
p = 5 * n + 1; // Calcule puis affecte la valeur obtenue à la variable p
```

Farida Ait-Ammar et Nicolas Leduc

21

Variables
Affecter une valeur aléatoire

- Il est possible de générer des nombres pseudo-aléatoires à l'aide de la librairie `commun.Outils`, fournie par votre enseignant, et de les enregistrer dans des variables. Pour ce faire, vous devez:
  - Importer la librairie `Outils` : `import commun.Outils;`
  - Appeler la fonction `Outils.Aleatoire.nextInt(int bound)` en lui fournissant la valeur maximale désirée pour le nombre aléatoire. Elle générera un nombre pseudo-aléatoire compris entre zéro inclus et la valeur inscrite en paramètre exclue. (`[0;bound[`).
  - Ainsi, pour obtenir un nombre entre `a` et `b` inclusivement (`[a;b]`), il faut appeler la fonction de la manière suivante:
    - `Outils.Aleatoire.nextInt(b - a + 1) + a`
- Exemple
  - On veut un entier compris entre 1 et 10 inclusivement:

```
int alea = Outils.Aleatoire.nextInt(10 - 1 + 1) + 1;
```

OU

```
int alea = Outils.Aleatoire.nextInt(10) + 1;
```

  - On veut un entier compris entre 15 et 25 inclusivement:

```
alea = Outils.Aleatoire.nextInt(25 - 15 + 1) + 15;
```

OU

```
alea = Outils.Aleatoire.nextInt(11) + 15;
```

Farida Ait-Ammar et Nicolas Leduc

22

Variables
Contrôler les valeurs aléatoires

- Si on veut contrôler la séquence pseudo-aléatoire afin de faciliter le débogage, on appelle la fonction `Outils.Aleatoire.setSeed(long seed)` une seule fois au début du programme, en lui fournissant la valeur de départ du générateur (`seed`).

```
import commun.Outils;

public class DemoAlea {
    public static void main(String[] args) {
        int al1, al2;

        al1 = Outils.Aleatoire.nextInt(10)+1;
        al2 = Outils.Aleatoire.nextInt(11)+15;
        System.out.print(al1 + " : " + al2);
    }
}
```

Génère des nombres différents à chaque exécution du programme, car le générateur pseudo-aléatoire se sert de l'horloge système pour choisir la valeur de départ.

```
import commun.Outils;

public class DemoSeed {
    public static void main(String[] args) {
        int al1, al2;

        Outils.Aleatoire.setSeed(12345);
        al1 = Outils.Aleatoire.nextInt(10)+1;
        al2 = Outils.Aleatoire.nextInt(11)+15;
        System.out.print(al1 + " : " + al2);
    }
}
```

À chaque exécution `al1` vaut `2` et `al2` vaut `21`, car le générateur pseudo-aléatoire démarre toujours avec la valeur fournie, soit `12345`. On vous indiquera la valeur de départ dans les ateliers et les travaux pratiques.

Farida Ait-Ammar et Nicolas Leduc

23

Opérateurs arithmétiques et priorités

Priorité	Opérateur	Dénomination	Effet	Exemple	Résultat (avec x = 7)
+		Appel de fonction	Appel remplacé par sa valeur de retour		
	( )	Parenthèses	Augmente la priorité de l'opération		
	– unaire	- unaire	Donne l'opposé d'un nombre	`-x`	-7
	*	Opérateur de multiplication	Multiplie deux valeurs	`x * 3`	21
	/	Opérateur de division	Divise deux valeurs	`x / 3`	2
	%	Opérateur modulo	Effectue la division et détermine le reste a % b = a – (a ÷ b) * b	`x % 3`	1
	+	Opérateur d'addition	Additionne deux valeurs	`x + 3`	10
	–	Opérateur de soustraction	Soustrait deux valeurs	`x - 3`	4
-	=	Opérateur d'affectation	Affecte une valeur à une variable	`x = 3`	Stocke la valeur `3` dans la variable `x`

La **priorité des opérations** précise **l'ordre** dans lequel les **calculs sont effectués** dans une expression complexe. Lorsqu'elle contient plusieurs opérateurs de **même priorité** (opérateurs adjacents ayant la même couleur de fond), les calculs s'effectuent de **gauche à droite**.

Farida Aït-Ammar et Nicolas Leduc

24

Variables: affectation Exemple

```
package sandbox;

import java.util.Scanner;

public class Facture {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner (System.in);

        float total, totalProduit, prixProduit;
        int quantiteProduit;

        System.out.print("Entrez le prix du produit : ");
        prixProduit = clavier.nextFloat();
        System.out.print("Entrez la quantite achetee : ");
        quantiteProduit = clavier.nextInt();
        totalProduit = prixProduit * quantiteProduit;
        total = totalProduit;
        total = total + totalProduit * TPS;
        total = total + totalProduit * TVQ;
        System.out.println("Le total de la facture est " + total + "$.");
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

3.2.4 Conversion entre types numériques

26

Conversions entre types numériques Conversion implicite

- Dans une expression arithmétique, il arrive que nous combinions des variables de types numériques différents. Dans ce cas, *Java* fait une conversion automatique de type pour obtenir le résultat.
  - Exemple : `2 / 10.0f` → `int / float` → tout est converti automatiquement en `float`
- *Java* fait une **conversion automatique** s'il n'y a **aucun risque de perte d'information**, donc si le type cible utilise plus d'octets que le type de départ. La hiérarchie des conversions est la suivante:
  - `byte` → `short` → `int` → `long` → `float` → `double`
- *Java* transforme le type de certaines variables, afin que tous les membres d'une sous-expression (groupe de deux opérandes) soient de même type, selon ces règles :
  - Si un des opérandes est de type `double`, l'autre sera converti en valeur de type `double`;
  - sinon, si un des opérandes est de type `float`, l'autre sera converti en valeur de type `float`;
  - sinon, si un des opérandes est de type `long`, l'autre sera converti en valeur de type `long`;
  - sinon, tous les opérandes seront convertis en valeur de type `int`.

Farida Ait-Ammar et Nicolas Leduc

27

Conversions entre types numériques Conversion implicite - Exemples

- *Java* peut multiplier un entier par un réel double et le résultat sera de type `double`.

```
int n = 2;
double x;
double y = 45.2;
double z = 25;   // int converti automatiquement en double
x = n * y;       // x vaut 90.4
```

  - *Java* convertit implicitement l'entier `n` en `double` avant d'effectuer la multiplication
- Cas d'erreurs soulevées par le compilateur:

```
int n ;
float y = 2;
n = 456.3; // Type mismatch: cannot convert from double to int
n = y * 2; // Type mismatch: cannot convert from float to int
n = 12L;   // Type mismatch: cannot convert from long to int
```

- Cas de la division
  - Pour obtenir une **division entière**, les **deux opérandes** doivent être de **type entier**, sinon la division se fera en virgule flottante:

```
15 / 2        division entière, le résultat est 7
15.0 / 2.0    division en virgule flottante, le résultat est 7.5
15 / 2.0      division en virgule flottante, le résultat est 7.5
15.0 / 2      division en virgule flottante, le résultat est 7.5
```

Farida Ait-Ammar et Nicolas Leduc

28

Conversion explicite (*casting*)

- *Java* refuse de faire une conversion de type lorsqu'il y a un risque de perdre de l'information, même si la valeur contenue dans la variable est dans la plage des valeurs admissibles du type cible.

```
float f = 2; // Enregistre un int dans un float
int i;
i = f;       // Erreur, même si la valeur dans f est un entier
```

- Dans certaines situations spécifiques, ce type de conversion peut être nécessaire. Toutefois, on tente de l'éviter en choisissant le type des variables et des constantes judicieusement.
- Pour éliminer l'erreur de l'exemple précédent, il est possible de faire une conversion explicite (*casting*) en indiquant le type cible entre parenthèses.

```
i = (int) f; // Conversion explicite de float en int
```

- Dans une conversion de réel vers entier, la partie décimale du nombre réel est tronquée et non arrondie. Il y a perte d'information dans ce cas.

Farida Ait-Ammar et Nicolas Leduc

3.3 Commentaire de programmation

30

Commentaires de programmation

- Les commentaires sont des portions du code source **ignorées** par le compilateur ou l'interpréteur.
- Les commentaires servent à donner **des explications** sur le code ou une **implémentation particulière**.
- Ils sont également utilisés pour **masquer une partie du code** en cours de développement.
- Ils peuvent être mis sous la forme de l'un des styles suivants :
  - blocs de commentaires;
  - commentaires sur une seule ligne;
  - commentaires de fin de ligne;

Farida Aït-Ammar et Nicolas Leduc

31

Commentaires de programmation

- Blocs de commentaires
  - Servent à expliquer l'utilité et les particularités des fichiers, des méthodes, des structures de données et des algorithmes. Ils sont précédés d'une ligne blanche, et sont présentés comme suit :

```
/*
 * Ceci est un bloc de commentaires.
 * Il peut comporter plusieurs lignes, chacune commençant par une
 * étoile entourée de deux espaces pour la lisibilité de l'ensemble.
 */
```

- Commentaires sur une seule ligne
  - Commentaires courts qui peuvent être placés sur une seule ligne indentée au niveau du code. Un commentaire en ligne devrait être précédé d'une ligne blanche.

```
/* Traiter la condition. */ ou
// Traiter la condition.
if (condition) {
   ...
}
```

- Commentaires de fin de ligne
  - Ils peuvent prendre deux formes, soit `/* ... */` ou `// ...`

```
x = 100;  // Expliquer pourquoi ici.
x = 300;  /* Expliquer pourquoi ici. */
```

  - La forme `//...` est aussi utilisée pour masquer une partie du code en cours de développement. Par exemple :

```
//for (int i = 0; i < limite; i++) {
//     s = s * 12; // commentaire
//}
```

Farida Ait-Ammar et Nicolas Leduc

3.4 Entrée et sortie au terminal

33

Entrées et sorties
Affichage au terminal

- Pour afficher un message dans le terminal, on utilise les instructions:
```
// Pseudo-code: AFFICHER(message)
System.out.print(message);   // Affiche le message
System.out.println(message); // Affiche le message suivi d'un retour à la ligne
```
- Les chaînes littérales sont placées entre guillemets (`" "`) pour les distinguer des expressions (variables et calculs), qui doivent être évaluées avant d'être affichées
- On utilise le signe **+**, équivalent à la virgule en pseudo-code, pour concaténer plusieurs messages.
```
int valeur = 22, nombre = 2;
System.out.print("Bonjour tout le monde.");     // Affiche le texte
System.out.print(valeur);                       // Affiche la valeur d'une variable
// Pseudo-code: AFFICHER("Le montant est de: ", valeur, "$")
System.out.print("Le montant est de: " + valeur + "$");
System.out.println(nombre + " items valent au total " + valeur + "$.");
```
- Il est aussi possible d'afficher le résultat d'une expression arithmétique. Dans ce cas, il est conseillé de la mettre entre parenthèses pour éviter les mauvaises surprises.
```
int a = 10, b = 5;
System.out.println(a + " fois " + b + " vaut " + (a * b));
System.out.println(a + " plus " + b + " vaut " + (a + b)); // Affiche 10 plus 5 vaut 15
System.out.println(a + " plus " + b + " vaut " + a + b);   // Affiche 10 plus 5 vaut 105
```

Farida Ait-Ammar et Nicolas Leduc

34

Entrées et sorties
Lecture dans le terminal

- La classe **`Scanner`** propose un ensemble de fonctions de lecture d'une donnée de type simple (`byte`, `short`, `int`, `long`, `float`, `double`, `boolean`) ou `String`.
- Pour l'utiliser, vous devez d'abord importer le paquetage **`java.util`**, en inscrivant la ligne suivante au début de votre fichier *.java* , car ce paquetage n'est pas chargé automatiquement par *Java*.
```
import java.util.*;
```
- Ensuite, vous devez créer une variable de type `Scanner` dans votre programme principal (`main()`):
```
Scanner clavier = new Scanner(System.in);
```
- La variable `clavier`, de type `Scanner`, peut être utilisée pour effectuer plusieurs lectures. Le paramètre `System.in` indique au compilateur que l'objet `clavier` doit balayer l'entrée standard, soit le clavier.
- Pour affecter à une variable une valeur lue au clavier, on utilise les méthodes de récupération de données. Il existe une méthode pour chaque type de donnée sauf pour le type `char`:
  - `nextBoolean()`, `nextByte()`, `nextShort()`, `nextInt()`, `nextLong()`, `nextFloat()`, `nextDouble()`
  - `next()`, `nextLine()` : Permet de lire respectivement un mot ou une ligne entière
- Exemple d'utilisation: lire un entier et l'affecter à la variable `nombre`
```
int nombre;
// Pseudo-code: LIRE(nombre)
nombre = clavier.nextInt();
```

Farida Ait-Ammar et Nicolas Leduc

35

Entrées et sorties
Lecture dans le terminal - caractère

- La lecture d'une variable de type `char` n'est pas prise en charge par la classe `Scanner`.
- On peut cependant lire un mot avec la méthode `next()` de la classe `Scanner`, puis récupérer le premier caractère de la chaîne avec la méthode `charAt(0)` de la classe `String`.

```
Scanner clavier = new Scanner(System.in);
char caractere;
System.out.print("Entrez un caractère: ");
caractere = clavier.next().charAt(0);
System.out.println("Le caractère saisi est: " + caractere);
```

Farida Ait-Ammar et Nicolas Leduc

36

Entrées et sorties
Lecture dans le terminal – mot/phrase

- Pour lire un mot, soit les caractères jusqu'à la prochaine espace, on utilise la fonction `next()` de la classe `Scanner`, alors que pour lire une suite de mots jusqu'au prochain retour à la ligne (fin de ligne), on utilise la fonction `nextLine()`.

```
Scanner clavier = new Scanner(System.in);
String mot, phrase, phrase2;
System.out.print("Entrez une phrase: ");
phrase = clavier.nextLine();
System.out.print("Entrez un mot: ");
mot = clavier.next();
System.out.print("Entrez une seconde phrase: ");
clavier.nextLine();    // Chaine vide: consomme le retour de next()
phrase2 = clavier.nextLine();
```

- **Attention**: les fonctions `next*()`, sauf `nextLine()`, ne consomment pas le retour à la ligne. Donc, si vous faites appel à `nextLine()` après avoir fait d'autres lectures, vous risquez de retrouver une chaîne vide, car le `Scanner` rencontrera le retour à la ligne de la lecture précédente. Dans ce cas, appelez une seconde fois *`nextLine()`* pour récupérer la phrase voulue.

Farida Ait-Ammar et Nicolas Leduc

37

Entrées et sorties
Lecture dans le terminal - localisation

- Pour utiliser le point comme séparateur décimal sur un système d'exploitation réglé en zone française, nous devons modifier la localité grâce à l'instruction:
```
clavier.useLocale(Locale.US);
```
- Exemple
```
Scanner clavier = new Scanner(System.in);
float x;
clavier.useLocale(Locale.US);
x = clavier.nextFloat();
```
- Pour revenir à la saisie des réels avec une virgule, on utilise:
```
clavier.useLocale(Locale.CANADA_FRENCH);
```

Farida Ait-Ammar et Nicolas Leduc

38

Entrées et sorties
Formater l'affichage

- Parfois, on veut formater l'affichage de certaines variables, par exemple, pour limiter le nombre de décimales visibles. Pour y arriver, on utilise l'utilitaire `String.format()`.
  - `String.format("chaine avec formatage", variable1, variable2, … )`
- Dans la chaîne, tous les caractères inscrits sont affichés sans modification, y compris les espaces, à l'exception des marques de formatage qui précisent comment les variables qui suivent sont affichées. Il faut mettre autant de variables en paramètres qu'il y a de marques de formatage dans la chaîne.
  - Une marque de formatage débute par `%`. Pour afficher un signe de pourcentage, il faut écrire `%%`.
  - Ensuite, on peut indiquer si on désire un affichage aligné à gauche en inscrivant `-`, car par défaut l'affichage est aligné à droite. Il n'est pas possible de centrer automatiquement l'affichage.
  - On peut ensuite indiquer le nombre minimum d'espaces à réserver pour afficher la variable.
  - Pour les nombres décimaux, on peut indiquer le nombre de décimales à la suite d'un point, par exemple `.2`.
  - Enfin, on indique le type de variable à afficher:
    `d` : affiche des données de type `byte, short, int, long`
    `f` : affiche des données de type `float, double`
    `s` : affiche des données de type `String`
- Exemples:
```
int entier = 400;
float reel = 3.1415f;
String mot = "Bonjour";
System.out.print(String.format("E=%5d R:%4.2f$ C->%-10s!", entier, reel, mot));
// Affiche: E=__400_R:3,14$_C->Bonjour___!     // Ici _ représente une espace
%5d   : Affiche un entier aligné à droite en lui réservant 5 espaces
%4.2f : Affiche un nombre réel avec deux décimales en lui réservant 4 espaces incluant la partie décimale.
%-10s : Affiche une chaine alignée à gauche en lui réservant 10 espaces.
```

Farida Ait-Ammar et Nicolas Leduc

39

Exemple: Facture

Pseudo-code

```
ALGORITHME Facture
CONSTANTES
      TPS = 5%
      TVQ = 9,975%


VARIABLES
      total, totalProduit, prixProduit TYPE NUMÉRIQUE
      quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
      AFFICHER("Entrez le prix du produit : ")
      LIRE(prixProduit)
      AFFICHER("Entrez la quantité achetée : ")
      LIRE(quantiteProduit)
      totalProduit ← prixProduit * quantiteProduit
      total ← totalProduit
      total ← total + totalProduit * TPS
      total ← total + totalProduit * TVQ
      AFFICHER("Le total de la facture est ", total)
FIN
```

Code *Java*

```
package sandbox;

import java.util.Scanner;

public class Facture {

    public static final float TPS = 0.05f;
    public static final float TVQ = 0.09975f;

    public static void main(String[] args) {
            Scanner clavier = new Scanner (System.in);

            float total, totalProduit, prixProduit;
            int quantiteProduit;

            System.out.print("Entrez le prix du produit : ");
            prixProduit = clavier.nextFloat();
            System.out.print("Entrez la quantité achetée : ");
            quantiteProduit = clavier.nextInt();
            totalProduit = prixProduit * quantiteProduit;
            total = totalProduit;
            total = total + totalProduit * TPS;
            total = total + totalProduit * TVQ;
            System.out.println("Le total de la facture est " + total);
      }
}
```


40

Exemple: Facture – 2 produits

```
package sandbox;

import java.util.Scanner;

public class Facture2Produits {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
          Scanner clavier = new Scanner (System.in);

          float total, totalProduit, prixProduit;
          int quantiteProduit;

          System.out.print("Entrez le prix du premier produit : ");
          prixProduit = clavier.nextFloat();
          System.out.print("Entrez la quantité achetée : ");
          quantiteProduit = clavier.nextInt();
          totalProduit = prixProduit * quantiteProduit;
          total = totalProduit;
          total = total + totalProduit * TPS;
          total = total + totalProduit * TVQ;
          System.out.print("Entrez le prix du second produit : ");
          prixProduit = clavier.nextFloat();
          System.out.print("Entrez la quantité achetée : ");
          quantiteProduit = clavier.nextInt();
          totalProduit = prixProduit * quantiteProduit;
          total = total + totalProduit;
          total = total + totalProduit * TPS;
          total = total + totalProduit * TVQ;
          System.out.println(String.format(
               "Le total de la facture est %.2f$.", total));
     }
}
```

3.5 Bibliothèque `FenetreDessin`

42

Structure d'un programme *Java* Utilisation de la classe `FenetreDessin`

```
// Package;

import java.awt.Color;
import commun.FenetreDessin;
// Imports;

public class ExempleGraphique {
    // Constantes;

    public static void main(String[] args) {
        FenetreDessin fd = new FenetreDessin("Test", 640, 480);

        // Variables;

        // Instructions;
    }

    // Autres méthodes
}
```

Farida Ait-Ammar et Nicolas Leduc

43

Entrées et sorties Affichage dans `FenetreDessin`

- Pour afficher du texte dans la fenêtre graphique, vous devez utiliser la méthode `dessineTexte()`:
  `public void dessineTexte(int x, int y, String aTexte);`
- Le coin inférieur gauche du premier caractère de la chaîne est placé à la position spécifiée par les paramètres `x` et `y`. Chaque caractère a une largeur de 8 pixels et une hauteur de 16 pixels.

- Le troisième paramètre est la chaîne à afficher. Elle est construite de la même façon que pour les messages destinés au terminal.

```
FenetreDessin fd = new FenetreDessin("Exemple texte", 640, 480);
fd.dessineTexte(100, 100, "Une première ligne");
fd.dessineTexte(100, 116, "Une seconde ligne");
fd.dessineTexte(100, 132, "Le total est de "  + total + "$.");
```

- **Attention**: Les retours de chariot n'ont aucun effet dans la fenêtre graphique.

Farida Ait-Ammar et Nicolas Leduc

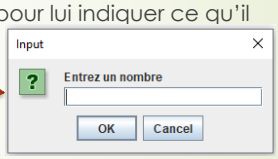
44

Entrées et sorties
Lecture dans `FenetreDessin`

- Les méthodes suivantes de `FenetreDessin` permettent de récupérer une entrée au clavier de l'usager:

```
public int litEntier(String aTexte);     // Retourne l'entier lu
public float litReel(String aTexte);     // Retourne le réel lu (utilise le . décimal)
public char litCaractere(String aTexte); // Retourne le premier caractère de la chaine lue
public String litMot(String aTexte);     // Retourne le premier mot de la chaine lue
public String litPhrase(String aTexte);  // Retourne la chaine entière
```

- Le paramètre de chacune de ces fonctions est le message à afficher à l'usager pour lui indiquer ce qu'il doit inscrire dans la boîte prévue.
- L'appel de ces fonctions fait apparaître une fenêtre de dialogue.



- Exemple d'utilisation:

```
FenetreDessin fd = new FenetreDessin("Exemple texte", 640, 480);
int entier;
char ch;
String nom;
entier = fd.litEntier("Entrez un nombre");
ch = litCaractere("Entrez un caractère");
nom = litMot("Entrez votre nom de famille");
```

Farida Ait-Ammar et Nicolas Leduc

45

Entrées et sorties
Actions dans `FenetreDessin`

- Lorsque la fenêtre de dessin est active, il est possible de lire les touches du clavier pressées par l'usager, une touche à la fois. On doit d'abord attendre qu'une touche soit pressée avant de pouvoir récupérer le caractère correspondant.

```
FenetreDessin fd = new FenetreDessin("Exemple", 640, 480);
char ch;
fd.clavierAttendreEvenement();
ch = fd.clavierCharactere();
```

- Par une procédure similaire, il est aussi possible de récupérer la position de la souris à la suite d'un clic de l'usager.

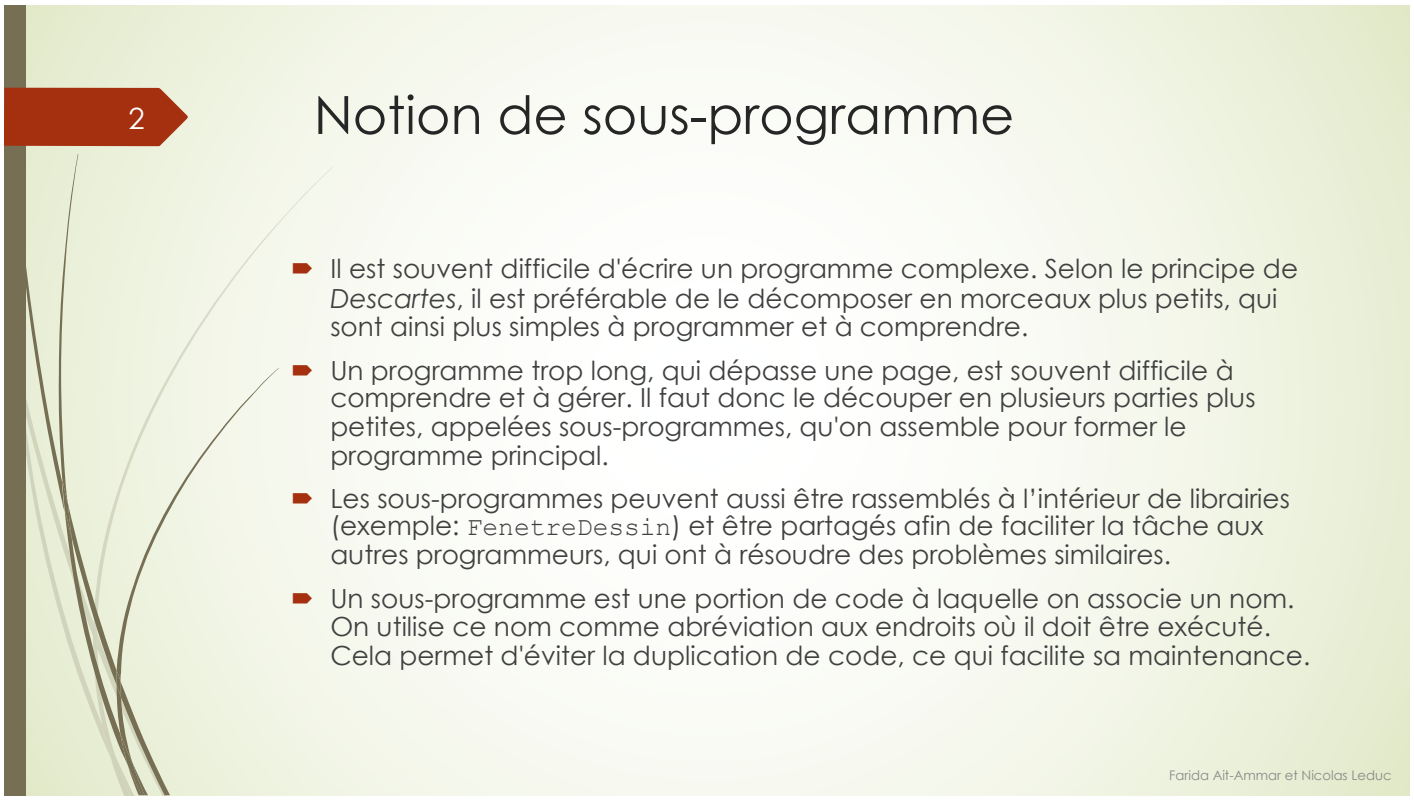
```
FenetreDessin fd = new FenetreDessin("Exemple", 640, 480);
int posX, posY;
fd.sourisAttendreEvenement();
posX = fd.sourisPositionX();
posY = fd.sourisPositionY();
```

Farida Ait-Ammar et Nicolas Leduc

4. Sous-programme

« Diviser chacune des difficultés que j'examinerais, en autant de parcelles qu'il se pourrait, et qu'il serait requis pour les mieux résoudre. »
— René DESCARTES

4.1 Définition



3

Analogie de la boîte noire - Extérieur

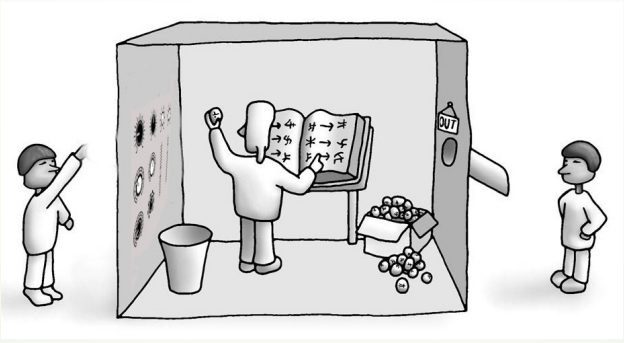
- Un sous-programme peut être traité comme une boîte noire. Vu de l'extérieur, on ne connaît pas le processus utilisé par la boîte pour fournir un résultat. Il est toutefois possible d'agir sur les boutons pour modifier la sortie.

Farida Ait-Ammar et Nicolas Leduc

4

Analogie de la boîte noire - Intérieur

- À l'intérieur de la boîte (sous-programme), nous pouvons seulement voir la **valeur** de chaque bouton pour réaliser notre travail. Nous n'avons aucun moyen de savoir ce qui se passe à l'extérieur (dans le reste du programme). Nous ne pouvons fournir qu'une seule **valeur**, qui correspond au résultat du travail réalisé.

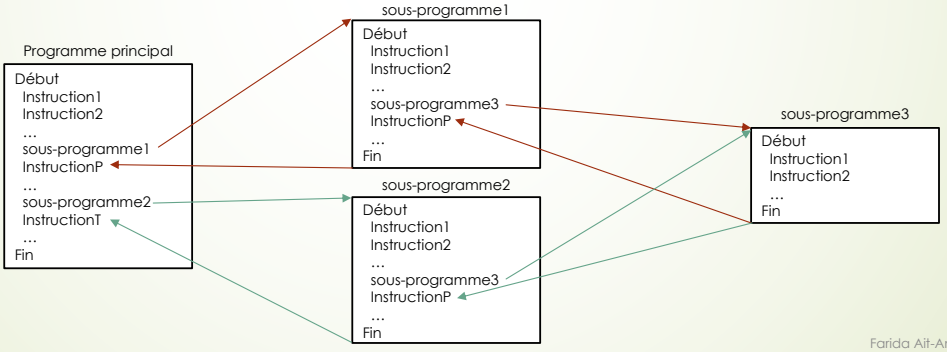


Farida Ait-Ammar et Nicolas Leduc

5

Notion de sous-programme

- Un sous-programme, tout comme un programme, possède
  - **un nom, des variables, un début, des instructions et une fin**
- Un sous-programme ne s'exécute que s'il est appelé par un programme principal ou un autre sous-programme.
- Lorsqu'un sous-programme est appelé, il s'exécute. À la fin de son exécution, le programme appelant se poursuit à l'instruction suivant l'appel.



Farida Ait-Ammar et Nicolas Leduc

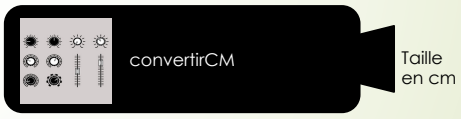
4.2 Procédure, fonction et paramètre

6

Types de sous-programmes

- **Procédure**
  - Réalise une suite d'actions lors de son appel.
  - Exemples: trier une liste de nombres, réaliser un affichage, ...

- **Fonction**
  - Réalise une suite d'actions puis **renvoie une valeur** au programme appelant.
  - Exemples: fonction qui retourne l'aire d'un rectangle, qui convertit la taille en cm, ...

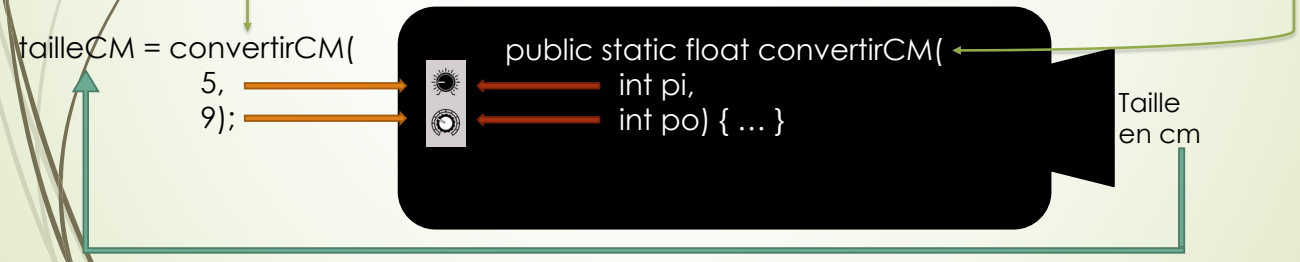


Farida Ait-Ammar et Nicolas Leduc

7

Paramètres

- Peu importe le type de sous-programme choisi, il est toujours possible de fournir des paramètres afin d'en contrôler le fonctionnement. Selon notre analogie, les paramètres correspondent aux boutons de la boîte noire.
- Lors de la définition du sous-programme, on doit indiquer le nom et le type de chaque paramètre (bouton) accessible entre parenthèses. Les paramètres sont séparés par des virgules.
- Lors de l'**appel** d'un sous-programme, il faut indiquer la **valeur** que doit prendre chaque paramètre (bouton). **La valeur peut provenir d'une variable, d'une expression ou même être une valeur littérale.**



Farida Ait-Ammar et Nicolas Leduc

8

Paramètres et valeur de retour

- À l'intérieur du sous-programme, nous inscrivons le nom de chaque paramètre (bouton) pour en connaître la valeur. Les **paramètres** s'utilisent comme des **variables initialisées lors de l'appel** du sous-programme.
- **Il n'y a aucun lien entre l'expression ou la variable utilisée lors de l'appel et le paramètre de la fonction**. La modification d'un paramètre n'a aucun effet à l'extérieur de la fonction. Une fois que le programme appelant a ajusté les boutons, il ne les regarde plus et attend seulement le résultat.
- Lorsque les traitements sont terminés, on doit retourner une valeur seulement dans le cas d'une fonction. Cette valeur est récupérée en utilisant l'opérateur d'affectation (=). Une fonction peut aussi être utilisée dans une expression mathématique.

Farida Ait-Ammar et Nicolas Leduc

9

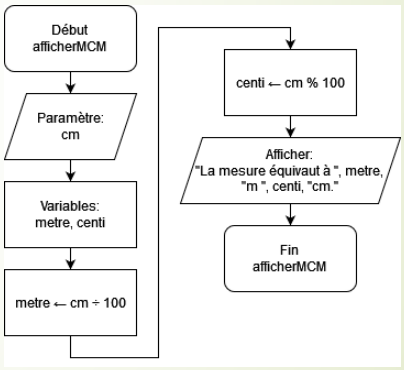
Exemple de sous-programme Procédure

- Sous-programme `afficherMCM()` : affiche la taille en mètres et centimètres équivalente au nombre de centimètres (`cm`) reçu en paramètre.

```
PROCÉDURE afficherMCM(cm : NUMÉRIQUE)

VARIABLES
    metre, centi  TYPE NUMÉRIQUE

DÉBUT
    metre ← cm ÷ 100
    centi ← cm % 100
    AFFICHER("La mesure équivaut à ", metre,
        "m ", centi, "cm.")
FIN
```



Farida Ait-Ammar et Nicolas Leduc

10

Exemple de sous-programme Fonction

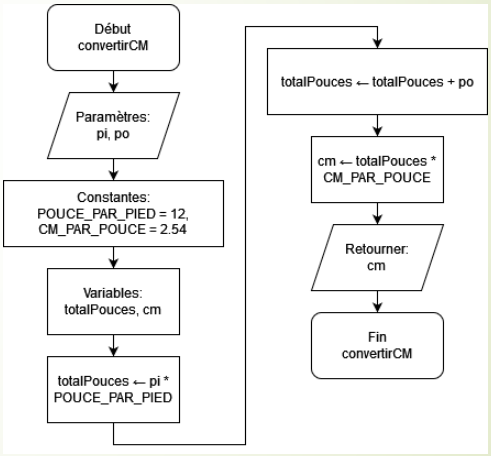
- Sous-programme `convertirCM()` : convertit en centimètres une taille en pieds (`pi`) et pouces (`po`).

```
FONCTION convertirCM(pi, po : NUMÉRIQUE) : NUMÉRIQUE

CONSTANTES
    POUCE_PAR_PIED = 12
    CM_PAR_POUCE = 2.54

VARIABLES
    totalPouces, cm TYPE NUMÉRIQUE

DÉBUT
    totalPouces ← pi * POUCE_PAR_PIED
    totalPouces ← totalPouces + po
    cm ← totalPouces * CM_PAR_POUCE
    RETOURNER(cm)
FIN
```



Farida Ait-Ammar et Nicolas Leduc

11

Exemple de sous-programme
Programme principal (`main()`)

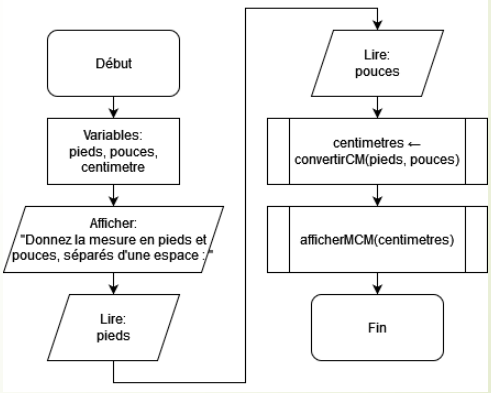
- Programme principal: appelle `convertirCM()` et `afficherMCM()`, dont les signatures sont les suivantes:

```
PROCÉDURE afficherMCM(cm : NUMÉRIQUE)
FONCTION convertirCM(pi, po : NUMÉRIQUE) : NUMÉRIQUE
```

```
ALGORITHME ConversionTaille

VARIABLES
    pieds, pouces, centimetres TYPE NUMÉRIQUE

DÉBUT
    AFFICHER("Donnez la mesure en pieds et
        pouces, séparés d'une espace : ")
    LIRE(pieds)
    LIRE(pouces)
    centimetres ← convertirCM(pieds, pouces)
    afficherMCM(centimetres)
FIN
```



Farida Ait-Ammar et Nicolas Leduc

4.3 Trace d'exécution

12

Sous-programme
Trace d'exécution

- La trace d'exécution, sous forme de tableau, est aussi utile pour vérifier le bon fonctionnement d'un sous-programme. Toutefois, elle devra être adaptée selon le type de sous-programme que nous désirons analyser.
- Nous devons avoir une colonne pour
  - le **numéro** de ligne
  - chaque **paramètre reçu** par le sous-programme
  - chaque **variable locale** déclarée dans le sous-programme
  - le **Terminal**, dans le cas d'une **procédure**, ou la valeur de **Retour**, dans le cas d'une **fonction**
- On ajoute une ligne au début de la trace, dont le numéro de ligne est `I`, soit initialisation, afin d'indiquer les valeurs reçues pour chaque paramètre lors de l'appel du sous-programme.
- On indique enfin, au-dessus du tableau, la séquence d'appels ainsi que les lignes correspondantes dans les différents sous-programmes, qui a mené à cette trace.
  - Exemple: `main():4 -> convertirCM()` indique que la trace correspond à l'appel de la fonction `convertirCM()` à partir de la ligne 4 du programme principal (`main()`)

Farida Ait-Ammar et Nicolas Leduc

13

Liens entre traces d'exécution

```
ALGORITHME ConversionTaille

VARIABLES
    pieds, pouces, centimetres TYPE NUMÉRIQUE

DÉBUT
1   AFFICHER("Donnez la mesure en pieds et
        pouces, séparés d'une espace : ")
2   LIRE(pieds)
3   LIRE(pouces)
4   centimetres ← convertirCM(pieds, pouces)
5   afficherMCM(centimetres)
FIN

FONCTION convertirCM(pi, po : NUMÉRIQUE) : NUMÉRIQUE

CONSTANTES
    POUCE_PAR_PIED = 12
    CM_PAR_POUCE = 2.54

VARIABLES
    totalPouces, cm TYPE NUMÉRIQUE

DÉBUT
6   totalPouces ← pi * POUCE_PAR_PIED
7   totalPouces ← totalPouces + po
8   cm ← totalPouces * CM_PAR_POUCE
9   RETOURNER(cm)
FIN
```

main()

#	pieds	pouces	centi metres	Terminal
1				Donnez la mesure en pieds et pouces, séparés d'une espace :
2	5			
3		9		
4	5	9	175	
5				

Copie de valeur

Copie de valeur

main():4 -> convertirCM()

#	pi	po	total Pouces	cm	Retour
1	5	9			
6			60		
7			69		
8				175	
9					175

Farida Ait-Ammar et Nicolas Leduc

14

Exemples de traces Programme principal

```
ALGORITHME ConversionTaille

VARIABLES
    pieds, pouces, centimetres TYPE NUMÉRIQUE

DÉBUT
1   AFFICHER("Donnez la mesure en pieds et
        pouces, séparés d'une espace : ")
2   LIRE(pieds)
3   LIRE(pouces)
4   centimetres ← convertirCM(pieds, pouces)
5   afficherMCM(centimetres)
FIN
```

main()

#	pieds	pouces	centi metres	Terminal
1				Donnez la mesure en pieds et pouces, séparés d'une espace :
2	5			
3		9		
4			175	
5				

Farida Ait-Ammar et Nicolas Leduc

15

Exemples de traces Sous-programmes

```
FONCTION convertirCM(pi, po : NUMÉRIQUE) : NUMÉRIQUE

CONSTANTES
     POUCE_PAR_PIED = 12
     CM_PAR_POUCE = 2.54

VARIABLES
     totalPouces, cm TYPE NUMÉRIQUE

DÉBUT
6    totalPouces ← pi * POUCE_PAR_PIED
7    totalPouces ← totalPouces + po
8    cm ← totalPouces * CM_PAR_POUCE
9    RETOURNER(cm)
FIN

PROCÉDURE afficherMCM(cm : NUMÉRIQUE)

VARIABLES
     metre, centi  TYPE NUMÉRIQUE

DÉBUT
10   metre ← cm ÷ 100
11   centi ← cm % 100
12   AFFICHER("La mesure équivaut à ", metre,
          "m ", centi, "cm.")
FIN
```

main():4 -> convertirCM()

#	pi	po	total Pouces	cm	Retour
1	5	9			
6			60		
7			69		
8				175	
9					175

main():5 -> afficherMCM()

#	cm	metre	centi	Terminal
1	175			
10		1		
11			75	
12				La mesure équivaut à 1m 75cm

Farida Ait-Ammar et Nicolas Leduc

4.4 Éléments d'une méthode en *Java*

17

Structure d'un programme *Java*

- Dans le cadre de ce cours, un programme *Java* est constitué d'une seule classe.
- Une classe contient des déclarations de constantes et de variables ainsi que des méthodes (sous-programmes), dont une méthode principale appelée `main()`.
- Une méthode contient des déclarations de constantes et de variables, des instructions élémentaires et des appels de méthodes prédéfinies ou non.

```
public class NomDeClasse {
    //déclaration de variables et constantes  de classe (globales)
    public static void main(String[] args) {
        // déclaration de constantes et variables locales

        //instructions élémentaires
        //appel de méthodes
    }

    public static void nomMethode(parametres){
        // déclaration de constantes et variables locales

        //instructions élémentaires
        //appel de méthodes
    }

    public static typeMethode nomMethode(parametres){
        // déclaration de constantes et variables  locales

        //instructions élémentaires
        //appel de méthodes
        return resultat;
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

18

Sous-programmes en *Java* Méthodes

- Caractéristiques d'une méthode
  - En-tête / Signature
    - Visibilité: détermine qui a accès à la méthode
    - Type de retour: type de la valeur de retour de la méthode
    - Nom: représentatif du comportement de la méthode
    - Paramètres: variables contenant les valeurs en entrée de la méthode
  - Corps: instructions à exécuter
    - Constantes
    - Variables
    - Instructions
    - Retour: retourne une valeur ayant le type indiqué dans la signature de la fonction
- Note: les méthodes n'ont pas nécessairement des paramètres.
  - Exemple: la méthode `nextInt()` de la classe `Scanner` est sans paramètres.

19

Éléments d'une méthode En-tête / Signature

- Elle permet au programmeur de comprendre l'utilité de la méthode et la façon de l'utiliser, sans avoir à se soucier de son implémentation. Il n'a besoin que de cette en-tête pour utiliser la méthode ou l'appeler. Le compilateur se sert de la signature pour valider le code entré par le programmeur.
- L'en-tête contient, dans l'ordre, la visibilité de la méthode, son type de retour, son nom et, entre parenthèses, ses paramètres séparés par des virgules.
- Exemple: `public static float convertirCM(int pi, int po)`
  - `public`: mot clé qui indique que la méthode est toujours accessible
  - `static`: mot clé qui indique que la méthode peut être appelée sans avoir à créer une instance de la classe
  - `float`: type de retour de la méthode.
    - ***type simple ou complexe***: il s'agit d'une **fonction** qui retourne une seule valeur
    - **`void`**: il s'agit d'une **procédure**, donc aucune valeur n'est retournée
  - `convertirCM`: nom de la méthode. Il suit les mêmes règles de nomenclature que les variables. Il doit être significatif et pertinent.
  - `(int pi, int po)`: **paramètres** qui permettent de transmettre des **valeurs** à la méthode. On les déclare comme les variables et ils doivent respecter la nomenclature des variables et être pertinents. S'il n'y a aucun paramètre, les parenthèses doivent tout de même être présentes: `()`

20

Appel de méthodes

- Pour utiliser les méthodes que vous avez écrites ou qui font partie de librairies, vous devez les appeler à partir de votre code. L'appel est l'endroit où l'on utilise la méthode.
- Un **appel** est constitué du **nom** de la méthode et des **arguments** qu'on lui passe. **L'ordre des paramètres doit être le même** que celui indiqué dans la **signature** de la méthode et leurs **types** doivent être **compatibles**. Lors de l'appel de la méthode, les valeurs que l'on passe en paramètres sont appelées arguments.
- **Procédure**: ne retourne aucune valeur. L'appel de procédure se retrouve seul sur une ligne.

```
fd.effaceFenetre();                 // La procédure ne reçoit aucun argument
System.out.println("Bonjour");      // La procédure reçoit une chaîne de caractères
fd.dessineSegment(10, 10, 50, 50);  // La procédure reçoit 4 arguments de type int
```

- **Fonction**: retourne une valeur. Son appel se retrouve habituellement dans une expression arithmétique ou à droite d'une affectation.

```
float taille, hyp, a = 3, b = 4;
taille = convertirCM(5, 9); // La fonction reçoit 2 arguments et retourne une valeur de type float
hyp = (float) Math.sqrt( Math.pow(a, 2) + b*b );  // Calcule l'hypoténuse d'un triangle rectangle
```

- Signatures:

```
public void effaceFenetre()
public void println(String x)
public void dessineSegment(int x1, int y1, int x2, int y2)
public static float convertirCM(int pi, int po)
public static double pow(double a, double b)
public static double sqrt(double a)
```

Farida Ait-Ammar et Nicolas Leduc

21

Éléments d'une méthode Définition

- La définition de la méthode est constituée de l'en-tête et du corps
- **Procédure**: aucune valeur de retour (`void`)

```
Visibilité void nomDeMethode (liste de paramètres) {
    // Déclarations de constantes locales, si nécessaire
    // Déclarations de variables locales, si nécessaire

    // Instructions (incluant appel de méthodes)
}
```

- **Fonction**: appel remplacé par la valeur renvoyée par `return` dans le programme appelant

```
Visibilité type nomDeMethode (liste de paramètres) {
    // Déclarations de constantes locales, si nécessaire
    // Déclarations de variables locales, si nécessaire

    // Instructions (incluant appel de méthodes)
    return <valeur>;
}
```

  - Une **fonction doit retourner une valeur** grâce au mot-clé **`return`**. Lorsque l'instruction `return` est rencontrée, la méthode évalue la valeur qui la suit, puis la renvoie au programme appelant. **Dès qu'un `return` est atteint, la fonction se termine** et les instructions suivantes ne sont pas exécutées.
  - Dans le cadre de ce cours, vos **fonctions ne doivent comporter qu'un seul `return`** placé à la dernière ligne.

Farida Ait-Ammar et Nicolas Leduc

22

Éléments d'une méthode
Définition - exemples

- **Procédure**

```
public static void afficherMCM(float cm) {
     int metre, centi;                                  // Variables

     metre = (int) cm / 100;
     centi = (int) cm % 100;
     System.out.println("La mesure équivaut à " + metre + "m " + centi + "cm.");
}
```

- **Fonction**

```
public static final int POUCE_PAR_PIED = 12;          // Constantes
public static final float CM_PAR_POUCE = 2.54f;

public static float convertirCM(int pi, int po) {
     int totalPouces;                                   // Variables
     float cm;

     totalPouces = pi * POUCE_PAR_PIED;
     totalPouces = totalPouces + po;
     cm = totalPouces * CM_PAR_POUCE;
     return cm;
}
```


23

Éléments d'une méthode
Documentation

- Les **méthodes doivent être documentées** avant l'en-tête. Pour chaque méthode, on doit spécifier: une **description**, les **paramètres** et le **résultat retourné**, s'il y a lieu.
- Des exemples de documentation des méthodes de la classe `Math` de la librairie `java` sont dis onibles à l'adresse:
  - [redacted]
- Voici comment on pourrait documenter la méthode `convertirCM()`. En inscrivant `/**` sur la ligne précédant l'en-tête, *Visual Studio Code* génère un gabarit de documentation. Il utilise ces informations pour offrir de l'aide lorsque l'on place sa souris au-dessus de l'appel d'une méthode.

```
/**
 * Convertit en centimètres une mesure en pieds et pouces
 *
 * @param pi Nombre entier de pieds de la mesure
 * @param po Nombre entier de pouces de la mesure
 *
 * @return Nombre de centimètres équivalent à la mesure
 */
public static float convertirCM(int pi, int po) {
    int totalPouces;
    float cm;

    totalPouces = pi * POUCE_PAR_PIED;
    totalPouces = totalPouces + po;
    cm = totalPouces * CM_PAR_POUCE;
    return cm;
}
```

```
centimetres = convertirCM(pieds, pouces);
afficherMCM(ce
}

/**
 * Convertit en ce
 *
 * @param pi Nombr
 * @param po Nombr
 *
 * @return Nombre
```

float sandbox.ConversionTaille.convertirCM(int pi, int po)

Convertit en centimètres une mesure en pieds et pouces

Parameters:
pi Nombre entier de pieds de la mesure
po Nombre entier de pouces de la mesure
Returns:
Nombre de centimètres équivalent à la mesure

Press 'F2' for focus

4.5 Ajout d'une méthode dans `Facture`

24

Ajout d'une fonction dans *Facture*

```
import java.util.Scanner;

public class Facture {
   public static final float TPS = 0.05f;
   public static final float TVQ = 0.09975f;

   public static void main(String[] args) {
      Scanner clavier = new Scanner(System.in);

      float total, totalProduit, prixProduit;
      int quantiteProduit;

      System.out.print("Entrez le prix du produit : ");
      prixProduit = clavier.nextFloat();
      System.out.print("Entrez la quantite achetee : ");
      quantiteProduit = clavier.nextInt();
      totalProduit = prixProduit * quantiteProduit;
      total = totalProduit;
      total = total + totalProduit * TPS;
      total = total + totalProduit * TVQ;
      System.out.println("Le total de la facture est " +
         total + "$.");
   }
}
```

Appel

Ajuster le nom des variables

```
import java.util.Scanner;

public class FactureFonction {
   public static final float TPS = 0.05f;
   public static final float TVQ = 0.09975f;

   public static void main(String[] args) {
      Scanner clavier = new Scanner(System.in);

      float total, prixProduit;
      int quantiteProduit;

      System.out.print("Entrez le prix du produit : ");
      prixProduit = clavier.nextFloat();
      System.out.print("Entrez la quantité achetée : ");
      quantiteProduit = clavier.nextInt();
      total = totalTaxes(prixProduit, quantiteProduit);
      System.out.println("Le total de la facture est " +
         total + "$.");
   }

   public static float totalTaxes(float prix,
                                  int quantite) {
      float totalProduit, coutTotal;

      totalProduit = prix * quantite;
      coutTotal = totalProduit;
      coutTotal = coutTotal + totalProduit * TPS;
      coutTotal = coutTotal + totalProduit * TVQ;
      return coutTotal;
   }
}
```

Farida Ait-Ammar et Nicolas Leduc

25

Exemple: *Facture* avec fonction

```
import java.util.Scanner;

public class FactureFonction {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);
        float total, prixProduit;
        int quantiteProduit;

        System.out.print("Entrez le prix du produit : ");
        prixProduit = clavier.nextFloat();
        System.out.print("Entrez la quantité achetée : ");
        quantiteProduit = clavier.nextInt();
        total = totalTaxes(prixProduit, quantiteProduit);
        System.out.println(String.format("Le total de la facture est %.2f$.", total));
    }

    public static float totalTaxes(float prix, int quantite) {
        float totalProduit, coutTotal;

        totalProduit = prix * quantite;
        coutTotal = totalProduit;
        coutTotal = coutTotal + totalProduit * TPS;
        coutTotal = coutTotal + totalProduit * TVQ;
        return coutTotal;
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

26

Exemple: *Facture* – 2 produits

```
import java.util.Scanner;

public class Facture2ProdFct {
     public static final float TPS = 0.05f, TVQ = 0.09975f;

     public static void main(String[] args) {
          Scanner clavier = new Scanner(System.in);
          float total, prixProduit;
          int quantiteProduit;

          System.out.print("Entrez le prix du premier produit : ");
          prixProduit = clavier.nextFloat();
          System.out.print("Entrez la quantité achetée : ");
          quantiteProduit = clavier.nextInt();
          total = totalTaxes(prixProduit, quantiteProduit);
          System.out.print("Entrez le prix du second produit : ");
          prixProduit = clavier.nextFloat();
          System.out.print("Entrez la quantité achetée : ");
          quantiteProduit = clavier.nextInt();
          total = total + totalTaxes(prixProduit, quantiteProduit);
          System.out.println(String.format("Le total de la facture est %,.2f$.", total));
     }

     public static float totalTaxes(float prix, int quantite) {
          float totalProduit, coutTotal;

          totalProduit = prix * quantite;
          coutTotal = totalProduit;
          coutTotal = coutTotal + totalProduit * TPS;
          coutTotal = coutTotal + totalProduit * TVQ;
          return coutTotal;
     }
}
```


27

Trace: *Facture* - 2 produits

`main()`

```
public static void main(String[] args) {
     Scanner clavier = new Scanner(System.in);
     float total, prixProduit;
     int quantiteProduit;

1    System.out.print("Entrez le prix du premier produit : ");
2    prixProduit = clavier.nextFloat();
3    System.out.print("Entrez la quantité achetée : ");
4    quantiteProduit = clavier.nextInt();
5    total = totalTaxes(prixProduit, quantiteProduit);
6    System.out.print("Entrez le prix du second produit : ");
7    prixProduit = clavier.nextFloat();
8    System.out.print("Entrez la quantité achetée : ");
9    quantiteProduit = clavier.nextInt();
10   total = total + totalTaxes(prixProduit, quantiteProduit);
11   System.out.println(String.format(
          "Le total de la facture est %,.2f$.", total));
}
```

`main()`

#	total	prix Produit	quantite Produit	Terminal
1				`Entrez le prix du premier produit :`
2		5		
3				`Entrez la quantité achetée :`
4			4	
5	23			
6				`Entrez le prix du second produit :`
7		15		
8				`Entrez la quantité achetée :`
9			2	
10	57,49			
11				`Le total de la facture est 57,49$.`

28

Trace: *Facture* - 2 produits `totalTaxes()`

```
public static float totalTaxes(float prix, int quantite) {
    float totalProduit, coutTotal;

12  totalProduit = prix * quantite;
13  coutTotal = totalProduit;
14  coutTotal = coutTotal + totalProduit * TPS;
15  coutTotal = coutTotal + totalProduit * TVQ;
16  return coutTotal;
}
```

`main():5 -> totalTaxes()`

#	prix	quantite	total Produit	cout Total	Retour
I	5	4			
12			20		
13				20	
14				21	
15				23	
16					23

`main():10 -> totalTaxes()`

#	prix	quantite	total Produit	cout Total	Retour
I	15	2			
12			30		
13				30	
14				31,50	
15				34,49	
16					34,49

Farida Ait-Ammar et Nicolas Leduc

4.6 Visibilité des variables

29

Visibilité des variables locales

- Chaque méthode, incluant la méthode `main()`, a son propre espace de variables créé par l'interpréteur *Java*.
- Ainsi, une **variable déclarée dans une méthode** n'est pas visible dans les autres méthodes et s'appelle une **variable locale**. Les **paramètres** d'une méthode font aussi partie de ses **variables locales**.
- Une variable locale n'existe que pendant le temps d'exécution de la méthode dans laquelle elle est déclarée (dans sa boîte noire). Elle est détruite à la fin de son exécution.
- Par conséquent, même si on réutilise un nom de variable dans différentes méthodes, par exemple `nombre` dans la page suivante, la modification d'une variable n'affecte aucune des autres déclarées dans les autres méthodes. Elles sont dans des boîtes indépendantes.
- Ceci est aussi valable pour la méthode principale `main()`. Une variable déclarée dans le `main()` n'est pas accessible dans les autres méthodes.
- **En général, les noms des paramètres et des variables à l'intérieur d'une méthode doivent être choisis indépendamment (différents) de ceux du programme principal, car la méthode pourrait être utilisée dans d'autres programmes. Aussi, il n'est pas recommandé d'utiliser le nom de la méthode pour nommer la variable contenant la valeur de retour.**

Farida Ait-Ammar et Nicolas Leduc

30

Visibilité des variables locales Exemple

```
public class VariableLocale {
    public static void main(String[] args) {
        int nombre = 0;
        System.out.println("Avant l'appel, nombre = " + nombre);
        modifier();
        System.out.println("Après l'appel, nombre = " + nombre);
    }

    public static void modifier() {
        int nombre = 100;              // Variable locale, indépendante de "nombre" du main()
        System.out.println("Dans la méthode, nombre = " + nombre);
    }                                  // Destruction de la variable locale "nombre »

    public static void modifier2() {
        nombre = 100;  // Erreur de compilation: "nombre cannot be resolved to a variable"
    }
}
```

- Résultat:

```
Avant l'appel, nombre = 0
Dans la méthode, nombre = 100
Après l'appel, nombre = 0
```

--> Seule la valeur locale dans la méthode `modifier()` a été affectée.

31

Passage des paramètres par valeur

- En *Java*, tous les **paramètres** d'une méthode font partie des **variables locales** de cette méthode. Ces variables sont **initialisées** à l'aide des **arguments** passés lors de l'appel de la méthode. (**Valeur** des boutons de la boîte)
- La méthode ne reçoit qu'une **copie des arguments** lors de l'appel, donc la modification de la valeur d'un paramètre dans la méthode n'a aucun impact sur la valeur des variables de l'appelant. (L'appelant ne regarde plus les boutons après les avoir positionnés) On dit que *Java* fait **un passage des paramètres par valeur**.
- Exemple:

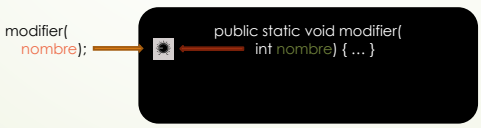
```
public class ParametreParValeur {
    public static void main(String[] args) {
        int nombre = 0;                        // Variable locale du programme principal
        System.out.println("Avant l'appel, nombre = " + nombre);
        modifier(nombre);                      // Passage de nombre par valeur.
        System.out.println("Après l'appel, nombre = " + nombre);
    }

    public static void modifier(int nombre) { // Reçoit une copie de l'argument --> nombre = 0
        nombre = 100;                          // Modifie la valeur locale de nombre
        System.out.println("Dans la méthode, nombre = " + nombre);
    }                                          // Destruction de la variable locale nombre
}
```

- Résultat:

```
Avant l'appel, nombre = 0
Dans la méthode, nombre = 100
Après l'appel, nombre = 0
```

--> La modification de la variable `nombre` en paramètre n'affecte pas le `main()`



32

Variable globale

- Une **variable globale** est **commune à tous les sous-programmes** et au programme principal. Il ne faut pas déclarer une autre variable, ou un paramètre, portant le même nom. Si cela se produit, c'est la **variable locale qui a priorité**.
- Une variable globale existe durant tout le temps d'exécution du programme. Toutes les méthodes définies à l'intérieur de la classe peuvent lire et modifier une variable globale.
- Une variable globale, appelée également variable de classe en *Java*, est déclarée à l'extérieur du programme principal (`main()`) et des autres méthodes.
- Exemple:

```
public class VariableGlobale {
    static int nombre;                          // Déclaration: variable globale nombre

    public static void main(String[] args) {
        nombre = 0;                             // Variable globale nombre = 0
        System.out.println("Avant l'appel, nombre = " + nombre);   // 0
        modifier();                             // Modifie la variable globale nombre
        System.out.println("Après l'appel, nombre = " + nombre);   // 100
    }

    public static void modifier() {
        nombre = 100;                           // Variable globale nombre = 100
        System.out.println("Dans la méthode, nombre = " + nombre); // 100
    }
}
```

- **Attention**: Il est fortement déconseillé d'utiliser des variables globales dans ce cours, car elles peuvent rendre le code difficile à comprendre et à déboguer.

Farida Ait-Ammar et Nicolas Leduc

4.7 Généralisation et polymorphisme

33

Généralisation

- Lorsque l'on écrit une méthode, il faut penser à son utilisation plus générale et non seulement au contexte du problème. Par exemple, voici trois énoncés et les fonctions associées, qui sont contextualisées:
  - Fonction qui calcule la surface d'un plancher connaissant sa longueur et sa largeur

```
public static int surfacePlancher(int lon, int lar) {
    return lon * lar;
}
```

  - Fonction qui calcule l'aire d'un mur connaissant la longueur de sa base et sa hauteur

```
public static int aireMur(int base, int haut) {
    return base * haut;
}
```

  - Fonction qui calcule la surface gazonnée d'un terrain connaissant la longueur de deux côtés

```
public static int surfaceGazon(int cote1, int cote2) {
    return cote1 * cote2;
}
```

- Toutefois, on remarque que c'est le même calcul qui est réalisé dans les trois cas, soit le calcul de l'aire d'un rectangle. Il est préférable de créer une seule fonction générique qui pourra être utilisée dans les tous les cas:

```
public static int aireRectangle(int longueur, int largeur) {
    return longueur * largeur;
}
```

Farida Ait-Ammar et Nicolas Leduc

34

Généralisation – Programme complet

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);

    int lon, lar, surfPlan;
    System.out.print("Indiquez longueur et largeur : ");
    lon = cl.nextInt();
    lar = cl.nextInt();
    surfPlan = aireRectangle(lon, lar);
    System.out.println("Surface du plancher : " + surfPlan);

    int base, haut, aireMur;
    System.out.print("Indiquez base et hauteur : ");
    base = cl.nextInt();
    haut = cl.nextInt();
    aireMur = aireRectangle(base, haut);
    System.out.println("Aire du mur : " + aireMur);

    int c1, c2, gazon;
    System.out.print("Indiquez deux côtés : ");
    c1 = cl.nextInt();
    c2 = cl.nextInt();
    gazon = aireRectangle(c1, c2);
    System.out.println("Surface du gazon : " + gazon);
}

public static int aireRectangle(int longueur, int largeur) {
    return longueur * largeur;
}
```


35

Polymorphisme

- Selon *Wikipedia*[1], le polymorphisme, du grec ancien *polús* (plusieurs) et *morphê* (forme), est le concept consistant à fournir une interface unique à des entités pouvant avoir différents types.
- En *Java*, il est possible d'avoir plusieurs méthodes ayant le même nom, en autant qu'elles aient des listes différentes de paramètres. Deux listes de paramètres sont considérées différentes si leurs nombres ou leurs types sont différents. Le nom des paramètres ne permet pas de différencier les méthodes.
- Par exemple, il est possible de déclarer les méthodes ci-dessous dans un même programme:

```
public static String inverser(String orig) { … }
public static int inverser(int orig) { … }          // Type différent
public static int inverser(int orig, int lon) { … } // Nombre différent
```

- Par contre, les déclarations suivantes sont illégales, car *Java* n'est pas en mesure de déterminer quelle méthode appeler étant donné que les listes de paramètres sont identiques:

```
public static int aire(int cote) { … }
public static float aire(int cote) { … }  // Type de retour n'a pas d'importance
public static float aire(int rayon) { … } // Nom du paramètre n'a pas d'importance
```

4.8 Exemple de résolution incluant des méthodes

36

Exemple de résolution

ÉNONCÉ : Votre client désire avoir un programme qui calcule le volume d'une boîte et qui affiche ce volume en litres et millilitres. Le calcul du volume doit être programmé dans une fonction appelée `calculVolume()`. Cette fonction reçoit trois paramètres de type entier, soit la longueur, la largeur et la hauteur de la boîte. Elle retourne un nombre entier représentant le volume calculé. Aucun affichage ni aucune lecture ne doit être effectué à l'intérieur de cette fonction. En ce qui concerne l'affichage du volume en litres et millilitres, il doit être réalisé dans une procédure qui ne retourne aucune valeur. Celle-ci reçoit un seul paramètre, soit le volume de la boîte en nombre entier, et s'appelle `afficheVolume()`. Votre programme principal (`main()`) doit demander les dimensions de la boîte en centimètres, soit sa longueur, sa largeur et sa hauteur. Il doit ensuite appeler la fonction pour calculer son volume et appeler enfin la procédure afin de l'afficher. En plus de coder votre programme, vous devez documenter vos deux méthodes.
Rappel : 1 cm^3 = 1 ml et 1 l = 1000 ml .

Exemple d'affichage au terminal :

```
Inscrivez les dimensions de la boîte en cm, soit sa longueur,
sa largeur et sa hauteur, séparées par des espaces : 43 32 21
Votre boîte contient 28 l et 896 ml.
```

Farida Ait-Ammar et Nicolas Leduc

37

Exemple de résolution – Étape 1 Décomposer le texte

- Vous devez isoler les informations concernant chaque méthode (fonction / procédure) et celles associées au programme principal (`main()`).

Votre client désire avoir un programme qui calcule le volume d'une boîte et qui affiche ce volume en litres et millilitres. Le calcul du volume doit être programmé dans une fonction appelée `calculVolume()`. Cette fonction reçoit trois paramètres de type entier, soit la longueur, la largeur et la hauteur de la boîte. Elle retourne un nombre entier représentant le volume calculé. Aucun affichage ni aucune lecture ne doit être effectué à l'intérieur de cette fonction. En ce qui concerne l'affichage du volume en litres et millilitres, il doit être réalisé dans une procédure qui ne retourne aucune valeur. Celle-ci reçoit un seul paramètre, soit le volume de la boîte en nombre entier, et s'appelle `afficheVolume()`. Votre programme principal (`main()`) doit demander les dimensions de la boîte en centimètres, soit sa longueur, sa largeur et sa hauteur. Il doit ensuite appeler la fonction pour calculer son volume et appeler enfin la procédure afin de l'afficher. En plus de coder votre programme, vous devez documenter vos deux méthodes. **Rappel** : 1 cm^3 = 1 ml et 1 l = 1000 ml .

Fonction (`calculVolume()`)
Procédure (`afficheVolume()`)
Programme principal (`main()`)
Autres informations

Farida Ait-Ammar et Nicolas Leduc

38

Exemple de résolution – Étape 2 Écrire les signatures

- À partir du texte associé à une méthode, il faut identifier les éléments faisant partie de sa signature afin de pouvoir l'écrire.

Le calcul du volume doit être programmé dans une fonction appelée `calculVolume()`. Cette fonction reçoit trois paramètres de type entier, soit la longueur, la largeur et la hauteur de la boîte. Elle retourne un nombre entier représentant le volume calculé. Aucun affichage ni aucune lecture ne doit être effectué à l'intérieur de cette fonction.

```
public static int calculVolume(int lon, int lar, int hau)
```

En ce qui concerne l'affichage du volume en litres et millilitres, il doit être réalisé dans une procédure qui ne retourne aucune valeur. Celle-ci reçoit un seul paramètre, soit le volume de la boîte en nombre entier, et s'appelle `afficheVolume()`.

```
public static void afficheVolume(int capacite)
```


39

Exemple de résolution – Étape 3 Préparer le fichier `.java`

- Vous devez maintenant créer la classe *Java* contenant le programme principal et le gabarit des méthodes que vous allez coder. Vous pouvez mettre le texte associé à chaque méthode en commentaire à l'intérieur de celles-ci afin de vous aider.

```
import java.util.Scanner;

public class VolumeCylindre {
     public static void main(String[] args) {
          Scanner clavier = new Scanner(System.in);
// Votre programme principal doit demander les dimensions de la boîte en centimètres, soit
// sa longueur, sa largeur et sa hauteur.  Il doit ensuite appeler la fonction pour calculer
// son volume et appeler enfin la procédure afin de l'afficher
     }

     public static int calculVolume(int lon, int lar, int hau) {
// Le calcul du volume doit être programmé dans une fonction appelée calculVolume().  Cette
// fonction reçoit trois paramètres de type entier, soit la longueur, la largeur et la
// hauteur de la boîte.  Elle retourne un nombre entier représentant le volume calculé.
// Aucun affichage ni aucune lecture ne doit être effectué à l'intérieur de cette fonction.
     }

     public static void afficheVolume(int capacite) {
// En ce qui concerne l'affichage du volume en litres et millilitres, il doit être réalisé
// dans une procédure qui ne retourne aucune valeur.  Celle-ci reçoit un seul paramètre,
// soit le volume de la boîte en nombre entier, et s'appelle afficheVolume().
     }
}
```

40

Exemple de résolution – Étape 4 (1/2) Coder les méthodes

- Écrivez le code et documentez chaque méthode **indépendamment** du reste du programme, une méthode à la fois. Assurez-vous qu'il n'y ait aucune erreur de compilation.

```
/**
 * Procédure qui affiche un volume en litres et millilitres
 *
 * @param capacite Capacité en millilitres ou en cm3
 */
public static void afficheVolume(int capacite) {
// En ce qui concerne l'affichage du volume en litres et millilitres,
// il doit être réalisé dans une procédure qui ne retourne aucune
// valeur.  Celle-ci reçoit un seul paramètre, soit le volume de la
// boîte en nombre entier, et s'appelle afficheVolume().

    int litre, millilitre;

    litre = capacite / 1000;
    millilitre = capacite % 1000;
    System.out.print("Votre boîte contient " + litre +
                     " l et " + millilitre + " ml.");
}
```

Farida Ait-Ammar et Nicolas Leduc

41

Exemple de résolution – Étape 4 (2/2) Coder les méthodes

```
/**
 * Fonction qui calcule le volume d'une boîte en cm3
 *
 * @param lon Longueur de la boîte en cm
 * @param lar Largeur de la boîte en cm
 * @param hau Hauteur de la boîte en cm
 *
 * @return Volume en cm3
 */
public static int calculVolume(int lon, int lar, int hau) {
// Le calcul du volume doit être programmé dans une fonction appelée
// calculVolume() .  Cette fonction reçoit trois paramètres de type
// entier, soit la longueur, la largeur et la hauteur de la
// boîte.  Elle retourne un nombre entier représentant le volume
// calculé.  Aucun affichage ni aucune lecture ne doit être effectué
// à l'intérieur de cette fonction.

    int vol;

    vol = lon * lar * hau;
    return vol;
}
```

Farida Ait-Ammar et Nicolas Leduc

42

Exemple de résolution – Étape 5 Coder le programme principal (`main()`)

- Il ne vous reste qu'à écrire le programme principal (`main()`), qui doit appeler vos méthodes. Le texte vous indique les principales étapes de l'algorithme que vous devez implémenter.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
// Votre programme principal doit demander les dimensions de la boîte
// en centimètres, soit sa longueur, sa largeur et sa hauteur.  Il
// doit ensuite appeler la fonction pour calculer son volume et
// appeler enfin la procédure afin de l'afficher.

    int longueur, largeur, hauteur;
    int volume;

    System.out.println("Inscrivez les dimensions de la boîte en cm, soit sa longueur,");
    System.out.print("sa largeur et sa hauteur, séparées par des espaces : ");
    longueur = clavier.nextInt();
    largeur = clavier.nextInt();
    hauteur = clavier.nextInt();
    volume = calculVolume(longueur, largeur, hauteur);
    afficheVolume(volume);
}
```

Farida Ait-Ammar et Nicolas Leduc

43

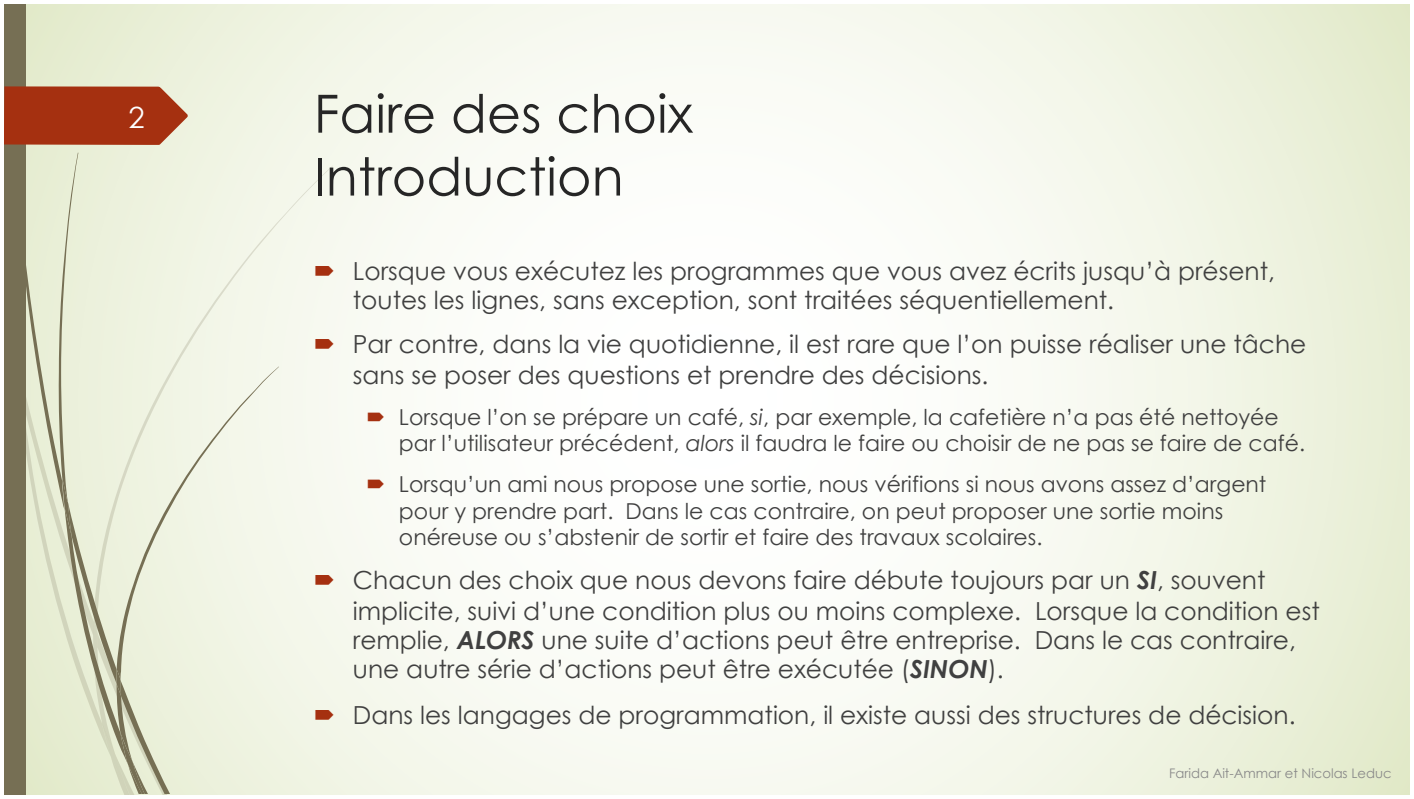
Exemple de résolution – Étape 6 Tester le programme

- Exécutez votre programme et assurez-vous que vous obtenez le même affichage que le ou les terminaux inclus dans l'énoncé de votre atelier ou de votre travail pratique.

Farida Ait-Ammar et Nicolas Leduc

5. Décision

5.1 Variable booléenne et opérateurs



3

Opérateurs relationnels (comparaison)

Opérateur	Dénomination	Effet	Exemple	Résultat
== À ne pas confondre avec le signe d'affectation (=)	opérateur d'égalité	Vérifie qu'une variable est égale à une valeur	`X == 3`	Retourne `true` si `X` est égal à 3, sinon `false`
!=	opérateur de différence	Vérifie qu'une variable est différente d'une valeur	`X != 3`	Retourne `true` si `X` est différent de 3, sinon `false`
<	opérateur d'infériorité stricte	Vérifie qu'une variable est strictement inférieure à une valeur	`X < 3`	Retourne `true` si `X` est inférieur à 3, sinon `false`
<=	opérateur d'infériorité	Vérifie qu'une variable est inférieure ou égale à une valeur	`X <= 3`	Retourne `true` si `X` est inférieur ou égal à 3, sinon `false`
>	opérateur de supériorité stricte	Vérifie qu'une variable est strictement supérieure à une valeur	`X > 3`	Retourne `true` si `X` est supérieur à 3, sinon `false`
>=	opérateur de supériorité	Vérifie qu'une variable est supérieure ou égale à une valeur	`X >= 3`	Retourne `true` si `X` est supérieur ou égal à 3, sinon `false`

Attention: Ces **opérateurs** sont seulement **valides** pour des variables de **types simples**. Ils n'auront pas l'effet décrit pour les types composés, comme *`String`*. De plus, l'opérateur d'**égalité** peut avoir un comportement **imprévisible** pour les nombres en **virgule flottante**. (Ex.: `54.321*10 == 543.21` en `float` donne `false`)

Farida Ait-Ammar et Nicolas Leduc

4

Opérateurs logiques

- Les opérateurs logiques permettent de combiner les opérateurs relationnels
  - **SI** il fait beau **ET** que j'ai de l'argent **ALORS** je vais à *La Ronde*.

Priorité	Opérateur	Dénomination	Effet	Table de vérité
+	**( )**		Priorise des groupes d'opérateurs	
	!	NON logique (¬)	Inverse l'état d'une variable booléenne	X / !X **false** **true** true false
	&&	ET logique (∧)	Vérifie que toutes les conditions sont réalisées	X / Y / X&&Y false false false false true false true false false **true true true**
-	**||**	OU logique (∨)	Vérifie qu'une des conditions est réalisée	X / Y / X||Y false false false **false true true** **true false true** **true true true**

- Il est possible de simplifier les expressions en utilisant les règles suivantes:

```
X && X → X        X && !X → false        X || X → X        X || !X → true
   !(!X) → X        !(X && Y) → !X || !Y        !(X || Y) → !X && !Y
```

Farida Ait-Ammar et Nicolas Leduc

5

Variable booléenne

- Les variables de type booléen (`boolean`) permettent d'enregistrer des informations qui ne peuvent prendre que la valeur vrai (`true`) ou faux (`false`). Elles peuvent aussi être utilisées pour stocker le résultat d'une expression booléenne.

```
int x = 5, y = 10;
boolean resExp, soleil = true;

resExp = x >= y;
System.out.print(soleil);          // Affiche true
System.out.print(!resExp);         // Affiche true
System.out.print(x < 5 && y > 5); // Affiche false
```

Farida Ait-Ammar et Nicolas Leduc

5.2 SI ... ALORS ... (`if _`)

7

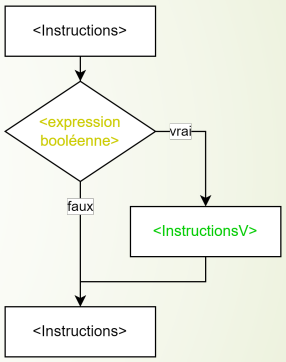
SI ... ALORS ...

- Pseudo-code

```
SI (<expression booléenne>) ALORS
    <InstructionsV>
FINSI
```

- *Java*

```
if (<expression booléenne>) {
    <InstructionsV>;
}
```



- Remarques:
  - La **condition** est écrite entre **parenthèses.**
  - Il ne faut pas mettre un point virgule à la fin de la ligne contenant le `if`, ni après le mot clé `else`.
  - Dans ce cours, les **accolades** sont **obligatoires** même si le `if` ou le `else` ne contient qu'une seule instruction.

Farida Ait-Ammar et Nicolas Leduc

8

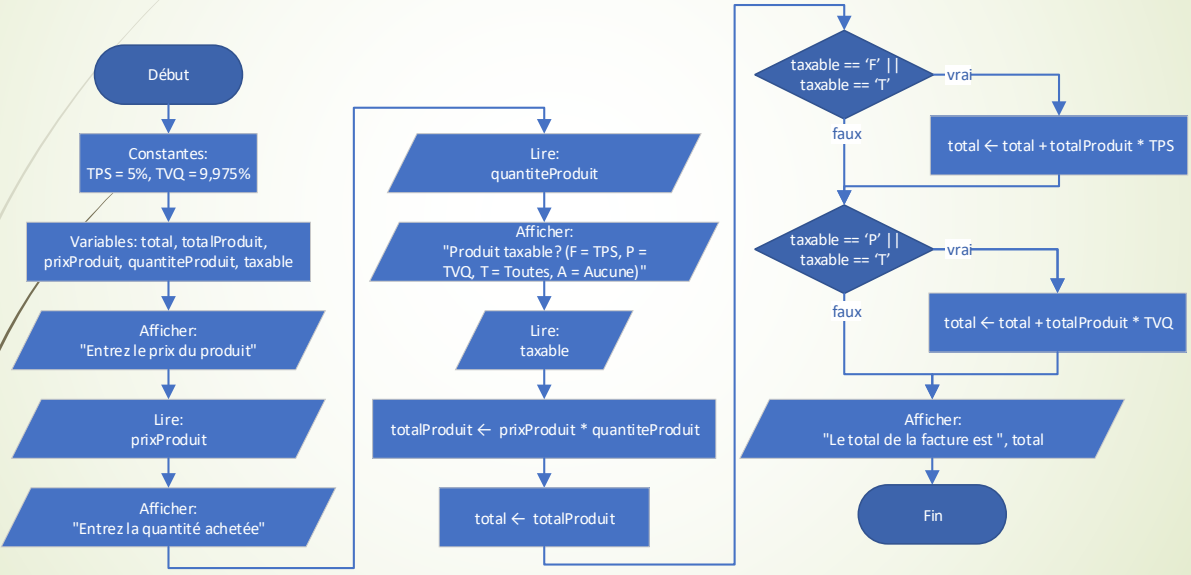
Facture – Sélection des taxes

```
ALGORITHME FactureSelTaxes
CONSTANTES
     TPS = 5%
     TVQ = 9,975%
VARIABLES
     total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
     taxable TYPE CARACTÈRE
DÉBUT
     AFFICHER("Entrez le prix du produit")
     LIRE(prixProduit)
     AFFICHER("Entrez la quantité achetée")
     LIRE(quantiteProduit)
     AFFICHER("Produit taxable? (F = TPS, P = TVQ, T = Toutes, A = Aucune)")
     LIRE(taxable)
     totalProduit ← prixProduit * quantiteProduit
     total ← totalProduit
     SI (taxable == 'F' || taxable == 'T') ALORS
          total ← total + totalProduit * TPS
     FINSI
     SI (taxable == 'P' || taxable == 'T') ALORS
          total ← total + totalProduit * TVQ
     FINSI
     AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

9

Facture – Sélection des taxes



Farida Ait-Ammar et Nicolas Leduc

10

Facture – Sélection des taxes

Achat de 5 produits à 10$ chacun. Seule la TPS doit être perçue sur ce type de produit.

```
ALGORITHME FactureSelTaxes
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
            quantiteProduit TYPE NUMÉRIQUE
      taxable TYPE CARACTÈRE
DÉBUT
1     AFFICHER("Entrez le prix du produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     AFFICHER("Produit taxable? (F = TPS, P = TVQ,
            T = Toutes, A = Aucune)")
6     LIRE(taxable)
7     totalProduit ← prixProduit * quantiteProduit
8     total ← totalProduit
9     SI (taxable == 'F' || taxable == 'T') ALORS
10          total ← total + totalProduit * TPS
      FINSI
11    SI (taxable == 'P' || taxable == 'T') ALORS
12          total ← total + totalProduit * TVQ
      FINSI
13    AFFICHER("Le total de la facture est ", total)
FIN
```

#	total	totProd	prixProd	qteProd	taxable	Terminal
1						Entrez le prix du produit
2			10			
3						Entrez la quantité achetée
4				5		
5						Produit taxable? (F = TPS, P = TVQ, T = Toutes, A = Aucune)
6					F	
7		50				
8	50					
9						
10	52,50					
11						
13						Le total de la facture est 52,50

Farida Ait-Ammar et Nicolas Leduc

11

Facture – Sélection des taxes

```
import java.util.Scanner;

public class FactureSelTaxes {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);

        float total, totalProduit, prixProduit;
        int quantiteProduit;
        char taxable;

        System.out.print("Entrez le prix du produit : ");
        prixProduit = clavier.nextFloat();
        System.out.print("Entrez la quantité achetée : ");
        quantiteProduit = clavier.nextInt();
        System.out.println("Produit taxable? (F = TPS, P = TVQ, T = Toutes, A = Aucune)");
        taxable = clavier.next().toUpperCase().charAt(0);
        totalProduit = prixProduit * quantiteProduit;
        total = totalProduit;
        if (taxable == 'F' || taxable == 'T') {
            total = total + totalProduit * TPS;
        }
        if (taxable == 'P' || taxable == 'T') {
            total = total + totalProduit * TVQ;
        }
        System.out.println(String.format("Le total de la facture est %.2f$.", total));
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

5.3 SI ... ALORS ... SINON ... (`if _ else _`)

12

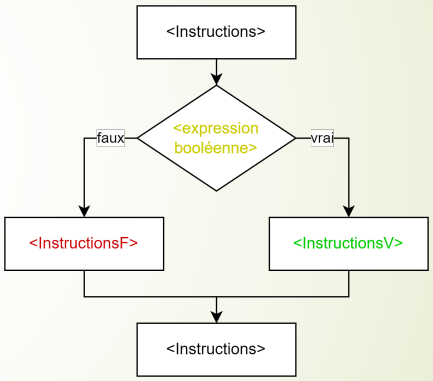
SI ... ALORS ... SINON ...

- Pseudo-code

```
SI (<expression booléenne>) ALORS
    <InstructionsV>
SINON
    <InstructionsF>
FINSI
```

- *Java*

```
if (<expression booléenne>) {
    <InstructionsV>;
} else {
    <InstructionsF>;
}
```



Farida Ait-Ammar et Nicolas Leduc

13

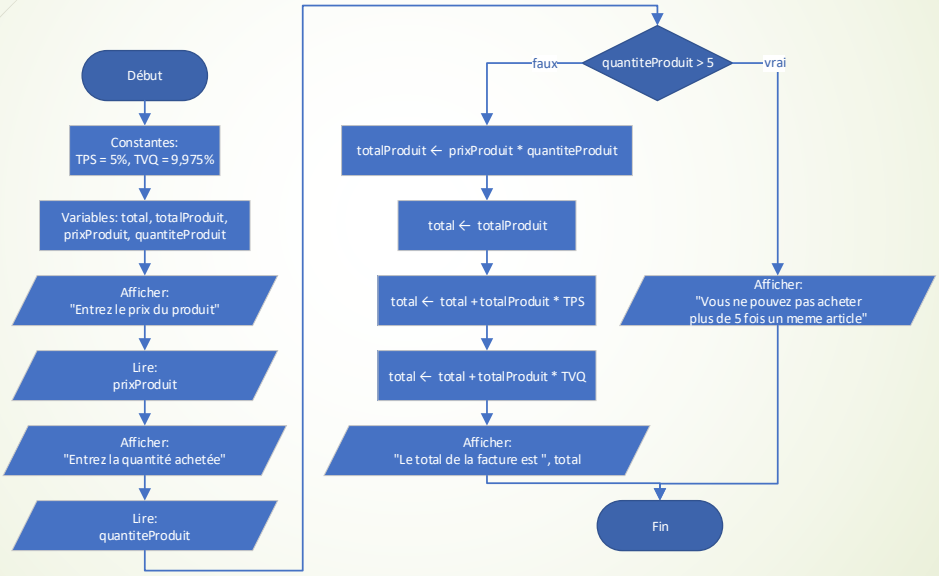
Facture – Limite d'achat (5 produits)

```
ALGORITHME FactureLimiteAchat
CONSTANTES
    TPS = 5%
    TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée")
    LIRE(quantiteProduit)
    SI (quantiteProduit > 5) ALORS
        AFFICHER("Vous ne pouvez pas acheter plus de 5 fois un même article")
    SINON
        totalProduit ← prixProduit * quantiteProduit
        total ← totalProduit
        total ← total + totalProduit * TPS
        total ← total + totalProduit * TVQ
        AFFICHER("Le total de la facture est ", total)
    FINSI
FIN
```

Farida Ait-Ammar et Nicolas Leduc

14

Facture – Limite d'achat (5 produits)



Farida Ait-Ammar et Nicolas Leduc

15

Facture – Limite d'achat (5 produits)

Achat de 10 produits à 10$ chacun.

```
ALGORITHME FactureLimiteAchat
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
            quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
1     AFFICHER("Entrez le prix du produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     SI (quantiteProduit > 5) ALORS
6           AFFICHER("Vous ne pouvez pas acheter plus
                  de 5 fois un même article")
7     SINON
8           totalProduit ← prixProduit *
                  quantiteProduit
9           total ← totalProduit
10          total ← total + totalProduit * TPS
11          total ← total + totalProduit * TVQ
12          AFFICHER("Le total de la facture est ",
                  total)
      FINSI
FIN
```

#	total	totProd	prixProd	qteProd	Terminal
1					Entrez le prix du produit
2			10		
3					Entrez la quantité achetée
4				10	
5					
6					Vous ne pouvez pas acheter plus de 5 fois un même article

Farida Ait-Ammar et Nicolas Leduc

16

Facture – Limite d'achat (5 produits)

```
import java.util.Scanner;

public class FactureLimiteAchat {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);

        float total, totalProduit, prixProduit;
        int quantiteProduit;

        System.out.print("Entrez le prix du produit : ");
        prixProduit = clavier.nextFloat();
        System.out.print("Entrez la quantité achetée : ");
        quantiteProduit = clavier.nextInt();
        if (quantiteProduit > 5) {
            System.out.println("Vous ne pouvez pas acheter plus de 5 fois un même article");
        } else {
            totalProduit = prixProduit * quantiteProduit;
            total = totalProduit;
            total = total + totalProduit * TPS;
            total = total + totalProduit * TVQ;
            System.out.println(String.format("Le total de la facture est %.2f$.", total));
        }
    }
}
```


17

Blocs d'instructions vides

- Vous devez éviter d'avoir des blocs d'instructions vides dans vos conditions.
- Si cela se produit à la suite d'un `if`, vous devez inverser votre condition afin que le bloc vide se retrouve dans le `else`.
- Vous pouvez éliminer un `else` avec son bloc d'instructions vide associé.
- Inverse des opérateurs de comparaison:

  `==` ↔ `!=` `<` ↔ `>=` `>` ↔ `<=`
- Inverse des opérateurs logiques (lois de De Morgan):

  `!(X && Y)` → `!X || !Y` `!(X || Y)` → `!X && !Y`

```
if (total < 100) {
    // Pas de rabais
} else {
    total = total - 10;
}
```

```
if (total >= 100) {
    total = total - 10;
} else {
    // Pas de rabais
}
```

```
if (total >= 100) {
    total = total - 10;
}
```

5.4 Décisions imbriquées

18

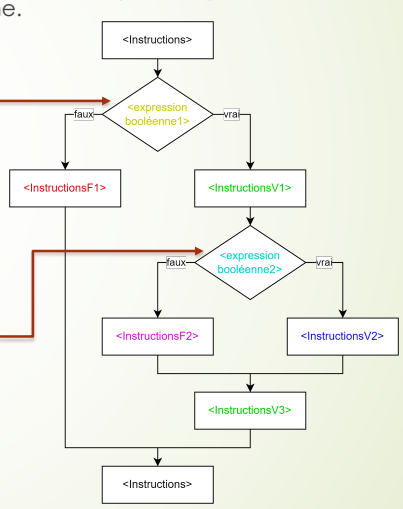
Décisions imbriquées

- Les blocs d'instructions contenus dans les structures `if` et `else` peuvent contenir des instructions simples, mais aussi d'autres instructions `if`. Il s'agit alors de décisions imbriquées. Dans ce cas, il faut bien respecter **l'indentation** pour simplifier la compréhension du programme ou de l'algorithme.
- Pseudo-code

```
SI (<expression booléenne1>) ALORS
    <InstructionsV1>
    SI (<expression booléenne2>) ALORS
      <InstructionsV2>
    SINON
      <InstructionsF2>
    FINSI
    <InstructionsV3>
SINON
    <InstructionsF1>
FINSI
```

- *Java*

```
if (<expression booléenne1>) {
    <InstructionsV1>;
    if (<expression booléenne2>) {
      <InstructionsV2>;
    } else {
      <InstructionsF2>;
    }
    <InstructionsV3>;
} else {
    <InstructionsF1>;
}
```



Farida Ait-Ammar et Nicolas Leduc

19

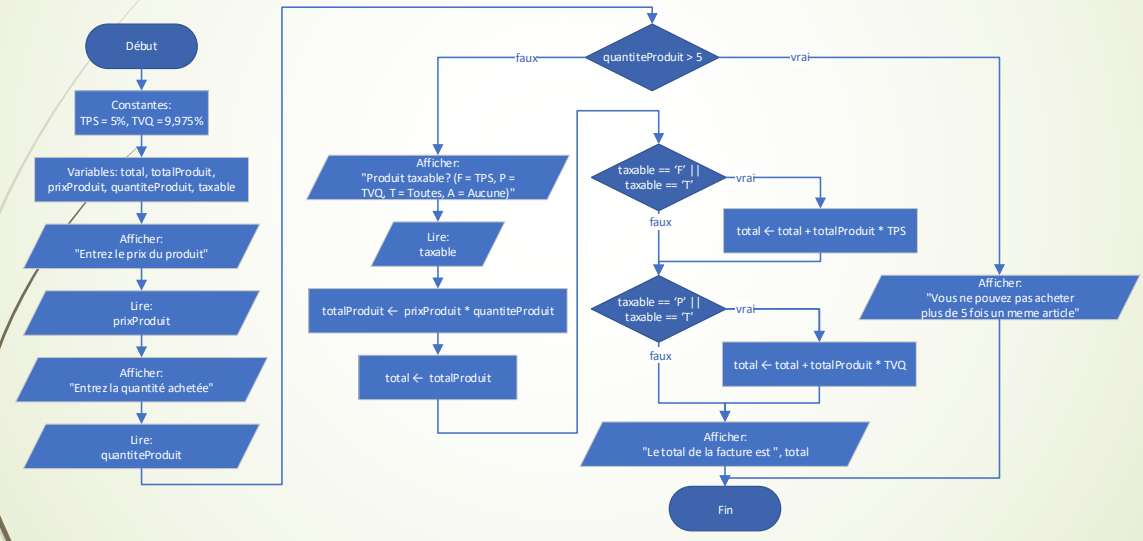
Facture – Taxes + limite

```
ALGORITHME FactureTaxesEtLimite
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
      taxable TYPE CARACTÈRE
DÉBUT
      AFFICHER("Entrez le prix du produit")
      LIRE(prixProduit)
      AFFICHER("Entrez la quantité achetée")
      LIRE(quantiteProduit)
      SI (quantiteProduit > 5) ALORS
            AFFICHER("Vous ne pouvez pas acheter plus de 5 fois un même article")
      SINON
            AFFICHER("Produit taxable? (F = TPS, P = TVQ, T = Toutes, A = Aucune)")
            LIRE(taxable)
            totalProduit ← prixProduit * quantiteProduit
            total ← totalProduit
            SI (taxable == 'F' || taxable == 'T') ALORS
                  total ← total + totalProduit * TPS
            FINSI
            SI (taxable == 'P' || taxable == 'T') ALORS
                  total ← total + totalProduit * TVQ
            FINSI
            AFFICHER("Le total de la facture est ", total)
      FINSI
FIN
```

Farida Ait-Ammar et Nicolas Leduc

20

Facture – Taxes + limite



21

Facture – Taxes + limite

Achat de 5 produits à 10$ chacun. Seule la TVQ doit être perçue sur ce type de produit.

```
ALGORITHME FactureTaxesEtLimite
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit, quantiteProduit
            TYPE NUMÉRIQUE
      taxable TYPE CARACTÈRE
DÉBUT
1     AFFICHER("Entrez le prix du produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     SI (quantiteProduit > 5) ALORS
6           AFFICHER("Vous ne pouvez pas acheter plus de 5
                  fois un même article")
7     SINON
8           AFFICHER("Produit taxable? (F = TPS, P = TVQ,
                  T = Toutes, A = Aucune)")
9           LIRE(taxable)
10          totalProduit ← prixProduit * quantiteProduit
11          total ← totalProduit
12          SI (taxable == 'F' || taxable == 'T') ALORS
13                total ← total + totalProduit * TPS
            FINSI
14          SI (taxable == 'P' || taxable == 'T') ALORS
15                total ← total + totalProduit * TVQ
            FINSI
16          AFFICHER("Le total de la facture est ", total)
      FINSI
FIN
```

#	total	totProd	prixProd	qteProd	taxable	Terminal
1						Entrez le prix du produit
2			10			
3						Entrez la quantité achetée
4				5		
5						
7						
8						Produit taxable? (F = TPS, P = TVQ, T = Toutes, A = Aucune)
9					P	
10		50				
11	50					
12						
14						
15	54,99					
16						Le total de la facture est 54,99

22

Facture – Taxes + limite

```
import java.util.Scanner;

public class FactureTaxesEtLimite {
     public static final float TPS = 0.05f, TVQ = 0.09975f;

     public static void main(String[] args) {
          Scanner clavier = new Scanner(System.in);
          float total, totalProduit, prixProduit;
          int quantiteProduit;
          char taxable;
          System.out.print("Entrez le prix du produit : ");
          prixProduit = clavier.nextFloat();
          System.out.print("Entrez la quantité achetée : ");
          quantiteProduit = clavier.nextInt();
          if (quantiteProduit > 5) {
               System.out.println("Vous ne pouvez pas acheter plus de 5 fois un même article");
          } else {
               System.out.println("Produit taxable? (F = TPS, P = TVQ, T = Toutes, A = Aucune)");
               taxable = clavier.next().toUpperCase().charAt(0);
               totalProduit = prixProduit * quantiteProduit;
               total = totalProduit;
               if (taxable == 'F' || taxable == 'T') {
                    total = total + totalProduit * TPS;
               }
               if (taxable == 'P' || taxable == 'T') {
                    total = total + totalProduit * TVQ;
               }
               System.out.println(String.format("Le total de la facture est %.2f$.", total));
          }
     }
}
```

Farida Ait-Ammar et Nicolas Leduc

5.5 SELON ... CAS ... DEFAUT ... (`switch _ case _ default _`)

23

SELON … CAS … DEFAUT …

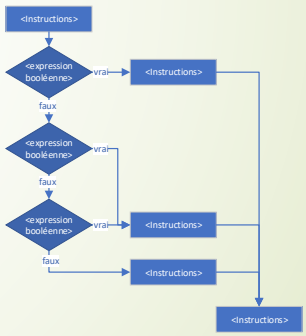
- Fonctionnement
  - L'expression est d'abord évaluée et sa valeur est recherchée parmi les étiquettes définies (`case`)
  - Si l'étiquette est trouvée, toutes les instructions qui la suivent sont exécutées jusqu'à ce que l'on atteigne l'instruction `break`, peu importe si d'autres `case` ont été définis dans l'intervalle.
  - Si aucune étiquette ne correspond à la valeur de l'expression, alors on exécute le bloc `default`.
- L'expression peut être de type `byte`, `short`, `int`, `char`, *énuméré* ou `String`.

Pseudo-code

```
SELON (expression)
     CAS etiquette1 :
          <Instructions>
          FINCAS
     CAS etiquette2:
     CAS etiquette3:
          <Instructions>
          FINCAS
     DEFAUT:
          <Instructions>
FINSELON
```

Java

```
switch (expression) {
     case etiquette1 :
          <Instructions>;
          break;
     case etiquette2:
     case etiquette3:
          <Instructions>;
          break;
     default:
          <Instructions>;
}
```



Farida Ait-Ammar et Nicolas Leduc

24

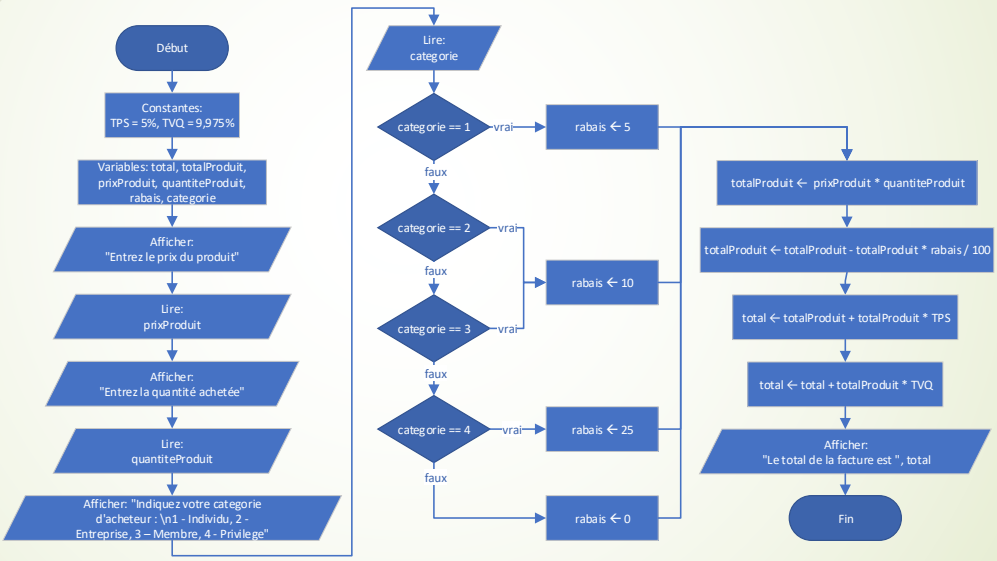
Facture – Catégorie de client

```
ALGORITHME FactureCategorie
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit, quantiteProduit, rabais, categorie TYPE NUMÉRIQUE
DÉBUT
      AFFICHER("Entrez le prix du produit")
      LIRE(prixProduit)
      AFFICHER("Entrez la quantité achetée")
      LIRE(quantiteProduit)
      AFFICHER("Indiquez votre catégorie d'acheteur : ")
      AFFICHER("1 - Individu, 2 - Entreprise, 3 - Membre, 4 - Privilège")
      LIRE(categorie)
      SELON (categorie)
            CAS 1 : rabais ← 5
                  FINCAS
            CAS 2 :
            CAS 3 : rabais ← 10
                  FINCAS
            CAS 4 : rabais ← 25
                  FINCAS
            DEFAUT : rabais ← 0
      FINSELON
      totalProduit ← prixProduit * quantiteProduit
      totalProduit ← totalProduit - totalProduit * rabais / 100
      total ← totalProduit + totalProduit * TPS
      total ← total + totalProduit * TVQ
      AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

25

Facture – Catégorie de client



Farida Ait-Ammar et Nicolas Leduc

26

Facture – Catégorie de client

Achat de 5 produits à 10$ chacun.
Le client a le statut *Privilège*.

```
ALGORITHME FactureCategorie
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit, quantiteProduit,
            rabais, categorie TYPE NUMÉRIQUE
DÉBUT
1     AFFICHER("Entrez le prix du produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     AFFICHER("Indiquez votre catégorie d'acheteur : ")
6     AFFICHER("1 - Individu, 2 - Entreprise, 3 - Membre,
            4 - Privilège")
7     LIRE(categorie)
8     SELON (categorie)
9           CAS 1 : rabais ← 5
                FINCAS
10          CAS 2 :
11          CAS 3 : rabais ← 10
                FINCAS
12          CAS 4 : rabais ← 25
                FINCAS
13          DEFAUT : rabais ← 0
      FINSELON
14    totalProduit ← prixProduit * quantiteProduit
15    totalProduit←totalProduit-totalProduit*rabais/100
16    total ← totalProduit + totalProduit * TPS
17    total ← total + totalProduit * TVQ
18    AFFICHER("Le total de la facture est ", total)
FIN
```

#	total	totProd	prixProd	qteProd	rabais	cat	Terminal
1							Entrez le prix du produit
2			10				
3							Entrez la quantité achetée
4				5			
5							Indiquez votre catégorie d'acheteur :
6							1 - Individu, 2 - Entreprise, 3 – Membre, 4 - Privilège
7						4	
8							
12					25		
14		50					
15		37,50					
16	39,38						
17	43,12						
18							Le total de la facture est 43,12

Farida Ait-Ammar et Nicolas Leduc

27

Facture – Catégorie de client

```
import java.util.Scanner;

public class FactureCategorie {
      public static final float TPS = 0.05f, TVQ = 0.09975f;

      public static void main(String[] args) {
            Scanner clavier = new Scanner(System.in);
            float total, totalProduit, prixProduit;
            int quantiteProduit, rabais, categorie;

            System.out.print("Entrez le prix du produit : ");
            prixProduit = clavier.nextFloat();
            System.out.print("Entrez la quantité achetée : ");
            quantiteProduit = clavier.nextInt();
            System.out.println("Indiquez votre catégorie d'acheteur : \n" +
                  "1 - Individu, 2 - Entreprise, 3 - Membre, 4 - Privilège");
            categorie = clavier.nextInt();
            switch (categorie) {
            case 1:
                  rabais = 5;
                  break;
            case 2:
            case 3:
                  rabais = 10;
                  break;
            case 4:
                  rabais = 25;
                  break;
            default:
                  rabais = 0;
            }
            totalProduit = prixProduit * quantiteProduit;
            totalProduit = totalProduit - totalProduit * rabais / 100;
            total = totalProduit + totalProduit * TPS;
            total = total + totalProduit * TVQ;
            System.out.println(String.format("Le total de la facture est %.2f$.", total));
      }
}
```

Farida Ait-Ammar et Nicolas Leduc

5.6 SI ... ALORS ... SINON SI ..... SINON ... (`if _ else if __ else _`)

28

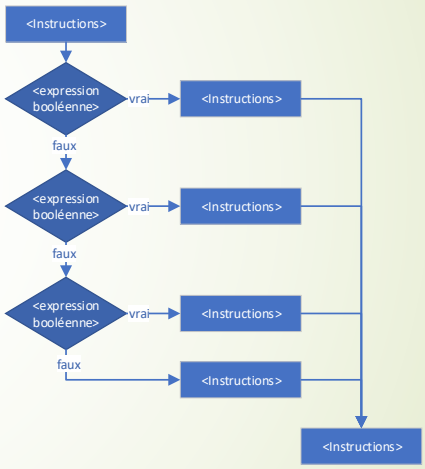
SI … ALORS … SINON SI ….. SINON …

- Pseudo-code

```
SI (<expression booléenne>) ALORS
    <Instructions>
SINON SI (<expression booléenne>) ALORS
    <Instructions>
SINON SI (<expression booléenne>) ALORS
    <Instructions>
SINON
    <Instructions>
FINSI
```

- *Java*

```
if (<expression booléenne>) {
    <Instructions>;          // vrai
} else if (<expression booléenne>) {
    <Instructions>;          // faux, vrai
} else if (<expression booléenne>) {
    <Instructions>;          // faux, faux, vrai
} else {
    <Instructions>;          // faux, faux, faux
}
```



Farida Ait-Ammar et Nicolas Leduc

29

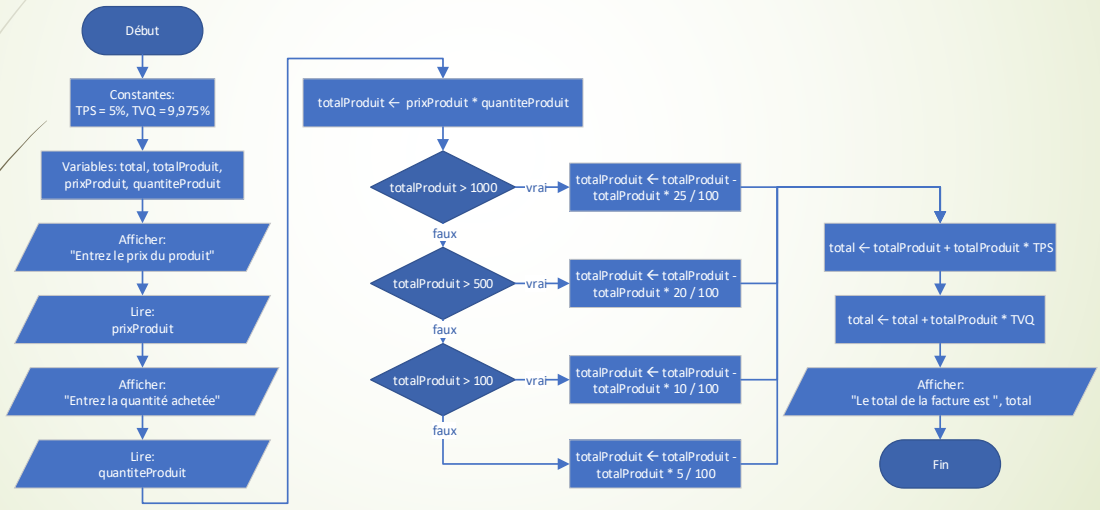
Facture – Rabais selon le montant

```
ALGORITHME FactureRabaisMontant
CONSTANTES
     TPS = 5%
     TVQ = 9,975%
VARIABLES
     total, totalProduit, prixProduit, quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
     AFFICHER("Entrez le prix du produit")
     LIRE(prixProduit)
     AFFICHER("Entrez la quantité achetée")
     LIRE(quantiteProduit)
     totalProduit ← prixProduit * quantiteProduit
     SI (totalProduit > 1000) ALORS
          totalProduit ← totalProduit - totalProduit * 25 / 100
     SINON SI (totalProduit > 500) ALORS
          totalProduit ← totalProduit - totalProduit * 20 / 100
     SINON SI (totalProduit > 100) ALORS
          totalProduit ← totalProduit - totalProduit * 10 / 100
     SINON
          totalProduit ← totalProduit - totalProduit * 5 / 100
     FINSI
     total ← totalProduit + totalProduit * TPS
     total ← total + totalProduit * TVQ
     AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

30

Facture – Rabais selon le montant



Farida Ait-Ammar et Nicolas Leduc

31

Facture – Rabais selon le montant

Achat de 25 produits à 10$ chacun.

```
ALGORITHME FactureRabaisMontant
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
            quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
1     AFFICHER("Entrez le prix du produit")
2     LIRE(prixProduit)
3     AFFICHER("Entrez la quantité achetée")
4     LIRE(quantiteProduit)
5     totalProduit ← prixProduit * quantiteProduit
6     SI (totalProduit > 1000) ALORS
7           totalProduit ← totalProduit -
                  totalProduit * 25 / 100
8     SINON SI (totalProduit > 500) ALORS
9           totalProduit ← totalProduit -
                  totalProduit * 20 / 100
10    SINON SI (totalProduit > 100) ALORS
11          totalProduit ← totalProduit -
                  totalProduit * 10 / 100
12    SINON
13          totalProduit ← totalProduit -
                  totalProduit * 5 / 100
      FINSI
14    total ← totalProduit + totalProduit * TPS
15    total ← total + totalProduit * TVQ
16    AFFICHER("Le total de la facture est ", total)
FIN
```

#	total	totProd	prixProd	qteProd	Terminal
1					Entrez le prix du produit
2			10		
3					Entrez la quantité achetée
4				25	
5		250			
6					
8					
10					
11		225			
14	236,25				
15	258,69				
16					Le total de la facture est 258,69

Farida Ait-Ammar et Nicolas Leduc

32

Facture – Rabais selon le montant

```
import java.util.Scanner;

public class FactureRabaisMontant {
     public static final float TPS = 0.05f, TVQ = 0.09975f;

     public static void main(String[] args) {
          Scanner clavier = new Scanner(System.in);

          float total, totalProduit, prixProduit;
          int quantiteProduit;

          System.out.print("Entrez le prix du produit : ");
          prixProduit = clavier.nextFloat();
          System.out.print("Entrez la quantité achetée : ");
          quantiteProduit = clavier.nextInt();
          totalProduit = prixProduit * quantiteProduit;
          if (totalProduit > 1000) {
               totalProduit = totalProduit - totalProduit * 25 / 100;
          } else if (totalProduit > 500) {
               totalProduit = totalProduit - totalProduit * 20 / 100;
          } else if (totalProduit > 100) {
               totalProduit = totalProduit - totalProduit * 10 / 100;
          } else {
               totalProduit = totalProduit - totalProduit * 5 / 100;
          }
          total = totalProduit + totalProduit * TPS;
          total = total + totalProduit * TVQ;
          System.out.println(String.format("Le total de la facture est %.2f$.", total));
     }
}
```

Farida Ait-Ammar et Nicolas Leduc

33

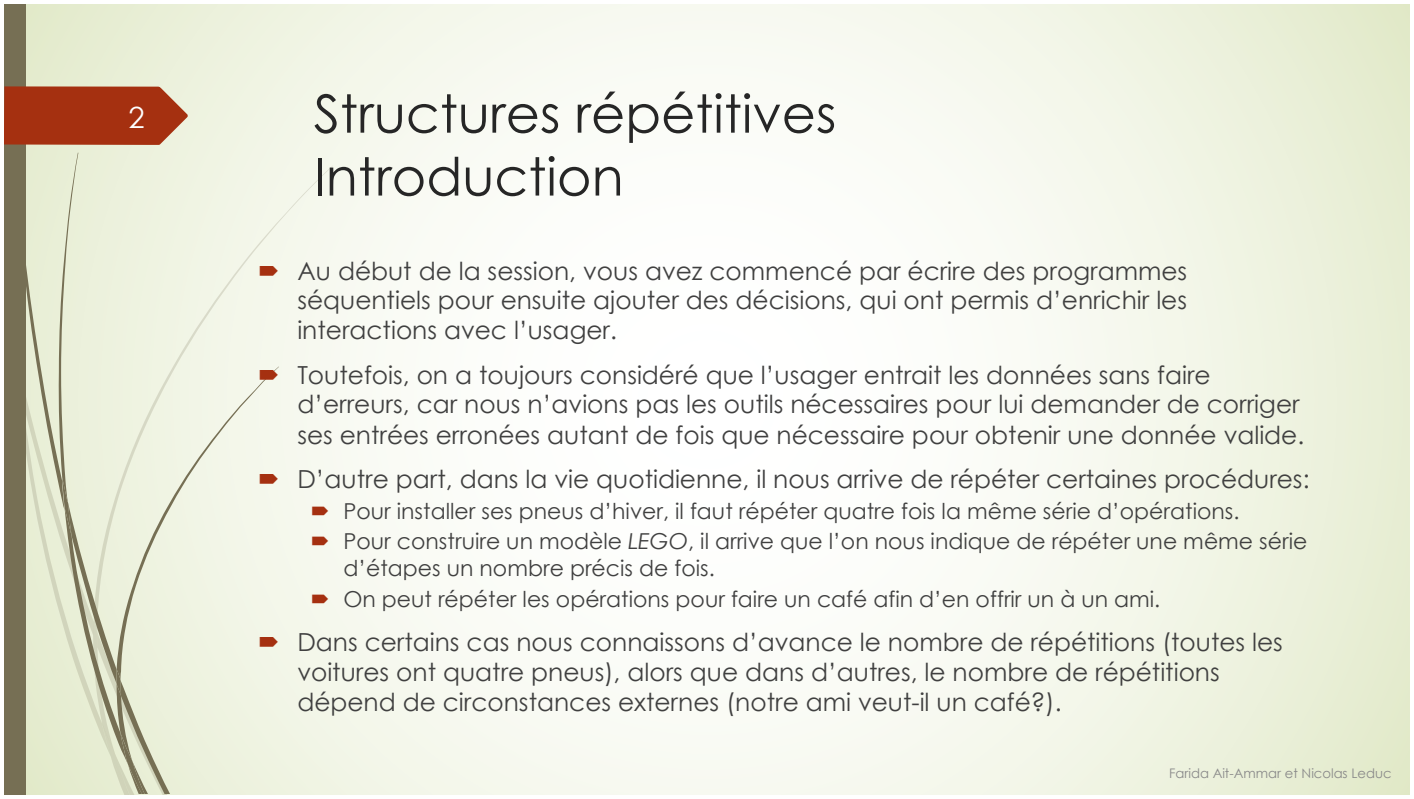
Choix de la structure de décision

- SELON … CAS … DEFAUT …
  - Variable de type caractère ou chaîne de caractères
    - Exemple: Nom du mois pour trouver le nombre de jours
  - Valeur entière ayant un nombre limité de valeurs possibles (max 20)
    - Exemple: Chiffre pour choisir une option dans un menu
- SI … ALORS … SINON SI ….. SINON …
  - Variable de type réel
    - Exemple: Rabais selon la valeur de l'achat (type réel)
  - Valeur entière avec un grand nombre de valeurs possibles (plus de 20)
    - Exemple: Mention selon la note arrondie de l'étudiant (type entier)

Farida Ait-Ammar et Nicolas Leduc

6. Répétition

6.1 Introduction



3

Structures répétitives Introduction

- Dans les langages de programmation, il existe aussi des structures permettant de faire des répétitions.
- Une **structure répétitive**, appelée également **structure itérative** ou **boucle**, permet de répéter une ou plusieurs instructions tant qu'une condition est respectée.
- Il existe trois types d'instructions répétitives en *Java*:
  - `while (TANTQUE … FAIRE)`
  - `do ... while (FAIRE … TANTQUE)`
  - `for (POUR)`

Farida Ait-Ammar et Nicolas Leduc

6.2 TANTQUE ... FAIRE ... (`while _`)

4

TANTQUE … FAIRE …

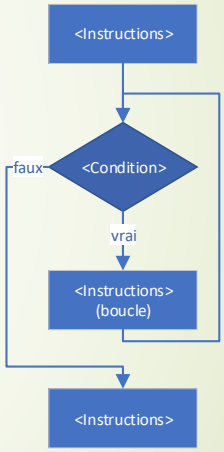
- Pseudo-code

```
TANTQUE (<expression booléenne>) FAIRE
    <Instructions>
FINTANTQUE
```

- *Java*

```
while (<expression booléenne>) {
    <Instructions>;
}
```

- Fonctionnement
  - 1 - Vérifier d'abord la condition
  - 2 - SI la condition est vraie ALORS
    - 2a - On exécute les instructions à répéter
    - 2b - On retourne à la vérification de la condition (étape 1)
  - 3 - SINON on quitte la boucle
- Note:
  - Il est possible que les instructions à répéter ne soient jamais exécutées.
  - Le nombre d'exécutions de la boucle est inconnu au départ.
  - Il ne faut pas mettre de point-virgule à la fin de l'instruction `while`.
  - La valeur de la variable faisant partie de la condition doit être modifiée à l'intérieur de la boucle, sinon on obtient une boucle infinie.



Farida Ait-Ammar et Nicolas Leduc

5

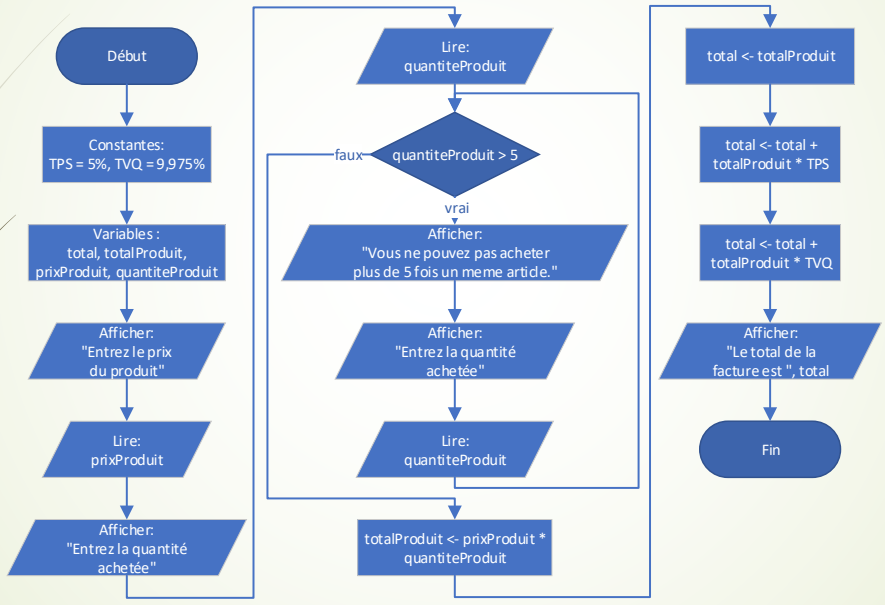
Facture - Limite d'achat (5 produits)

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%, TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit TYPE NUMÉRIQUE
DÉBUT
    AFFICHER("Entrez le prix du produit : ")
    LIRE(prixProduit)
    AFFICHER("Entrez la quantité achetée : ")
    LIRE(quantiteProduit)
    TANTQUE (quantiteProduit > 5) FAIRE
        AFFICHER("Vous ne pouvez pas acheter plus de 5 fois un même article.")
        AFFICHER("Réentrez la quantité achetée : ")
        LIRE(quantiteProduit)
    FINTANTQUE
    totalProduit ← prixProduit * quantiteProduit
    total ← totalProduit
    total ← total + totalProduit * TPS
    total ← total + totalProduit * TVQ
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

6

Facture - Limite d'achat (5 produits)



Farida Ait-Ammar et Nicolas Leduc

7

Facture - Limite d'achat (5 produits)

Achat de 2 produits à 10$ chacun. L'usager entre une quantité de 11, puis de 6 et enfin de 2 produits.

```
ALGORITHME CalculFacture
CONSTANTES
        TPS = 5%, TVQ = 9,975%
VARIABLES
        total, totalProduit, prixProduit,
        quantiteProduit TYPE NUMÉRIQUE
DÉBUT
1       AFFICHER("Entrez le prix du produit : ")
2       LIRE(prixProduit)
3       AFFICHER("Entrez la quantité achetée : ")
4       LIRE(quantiteProduit)
5       TANTQUE (quantiteProduit > 5) FAIRE
6               AFFICHER("Vous ne pouvez pas
                        acheter plus de 5 fois un même
                        article.")
7               AFFICHER("Réentrez la quantité
                        achetée : ")
8               LIRE(quantiteProduit)
        FINTANTQUE
9       totalProduit ← prixProduit * quantiteProduit
10      total ← totalProduit
11      total ← total + totalProduit * TPS
12      total ← total + totalProduit * TVQ
13      AFFICHER("Le total de la facture est ", total)
FIN
```

#	total	totProd	prixProd	qteProd	Terminal
1					Entrez le prix du produit :
2			10		
3					Entrez la quantité achetée :
4				11	
5					
6					Vous ne pouvez pas acheter plus de 5 fois un même article.
7					Réentrez la quantité achetée :
8				6	
5					
6					Vous ne pouvez pas acheter plus de 5 fois un même article.
7					Réentrez la quantité achetée :
8				2	
5					
9		20			
10	20				
11	21.00				
12	23.00				
13					Le total de la facture est 23.00

Farida Ait-Ammar et Nicolas Leduc

8

Facture - Limite d'achat (5 produits)

```
import java.util.Scanner;

public class FactureLimiteBoucle {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);

        float total, totalProduit, prixProduit;
        int quantiteProduit;

        System.out.println("Entrez le prix du produit : ");
        prixProduit = clavier.nextFloat();
        System.out.println("Entrez la quantité achetée : ");
        quantiteProduit = clavier.nextInt();
        while (quantiteProduit > 5){
            System.out.println("Vous ne pouvez pas acheter plus de 5 fois un même article.");
            System.out.println("Réentrez la quantité achetée : ");
            quantiteProduit = clavier.nextInt();
        }
        totalProduit = prixProduit * quantiteProduit;
        total = totalProduit;
        total = total + totalProduit * TPS;
        total = total + totalProduit * TVQ;
        System.out.println(String.format("Le total de la facture est %.2f$.", total));
    }
}
```

Condition: elle est évaluée avant chaque exécution du corps de la boucle. Si elle est vraie, le corps de la boucle est exécuté une nouvelle fois, sinon on sort de la boucle

Corps de la boucle

Farida Ait-Ammar et Nicolas Leduc

9

Affichage → Boucle `while`

- Vous devez écrire le programme correspondant à l'affichage suivant :

```
Entrez un nombre entre 1 et 10 : 15
Le nombre doit être entre 1 et 10!
Veuillez entrer un autre nombre : 0
Le nombre doit être entre 1 et 10!
Veuillez entrer un autre nombre : 25
Le nombre doit être entre 1 et 10!
Veuillez entrer un autre nombre : 7
Le nombre entré est 7
```

- On remarque que les lignes en gras se répètent, donc elles doivent se retrouver dans le corps de la boucle (à l'intérieur des accolades). Les autres lignes ne doivent pas faire partie de la boucle.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    int nombre;

    System.out.print("Entrez un nombre entre
        1 et 10 : ");
    nombre = cl.nextInt();
    while (nombre < 1 || nombre > 10) {
        System.out.println("Le nombre doit
            être entre 1 et 10!");
        System.out.print("Veuillez entrer un
            autre nombre : ");
        nombre = cl.nextInt();
    }
    System.out.println("Le nombre entré
        est " + nombre);
}
```

Farida Ait-Ammar et Nicolas Leduc

6.3 FAIRE ... TANTQUE ... (`do _ while _`)

10

FAIRE ... TANTQUE ...

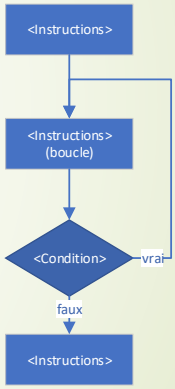
- Pseudo-code

```
FAIRE
    <Instructions>
TANTQUE (<expression booléenne>)
```

- *Java*

```
do {
    <Instructions>;
} while (<expression booléenne>);
```

- Fonctionnement
  - Le fonctionnement est similaire à celui de la boucle TANTQUE...
  - **Toutefois**, la condition n'est évaluée qu'après avoir exécuté les instructions de la boucle.
- Note:
  - Le corps de la boucle s'exécute toujours **au moins une fois.**
  - Le nombre d'exécutions de la boucle est inconnu au départ.
  - La valeur de la variable faisant partie de la condition doit être modifiée à l'intérieur de la boucle, sinon on obtient une boucle infinie.



Farida Ait-Ammar et Nicolas Leduc

11

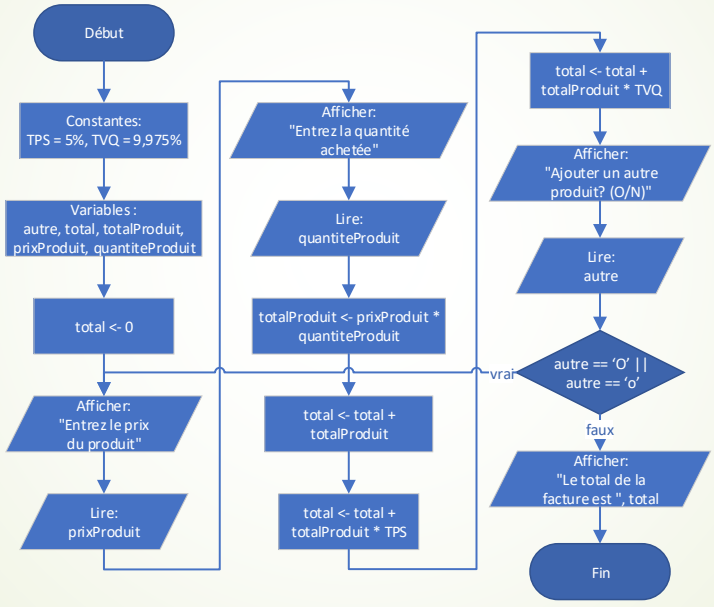
Facture – Autre produit

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%, TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit, quantiteProduit TYPE NUMÉRIQUE
    autre TYPE CARACTÈRE
DÉBUT
    total ← 0
    FAIRE
        AFFICHER("Entrez le prix du produit")
        LIRE(prixProduit)
        AFFICHER("Entrez la quantité achetée")
        LIRE(quantiteProduit)
        totalProduit ← prixProduit * quantiteProduit
        total ← total + totalProduit
        total ← total + totalProduit * TPS
        total ← total + totalProduit * TVQ
        AFFICHER("Ajouter un autre produit? (O/N)")
        LIRE(autre)
    TANTQUE (autre == 'O' || autre == 'o')
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

12

Facture – Autre produit



Farida Ait-Ammar et Nicolas Leduc

13

Facture – Autre produit

```
ALGORITHME CalculFacture
CONSTANTES
        TPS = 5%, TVQ = 9,975%
VARIABLES
        total, totalProduit, prixProduit,
        quantiteProduit TYPE NUMÉRIQUE
        autre TYPE CARACTÈRE
DÉBUT
1       total ← 0
        FAIRE
2               AFFICHER("Entrez le prix du produit")
3               LIRE(prixProduit)
4               AFFICHER("Entrez la quantité achetée")
5               LIRE(quantiteProduit)
6               totalProduit ← prixProduit *
                        quantiteProduit
7               total ← total + totalProduit
8               total ← total + totalProduit * TPS
9               total ← total + totalProduit * TVQ
10              AFFICHER("Ajouter un autre produit?
                        (O/N)")
11              LIRE(autre)
12      TANTQUE (autre == 'O' | | autre == 'o')
13      AFFICHER("Le total de la facture est ", total)
FIN
```

2 achats: 5 produits à 10$ chacun et 2 produits à 5$ chacun.

#	total	totProd	prixProd	qteProd	autre	Terminal
1	0					
2						Entrez le prix du produit
3			10			
4						Entrez la quantité achetée
5				5		
6		50				
7	50					
8	52.50					
9	57.49					
10						Ajouter un autre produit? (O/N)
11					O	
12						
2						Entrez le prix du produit
3			5			
4						Entrez la quantité achetée
5				2		
6		10				
7	67.49					
8	67.99					
9	68.98					
10						Ajouter un autre produit? (O/N)
11					N	
12						
13						Le total de la facture est 68.98

Farida Ait-Ammar et Nicolas Leduc

14

Facture – Autre produit

```
import java.util.Scanner;

public class FactureAutre {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);

        float total, totalProduit, prixProduit;
        int quantiteProduit;
        char autre;

        total = 0;
        do {
            System.out.println("Entrez le prix du produit :");
            prixProduit = clavier.nextFloat();
            System.out.println("Entrez la quantité achetée :");
            quantiteProduit = clavier.nextInt();
            totalProduit = prixProduit * quantiteProduit;
            total = total + totalProduit;
            total = total + totalProduit * TPS;
            total = total + totalProduit * TVQ;
            System.out.print("Ajouter un autre produit? (O/N)");
            autre = clavier.next().charAt(0);
        } while (autre == 'O' || autre == 'o');
        System.out.println(String.format("Le total de la facture est %.2f$.", total));
    }
}
```

Corps de la boucle. Il est exécuté au moins une fois dans la boucle `do... while`

Condition: elle est évaluée après chaque exécution du corps de la boucle. Si elle est vraie, le corps de la boucle est exécuté une nouvelle fois, sinon on sort de la boucle

Farida Ait-Ammar et Nicolas Leduc

15

Affichage → Boucle `do...while`

- Vous devez écrire le programme correspondant à l'affichage suivant :

```
Lecture de nombres (0 : quitter)
Entrez un nombre : 7
Nombre inscrit = 7
Entrez un nombre : 3
Nombre inscrit = 3
Entrez un nombre : 0
Nombre inscrit = 0
Fin du programme
```

- On remarque que les lignes en gras se répètent, donc elles doivent se retrouver dans le corps de la boucle (à l'intérieur des accolades). Les autres lignes ne doivent pas faire partie de la boucle.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    int nombre;

    System.out.println("Lecture de
        nombres (0 : quitter)");
    do {
        System.out.print("Entrez un
            nombre : ");
        nombre = cl.nextInt();
        System.out.println("Nombre
            inscrit = " + nombre);
    } while (nombre != 0);
    System.out.println("Fin du
        programme");
}
```

Farida Ait-Ammar et Nicolas Leduc

6.4 Opérateur : complément

16

Opérateurs d'affectation et d'incrémentation

Opérateur	Exemple	Équivalence	Commentaire
=	x = 1	x = 1	Affectation: Place la valeur 1 dans la zone mémoire x
+=	x += 3	x = x + (3)	Attention: Les opérateurs d'affectation sont exécutés en dernier dans l'évaluation d'une expression et la priorité des opérations peut sembler ne pas être respectée. La transformation suivante est appliquée (notez les ()): <var> <op>= <expression> <var> = <var> <op> (<expression>)
-=	x -= 5	x = x - (5)	
*=	x *= 7	x = x * (7)	
/=	x /= 9	x = x / (9)	
%=	x %= 11	x = x % (11)	
++	x++	x = x + 1	Post incrément: effectué après l'évaluation de la variable
	++x	x = x + 1	Pré incrément: effectué avant l'évaluation de la variable
--	x--	x = x - 1	Post décrément: effectué après l'évaluation de la variable
	--x	x = x - 1	Pré décrément: effectué avant l'évaluation de la variable

Farida Ait-Ammar et Nicolas Leduc

17

Priorité des opérations

- La **priorité des opérations** précise **l'ordre** dans lequel les **calculs sont effectués** dans une expression complexe.
- Les priorités sont (du + au – prioritaire):
  - Appel de fonction
  - Parenthèses ( )
  - Opérateur –(unaire), ++, --
  - Multiplication, division, modulo (*, /, %)
  - Addition, soustraction (+ , –)
  - Affectation (= , += , -= , *= , /= , %=)
- Lorsqu'une expression contient plusieurs opérateurs de **même priorité**, les calculs s'effectuent de **gauche à droite**.

Farida Ait-Ammar et Nicolas Leduc

6.5 POUR ... FAIRE ... (`for _`)

18

POUR ... FAIRE ...

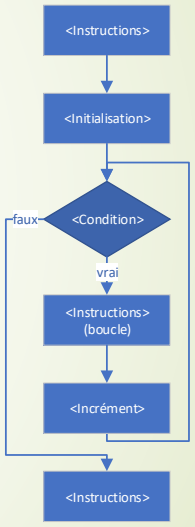
- Pseudo-code

```
POUR <Variable> ALLANT DE <Début> À <Fin> FAIRE
    <Instructions>
FINPOUR
```

- *Java*

```
for (<Initialisation>; <Condition>; <Incrément>) {
    <Instructions>;
}
```

- Fonctionnement
  - La boucle `for` est utilisée lorsque l'on connaît à l'avance le nombre de fois (itérations) qu'un bloc d'instructions doit être exécuté.
  - `<Initialisation>`
    - Permet d'initialiser la variable représentant le compteur de la boucle. **Exécutée une seule fois** avant d'entrer dans la boucle.
  - `<Condition>`
    - Définit la condition à vérifier pour continuer à exécuter la boucle. Elle **est examinée avant chaque tour de boucle**, y compris au premier tour de boucle. Si elle est vraie, le corps de la boucle est exécuté une nouvelle fois, sinon on sort de la boucle.
  - `<Incrément>`
    - Instruction **exécutée à la fin de chaque tour de boucle**. Permet de changer la valeur du compteur de boucle.
- **Attention: vous ne devez pas modifier la valeur du compteur à l'intérieur de la boucle.**



Farida Ait-Ammar et Nicolas Leduc

19

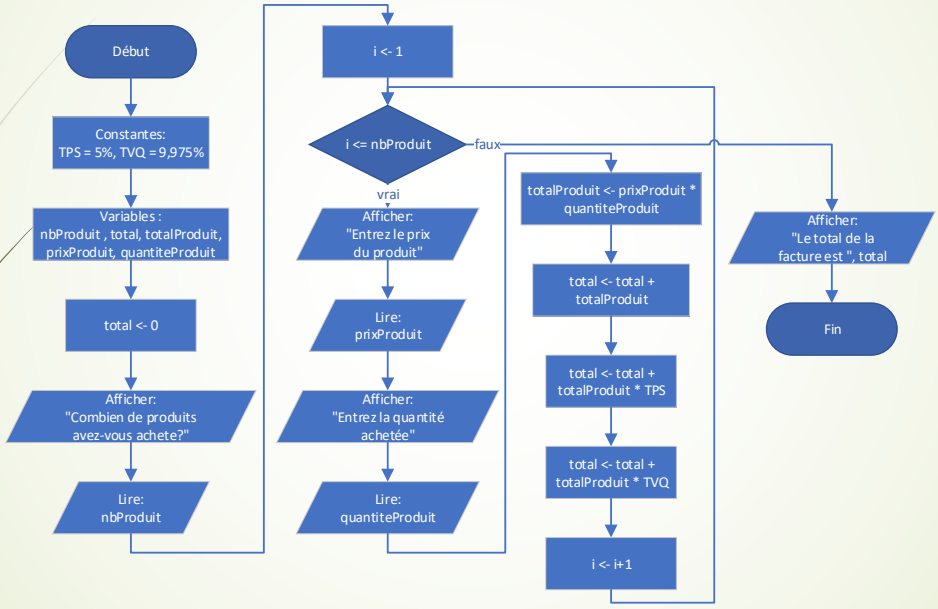
Facture – Achat de *n* produits

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%, TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit,
    quantiteProduit, nbProduit TYPE NUMÉRIQUE
DÉBUT
    total ← 0
    AFFICHER("Combien de produits avez-vous acheté?")
    LIRE(nbProduit)
    POUR i ALLANT DE 1 À nbProduit FAIRE
        AFFICHER("Entrez le prix du produit")
        LIRE(prixProduit)
        AFFICHER("Entrez la quantité achetée")
        LIRE(quantiteProduit)
        totalProduit ← prixProduit * quantiteProduit
        total ← total + totalProduit
        total ← total + totalProduit * TPS
        total ← total + totalProduit * TVQ
    FINPOUR
    AFFICHER("Le total de la facture est ", total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

20

Facture – Achat de *n* produits



Farida Ait-Ammar et Nicolas Leduc

21

Facture – Achat de *n* produits

2 achats: 5 produits à 10$ chacun et
2 produits à 5$ chacun.

```
ALGORITHME CalculFacture
CONSTANTES
        TPS = 5%, TVQ = 9,975%
VARIABLES
        total, totalProduit, prixProduit,
        quantiteProduit, nbProduit TYPE NUMÉRIQUE
DÉBUT
1       total ← 0
2       AFFICHER("Combien de produits avez-vous
                acheté?")
3       LIRE(nbProduit)
4       POUR i ALLANT DE 1 À nbProduit FAIRE
5               AFFICHER("Entrez le prix du produit")
6               LIRE(prixProduit)
7               AFFICHER("Entrez la quantité achetée")
8               LIRE(quantiteProduit)
9               totalProduit ← prixProduit *
                        quantiteProduit
10              total ← total + totalProduit
11              total ← total + totalProduit * TPS
12              total ← total + totalProduit * TVQ
        FINPOUR
13      AFFICHER("Le total de la facture est ", total)
FIN
```

#	total	totProd	prixProd	qteProd	nbProd	i	Terminal
1	0						
2							Combien de produits avez-vous acheté?
3					2		
4						1	
5							Entrez le prix du produit
6			10				
7							Entrez la quantité achetée
8				5			
9		50					
10	50						
11	52.50						
12	57.49						
4						2	
5							Entrez le prix du produit
6			5				
7							Entrez la quantité achetée
8				2			
9		10					
10	67.49						
11	67.99						
12	68.98						
4						3	
13							Le total de la facture est 68.98

22

Facture – Achat de *n* produits

```
import java.util.Scanner;

public class FactureNbProduits {
    public static final float TPS = 0.05f, TVQ = 0.09975f;

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);

        float total, totalProduit, prixProduit;
        int quantiteProduit, nbProduit;

        total = 0;
        System.out.print("Combien de produits avez-vous acheté?");
        nbProduit = clavier.nextInt();
        for (int i = 0; i < nbProduit; ++i) {
            System.out.println("Entrez le prix du produit :");
            prixProduit = clavier.nextFloat();
            System.out.println("Entrez la quantité achetée :");
            quantiteProduit = clavier.nextInt();
            totalProduit = prixProduit * quantiteProduit;
            total = total + totalProduit;
            total = total + totalProduit * TPS;
            total = total + totalProduit * TVQ;
        }
        System.out.println(String.format("Le total de la facture est %.2f$.", total));
    }
}
```

Corps de la boucle

23

Affichage → Boucle `for`

- Vous devez écrire le programme correspondant à l'affichage suivant :

```
Vous devez entrer 3 nombres.
Nombre #1 ? 4
Nombre entré : 4
Nombre #2 ? 8
Nombre entré : 8
Nombre #3 ? 2
Nombre entré : 2
Merci!
```

- On remarque que les lignes en gras se répètent, donc elles doivent se retrouver dans le corps de la boucle (à l'intérieur des accolades). Les autres lignes ne doivent pas faire partie de la boucle.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    int nombre;

    System.out.println("Vous devez entrer
        3 nombres.");
    for (int i = 1; i <= 3; ++i) {
        System.out.print("Nombre #" + i
            + " ? ");
        nombre = cl.nextInt();
        System.out.println("Nombre entré : "
            + nombre);
    }
    System.out.println("Merci!");
}
```

Farida Ait-Ammar et Nicolas Leduc

24

Dérouler une boucle `for`

- Boucle `for`

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    int nombre;

    System.out.println("Vous devez entrer
        3 nombres.");
    for (int i = 1; i <= 3; ++i) {
        System.out.print("Nombre #" + i + " ? ");
        nombre = cl.nextInt();
        System.out.println("Nombre entré : "
            + nombre);
    }
    System.out.println("Merci!");
}
```

- Code équivalent

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in)
    int nombre;

    System.out.println("Vous devez entrer
        3 nombres.");
    System.out.print("Nombre #1 ? ");
    nombre = cl.nextInt();
    System.out.println("Nombre entré : " + nombre);
    System.out.print("Nombre #2 ? ");
    nombre = cl.nextInt();
    System.out.println("Nombre entré : " + nombre);
    System.out.print("Nombre #3 ? ");
    nombre = cl.nextInt();
    System.out.println("Nombre entré : " + nombre);
    System.out.println("Merci!");
}
```

Farida Ait-Ammar et Nicolas Leduc

6.6 Imbrication, équivalence et choix de boucle

25

Boucles imbriquées

- Comme pour les décisions, les blocs d'instructions contenus dans les structures `while`, `do … while` et `for` peuvent contenir des instructions simples, mais aussi d'autres instructions de répétition ou de décision.
- Dans ce cas, il faut bien respecter l'indentation pour simplifier la compréhension du programme ou de l'algorithme (*ALT+MAJ+F*). Il faut aussi s'assurer que les accolades délimitent correctement les blocs de code à répéter.
- Le nombre de niveaux d'indentation n'est pas limité, mais il est recommandé d'utiliser des appels de méthodes afin de réduire le nombre de niveaux dans la méthode appelante pour ainsi faciliter la compréhension du code et le débogage.

Farida Ait-Ammar et Nicolas Leduc

26

Facture – Boucles imbriquées

```
ALGORITHME CalculFacture
CONSTANTES
    TPS = 5%, TVQ = 9,975%
VARIABLES
    total, totalProduit, prixProduit,
    quantiteProduit, nbProduit TYPE NUMÉRIQUE
    taxable TYPE CARACTÈRE
DÉBUT
    total ← 0
    AFFICHER("Combien de produits avez-vous
        acheté?")
    LIRE(nbProduit)
    POUR i ALLANT DE 1 À nbProduit FAIRE
        AFFICHER("Entrez le prix du produit")
        LIRE(prixProduit)
        AFFICHER("Entrez la quantité achetée")
        LIRE(quantiteProduit)
        TANTQUE (quantiteProduit > 5) FAIRE
            AFFICHER("Vous ne pouvez pas
                acheter plus de 5 fois un
                même article.")
            AFFICHER("Entrez la quantité
                achetée :")
            LIRE(quantiteProduit)
        FINTANTQUE
        AFFICHER("Produit taxable? (F = TPS,
            P = TVQ, T = Toutes, A = Aucune)")
        LIRE(taxable)
        totalProduit ← prixProduit *
            quantiteProduit
        SI (totalProduit > 1000) ALORS
            totalProduit ← totalProduit -
                totalProduit * 20 / 100
        SINON SI (totalProduit > 100) ALORS
            totalProduit ← totalProduit -
                totalProduit * 10 / 100
        SINON
            totalProduit ← totalProduit -
                totalProduit * 5 / 100
        FINSI
        total ← total + totalProduit
        SI (taxable=='F'||taxable=='T') ALORS
            total ← total + totalProduit * TPS
        FINSI
        SI(taxable=='P'||taxable=='T') ALORS
            total ← total + totalProduit * TVQ
        FINSI
    FINPOUR
    AFFICHER("Le total de la facture est ",
        total)
FIN
```

Farida Ait-Ammar et Nicolas Leduc

27

Facture – Boucles imbriquées

```
import java.util.Scanner;
public class FactureImbriquee {
      public static final float TPS = 0.05f;
      public static final float TVQ = 0.09975f;

      public static void main(String[] args) {
            Scanner clavier = new Scanner(System.in);
            float total, totalProduit, prixProduit;
            int quantiteProduit, nbProduit;
            char taxable;

            total = 0;
            System.out.print("Combien de produits avez-
                  vous acheté?");
            nbProduit = clavier.nextInt();
            for (int i = 0; i < nbProduit; ++i) {
                  System.out.println("Entrez le prix du
                        produit :");
                  prixProduit = clavier.nextFloat();
                  System.out.println("Entrez la quantité
                        achetée :");
                  quantiteProduit = clavier.nextInt();
                  while (quantiteProduit > 5) {
                        System.out.println("Vous ne pouvez
                              pas acheter plus de 5 fois un
                              même article.");
                        System.out.println("Entrez la
                              quantite achetee :");
                        quantiteProduit = clavier.nextInt();
                  }
                  System.out.println("Produit taxable? (F = TPS,
                        P = TVQ, T = Toutes, A = Aucune)");
                  taxable = clavier.next().toUpperCase().
                        charAt(0);
                  totalProduit = prixProduit * quantiteProduit;
                  if (totalProduit > 1000) {
                        totalProduit = totalProduit -
                              totalProduit * 20 / 100;
                  } else if (totalProduit > 100) {
                        totalProduit = totalProduit -
                              totalProduit * 10 / 100;
                  } else {
                        totalProduit = totalProduit -
                              totalProduit * 5 / 100;
                  }
                  total = total + totalProduit;
                  if (taxable == 'F' || taxable == 'T') {
                        total = total + totalProduit * TPS;
                  }
                  if (taxable == 'P' || taxable == 'T') {
                        total = total + totalProduit * TVQ;
                  }
            }
            System.out.println(String.format("Le total de la
                  facture est %.2f$.", total));
      }
}
```


28

Équivalence de boucles

Boucle **`for`**

- Code *Java*

```
<Instructions>;
for (<Initialisation>; <Condition>; <Incrément>) {
      <Instructions>; // boucle
}
<Instructions>;
```

- Exemple

```
int aleatoire;

System.out.println("Vos nombres chanceux sont");

for (int i = 1; i <= 6; i++) {

      // génère un nombre aléatoire entre 1 et 49
      aleatoire = Outils.Aleatoire.nextInt(49)+1;
      System.out.print(aleatoire + " ");

}

System.out.println("\nBonne chance!");
```

Boucle **`while`**

- Code *Java*

```
<Instructions>;
<Initialisation>;
while (<Condition>) {
      <Instructions>; // boucle
      <Incrément>;
}
<Instructions>;
```

- Exemple

```
int aleatoire;

System.out.println("Vos nombres chanceux sont");

int i = 1;
while (i <= 6) {
      // génère un nombre aléatoire entre 1 et 49
      aleatoire = Outils.Aleatoire.nextInt(49)+1;
      System.out.print(aleatoire + " ");
      i++;
}

System.out.println("\nBonne chance!");
```

29

Choix du type de boucle

Type	Situations
`while`	• Validation des données et corrections par l'usager • Traitement des touches du clavier disponibles ou non • Parcours d'une liste d'éléments de taille inconnue
`do … while`	• Demande d'éléments supplémentaires • Gestion d'un menu
`for`	• Traitement d'un tableau de taille connue • Nombre d'éléments à traiter connu à l'avance • Affichage de graphiques ou de tableaux

Farida Ait-Ammar et Nicolas Leduc

7. Tableau unidimensionnel

7.1 Déclaration et initialisation

2

Introduction

- Dans certaines situations, on manipule un ensemble de données similaires, qui ont toutes le même type:
  - Liste de notes des étudiants
  - Liste de prix pour les articles d'un magasin
  - Valeur en y pour différentes valeurs de x d'une fonction mathématique
- Un tableau permet de manipuler un ensemble de données, qui ont toutes le même type, comme un tout.
  - Exemple: si l'on veut traiter un ensemble de dix prix, on pourrait créer une seule variable de type tableau de `float` au lieu de créer dix variables de type `float`: (`prix1`, `prix2`, ..., `prix10`).

10 variables

```
float prix1,
  prix2, prix3,
  prix4, prix5,
  prix6, prix7,
  prix8, prix9,
  prix10;
```

1 tableau

```
float[] prix;
```

60.23
54.89
78.00
10.50
78.23
78.00
10.50
78.00
10.50
78.00

Farida Ait-Ammar et Nicolas Leduc

3

Déclaration

- Comme toute variable, un tableau doit être déclaré.
- Un tableau se déclare comme n'importe quelle variable. On doit cependant **ajouter une paire de crochets `[]` entre le type et le nom de la variable** pour indiquer au compilateur que la variable représente un tableau.
- Syntaxe de déclaration d'un tableau:

```
TypeTableau[] nomTableau;
```

- Exemples:

```
float[] prix;
String[] nomProduits;
```

- Note: *Java* autorise aussi de placer les crochets après le nom de la variable.

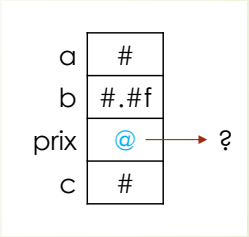
Farida Ait-Ammar et Nicolas Leduc

4

Références

- Les tableaux, tout comme le type `String`, sont des types évolués, qui se manipulent différemment des types simples.
- Une variable de type évolué ne stocke pas directement sa valeur, mais une référence (adresse mémoire) vers l'endroit où est stockée sa vraie valeur.
- Quand le compilateur rencontre la déclaration d'un tableau comme `float[] prix`, il réserve un espace mémoire portant le nom du tableau (ici `prix`), qui contiendra l'adresse de la première case du tableau une fois initialisé.

```
int a;
float b;
float[] prix;
int c;
```



Farida Ait-Ammar et Nicolas Leduc

5

Initialisation

- Il existe deux façons d'initialiser un tableau en *Java*:
  - Sans définir ses éléments:

```
float[] prix;                  // déclaration
prix = new float[10];          // initialisation
                        OU
float[] prix = new float[10]; // déclaration et initialisation
```

L'opérateur `new` réserve le nombre de cases mémoires consécutives indiqué entre crochets. De plus, il initialise le tableau selon le type:

```
byte, short, int, long            -> 0
float, double                     -> 0.0
boolean                           -> false
char                              -> '\u0000'
objet quelconque (incluant String) -> null
```

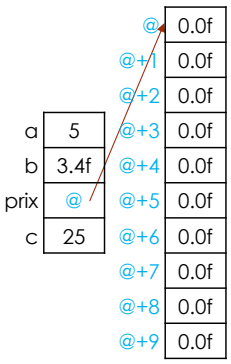
  - En donnant ses éléments lors de la déclaration:

```
float[] prix = {2.5f, 1.25f, 0.50f, 3.0f};
String[] nomProduits = {"Patate", "Carotte", "Navet", "Pomme"};
```

Les syntaxes suivantes sont aussi valides:

```
float[] prix = new float[] {2.5f, 1.25f, 0.50f, 3.0f};
String[] nomProduits;
nomProduits = new String[] {"Patate", "Carotte", "Navet", "Pomme"};
```

```
int a = 5;
float b = 3.4f;
float[] prix;
prix = new float[10];
int c = 25;
```



Farida Ait-Ammar et Nicolas Leduc

6

Initialisation différée

- Une fois que l'opérateur `new` est exécuté, la taille du tableau est fixée et on ne peut plus la modifier.
- Il est cependant possible de spécifier la taille du tableau à l'aide d'une variable au lieu d'une valeur littérale, ce qui permet de créer un tableau selon les besoins de l'usager. Exemple:

```
Scanner clavier = new Scanner(System.in);
float[] prix;
int nbProduits;
System.out.print("Nombre de produits ? ");
nbProduits = clavier.nextInt();   // Lecture de la taille du tableau
prix = new float[nbProduits];     // Réservation de l'espace mémoire
nbProduits = 25;                  // Aucune influence sur la taille du tableau
```

- Dans l'exemple précédent, on lit la taille du tableau avant de réserver les espaces mémoire par l'opérateur `new`. Ainsi, la taille du tableau varie à chaque exécution. Elle est toutefois fixée lors de l'exécution du `new` et toute modification subséquente de la variable utilisée pour spécifier la taille n'a aucune influence sur le tableau.

Farida Ait-Ammar et Nicolas Leduc

7.2 Accès aux éléments

7

Taille et bornes

- Un **tableau est un objet** caractérisé par des **attributs** (données) et des **méthodes**. L'attribut **`length`** nous donne la **taille du tableau**.
- Pour connaître la taille d'un tableau, il suffit de placer le terme **`.length`** après le nom du tableau.

```
float[] prix = {2.5f, 1.25f, 0.50f, 3.0f};
System.out.print(prix.length);  // Affiche 4
```

- Pour un tableau de *n* éléments, les indices varient de *0* à *n-1*. Par exemple, les indices du tableau `prix`, qui contient *4* éléments, vont de *0* à *3*.

Farida Ait-Ammar et Nicolas Leduc

8

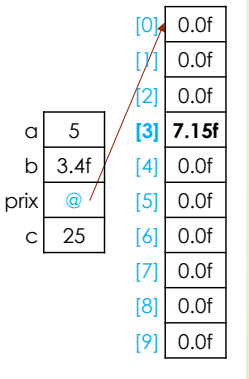
Accès direct aux éléments

- Chaque case d'un tableau est identifiée par un indice(numéro), qui correspond au décalage par rapport à la première case.
  - L'indice de la première case est *0* (et non 1).
  - L'indice de la deuxième case est *1*, ainsi de suite
  - L'indice de la dernière case est *n-1* pour *n* éléments
- Pour stocker la valeur *7,15* dans la quatrième case:

```
prix[3] = 7.15f; // Indice de la 4e case -> 3
```

- Si on tente d'accéder à un élément avec un indice hors des limites du tableau, la machine *Java* génère une exception de type *ArrayIndexOutOfBounds* et le programme s'arrête.

```
int a = 5;
float b = 3.4f;
float[] prix;
prix = new float[10];
int c = 25;
```



Farida Ait-Ammar et Nicolas Leduc

9

Accès séquentiel aux éléments

- Les **éléments** d'un tableau sont **ordonnés** grâce à leurs **indices**. On peut donc les manipuler avec des **instructions itératives (boucles)**.

- Exemple: Lire dix prix et les stocker dans un tableau.

```
public static final int NB_PRIX = 10;
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    float[] prix = new float[NB_PRIX];
    for (int i = 0; i < prix.length; ++i) {
        System.out.print("Prix #" + (i+1) + " ? ");
        prix[i] = clavier.nextFloat();
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

10

Accès séquentiel aux éléments

- Exemple: Afficher cinq prix stockés dans un tableau.

```
public static void main(String[] args) {
    float[] prix = {2.5f, 3.2f, 1.1f, 8.6f, 1.5f};
    for (int i = 0; i < prix.length; ++i) {
        System.out.println(String.format("Prix #%d : %.2f", i+1, prix[i]));
    }
}
```

- *Java* propose une **syntaxe alternative** pour la boucle **`for`** :

```
for (float unPrix : prix) {
    System.out.println(unPrix);
}
```

  - **Fonctionnement**: Le tableau est parcouru du premier élément jusqu'au dernier et la valeur de chaque case est enregistrée tour à tour dans la variable `unPrix`. Ainsi, la gestion du compteur est transparente.
  - **Désavantages**: **Ne permet pas** de modifier le contenu des cases, d'itérer sur plus d'un tableau, d'accéder à l'élément suivant ou précédent, de parcourir le tableau en ordre inverse.

Farida Ait-Ammar et Nicolas Leduc

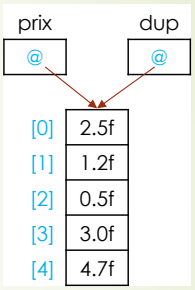
7.3 Copie et calcul

11

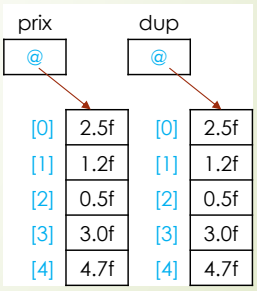
Copie de tableaux

- Pour copier un tableau, il ne suffit pas d'utiliser l'opérateur = :

```
float[] prix = {2.5f, 1.2f, 0.5f, 3.0f, 4.7f};
float[] dup = new float[5];      // Inutile dans ce cas
dup = prix;  // Copie la référence et non les éléments
```



- Il existe deux principales méthodes pour copier les éléments:
  - Méthode `clone()`
    - Copie en surface: fait une copie des types simples et des références aux objets uniquement. Vous n'aurez pas de véritable copie de vos objets.
  - Boucle `for`
    - Permet de contrôler la copie afin de dupliquer correctement les objets.



Farida Ait-Ammar et Nicolas Leduc

12

Copie de tableaux - Exemple

```
public static void main(String[] args) {
    String[] nomProd = {"Ananas","Brocoli","Carotte","Navet","Patate",
                        "Poire","Pomme","Raisin","Tomate"};
    String[] nomCopie = new String[nomProd.length];

    for (int i = 0; i < nomProd.length; ++i) {
        nomCopie[i] = nomProd[i];  // Copie de chaque élément
    }

//  nomCopie = nomProd.clone();    // Utilisation de la méthode clone()

    nomCopie[3] = "Banane";        // N'affecte pas le tableau nomProd
    for (int i = 0; i < nomProd.length; ++i) {
        System.out.println(nomProd[i] + " : " + nomCopie[i]);
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

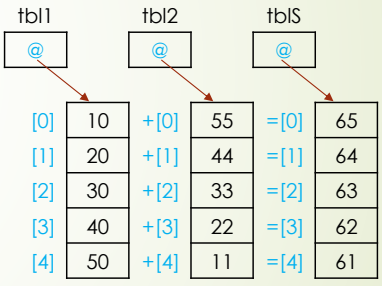
13

Opérations arithmétiques Plusieurs tableaux

- Les opérations arithmétiques directes sur les tableaux ne sont pas possibles.
- Exemple: addition de deux tableaux:

```
int[] tbl1 = {10, 20, 30, 40, 50};
int[] tbl2 = {55, 44, 33, 22, 11};
int[] tblS = new int[tbl1.length];

tblS = tbl1 + tbl2; // ERREUR : @ + @ = ???
```



- Il faut additionner deux (ou plusieurs) tableaux élément par élément:

```
for (int i = 0; i < tblS.length; i++) {
    tblS[i] = tbl1[i] + tbl2[i]; // CORRECT
}
```

Farida Ait-Ammar et Nicolas Leduc

7.4 Algorithmes : tableau non ordonné

7.4.1 Somme et moyenne

14

Calculs de sommes et de moyennes

- En utilisant un tableau, calculez la somme et la moyenne des neuf prix suivants:
  1.9, 2.8, 3.7, 4.6, 5.5, 6.4, 7.3, 8.2, 9.1

```
public static void main(String[] args) {
    float[] prix = {1.9f, 2.8f, 3.7f, 4.6f, 5.5f, 6.4f, 7.3f, 8.2f, 9.1f};
    float somme = 0;
    float moyenne;
    for (int i = 0; i < prix.length; ++i) {
        somme += prix[i];
    }
    moyenne = somme / prix.length;
    System.out.println("Somme = " + somme);
    System.out.println("Moyenne = " + moyenne);
}

// Solution alternative
public static void main(String[] args) {
    float[] prix = {1.9f, 2.8f, 3.7f, 4.6f, 5.5f, 6.4f, 7.3f, 8.2f, 9.1f};
    float somme = 0;
    float moyenne;
    for (float unPrix : prix) {
        somme += unPrix;
    }
    moyenne = somme / prix.length;
    System.out.println("Somme = " + somme);
    System.out.println("Moyenne = " + moyenne);
}
```

Farida Ait-Ammar et Nicolas Leduc

7.4.2 Recherche d'occurrence

15

Recherche séquentielle Première occurrence

- Votre employeur vous a fourni une liste de produits ainsi que la quantité achetée par un client, entre parenthèses:
  - Patate(3), Carotte(5), Navet(1), Brocoli(2), Tomate(4), Pomme(5), Poire(5), Raisin(2), Ananas(1)
- Il vous demande de trouver et d'afficher le nom du premier produit pour lequel la quantité achetée est égale à deux.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    String[] nomProd = {"Patate", "Carotte", "Navet", "Brocoli",
                        "Tomate", "Pomme", "Poire", "Raisin", "Ananas"};
    int[] quant = {3, 5, 1, 2, 4, 5, 5, 2, 1};
    int valeurCherchee;
    System.out.print("Quelle est la quantité recherchée? ");
    valeurCherchee = clavier.nextInt();
    int pos = 0;
    while (pos < quant.length &&                // Tant qu'on a pas dépassé la taille du tableau et que
           quant[pos] != valeurCherchee) {      // la case actuelle ne contient pas la valeur cherchée
        ++pos;                                  // On passe à la case suivante
    }
    if (pos < quant.length) {                   // La valeur a été trouvée, car pos est dans le tableau
        System.out.println("Produit trouvé : " + nomProd[pos]);
    } else {                                    // La valeur n'a pas été trouvée
        System.out.println("Aucun produit trouvé");
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

16

Recherche séquentielle Ensemble des occurrences

- Votre employeur vous a fourni une liste de produits ainsi que la quantité achetée par un client, entre parenthèses:
  - Patate(3), Carotte(5), Navet(1), Brocoli(2), Tomate(4), Pomme(5), Poire(5), Raisin(2), Ananas(1)
- Il vous demande de trouver et d'afficher le nom de tous les produits dont la quantité achetée est d'au moins de trois.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    String[] nomProd = {"Patate", "Carotte", "Navet", "Brocoli",
                        "Tomate", "Pomme", "Poire", "Raisin", "Ananas"};
    int[] quant = {3, 5, 1, 2, 4, 5, 5, 2, 1};
    int nbTrouve = 0;
    int valeurCherchee;
    System.out.print("Quelle est la quantité recherchée? ");
    valeurCherchee = clavier.nextInt();
    for (int i = 0; i < quant.length; ++i) {
        if (quant[i] >= valeurCherchee) {
            System.out.println("Produit : " + nomProd[i] + "(" + quant[i] + ")");
            nbTrouve++;
        }
    }
    if (nbTrouve > 0) {
        System.out.println(nbTrouve + " produit(s) trouvé(s)");
    } else {
        System.out.println("Aucun produit trouvé");
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

7.4.3 Minimum et maximum

17

Recherche du minimum et du maximum

- Votre employeur vous a fourni une liste de produits ainsi que la quantité achetée par un client, entre parenthèses:
  - Patate(3), Carotte(5), Navet(1), Brocoli(2), Tomate(4), Pomme(5), Poire(5), Raisin(2), Ananas(1)
- Il vous demande d'afficher la quantité minimale et maximale achetée ainsi que le nom du premier produit correspondant à chacune de ces quantités.

```
public static void main(String[] args) {
     String[] nomProd = {"Patate", "Carotte", "Navet", "Brocoli",
                         "Tomate", "Pomme", "Poire", "Raisin", "Ananas"};
     int[] quant = {3, 5, 1, 2, 4, 5, 5, 2, 1};
     int posMin = 0;                                  // Au départ, la 1ière valeur est le min
     int posMax = 0;                                  // Au départ, la 1ière valeur est le max
     for (int i = 1; i < quant.length; ++i) {         // On commence par la 2e valeur (i=1)
          if (quant[i] < quant[posMin]) {             // Si on trouve une valeur plus petite
               posMin = i;                            // Mise à jour de la position du min
          }
          if (quant[i] > quant[posMax]) {             // Si on trouve une valeur plus grande
               posMax = i;                            // Mise à jour de la position du max
          }
     }
     System.out.println("La quantité minimale achetée est de " + quant[posMin]);
     System.out.println("et elle a été observée pour le produit : " + nomProd[posMin]);
     System.out.println("La quantité maximale achetée est de " + quant[posMax]);
     System.out.println("et elle a été observée pour le produit : " + nomProd[posMax]);
}
```

Farida Ait-Ammar et Nicolas Leduc

18

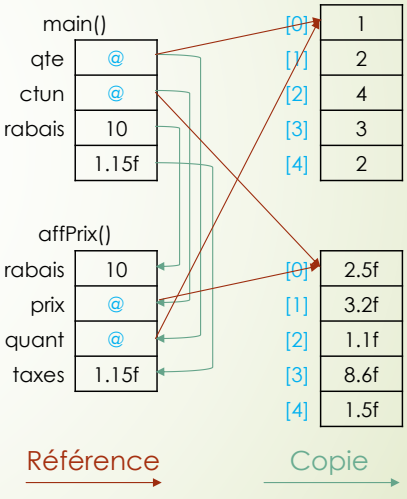
Tableaux et méthodes Paramètre

- La spécification d'un paramètre de type tableau se fait sans préciser sa taille, avec des crochets vides `[]` . On peut obtenir la taille du tableau à l'aide de l'attribut `length`.

```
public static void affPrix(int rabais, float[] prix,
                           int[] quant, float taxes) {
     for (int i = 0; i < prix.length; ++i) {
          System.out.println("Article #" + (i+1) + " : " +
               prix[i]*quant[i]*(1-rabais/100.0f)*taxes);
     }
}

public static void main(String[] args) {
     int[] qte = {1, 2, 4, 3, 2};
     float[] ctun = {2.5f, 3.2f, 1.1f, 8.6f, 1.5f};
     int rabais = 10;
     affPrix(rabais, ctun, qte, 1.15f);
}
```

- Lorsque l'on transmet un nom de tableau (sans `[]`) en argument d'une méthode, on transmet une **copie de la référence** au tableau. Ainsi, cette copie pointe toujours vers le même objet (les mêmes cases du tableau). La méthode agit alors directement sur l'objet tableau et non pas sur une copie.



Farida Ait-Ammar et Nicolas Leduc

7.4.4 Éléments distincts, ajout et retrait

19

Initialisation - Éléments tous différents

- Votre employeur désire avoir un logiciel pour lui permettre de faire son inventaire. Il veut inscrire le nom de chaque produit et sa quantité dans des tableaux séparés. Il a un maximum de quinze produits différents. Vous devez vous assurer qu'il n'inscrit pas deux fois le même produit.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    String[] nomProd = new String[15];
    // {"Navet","Ananas","Brocoli","Raisin","Patate","Tomate","Poire","Pomme","","","","","","","","};
    int[] quant = new int[15]; // {1,1,2,2,3,4,5,5,-1,-1,-1,-1,-1,-1,-1};
    int nbProd = 0; // 8;
    String nouveauProd;
    int nouvelleQte;
    for (int posProchain = nbProd; posProchain < nomProd.length; posProchain++) {
        System.out.print("Entrez le nom d'un produit : ");
        nouveauProd = clavier.next();
        while (trouve(nouveauProd, nomProd) != -1) {
            System.out.println("Produit déjà existant");
            System.out.print("Entrez nom produit : ");
            nouveauProd = clavier.next();
        }
        System.out.print("Entrez quantité inventaire : ");
        nouvelleQte = clavier.nextInt();
        nomProd[posProchain] = nouveauProd;
        quant[posProchain] = nouvelleQte;
        nbProd++;
    }
    System.out.println("Merci");
}
```

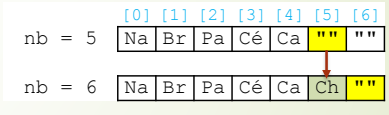
```
// Recherche première occurrence (p15)
public static int trouve(
String strCherche, String[] strTable) {
    int pos = 0;
    while (pos < strTable.length &&
    (strTable[pos] == null ||
    !strTable[pos].equals(strCherche))) {
        ++pos;
    }
    if (pos == strTable.length) {
        pos = -1; // Valeur non trouvée
    }
    return pos;
}
```


20

Ajout – Tableau non ordonné

- Votre employeur désire avoir un logiciel pour lui permettre de faire son inventaire. Il veut inscrire le nom de chaque produit et sa quantité dans des tableaux séparés. Il a un maximum de sept produits différents.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    String[] nomProd = {"Navet","Brocoli","Patate","Céleri","Carotte","",""};
    int[] quant = {1,2,3,4,5,-1,-1}; // -1 ou "" indique une case vide dans ce cas
    int nbItems = 5;                 // Nombre de cases utilisées
    String nomItem;
    int qteItem;
    System.out.print("Indiquez l'item à ajouter : ");
    nomItem = clavier.next();
    System.out.print("Indiquez la quantité : ");
    qteItem = clavier.nextInt();
    if (nbItems < nomProd.length) { // S'il reste de la place
        nomProd[nbItems] = nomItem; // Ajouter l'item dans la
        quant[nbItems] = qteItem;   // première case vide
        nbItems++;
    }
    for (int i = 0; i < nomProd.length; ++i) {
        System.out.println(nomProd[i] + " : " + quant[i]);
    }
}
```

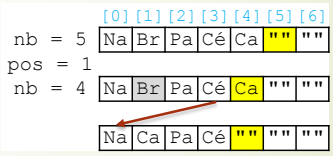


21

Retrait – Tableau non ordonné

- Votre employeur désire avoir un logiciel pour lui permettre de faire son inventaire. Il veut inscrire le nom de chaque produit et sa quantité dans des tableaux séparés. **De plus, il veut être en mesure de supprimer des produits.** Il a un maximum de sept produits différents.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    String[] nomProd = {"Navet","Brocoli","Patate","Céleri","Carotte","",""};
    int[] quant = {1,2,3,4,5,-1,-1};          // -1 ou "" indique une case vide dans ce cas
    int nbItems = 5;                          // Nombre de cases utilisées
    String nomItem;
    int posItem;
    System.out.print("Indiquez l'item à retirer : ");
    nomItem = clavier.next();
    posItem = trouve(nomItem, nomProd);       // Voir p19
    if (posItem != -1) {  // Si l'item existe dans le tableau
        nbItems--;        // Libérer la dernière case du tableau
        nomProd[posItem] = nomProd[nbItems]; // en copiant le
        nomProd[nbItems] = ""; // dernier item à la place de
        quant[posItem] = quant[nbItems];      // l'item à enlever
        quant[nbItems] = -1;   // et mettre -1 ou "" à la place
    }
    for (int i = 0; i < nomProd.length; ++i) {
        System.out.println(nomProd[i] + " : " + quant[i]);
    }
}
```



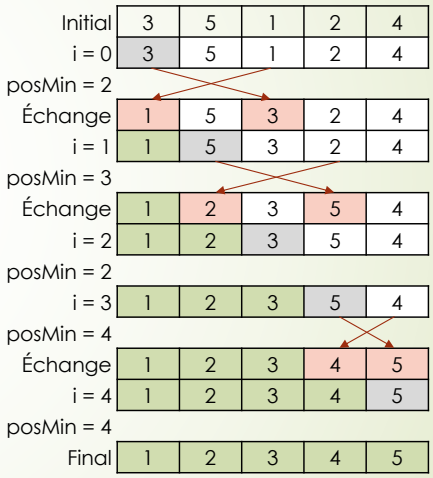
Farida Ait-Ammar et Nicolas Leduc

7.5 Tri par échange

22

Tri par échange d'un tableau Algorithme

- Votre employeur vous a fourni une liste de produits ainsi que la quantité achetée par un client, entre parenthèses:
  - Patate(3), Carotte(5), Navet(1), Brocoli(2), Tomate(4)
- Il vous demande de trier les quantités achetées en ordre croissant.
- Stratégie de résolution:
  - POUR chaque case du tableau (i de 0 à n-1) FAIRE
    - Rechercher la valeur minimale en partant de la case courante (i) jusqu'à la fin du tableau (n-1)
    - Échanger la valeur de la case courante (i) et la valeur de la case contenant le minimum



Farida Ait-Ammar et Nicolas Leduc

23

Tri par échange d'un tableau
Code *Java*

```
public static void main(String[] args) {
    int[] quant = {3, 5, 1, 2, 4, 5, 5, 2, 1};
    int posMin;
    int temp;

    for (int i = 0; i < quant.length; ++i) {        // Pour chaque case successive du tableau
        posMin = i;                                 // On choisit la case actuelle (i) comme minimum initial
        for (int j = i+1; j < quant.length; ++j) { // Pour toutes les cases suivantes (i+1 à n)
            if (quant[j] < quant[posMin]) {         // On recherche le minimum
                posMin = j;
            }
        }
        if (posMin != i) {                          // Si la case actuelle ne contient pas le minimum
            temp = quant[i];                        // Alors on échange les valeurs des cases i et posMin
            quant[i] = quant[posMin];
            quant[posMin] = temp;
        }
    }

    // Affichage du tableau trié
    for (int val : quant) {
        System.out.print(val + ", ");
    }
}
```


24

Tri par échange – plusieurs tableaux
Code *Java*

- Votre employeur vous a fourni une liste de produits ainsi que la quantité achetée par un client, entre parenthèses:
  - Patate(3), Carotte(5), Navet(1), Brocoli(2), Tomate(4), Pomme(5), Poire(5), Raisin(2), Ananas(1)
- Il vous demande d'afficher le nom des produits triés en ordre croissant selon les quantités achetées.

```
public static void main(String[] args) {
    String[] nomProd = {"Patate","Carotte","Navet","Brocoli","Tomate","Pomme","Poire","Raisin","Ananas"};
    int[] quant = {3, 5, 1, 2, 4, 5, 5, 2, 1};
    int posMin;
    int tempInt;
    String tempStr;

    for (int i = 0; i < quant.length; ++i) {
        posMin = i;
        for (int j = i+1; j < quant.length; ++j) {
            if (quant[j] < quant[posMin]) { posMin = j; }              // On trie selon la quantité
        }
        if (posMin != i) {          // On doit échanger les éléments dans TOUS les tableaux manuellement
            tempInt = quant[i];
            quant[i] = quant[posMin];                                  // Échange de la quantité
            quant[posMin] = tempInt;
            tempStr = nomProd[i];
            nomProd[i] = nomProd[posMin];                              // Échange du nom
            nomProd[posMin] = tempStr;
        }
    }
    for (int i = 0; i < quant.length; ++i) { System.out.print(nomProd[i] + "(" + quant[i] + "), "); }
}
```

7.6 Algorithmes : tableau ordonné

7.6.1 Recherche d'occurrence

25

Recherche directe
Tableau trié

- Votre employeur vous a fourni une **liste triée** de produits ainsi que la quantité achetée par un client, entre parenthèses:
  - Navet(1), Ananas(1), Brocoli(2), Raisin(2), Patate(3), Tomate(4), Poire(5), Pomme(5), Carotte(5)
- Il voudrait connaître la quantité minimale, maximale et médiane de produits achetés par le client. Il vous rappelle que la médiane se retrouve dans la case centrale d'un tableau trié.

```
public static void main(String[] args) {
    int[] quant = {1,1,2,2,3,4,5,5,5};
    int minimum;
    int maximum;
    float mediane;

    minimum = quant[0];
    maximum = quant[quant.length-1];
    if (quant.length%2 == 0) {   // Nombre de cases pair
        mediane = (quant[quant.length/2-1] + quant[quant.length/2])/2.0f;
    } else {                     // Nombre de cases impair
        mediane = quant[quant.length/2];
    }
    System.out.println("Minimum : " + minimum);
    System.out.println("Médiane : " + mediane);
    System.out.println("Maximum : " + maximum);
}
```

length / 2 = 3

0	1	2	3	4	5

length / 2 = 2

0	1	2	3	4

Farida Ait-Ammar et Nicolas Leduc

26

Recherche séquentielle
Tableau ordonné

- Votre employeur vous a fourni une liste triée de produits ainsi que la quantité achetée par un client, entre parenthèses:
  - Navet(1), Ananas(1), Brocoli(2), Raisin(2), Patate(3), Tomate(4), Poire(5), Pomme(5), Carotte(5)
- Il vous demande d'afficher le premier produit trouvé qui correspond à la quantité lue. Étant donné que la liste est triée, vous devez éviter de parcourir systématiquement tout le tableau.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    String[] nomProd = {"Navet", "Ananas", "Brocoli", "Raisin",
                        "Patate", "Tomate", "Poire", "Pomme", "Carotte"};
    int[] quant = {1, 1, 2, 2, 3, 4, 5, 5, 5};
    int pos = 0;
    int valeurCherchee;

    System.out.print("Quelle est la quantité recherchée? ");
    valeurCherchee = clavier.nextInt();
    // La boucle s'arrête dès que l'on trouve ou dépasse la valeur recherchée
    while (pos < quant.length && quant[pos] < valeurCherchee) {
        ++pos;
    }
    // Si on n'a pas atteint la fin du tableau et que la valeur est celle recherchée
    if (pos < quant.length && quant[pos] == valeurCherchee) {
        System.out.println("Produit trouvé : " + nomProd[pos]);
    } else {
        System.out.println("Aucun produit trouvé");
    }
}
```

Farida Ait-Ammar et Nicolas Leduc

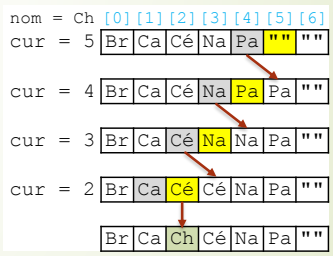
7.6.2 Ajout et retrait

27

Ajout – Tableau ordonné

- Votre employeur désire ajouter des produits à son inventaire (maximum de 7). Toutefois, il vous indique que les informations sont triées par nom de produit et il veut conserver cet ordre à la suite d'un ajout.

```
public static void main(String[] args) {
     Scanner clavier = new Scanner(System.in);
     String[] nomProd = {"Brocoli","Carotte","Céleri","Navet","Patate","",""};
     int[] quant = {2,5,4,1,3,-1,-1};
     int nbItems = 5;
     String nomItem;
     int qteItem;
     System.out.print("Indiquez l'item à ajouter : ");
     nomItem = clavier.next();
     System.out.print("Indiquez la quatité : ");
     qteItem = clavier.nextInt();
     if (nbItems < nomProd.length) {
          int curPos = nbItems;          // On débute à la première case vide
          // Tant que item à ajouter < item de la case précédente
          while (curPos > 0 && nomItem.compareTo(nomProd[curPos-1]) < 0) {
               nomProd[curPos] = nomProd[curPos-1];       // On le décale
               quant[curPos] = quant[curPos-1];           // vers la droite
               curPos--;                  // On se déplace d'une case à gauche
          }
          nomProd[curPos] = nomItem;
          quant[curPos] = qteItem;     // Ajout de l'item dans la case libre
          nbItems++;
     }
     for (int i = 0; i < nomProd.length; ++i) {
          System.out.println(nomProd[i] + " : " + quant[i]);
     }
}
```



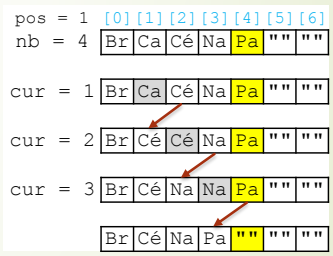
Farida Ait-Ammar et Nicolas Leduc

28

Retrait – Tableau ordonné

- Votre employeur désire supprimer des produits de son inventaire (maximum de 7). Toutefois, il vous indique que les informations sont triées par nom de produit et il veut conserver cet ordre à la suite d'un retrait.

```
public static void main(String[] args) {
     Scanner clavier = new Scanner(System.in);
     String[] nomProd = {"Brocoli","Carotte","Céleri","Navet","Patate","",""};
     int[] quant = {2,5,4,1,3,-1,-1};
     int nbItems = 5;
     String nomItem;
     int posItem;
     System.out.print("Indiquez l'item à retirer : ");
     nomItem = clavier.next();
     posItem = trouve(nomItem, nomProd);            // Voir p19
     if (posItem != -1) {
          nbItems--;                // Pointe sur la case du dernier item
          // À partir de l'item à effacer jusqu'au dernier item
          for (int curPos = posItem; curPos < nbItems; curPos++) {
               nomProd[curPos] = nomProd[curPos+1];   // On décale le
               quant[curPos] = quant[curPos+1];       // suivant à gauche
          }
          nomProd[nbItems] = "";    // On libère la case du dernier item
          quant[nbItems] = -1;
     }
     for (int i = 0; i < nomProd.length; ++i) {
          System.out.println(nomProd[i] + " : " + quant[i]);
     }
```



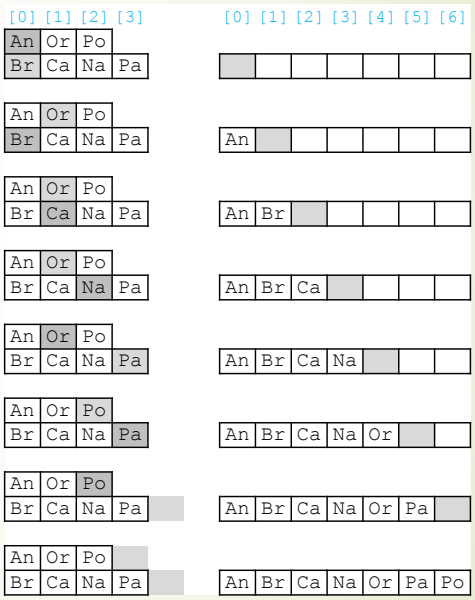
Farida Ait-Ammar et Nicolas Leduc

7.7 Fusion de tableaux

29

Fusion de tableaux

- Pour fusionner deux (ou plusieurs) tableaux triés dans un nouveau tableau trié, il suffit de tous les parcourir en parallèle et de copier la valeur minimale à chaque tour de boucle.
- Lorsque l'on arrive au bout d'un tableau, il ne reste qu'à copier tous les éléments restant de l'autre tableau.
- Si les tableaux sources ne sont pas triés, il n'est pas possible d'obtenir directement un tableau trié lors de la fusion. Dans ce cas, on peut simplement concaténer les éléments des différents tableaux en les parcourant à tour de rôle.



Farida Ait-Ammar et Nicolas Leduc

30

Fusion de tableaux

```
public static void main(String[] args) {
     String[] fruits = {"Ananas","Orange","Pomme","Raisin","Tomate"};
     String[] legumes = {"Brocoli","Carotte","Navet","Patate"};
     String[] nomProd = new String[fruits.length + legumes.length];
     int posFruits = 0, posLegumes = 0, posProd = 0;

     if (fruits.length + legumes.length <= nomProd.length) { // Assez de place?
          // Tant qu'il reste des éléments dans les deux tableaux
          while (posFruits < fruits.length && posLegumes < legumes.length) {
               // On copie la plus petite valeur et on avance dans le tableau
               if (fruits[posFruits].compareTo(legumes[posLegumes]) < 0) {
                    nomProd[posProd] = fruits[posFruits];
                    posFruits++;
               } else {
                    nomProd[posProd] = legumes[posLegumes];
                    posLegumes++;
               }
               posProd++; // On avance toujours dans le tableau fusionné
          }
          // Complète la fusion en concaténant les éléments restants
          while (posFruits < fruits.length) {
               nomProd[posProd++] = fruits[posFruits++];
          }
          while (posLegumes < legumes.length) {
               nomProd[posProd++] = legumes[posLegumes++];
          }
          while (posProd < nomProd.length) {
               nomProd[posProd++] = ""; // On efface les cases superflues
          }
     }
     for (int i = 0; i < nomProd.length; ++i) { System.out.print(nomProd[i] + " - "); }
}
```

Fusion de tableaux **triés**

Valide pour **toutes** les fusions

Farida Ait-Ammar et Nicolas Leduc

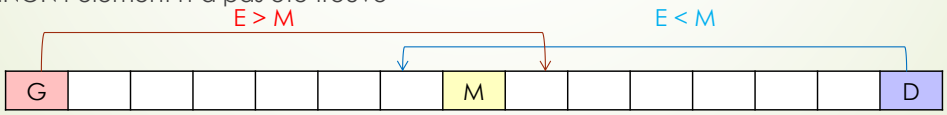
7.8 Recherche binaire

31

Recherche binaire

- La recherche séquentielle fonctionne, mais elle n'est pas très efficace dans le cas d'un tableau trié. En effet, il existe d'autres méthodes de recherche, tel que la recherche binaire, qui sont beaucoup plus rapides pour de grands tableaux.
  - Par exemple, pour un tableau de 1 000 000 d'éléments, il faut faire 500 000 boucles en moyenne dans le cas de la recherche séquentielle, alors que 20 boucles ($\log_2 N$) sont suffisantes à l'aide d'une recherche binaire.
- La recherche binaire implémente la stratégie *diviser pour régner*. À chaque étape, on réduit de moitié le nombre de candidats en déplaçant les bornes de la recherche.

Initialiser les bornes de gauche et de droite aux extrémités du tableau
Initialiser le milieu avec une division entière: (gauche + droite) / 2
TANT QUE l'élément n'est pas trouvé au milieu et que les bornes ne se croisent pas FAIRE
    SI l'élément < milieu ALORS déplacer la borne de droite à gauche du milieu
    SI l'élément > milieu ALORS déplacer la borne de gauche à droite du milieu
    Mettre à jour le milieu
SI les bornes ne se croisent pas ALORS l'élément cherché est à la position du milieu
SINON l'élément n'a pas été trouvé



Farida Ait-Ammar et Nicolas Leduc

32

Recherche binaire

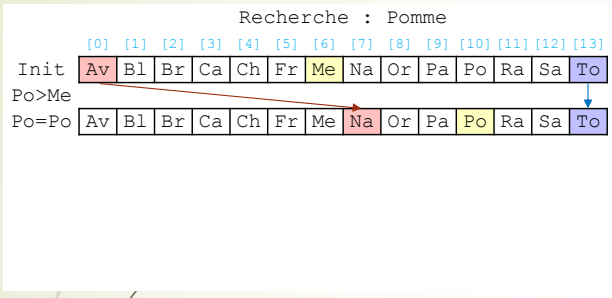
```
public static void main(String[] args) {
     Scanner clavier = new Scanner(System.in);
     String[] nomProd = {"Avocat","Bleuet","Brocoli","Carotte","Chou","Fraise","Melon",
                         "Navet","Orange","Patate","Pomme","Raisin","Salade","Tomate",""};
     int[] quant = {1, 4, 2, 5, 1, 3, 1, 1, 5, 4, 7, 2, 1, 6, -1};
     int nbElem = 14;
     String prodCherche;

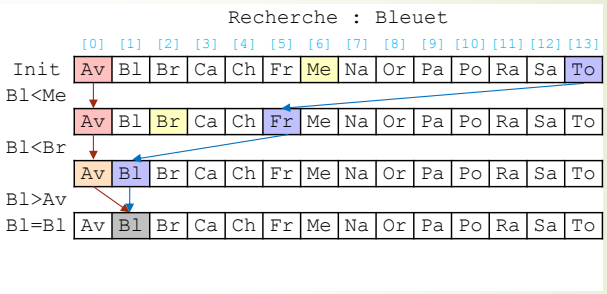
     System.out.print("Entrez le nom d'un produit : ");
     prodCherche = clavier.next();
     int posGa = 0;
     int posDr = nbElem - 1;
     int posMi = (posGa + posDr) / 2;      // Division entière pour trouver le milieu
     System.out.println(posGa + " : " + posMi + " : " + posDr);              // Affichage des bornes
     // On s'arrête lorsque les bornes se croisent ou lorsqu'on a trouvé le produit au milieu
     while (posGa <= posDr && !prodCherche.equals(nomProd[posMi])) {
          if (prodCherche.compareTo(nomProd[posMi]) < 0) {
               posDr = posMi - 1;          // Si on est à gauche du milieu, on déplace la borne de droite
          } else {
               posGa = posMi + 1;          // Si on est à droite du milieu, on déplace la borne de gauche
          }
          posMi = (posGa + posDr) / 2;     // Mise à jour du milieu
          System.out.println(posGa + " : " + posMi + " : " + posDr);
     }
     if (posGa <= posDr) {                 // Si les bornes ne se croisent pas, on a trouvé
          System.out.println("Produit trouvé: " + nomProd[posMi] + " (" + quant[posMi] + ")");
     } else {
          System.out.println("Aucun produit trouvé");
     }
}
```

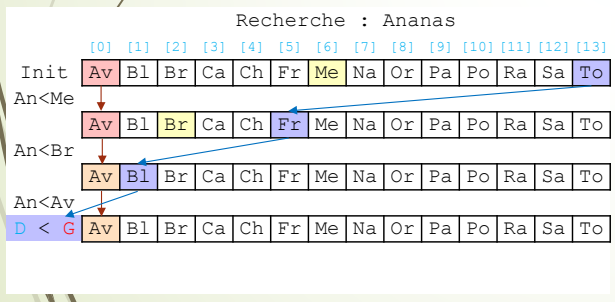
Farida Ait-Ammar et Nicolas Leduc

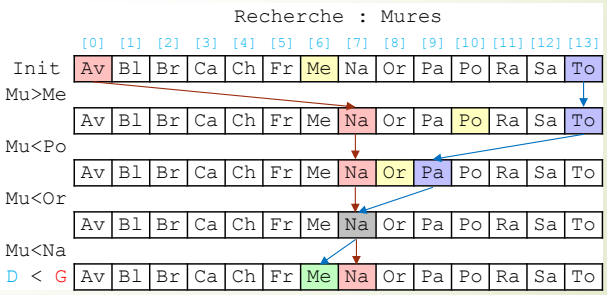
33

Recherche binaire









Farida Ait-Ammar et Nicolas Leduc

8. Tableau multidimensionnel

8.1 Déclaration et initialisation

2

Introduction

- Lorsque l'on traite de l'information, il arrive régulièrement que celle-ci se trouve sous forme de tableau. On pense, par exemple, aux notes des étudiants, à la couleur de chaque pixel d'une image, etc.
- L'utilisation des tableaux unidimensionnels est alors difficile, car il faut créer un tableau distinct pour chaque ligne ou colonne du tableau de données.
  - Un tableau pour chaque cours
  - Un tableau pour chaque ligne de l'image !!!
- Heureusement, il est possible de créer des tableaux multidimensionnels en *Java*.
- Un tableau multidimensionnel regroupe, sous forme de table, des données de même type.

	ZC5	ZC6	ZF5
NL	78	85	62
FA	92	88	95
FE	67	53	61
MB	43	35	71
FB	77	76	28

	ZC5	ZC6	ZF5
NL	78	85	62
FA	92	88	95
FE	67	53	61
MB	43	35	71
FB	77	76	28

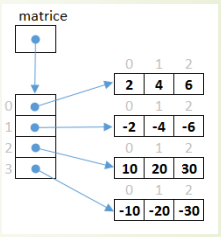
Farida Ait-Ammar
Nicolas Leduc

3

Déclaration

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- *Java* permet de simuler un tableau à plusieurs dimensions par la création de tableaux de tableaux.
- Pour représenter une matrice (tableau en 2D), il faut créer un tableau qui contiendra autant de cases qu'il y a de lignes dans la matrice. Chaque case contiendra alors un tableau représentant une ligne.



- Pour déclarer un tableau multidimensionnel, il faut mettre autant de crochets qu'il y a de dimensions (maximum de 255) :

```
int[][] notes; // Notes des cours pour chaque étudiant (2D)
float[][][] temp3D; // Températures dans un volume (3D)
int[][][][] video;  // Temps, RGB, Y, X (4D)
```

- Il est possible de placer les crochets de différentes façons pour obtenir le même résultat :

```
int[][] notes; // Obligatoire dans ce cours
int[] notes[];
int notes[][];
```

Farida Ait-Ammar
Nicolas Leduc

4

Initialisation

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Une fois que le nombre de dimensions est déterminé, il faut initialiser le tableau en indiquant l'espace à réserver pour chacune des dimensions.
- Si le tableau est régulier, c'est-à-dire que toutes ses dimensions ont des tailles fixes, on peut utiliser l'opérateur `new` comme pour les tableaux 1D :

```
int[][] notes = new int[5][3];              // 5 lignes et 3 colonnes
int[][] image = new int[1080][1920];        // Image de 1920x1080 pixels
float[][][][] varTemp3D = new float[60][25][25][25];  // Variation temp 3D
```

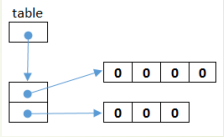
- L'opérateur `new` initialise le tableau avec les valeurs par défaut selon le type, comme pour les tableaux unidimensionnels.
- Il est aussi possible d'initialiser directement le tableau avec des valeurs spécifiques en imbriquant des accolades :

```
int[][] notes = {{78,85,62},
                 {92,88,95},
                 {67,53,61},
                 {43,35,71},
                 {77,76,28}}; // 5 lignes et 3 colonnes
```

Farida Ait-Ammar
Nicolas Leduc

5

Initialisation Tableau irrégulier



- Il est aussi possible de créer des tableaux dans lesquels les lignes ont des tailles variables. Dans ce cas, il faut les initialiser par étapes :

```
int[][] triangle = new int[5][]; // Initialisation du nombre de lignes

triangle[0] = new int[1]; //---------------
triangle[1] = new int[2]; // Initialise chaque ligne individuellement
triangle[2] = new int[3]; //
triangle[3] = new int[4]; // Longueur de ligne = numéro de ligne
triangle[4] = new int[5]; //---------------

// Initialisation équivalente à l'aide d'une boucle
for (int i = 0; i < triangle.length; ++i) { // Pour chaque ligne
    triangle[i] = new int[i+1]; // Initialisation du tableau de la ligne
}

// Initialisation équivalente en utilisant les accolades
int[][] triangle = {new int[1], new int[2], new int[3], new int[4], new int[5]};

// Initialisation avec des valeurs spécifiques (triangle de Pascal)
int[][] triangle = {{1}, {1,1}, {1,2,1}, {1,3,3,1}, {1,4,6,4,1}};
```

Farida Ait-Ammar
Nicolas Leduc

6

Déclaration et initialisation Exemples

- Déclarez et initialisez les tableaux suivants :

Tableau de 15 semaines de 7 jours pour stocker votre temps d'utilisation internet	`int[][] conso;` `conso = new int[15][7];`
Matrice de nombres à virgule flottante de 4 lignes et 3 colonnes	`float[][] mat = new float[4][3];`
Tableau dont la première ligne contient les valeurs 10, 5, 15 et la deuxième, 5, 25, 0	`int[][] valeur = {{10, 5, 15},{5, 25, 0}};`
Tableau qui peut être adressé par jour, mois et année pour les 10 dernières années et qui contient le nombre de minutes d'ensoleillement	`int[][][] minSoleil;` `minSoleil = new int[10][12][31];`
Grille de Sudoku	`int[][] sudoku = new int[9][9];`
Grille de mot croisés ayant 10 colonnes et 12 lignes	`char[][] mots = new char[12][10];`
Matrice contenant les valeurs (11, 21, 31) dans la colonne 1, (12, 22, 32) dans la colonne 2 et (13, 23, 33) dans la colonne 3.	`int[][] mat3 = {{11,12,13},{21,22,23},{31,32,33}};`

Farida Ait-Ammar
Nicolas Leduc

8.2 Accès aux éléments

7

Taille

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Comme pour les tableaux unidimensionnels, l'attribut `length` permet de déterminer la taille d'un tableau. Toutefois, il n'est possible de déterminer la taille que d'une dimension à la fois.
- Pour un tableau 2D, on peut déterminer le nombre de lignes en inscrivant :

```
notes.length;       // Retourne 5
```

- Pour connaître le nombre de colonnes, il faut interroger une ligne en particulier :

```
notes[0].length;    // Retourne 3, soit la longueur de la première ligne
```

- Lors du parcours de tableaux irréguliers, il est important de déterminer la longueur de chaque ligne afin d'éviter les accès à l'extérieur du tableau.

```
// Déterminez l'affichage à la console
int[][] image = new int[480][640];
int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
int[][] triPa = {{1}, {1,1}, {1,2,1}, {1,3,3,1}, {1,4,6,4,1}};

System.out.println(image.length);       // 480
System.out.println(image[0].length);    // 640
System.out.println(notes.length);       // 5
System.out.println(notes[1].length);    // 3
System.out.println(triPa.length);       // 5
System.out.println(triPa[3].length);    // 4
```

Farida Ait-Ammar
Nicolas Leduc

8

Accès aux éléments

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- En général, pour accéder à un élément d'un tableau multidimensionnel, il faut indiquer l'index de chacune de ses dimensions :

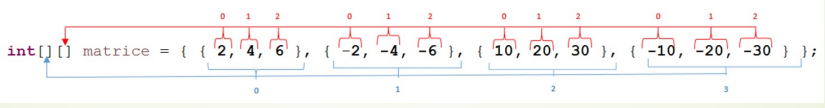
```
int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
notes[2][0];       // Accède à l'élément de la 3e ligne et 1ere colonne : 67
notes[2][0] = 69; // Affecte la valeur 69 à l'élément en position 2;0
```

- Les index vont de zéro à la taille de la dimension - 1. Si on a 5 lignes et 3 colonnes, les index vont de 0 à 4 et de 0 à 2 respectivement.
- Il faut éviter d'accéder à l'extérieur du tableau sinon l'exception `ArrayIndexOutOfBoundsException` est lancée et votre programme s'arrête.
- Il est aussi possible d'extraire une ligne entière sous forme d'un tableau unidimensionnel. Par contre, il n'existe pas de manière directe d'extraire une colonne.

```
int[] unEleve = notes[3];              // Permet d'accéder à la 4e ligne directement
```

- Déterminez l'affichage en tenant compte des instructions précédentes :

```
System.out.println(notes[4][2]);       // 28
System.out.println(notes[2][4]);       // ArrayIndexOutOfBoundsException
System.out.println(unEleve.length);    // 3
System.out.println(unEleve[1]);        // 35
unEleve[1] = 85;
System.out.println(notes[2][0]);       // 69
System.out.println(notes[3][1]);       // 85
```



Farida Ait-Ammar
Nicolas Leduc

8.3 Parcours par ligne ou colonne

9

Parcours d'un tableau 2D

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Parcours d'une ligne
  - Pour parcourir une ligne, il faut fixer le numéro de la ligne (`i`) et incrémenter les colonnes (`j`)

```
int i = 3;                                      // Sélection de la ligne
// Initialisation des variables / Affichage d'un en-tête
for (int j = 0; j < notes[i].length; ++j) {  // Parcours de la ligne (colonnes)
    // Traitement de chacun des éléments notes[i][j]
}
// Calcul final / Affichage du résultat / Fin de ligne
```

- Parcours d'une colonne
  - Pour parcourir une colonne, il faut fixer le numéro de la colonne (`j`) et incrémenter les lignes (`i`)

```
int j = 1;                                      // Sélection de la colonne
// Initialisation des variables / Affichage d'un en-tête
for (int i = 0; i < notes.length; ++i) {     // Parcours de la colonne (lignes)
    // Traitement de chacun des éléments notes[i][j]
}
// Calcul final / Affichage du résultat / Fin de ligne
```

Farida Ait-Ammar
Nicolas Leduc

10

Parcours d'un tableau 2D

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Parcours de l'ensemble des lignes

```
for (int i = 0; i < notes.length; ++i) {        // Sélection séquentielle des lignes
    // Initialisation des variables / Affichage d'un en-tête
    for (int j = 0; j < notes[i].length; ++j) { // Parcours de la ligne (colonnes)
        // Traitement de chacun des éléments notes[i][j]
    }
    // Calcul final / Affichage du résultat / Fin de ligne
}
```

- Parcours de l'ensemble des colonnes

```
for (int j = 0; j < notes[0].length; ++j) {     // Sélection séquentielle des colonnes
    // Initialisation des variables / Affichage d'un en-tête
    for (int i = 0; i < notes.length; ++i) {    // Parcours de la colonne (lignes)
        // Traitement de chacun des éléments notes[i][j]
    }
    // Calcul final / Affichage du résultat / Fin de ligne
}
```

- Parcours de tout le tableau
  - On utilise habituellement l'algorithme du parcours de l'ensemble des lignes

Farida Ait-Ammar
Nicolas Leduc

8.4 Lecture et affichage

11 Remplir un tableau 2D par ligne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- On vous demande d'écrire un programme qui va lire les notes dans les trois cours pour chacun des cinq étudiants.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    int[][] notes = new int[5][3];

    for (int i = 0; i < notes.length; ++i) {        // Sélection séquentielle des lignes
        System.out.println("Notes étudiant #" + (i+1)); // Affichage d'un en-tête
        for (int j = 0; j < notes[i].length; ++j) { // Parcours de la ligne (cours)
            System.out.print("Cours #" + (j+1) + " : ");
            notes[i][j] = clavier.nextInt();        // Lecture dans notes[i][j]
        }
        System.out.println(); // Fin de ligne
    }

    Outils.print(notes); // Affichage du tableau
}
```

```
Notes étudiant #1
Cours #1 : 78
Cours #2 : 85
Cours #3 : 62

Notes étudiant #2
Cours #1 : 92
...
```

Farida Ait-Ammar
Nicolas Leduc

12 Remplir un tableau 2D par colonne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- On vous demande d'écrire un programme qui va lire les notes des cinq étudiants pour chacun des trois cours.

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    int[][] notes = new int[5][3];

    for (int j = 0; j < notes[0].length; ++j) {  // Sélection séquentielle des colonnes
        System.out.println("Notes cours #" + (j+1)); // Affichage d'un en-tête
        for (int i = 0; i < notes.length; ++i) { // Parcours de la colonne (étudiants)
            System.out.print("Étudiant #" + (i+1) + " : ");
            notes[i][j] = clavier.nextInt();     // Lecture dans notes[i][j]
        }
        System.out.println(); // Fin de ligne
    }

    Outils.print(notes); // Affichage du tableau
}
```

```
Notes cours #1
Étudiant #1 : 78
Étudiant #2 : 92
Étudiant #3 : 67
Étudiant #4 : 43
Étudiant #5 : 77

Notes cours #2
Étudiant #1 : 85
...
```

Farida Ait-Ammar
Nicolas Leduc

13

Afficher toutes les lignes

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Vous devez écrire un programme qui affiche le tableau des notes tel que les notes d'un même étudiant pour les différents cours apparaissent sur une même ligne.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};

    // Affichage des en-têtes de colonnes d'affichage
    System.out.println(String.format("%4s  %s", "", "Cours"));
    System.out.print(String.format("%4s  ", "Etu"));
    for (int j = 0; j < notes[0].length; ++j) {          // Pour chaque cours
        System.out.print(String.format("%2d ", j+1));    // On inscrit son #
    }
    System.out.println();

    // Affichage du tableau
    for (int i = 0; i < notes.length; ++i) {             // Pour chaque ligne (étudiant)
        System.out.print(String.format("%4s  ", i+1));   // En-tête de ligne d'affichage
        for (int j = 0; j < notes[i].length; ++j) {      // Parcours de la ligne (cours)
            System.out.print(String.format("%2d ", notes[i][j]));
        }
        System.out.println();                            // Fin de ligne
    }
```

```
      Cours
Etu   1  2  3
  1   78 85 62
  2   92 88 95
  3   67 53 61
  4   43 35 71
  5   77 76 28
```

Farida Ait-Ammar
Nicolas Leduc

14

Afficher toutes les colonnes

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Vous devez écrire un programme qui affiche le tableau des notes tel que les notes d'un même cours pour les différents étudiants apparaissent sur une même ligne.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};

    // Affichage des en-têtes de colonnes d'affichage
    System.out.println(String.format("%5s %s", "", "Etudiant"));
    System.out.print(String.format("%5s ", "Cours"));
    for (int i = 0; i < notes.length; ++i) {             // Pour chaque étudiant
        System.out.print(String.format("%2d ", i+1)); // On inscrit son #
    }
    System.out.println();

    // Affichage du tableau
    for (int j = 0; j < notes[0].length; ++j) {          // Pour chaque colonne (cours)
        System.out.print(String.format("%3s   ", j+1)); // En-tête de ligne d'affichage
        for (int i = 0; i < notes.length; ++i) {         // Parcours de la colonne (étudiants)
            System.out.print(String.format("%2d ", notes[i][j]));
        }
        System.out.println();                            // Fin de ligne d'affichage
    }
}
```

```
       Etudiant
Cours  1  2  3  4  5
  1    78 92 67 43 77
  2    85 88 53 35 76
  3    62 95 61 71 28
```

Farida Ait-Ammar
Nicolas Leduc

8.5 Paramètre de type tableau

15

Tableaux et méthodes

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Comme pour les tableaux unidimensionnels, il est possible de passer un tableau multidimensionnel en paramètre à une méthode. Il suffit de mettre le nombre de crochets appropriés, correspondant au nombre de dimensions du tableau, sans indiquer la taille de chaque dimension.
- Comme le tableau est passé par référence, toute modification du tableau à l'intérieur de la méthode sera visible par le programme appelant.
- Exemple :

```
// Procédure qui reçoit un tableau 2D en paramètre
public static void incVal(int[][] table) { ... }

public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    incVal(notes); // Appel de la méthode en passant le tableau notes
}
```

Farida Ait-Ammar
Nicolas Leduc

16

Tableaux et méthodes Exemple

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Déterminez l'affichage :

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int[][] nouvNotes;

    System.out.println(notes[2][0]);       // 67
    System.out.println(notes[3][1]);       // 35
    nouvNotes = incVal(notes);
    System.out.println(notes[2][0]);       // 62
    System.out.println(notes[3][1]);       // 35
    System.out.println(nouvNotes[2][0]);   // 72
    System.out.println(nouvNotes[3][1]);   // 40
}

public static int[][] incVal(int[][] table) {
    int[][] nouvTable = new int[table.length][table[0].length];
    for (int i = 0; i < table.length; ++i) {
        for (int j = 0; j < table[i].length; ++j) {
            nouvTable[i][j] = table[i][j] + 5;
        }
    }
    table[2][0] -= 5;
    return nouvTable;
}
```

Farida Ait-Ammar
Nicolas Leduc

8.6 Somme et moyenne

17

Somme et moyenne
Une ligne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Calculez la moyenne des notes d'un étudiant pour l'ensemble de ses cours. (moyenne d'une ligne)

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    float somme;

    int i = 1;                              // Sélection de la ligne -> 2e étudiant
    somme = 0;                              // Initialisation de somme
    for (int j = 0; j < notes[i].length; ++j) {  // Parcours de la ligne (cours)
        somme += notes[i][j];               // Traitement : calcul de la somme
    }

    System.out.println("Moyenne de l'étudiant #" + (i+1) +
                       " : " + somme/notes[i].length);
}
```

Farida Ait-Ammar
Nicolas Leduc

18

Somme et moyenne
Une ligne – solution alternative

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Calculez la moyenne des notes d'un étudiant pour l'ensemble de ses cours. (moyenne d'une ligne)

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int numEtu = 3;                                  // 4e étudiant
    float somme = 0;

    int[] etudiant = notes[numEtu];                  // Extraire une ligne (étudiant)
    for (int i = 0; i < etudiant.length; ++i) {      // Somme d'un tableau 1D
        somme += etudiant[i];
    }

    System.out.println("Moyenne de l'étudiant #" + (numEtu+1) +
                       " : " + somme/etudiant.length);
}
```

Farida Ait-Ammar
Nicolas Leduc

19

Somme et moyenne
Toutes les lignes

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Déterminez la moyenne générale de chaque étudiant.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    float somme;

    for (int i = 0; i < notes.length; ++i) {          // Pour chaque ligne (étudiant)
        somme = 0;                                    // Initialisation de somme
        for (int j = 0; j < notes[i].length; ++j) {   // Parcours de la ligne (cours)
            somme += notes[i][j];                     // Traitement : calcul de la somme
        }
        System.out.println("Moyenne de l'étudiant #" + (i+1) +
                           " : " + somme/notes[i].length);
    }
}
```

```
Moyenne de l'étudiant #1 : 75.0
Moyenne de l'étudiant #2 : 91.666664
Moyenne de l'étudiant #3 : 60.333332
Moyenne de l'étudiant #4 : 49.666668
Moyenne de l'étudiant #5 : 60.333332
```

Farida Ait-Ammar
Nicolas Leduc

20

Somme et moyenne
Une colonne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Calculez la moyenne des notes de tous les étudiants inscrits au cours spécifié. (moyenne d'une colonne)

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    float somme;

    int j = 1;                                    // Sélection de la colonne -> 2e cours
    somme = 0;                                    // Initialisation de somme
    for (int i=0; i<notes.length; ++i) {          // Parcours de la colonne (étudiants)
        somme += notes[i][j];                     // Traitement : calcul de la somme
    }

    System.out.println("Moyenne du cours #" + (j+1) +
                       " : " + somme/notes.length);
}
```

Farida Ait-Ammar
Nicolas Leduc

21

Somme et moyenne Toutes les colonnes

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Calculez la moyenne générale de tous les cours.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    float somme;

    for (int j = 0; j < notes[0].length; ++j) {  // Pour chaque colonne (cours)
        somme = 0;                               // Initialisation de somme
        for (int i=0; i<notes.length; ++i) { // Parcours de la colonne (étudiants)
            somme += notes[i][j];                // Traitement : calcul de somme
        }
        System.out.println("Moyenne du cours #" + (j+1) +
                           " : " + somme/notes.length);
    }
}
```

```
Moyenne du cours #1 : 71.4
Moyenne du cours #2 : 67.4
Moyenne du cours #3 : 63.4
```

Farida Ait-Ammar
Nicolas Leduc

22

Somme et moyenne Tableau complet

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Calculez la moyenne globale de toutes les notes.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    float somme = 0;

    for (int i = 0; i < notes.length; ++i) {     // Pour chaque ligne (étudiant)
        for (int j=0; j<notes[i].length; ++j) {  // Parcours de la ligne (cours)
            somme += notes[i][j];
        }
    }

    System.out.println("Moyenne de toutes les notes : " +
                       + somme/(notes.length*notes[0].length));

}
```

Farida Ait-Ammar
Nicolas Leduc

8.7 Minimum et maximum

23

Minimum et maximum
Une ligne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Déterminez la note minimale et maximale obtenue pour l'étudiant spécifié.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int jMin, jMax;

    int i = 1;                                  // Sélection ligne -> 2e étudiant
    jMin = 0;
    jMax = 0;                                   // Min et max première note?
    for (int j = 0; j < notes[i].length; ++j) { // Parcours de la ligne (cours)
        if (notes[i][j] < notes[i][jMin]) {     // Vérification min
            jMin = j;
        }
        if (notes[i][j] > notes[i][jMax]) {     // Vérification max
            jMax = j;
        }
    }

    System.out.println("Etudiant #" + (i+1) + " - " + notes[i][jMin] +
                       " : " + notes[i][jMax]);
}
```

Farida Ait-Ammar
Nicolas Leduc

24

Minimum et maximum
Une ligne – solution alternative

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Déterminez la note minimale et maximale obtenue pour l'étudiant spécifié.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int numEtu = 1;                             // 2e étudiant
    int posMin = 0, posMax = 0;

    int[] etudiant = notes[numEtu];             // Extraire une ligne (étudiant)
    for (int i = 1; i < etudiant.length; ++i) { // Min et max sur tableau 1D
        if (etudiant[i] < etudiant[posMin]) {
            posMin = i;
        }
        if (etudiant[i] > etudiant[posMax]) {
            posMax = i;
        }
    }

    System.out.println("Etudiant #" + (numEtu+1) + " - " + etudiant[posMin]
                       + " : " + etudiant[posMax]);
}
```

Farida Ait-Ammar
Nicolas Leduc

25

Minimum et maximum
Une colonne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Déterminez la note minimale et maximale pour le cours spécifié.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int iMin, iMax;

    int j = 1;                                  // Sélection colonne -> 2e cours
    iMin = 0;
    iMax = 0;                                   // Min et max premier étudiant?
    for (int i = 0; i < notes.length; ++i) {    // Parcours de la colonne (étudiants)
        if (notes[i][j] < notes[iMin][j]) {     // Vérification min
            iMin = i;
        }
        if (notes[i][j] > notes[iMax][j]) {     // Vérification max
            iMax = i;
        }
    }

    System.out.println("Cours #" + (j+1) + " - " + notes[iMin][j] +
                       " : " + notes[iMax][j]);
}
```

Farida Ait-Ammar
Nicolas Leduc

26

Minimum et maximum
Tableau complet

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Trouvez la position des notes minimale et maximale globales dans le tableau.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int iMin = 0, jMin = 0, iMax = 0, jMax = 0;       // Min et max sur 1ier élément

    for (int i = 0; i < notes.length; ++i) {          // Pour chaque ligne (étudiant)
        for (int j = 0; j < notes[i].length; ++j) {   // Parcours de la ligne (cours)
            if (notes[i][j] < notes[iMin][jMin]) {    // Vérification minimum
                iMin = i;
                jMin = j;
            }
            if (notes[i][j] > notes[iMax][jMax]) {    // Vérification maximum
                iMax = i;
                jMax = j;
            }
        }
    }

    System.out.println("La note minimale est : " + notes[iMin][jMin]);
    System.out.println("La note maximale est : " + notes[iMax][jMax]);
}
```

Farida Ait-Ammar
Nicolas Leduc

8.8 Recherche de toutes les occurrences

27

Recherche toutes les occurrences Toutes les lignes

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Comptez le nombre d'échecs (notes < 60) pour chaque étudiant.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int compteEchec;

    for (int i = 0; i < notes.length; ++i) {          // Pour chaque ligne (étudiant)
        compteEchec = 0;                              // Initialisation de compteEchec
        for (int j = 0; j < notes[i].length; ++j) {   // Parcours de la ligne (cours)
            if (notes[i][j] < 60) {
                ++compteEchec;                        // Comptabilisation des échecs
            }
        }
        System.out.println("Échecs de l'étudiant #" + (i+1) + " : " + compteEchec);
    }
}
```

Farida Ait-Ammar
Nicolas Leduc

28

Recherche toutes les occurrences Toutes les colonnes

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Comptez le nombre d'échecs (notes < 60) dans chacun des cours.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int compteEchec;

    for (int j = 0; j < notes[0].length; ++j) {   // Pour chaque colonne (cours)
        compteEchec = 0;                          // Initialisation de compteEchec
        for (int i = 0; i < notes.length; ++i) {  // Parcours colonne (étudiants)
            if (notes[i][j] < 60) {
                ++compteEchec;                    // Comptabilisation des échecs
            }
        }
        System.out.println("Échecs du cours #" + (j+1) + " : " + compteEchec);
    }
}
```

Farida Ait-Ammar
Nicolas Leduc

29

Recherche toutes les occurrences Tableau complet

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Comptez le nombre global d'échecs (notes < 60) dans tout le tableau.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int compteEchec = 0;                                // Aucun échec au départ

    for (int i = 0; i < notes.length; ++i) {            // Pour chaque ligne (étudiant)
        for (int j = 0; j < notes[i].length; ++j) {     // Parcours de la ligne (cours)
            if (notes[i][j] < 60) {
                ++compteEchec;                          // Comptabilisation des échecs
            }
        }
    }

    System.out.println("Le nombre d'échecs est : " + compteEchec);
}
```

Farida Ait-Ammar
Nicolas Leduc

8.9 Recherche de la première occurrence

30

Recherche première occurrence Une ligne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Trouvez le premier cours dans lequel l'étudiant indiqué est en échec.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};

    int i = 1;                                      // Sélection de la ligne -> 2e étudiant
    int j = 0;
    while (j < notes[i].length && notes[i][j] >= 60) { // Parcours de la ligne (cours)
        j++;
    }
    System.out.print("Premier échec étudiant #" + (i+1) + " : ");
    if (j < notes[i].length) {                      // Échec trouvé : j dans tableau
        System.out.println(j+1);
    } else {                                        // Échec non trouvé
        System.out.println("Aucun");
    }
}
```

Farida Ait-Ammar
Nicolas Leduc

31 Recherche première occurrence Une ligne – solution alternative

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Trouvez le premier cours dans lequel l'étudiant indiqué est en échec.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int numEtu = 3;                                  // 4e étudiant

    int[] etudiant = notes[numEtu];                  // Extraire une ligne (étudiant)
    int pos = 0;                                     // Première occurrence tableau 1D
    while (pos < etudiant.length && etudiant[pos] >= 60) {
        pos++;
    }
    System.out.print("Premier échec étudiant #" + (numEtu+1) + " : ");
    if (pos < etudiant.length) {
        System.out.println(pos+1);
    } else {
        System.out.println("Aucun");
    }
}
```

Farida Ait-Ammar
Nicolas Leduc

32 Recherche première occurrence Une colonne

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

- Trouvez le premier étudiant qui est en échec pour le cours indiqué.

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};

    int j = 1;                                  // Sélection colonne -> 2e cours
    int i = 0;
    while (i < notes.length && notes[i][j] >= 60) { // Parcours colonne (étudiants)
        i++;
    }
    System.out.print("Premier échec cours #" + (j+1) + " : ");
    if (i < notes.length) {                     // Échec trouvé : i dans tableau
        System.out.println(i+1);
    } else {                                    // Échec non trouvé
        System.out.println("Aucun");
    }
}
```

Farida Ait-Ammar
Nicolas Leduc

8.10 Tri par échange

33 Tri de toutes les lignes

- Vous devez trier les étudiants par ordre croissant de note par rapport au cours indiqué. Attention, les notes des autres cours doivent être déplacées afin de suivre l'étudiant associé.

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int iMin;
    int jCleTri = 2;                                    // Cours à trier (clé de tri)

    // Routine de tri par sélection
    for (int i = 0; i < notes.length; ++i) {
        iMin = i;
        for (int k = i+1; k < notes.length; ++k) { // Recherche min dans lignes suivantes
            if (notes[k][jCleTri] < notes[iMin][jCleTri]) {
                iMin = k;
            }
        }
        if (iMin != i) {
            echangeLigne(notes, i, iMin);
        }
    }

    // Affichage du tableau trié
    for (int i = 0; i < notes.length; ++i) {
        for (int j = 0; j < notes[i].length; ++j) {
            System.out.print(notes[i][j] + " ");
        }
        System.out.println();
    }
}
```

```
77 76 28
67 53 61
78 85 62
43 35 71
92 88 95
```

```
public static void echangeLigne(
    int[][] grille, int i1, int i2) {
    int[] ligneTemp;

    ligneTemp = grille[i1];
    grille[i1] = grille[i2];
    grille[i2] = ligneTemp;
}
```

Farida Ait-Ammar
Nicolas Leduc

34 Tri de toutes les colonnes

- Triez les cours selon les notes qu'un étudiant précis a obtenues. N'oubliez pas de déplacer les notes de tous les autres étudiants afin de respecter le nouvel ordre des cours.

	[][0]	[][1]	[][2]
[0][]	78	85	62
[1][]	92	88	95
[2][]	67	53	61
[3][]	43	35	71
[4][]	77	76	28

```
public static void main(String[] args) {
    int[][] notes = {{78,85,62}, {92,88,95}, {67,53,61}, {43,35,71}, {77,76,28}};
    int jMin;
    int iCleTri = 2;                                    // Étudiant à trier (clé de tri)

    // Routine de tri par sélection
    for (int j = 0; j < notes[iCleTri].length; ++j) {
        jMin = j;
        for (int k=j+1; k<notes[iCleTri].length; ++k) {// Recherche min dans cols suivantes
            if (notes[iCleTri][k] < notes[iCleTri][jMin]) {
                jMin = k;
            }
        }
        if (jMin != j) {
            echangeColonne(notes, j, jMin);
        }
    }

    // Affichage du tableau trié
    for (int i = 0; i < notes.length; ++i) {
        for (int j = 0; j < notes[i].length; ++j) {
            System.out.print(notes[i][j] + " ");
        }
        System.out.println();
    }
}
```

```
85 62 78
88 95 92
53 61 67
35 71 43
76 28 77
```

```
public static void echangeColonne(
    int[][] grille, int j1, int j2) {
    int valTemp;

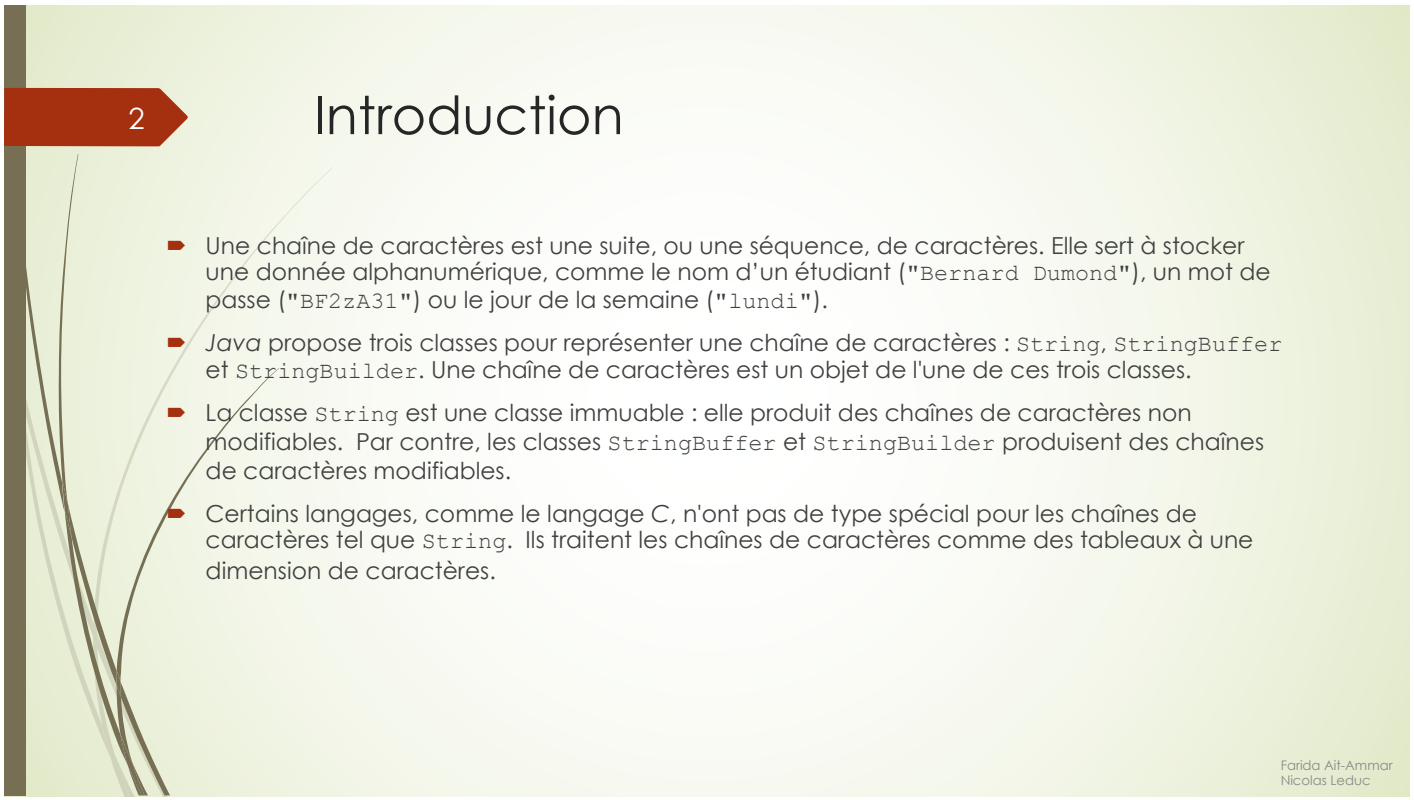
    for (int i=0; i<grille.length; ++i) {
        valTemp = grille[i][j1];
        grille[i][j1] = grille[i][j2];
        grille[i][j2] = valTemp;
    }
}
```

Farida Ait-Ammar
Nicolas Leduc

9. Chaîne de caractères

9.1 Classe `String`

9.1.1 Introduction



4

Classe `String` - Introduction

- La classe `String` permet de gérer les chaînes de caractères. Toutes les chaînes délimitées par des guillemets, comme `"abc"`, sont des instances de cette classe.
- Il n'est pas nécessaire d'utiliser le mot clé `new` pour instancier un objet de la classe `String`. Comme elle est un type de données très utilisé, *Java* permet une déclaration plus concise.
  - Exemple : Toutes ces déclarations sont équivalentes.

```
String str = "abc";  String str = new String("abc");  char data[] = {'a', 'b', 'c'};
                                                       String str = new String(data);
```

- La classe `String` se trouve dans le paquetage de base `java.lang`, qui contient toutes les classes fondamentales, comme la classe `Math` et la classe `System`. Par conséquent, son utilisation dans les programmes ne nécessite pas d'importation.
- La classe `String` comporte plusieurs constructeurs et une quarantaine de méthodes qui permettent des traitements comme la comparaison des chaînes, la recherche dans les chaînes, l'extraction des sous-chaînes et la conversion d'autres types de données en chaîne. Vous pouvez consulter la documentation de cette classe en cliquant sur ce lien :

Farida Ait-Ammar
Nicolas Leduc

5

Bloc de texte

- Depuis la version 15 de *Java*, il est possible d'initialiser une chaîne contenant plusieurs lignes de texte à l'aide de la notation du triple guillemets (`"""`) pour indiquer un bloc de texte. Celui-ci est transformé en une chaîne de type `String` qui inclut les caractères de contrôle nécessaires, par exemple, les retours de chariot sont convertis en `\n`.
- Exemple :

```
String chaine = """
    Ceci est
    un texte
    de trois lignes""";
```

  - est équivalent à :

```
String chaine = "Ceci est\n" +
    "un texte\n" +
    "de trois lignes";
```

  - Les deux déclarations précédentes produisent la chaîne suivante :

```
String chaine = "Ceci est\nun texte\nde trois lignes";
```

- Cette notation peut-être avantageuse pour initialiser une chaîne contenant plusieurs lignes, comme un menu, car elle simplifie la syntaxe et améliore la lisibilité du code.

Farida Ait-Ammar
Nicolas Leduc

9.1.2 Constructeur

6

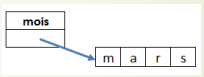
Construction d'un objet `String`

- Tout comme les tableaux, les variables de type `String` sont des pointeurs. Elles contiennent une référence (adresse) à l'objet associé en mémoire. Par exemple, la variable `mois`, ci-dessous, contient une référence à la suite de caractères contigus `mars`

```
String mois = "mars";
```

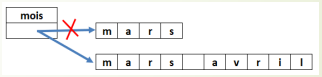
est équivalent à

```
String mois = new String("mars");
```



- Un objet `String` contient un tableau de caractères *Unicode* qui stocke la chaîne inscrite entre guillemets. Une fois cet objet construit, il n'est plus possible de le modifier. On dit qu'il est **immuable**, ce qui signifie qu'il est en **lecture seule**.
- Par conséquent, la moindre modification d'une chaîne ne peut se faire qu'en créant une nouvelle chaîne. Par exemple :

```
mois += " avril";
```



- L'instruction d'affectation ci-dessus provoque la création d'une nouvelle chaîne contenant `mars avril`. La variable `mois` contiendra la référence vers cette nouvelle chaîne.

Farida Ait-Ammar
Nicolas Leduc

7

Quelques constructeurs de la classe `String`

Constructeur	Exemple
`public String()` Initialise une chaîne de caractères vide (longueur de 0).	`String ch = new String();`

`public String(String original)`
Initialise une chaîne contenant les mêmes caractères que la chaîne `original` donnée en argument.
	`String ch = new String("Bonjour le monde");`
	`String ch1 = new String(ch);`
	`ch1` contient une référence à la chaîne `Bonjour le monde`

`public String(char[] value)`
Construit une nouvelle chaîne à partir des caractères d'un tableau. Les modifications subséquentes effectuées dans le tableau n'affectent pas la chaîne obtenue.
	`char[] tableau = {'b', 'o', 'n', 'j', 'o', 'u', 'r'};`
	`String ch = new String(tableau);`
	`ch` contient une référence à la chaîne `bonjour`

`public String(char[] value, int offset, int count)`
Construit une chaîne de caractères à partir d'une partie d'un tableau de caractères (`value`). L'argument `offset` spécifie l'indice du premier caractère copié et `count` spécifie le nombre de caractères copiés dans la chaîne.
	`char[] tableau = {'b', 'o', 'n', 'j', 'o', 'u', 'r'};`
	`String ch = new String(tableau, 3, 4);`
	`ch` contient une référence à la chaîne `jour`

Farida Ait-Ammar
Nicolas Leduc

23

Obtenir une chaîne à partir de données de différents types

- La méthode *valueOf()* retourne la représentation, sous forme de chaîne de caractères, de l'argument passé en paramètre. L'argument peut être de type `int`, `long`, `float`, `double`, `char`, `boolean`, `char[]`, ou `Object`.
  - ```
    public static String valueOf(boolean b)
    public static String valueOf(char c)
    public static String valueOf(double d)
    public static String valueOf(float f)
    public static String valueOf(int i)
    public static String valueOf(long l)
    public static String valueOf(Object obj)
    public static String valueOf(char[] data)
    public static String valueOf(char[] data, int offset, int count)
    ```
- Exemples :

```
int age = 12;
String ageStr = String.valueOf(age);              // ageStr contient 12
boolean continuer = false;
String continuerStr = String.valueOf(continuer);  // continuerStr contient false
float note = 80.75f;
String noteStr = String.valueOf(note);            // noteStr contient 80.75
char reponse = 'N';
String reponseStr = String.valueOf(reponse);      // reponseStr contient N
char[] tabVoyelles = {'a', 'e', 'i', 'o', 'u', 'y'};
String voyellesStr = String.valueOf(tabVoyelles); // voyellesStr contient aeiouy
String voyStr = String.valueOf(tabVoyelles,2,3);  // voyStr contient iou
```

Farida Ait-Ammar
Nicolas Leduc

9.1.3 Comparaison

8

Longueur d'une chaîne

- **`public int length()`**
  - Retourne le nombre de caractères contenus dans la chaîne. Exemples :

```
String ch1 = "mars", ch2 = "";
int t1 = ch1.length();        // t1 contient la valeur 4
int t2 = ch2.length();        // t2 contient la valeur 0
int t3 = "abc".length();      // t3 contient la valeur 3
```

Attention! Pour connaître la taille d'un tableau, on utilise `tableau.length`, sans parenthèses, puisque `length` désigne un attribut public du tableau. Par contre, pour connaître la taille d'une chaîne de caractères, on utilise `chaine.length()`, avec des parenthèses, puisque `length()` est une méthode publique de la chaîne.

- **`public boolean isEmpty()`**
  - Retourne `true` si le nombre de caractères dans la chaîne est de zéro et `false` dans le cas contraire. Exemples :

```
String ch1 = "Allo", ch2 = "";
if(!ch1.isEmpty()) { ... } // Pour vérifier si ch1 contient au moins un caractère, ce qui est vrai.
if(ch2.isEmpty()) { ... }  // Pour vérifier si ch2 ne contient aucun caractère, ce qui est vrai.
```

  - Ces deux instructions sont équivalentes : `ch1.isEmpty()` et `ch1.lenght() == 0`
- **`public boolean isBlank()`**
  - Retourne `true` si la chaîne est vide ou si elle ne contient que des espaces blancs et `false` dans le cas contraire. Exemples :

```
String ch1 = " Allo ", ch2 = " \n \t ";
if(!ch1.isBlank()) { ... } // Pour vérifier si ch1 contient au moins un caractère, ce qui est vrai.
if(ch2.isBlank()) { ... }  // Pour vérifier si ch2 ne contient que des espaces, ce qui est vrai.
```

Farida Ait-Ammar
Nicolas Leduc
```

9

Égalité de deux chaînes (1/2)

- On ne doit pas utiliser l'opérateur `==`, ni `!=`, pour vérifier si deux chaînes de caractères sont égales, soit qu'elles contiennent les mêmes caractères, ou différentes.
  `if(chaine1 == chaine2)` ne compare pas les caractères contenus dans `chaine1` et `chaine2`, mais plutôt les adresses mémoire des objets pointés par les deux variables.
- Pour comparer si deux chaînes de caractères sont égales, on utilise la méthode `equals()` ou la méthode `equalsIgnoreCase()`. Cette dernière ne tient pas compte de la différence entre les majuscules et les minuscules.
- **`public boolean equals(Object anObject)`**
  `public boolean equalsIgnoreCase(String anotherString)`
  `public boolean contentEquals(CharSequence cs)`
  - Compare la chaîne utilisée pour appeler la méthode avec la chaîne passée en paramètre. Elle retourne `true` si les deux chaînes contiennent les mêmes caractères et `false` dans le cas contraire.
    `equals()` tient compte de la différence entre les majuscules et les minuscules.
    `equalsIgnoreCase()` ne tient pas compte de la différence entre les majuscules et les minuscules.
    `contentEquals()` comme `equals()`, mais s'assure que le paramètre est une chaîne de caractères.
- Exemples :

```
String ch1 = "ab"; String ch2 = "Ab"; String ch3 = "a" + "b";
if(ch1.equals(ch2)) { ... }             //false
if(ch1.equals(ch3)) { ... }             //true
if(ch1.equalsIgnoreCase(ch2)) { ... }   //true
if(ch1.equalsIgnoreCase(ch3)) { ... }   //true
```

Farida Ait-Ammar
Nicolas Leduc

10

Égalité de deux chaînes (2/2)

- Les littéraux de type `String` occupent une zone mémoire unique appelée le ***Pool des littéraux***. Par conséquent, deux chaînes créées avec les mêmes littéraux auront la même référence (même adresse mémoire). Exemple :

```
String ch1 = "ab";
String ch2 = "ab";
if (ch1 == ch2) { ... }  // est vrai
```

- Toutefois, il est préférable de comparer l'égalité des chaînes de caractères avec la méthode `equals()`.
- Exemple : pour comparer deux mots de passe, on utilise la méthode `equals()`, alors que pour comparer deux noms d'utilisateurs, on utilise la méthode `equalsIgnoreCase()`, puisqu'ils peuvent être écrits en majuscules et/ou en minuscules.

```
public static final String USER = "Admin", PASSWORD = "Q3Nb6Rt*";
public boolean estValide(String utilisateur, String motPasse) {
          return utilisateur.equalsIgnoreCase(USER) && motPasse.equals(PASSWORD);
}
```

Farida Ait-Ammar
Nicolas Leduc

11

Comparaison lexicographique (1/3)

- **La comparaison lexicographique** consiste à comparer deux chaînes, caractère par caractère, pour savoir laquelle des deux chaînes précède l'autre. La comparaison est basée sur la valeur *Unicode* de chaque caractère dans les deux chaînes. Ainsi les majuscules sont classées avant les minuscules, les chiffres avant les lettres, etc. Vous pouvez consulter la table *Unicode* en cliquant sur ce lien [redacted]. Par exemple, si on classe la liste de chaînes de caractères `Ab`, `ab`, `9a` et `Cd` selon l'ordre lexicographique, on obtient : `9a`, `Ab`, `Cd` et `ab`.

  Les méthodes `compareTo()` et `compareToIgnoreCase()` permettent de faire des comparaisons lexicographiques de deux chaînes.

- **`public int compareTo(String anotherString)`**
  `public int compareToIgnoreCase(String str)`
  - Compare la chaîne appelant la méthode avec celle passée en argument selon l'ordre lexicographique. Elle retourne :
    - un entier négatif (< 0) si la chaîne appelant arrive avant celle en argument
    - un entier positif (> 0) si la chaîne appelant arrive après celle en argument
    - 0 si les deux chaînes sont égales. On a alors `chaine1.equals(chaine2)` vaut `true`.
  - `compareTo()` tient compte de la différence entre les majuscules et les minuscules.
    `compareToIgnoreCase()` ne tient pas compte de la différence entre les majuscules et les minuscules.
- Exemples :

```
String ch1 = "artichaut", ch2 = "tomate", ch3 = "ar" + "tichaut", ch4 = "ARtichaut";
ch1.compareTo(ch2); // retourne un nombre négatif (-19 dans ce cas), car ch1 vient avant ch2
ch2.compareTo(ch1); // retourne un nombre positif (19 dans ce cas), car ch2 vient après ch1
ch1.compareTo(ch3); // retourne 0, car ch1 est identique à ch3
ch1.compareToIgnoreCase(ch4); // retourne 0, car ch1 est identique à ch4 si on ignore la
                              // différence entre les majuscules et les minuscules
ch1.compareTo(ch4); // retourne un nombre positif (32), car artichaut, vient après ARtichaut
```

Farida Ait-Ammar
Nicolas Leduc

12

Comparaison lexicographique (2/3)

- Il existe une certaine similitude entre la comparaison des chaînes de caractères et la comparaison des nombres, comme le montre le tableau ci-dessous.

Nombres	Chaînes	Remarques
`n1 < n2` `n1` est plus petit que `n2`	`str1.compareTo(str2) < 0` `str1` vient avant `str2`	• Le même opérateur d'infériorité < • Les opérandes dans le même ordre
`n1 > n2` `n1` est plus grand que `n2`	`str1.compareTo(str2) > 0` `str1` vient après `str2`	• Le même opérateur de supériorité > • Les opérandes dans le même ordre

`n1 == n2`
`n1` est égal à `n2`
	`str1.compareTo(str2) == 0`
	équivalent à
	`str1.equals(str2)` est `true`
	`str1` a les mêmes caractères que `str2`
		• Le même opérateur d'égalité
		• Ici l'ordre n'a pas d'importance

`n1 != n2`
`n1` différent de `n2`
	`str1.compareTo(str2) != 0`
	équivalent à
	`!str1.equals(str2)` est `true`
	`str1` n'a pas les mêmes caractères que `str2`
		• Le même opérateur d'inégalité
		• Ici l'ordre n'a pas d'importance

Farida Ait-Ammar
Nicolas Leduc

13

Comparaison lexicographique (3/3)

Exemple :

- La fonction `ordreStr()`, ci-dessous, détermine l'ordre de trois chaînes de caractères. Elle retourne la chaîne `croissant` si elles sont dans l'ordre croissant, `décroissant` si elles sont dans l'ordre décroissant, `constant` si les trois chaînes sont égales et `quelconque` dans les autres cas.

```
public static String ordreStr(String ch1, String ch2, String ch3) {
     String ordre = "quelconque";
     if (ch1.compareTo(ch2) <= 0 && ch2.compareTo(ch3) <= 0) { ordre = "croissant"; }
     if (ch1.compareTo(ch2) >= 0 && ch2.compareTo(ch3) >= 0) { ordre = "décroissant"; }
     // peut s'écrire if (ch1.equals(ch2) && ch2.equals(ch3)) {
     if (ch1.compareTo(ch2) == 0 && ch2.compareTo(ch3) == 0) { ordre = "constant"; }
     return ordre;
}
```

- Un exemple d'appel de la fonction `ordreStr()` :

```
public static void main(String[] args) {
     Scanner clavier = new Scanner(System.in);
     String mot1, mot2, mot3;
     System.out.print("Entrez trois mots séparés par des espaces : ");
     mot1 = clavier.next();   mot2 = clavier.next();   mot3 = clavier.next();
     System.out.println("L'ordre des mots est " + ordreStr(mot1, mot2, mot3));
}
```

- Le résultat de l'exécution au terminal :

```
Entrez trois mots séparés par des espaces : arbre fleur rose
L'ordre des mots est croissant
```

Farida Ait-Ammar
Nicolas Leduc

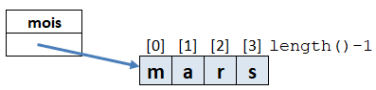
9.1.4 Recherche de caractère

14

Recherche de caractères (1/5)

- Les caractères d'une chaîne sont indexés. Les indices vont de `0` à `length()-1`, comme pour les tableaux.
Exemple :

```
String mois = "mars";
```



- Ces indices permettent aux méthodes de localiser un caractère ou une suite de caractères dans une chaîne.
  - Exemple : `mois.charAt(0)` retourne le caractère `m` et `mois.charAt(3)` retourne le caractère `s`
- La classe `String` possède plusieurs méthodes permettant de localiser un caractère ou une suite de caractères dans une chaîne :

`charAt()`	Extraire un caractère connaissant sa position
`substring()`	Extraire une sous-chaîne à partir d'un index
`indexOf()`	Rechercher la première occurrence d'une sous-chaîne ou d'un caractère
`lastIndexOf()`	Rechercher la dernière occurrence d'une sous-chaîne ou d'un caractère
`contains()`	Savoir si une chaîne contient une sous-chaîne
`startsWith()`	Savoir si une chaîne débute par une suite de caractères
`endsWith()`	Savoir si une chaîne se termine par une suite de caractères

- Ces méthodes sont détaillées dans les diapositives suivantes.

Farida Ait-Ammar
Nicolas Leduc

15

Recherche de caractères (2/5)

- Extraire un caractère connaissant sa position :
- **public char charAt(int index)**
  - Retourne le caractère à la position *index* de la chaîne. Si `index` n'existe pas dans la chaîne, la méthode lance une exception de type `StringIndexOutOfBoundsException`.
  - Exemple :

```
String ch = "Bonjour";
char c1 = ch.charAt(0);       // c1 contient B
char c2 = ch.charAt(6);       // c2 contient r
char c3 = ch.charAt(10);      // lance une exception de type StringIndexOutOfBoundsException.
```

- Extraire un tableau de caractères :
- **public char[] toCharArray()**
  public void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)
  - Permet de transformer la chaîne ou une sous chaîne en un tableau de caractères.
    `toCharArray()` transforme toute la chaîne en un tableau de caractères.
    `getChars()` extrait une sous chaîne débutant à la position `srcBegin` et se terminant à `srcEnd`, non inclus. Les caractères résultants sont placés dans le tableau `dst` à partir de la position `dstBegin`.
  - Exemple :

```
String ch1 = "Bonjour";
char[] chTab = {'T','o','m','e'};
char[] chTab1 = ch1.toCharArray();      // chTab1 contient {'B','o','n','j','o','u','r'}
ch1.getChars(4, 7, chTab, 1);           // chTab contient {'T','o','u','r'}
```


16

Recherche de caractères (3/5)

- Extraire une sous-chaîne à partir d'un indice :

[0]	[1]	[2]	[3]	[4]	[5]	[6]
b	o	n	j	o	u	r

- **public String substring(int beginIndex)**
  public String substring(int beginIndex, int endIndex)
  - Retourne une sous-chaîne composée des caractères de la chaîne à partir de l'indice *beginIndex* jusqu'à `endIndex-1` ou jusqu'au dernier caractère si `endIndex` n'est pas indiqué. Ainsi, le caractère à la position `endIndex` est exclu.
  - Lance une exception de type `StringIndexOutOfBoundException` si
    - *beginIndex* est négatif ou s'il est plus grand que la longueur de la chaîne.
    - `endIndex` est plus grand que la longueur de la chaîne ou plus petit que `beginIndex`.
  - Exemples :

```
String mot = "Bonjour";
String mot1 = mot.substring(3);          // mot1 contient jour
String mot2 = mot.substring(7);          // mot2 contient "", 7 est égal à mot.length()
String mot3 = "Au revoir".substring(3); // mot3 contient revoir
mot.substring(8);                        // Exception StringIndexOutOfBoundsException
String mot4 = mot.substring(1, 3);       // mot4 contient on. j à l'indice 3 est exclu
String mot5 = mot.substring(2, 8);       // Exception, car endIndex > mot.length()
"Allo".substring(3, 2);                  // Exception, car beginIndex > endIndex
"Allo".substring(-1, 2);                 // Exception, car beginIndex est négatif
```

17

Recherche de caractères (4/5)

[0]	[1]	[2]	[3]	[4]	[5]
b	o	n	b	o	n

- Rechercher la première (ou dernière) occurrence d'une sous-chaîne :
- **`public int indexOf(int ch)`**
  `public int indexOf(int ch, int fromIndex)`
  `public int lastIndexOf(int ch)`
  `public int lastIndexOf(int ch, int fromIndex)`
  `public int indexOf(String str)`
  `public int indexOf(String str, int fromIndex)`
  `public int lastIndexOf(String str)`
  `public int lastIndexOf(String str, int fromIndex)`
  - Retourne la position dans la chaîne utilisée lors de l'appel de la méthode de la première (`indexOf()`) ou de la dernière (`lastIndexOf()`) occurrence de la sous-chaîne ou du caractère passé en argument. On peut aussi indiquer la position de départ pour la recherche (`fromIndex`). Si aucune occurrence n'est trouvée, la méthode retourne `-1`.
  - Le paramètre `ch` est un identifiant numérique *Unicode* (`0x0000` à `0xFFFF` inclus). Il représente un seul caractère que l'on peut noter de plusieurs façons. Par exemple, le caractère `o` a comme identifiant *Unicode* `u006F` ou `111` en décimal. Il peut être noté `'o'`, `'\u006F'` ou `111`. On peut ainsi écrire `str.indexOf('o')`, `str.indexOf('\u006F')` ou `str.indexOf(111)`.
  - Exemples :

```
String ch = "bonbon";
int n1 = ch.indexOf('n');          // n1 contient 2.  Indice du premier n.
int n2 = ch.indexOf("ont");        // n2 contient -1. ont est introuvable.
int n3 = ch.indexOf('o', 2);       // n3 contient 4.  Repère o à partir de l'indice 2.
int n4 = ch.indexOf("bon", 1);     // n4 contient 3.  Repère bon à partir de l'indice 1.
int n5 = ch.lastIndexOf('w');      // n5 contient -1. w est introuvable.
int n6 = ch.lastIndexOf("on");     // n6 contient 4.  Indice du dernier on.
int n7 = ch.lastIndexOf('n', 1);   // n7 contient -1. Aucun n à l'indice 1 ou précédents.
int n8 = ch.lastIndexOf("on", 3);  // n8 contient 1.  Indice du dernier on avant l'indice 3.
```

Farida Ait-Ammar
Nicolas Leduc

18

Recherche de caractères (5/5)

- Savoir si une chaîne contient une sous-chaîne :
- **`public boolean contains(CharSequence s)`**
  - Retourne `true` si et seulement si la chaîne contient la séquence de caractères `s` et `false` dans le cas contraire.
  - Exemple :

```
String ch1 = "Bonjour", ch2 = "jour";
if (ch1.contains(ch2)) { … }       // est vrai
if (ch1.contains("bon")) { … }     // est faux, car ch1 contient Bon mais pas bon
```

- Savoir si une chaîne débute ou se termine par une suite de caractères :
- **`public boolean startsWith(String prefix)`**
  `public boolean startsWith(String prefix, int toffset)`
  `public boolean endsWith(String suffix)`
  - Teste si la chaîne appelant la méthode débute (`startsWith()`) par `prefix` ou se termine (`endsWith()`) par `suffix`. Il est aussi possible d'indiquer un décalage (`toffset`) pour le cas de la recherche du préfixe de la chaîne.
  - Exemple : Le programme ci-dessous lit une chaîne de caractères qui représente un titre *HTML* de niveau 1 et indique si la chaîne est bien formatée ou non. Un titre de niveau 1 est bien formaté s'il commence par `<h1>` et se termine par `</h1>`. Par exemple, `<h1> Le langage de programmation Java </h1>` est bien formaté.

```
public static void main(String args[]) {
     Scanner clavier = new Scanner(System.in);
     System.out.print("Entrez un titre de niveau 1 : ");
     String ligne = clavier.nextLine();
     if (ligne.startsWith("<h1>") && ligne.endsWith("</h1>")) {
          System.out.println("Le titre est bien formaté.");
     } else {
          System.out.println("Le titre n'est pas bien formaté.");
     }
}
```

Farida Ait-Ammar
Nicolas Leduc

9.1.5 Transformation de chaîne

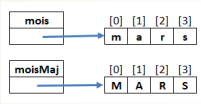
19

Méthodes de transformation de chaînes de caractères (1/4)

- Les méthodes de transformation de chaînes de caractères ne modifient pas la chaîne originale, puisque les chaînes de caractères sont immuables (non modifiables). Elles retournent donc une nouvelle chaîne.
- Exemples :

```
String mois = "mars";
String moisMaj = mois.toUpperCase(); // moisMaj contient une référence à MARS
```

L'instruction `mois.toUpperCase()` ne modifie pas la chaîne `mars`, mais crée plutôt une nouvelle chaîne contenant `MARS`.



- La classe `String` possède plusieurs méthodes de transformation de chaînes :

`strip()`	Supprimer les blancs
`toLowerCase()`	Convertir les caractères en minuscule
`toUpperCase()`	Convertir les caractères en majuscule
`replace()`	Remplacer des caractères
`split()`	Décomposer une chaîne en un tableau de chaînes de caractères
`repeat()`	Concaténer des copies de la chaîne

- Ces méthodes sont détaillées dans les diapositives suivantes.

Farida Ait-Ammar
Nicolas Leduc

20

Méthodes de transformation de chaînes de caractères (2/4)

- Suppression des blancs au début et/ou à la fin de la chaîne :
- **`public String trim()`**
  `public String strip()`
  `public String stripLeading()`
  `public String stripTrailing()`
  - Retourne une copie de la chaîne sans les blancs de début et/ou de fin. Les méthodes `strip*()` éliminent les espaces blancs définis par *Unicode* alors que `trim()` efface les caractères dont la valeur *Unicode* est inférieure ou égale au caractère d'espacement (`\u0020`). Les espaces blancs incluent, entre autres, le caractère de tabulation horizontale (`\t`), le caractère de changement de ligne/nouvelle ligne (`\n`) et le caractère espace (' ').
    `trim()` Efface les caractères <= `\u0020` en début et en fin de chaîne incluant les caractères de contrôle
    `strip()` Efface les espaces *Unicode* en début et en fin de chaîne
    `stripLeading()` Efface les espaces *Unicode* uniquement en début de chaîne
    `stripTrailing()` Efface les espaces *Unicode* uniquement en fin de chaîne
  - Exemples :

```
String mot = "\n Au revoir \t";
String mot1 = mot.strip(); // mot1 contient "Au revoir". Les blancs (\t, \n et ' ') sont retirés.
String mot2 = "  Allo  ".stripLeading(); // mot2 contient "Allo  ".
```

- Passages en majuscules ou en minuscules :
- **`public String toLowerCase()`**
  `public String toUpperCase()`
  - Crée une nouvelle chaîne en remplaçant les caractères majuscules par les caractères minuscules équivalents (`toLowerCase()`) ou l'inverse (`toUpperCase()`).
  - Exemples :

```
String ch = "NOM et Prénom";
String chMin = ch.toLowerCase(); // chMin contient nom et prénom
String chMaj = ch.toUpperCase(); // chMaj contient NOM ET PRÉNOM
```

Farida Ait-Ammar
Nicolas Leduc

21 Méthodes de transformation de chaînes de caractères (3/4)

- Remplacement de caractères :
- **`public String replace(char oldChar, char newChar)`**
  `public String replace(CharSequence target, CharSequence replacement)`
  - Crée une chaîne en remplaçant **toutes** les occurrences du caractère `oldChar` ou de la chaîne `target` par le caractère `newChar` ou la chaîne `replacement` respectivement.
  - Exemples :

```
String ch = "a; b; c; d; e";
String ch1 = ch.replace(';', ',');          // ch1 contient a, b, c, d, e
String ch2 = "bonbon".replace("on", "ang"); // ch2 contient bangbang
```

- Remplacement de caractères à l'aide d'une expression régulière :
- **`public String replaceFirst(String regex, String replacement)`**
  `public String replaceAll(String regex, String replacement)`
  - Crée une chaîne en remplaçant la première (`replaceFirst()`) ou toutes (`replaceAll()`) les occurrences de l'expression régulière `regex` par la chaîne `replacement`. Dans les expressions régulières, que nous verrons par la suite, certains caractères, comme `[`, `]`, `+` et `*`, ont une signification particulière. Il faut donc être prudent lors de l'utilisation de ces fonctions, car le résultat pourrait être surprenant.
  - Exemples :

```
String ch = "bonbon";
String ch1 = ch.replaceFirst("bon", "Ga");              // ch1 contient Gabon
String ch2 = ch.replaceAll("on", "ang");                // ch2 contient bangbang
String ch3 = "b[on]b[on]".replaceAll("[on]", "on");     // ch3 contient b[onon]b[onon]
String ch4 = "b[on]b[on]".replaceAll("\\[on\\]", "on"); // ch4 contient bonbon
```

Farida Ait-Ammar
Nicolas Leduc

22 Méthodes de transformation de chaînes de caractères (4/4)

- Décomposer une chaîne en un tableau de chaînes de caractères :
- **`public String[] split(String regex)`**
  `public String[] split(String regex, int limit)`
  - Découpe la chaîne en fonction d'un séparateur donné (`regex`) et retourne un tableau de chaînes de caractères. Le nombre maximal de chaînes extraites peut être spécifié (`limit`). Le séparateur est une expression régulière représentant une chaîne de caractères. Pour l'instant, nous considérons le séparateur comme une chaîne composée d'un seul caractère.
  - Exemples :

```
String phrase = "Le langage de programmation Java";
String[] mots = phrase.split(" ");          // Récupère les sous-chaînes séparées par une espace.
// mots contient le tableau : {"Le", "langage", "de", "programmation", "Java"}
String[] mots2 = "Samuel:Jean:25:7:2002".split(":", 3); // Limite la taille du tableau à 3 cases.
// mots2 contient {"Samuel", "Jean", "25:7:2002"}.
// Les deux premières cases contiennent chacune un mot et la troisième, le reste de la chaîne.
```

- Combiner des chaînes à l'aide d'un séparateur :
- **`public static String join(CharSequence delimiter, CharSequence ... elements)`**
  - Concatène une liste de chaînes en ajoutant le séparateur `delimiter` entre chacune des chaînes. La fonction peut recevoir, en paramètre, un tableau de chaînes ou une liste de chaînes séparées par des virgules. Elle fait l'inverse de `split()`.
  - Exemples :

```
String[] tabCh = {"Samuel", "Jean", "25", "7", "2002"};
String ch1 = String.join(":", tabCh);           // ch1 contient Samuel:Jean:25:7:2002
String ch2 = String.join("-", "A", "BC", "D");  // ch2 contient A-BC-D
```

- Concaténer des copies de la chaîne :
- **`public String repeat(int count)`**
  - Concatène un certain nombre de copies (`count`) de la chaîne et retourne la chaîne ainsi créée.
  - Exemples :

```
String ch = "abc";
String ch1 = ch.repeat(3);    // ch1 contient abcabcabc
String ch2 = "*".repeat(5);   // ch2 contient *****
```

Farida Ait-Ammar
Nicolas Leduc

24

Classe `String` - Exemple

- La méthode `plurielMotOu()` suivante reçoit, en paramètre, un mot qui se termine par le suffixe *ou*. Elle retourne une chaîne de caractères représentant le pluriel de ce mot. Par exemple, le pluriel de *bijou* est *bijoux*.
- Les mots en *ou* prennent un *s* au pluriel à l'exception des mots suivants, qui prennent un *x* : bijou, hibou, caillou, chou, genou, pou et joujou.

```
public static String plurielMotOu(String mot) {
    String exceptions = " bijou hibou caillou chou genou pou joujou ";
    mot = mot.toLowerCase();
    String motPluriel = "";
    if (mot.endsWith("ou")) {
        if (exceptions.indexOf(" " + mot + " ") != -1) {
            motPluriel = mot + 'x';
        } else {
            motPluriel = mot + 's';
        }
    }
    return motPluriel;
}
```

- Exemple d'appel de la méthode `plurielMotOu()` :

```
public static void main(String[] args) {
    String mot1 = "caillou";
    System.out.println(plurielMotOu(mot1)); // Affiche cailloux
}
```

Farida Ait-Ammar
Nicolas Leduc

9.2 Classe `StringBuilder`

9.2.1 `StringBuffer` vs `StringBuilder`

26

Classes `StringBuffer` et `StringBuilder`

- Nous avons appris précédemment que les objets de type `String` sont immuables (non modifiables). Par conséquent, la moindre modification d'une chaîne ne peut se faire qu'en créant une nouvelle chaîne.
- Par exemple, chaque concaténation à l'aide de l'opérateur `+` crée une nouvelle chaîne, ce qui engendre un gaspillage du temps d'exécution (*CPU*) et de l'espace mémoire.

```
String ch = "Bonjour ";   // Crée une chaîne contenant "Bonjour "
ch += "tout ";            // Crée une autre chaîne contenant "Bonjour tout "
ch += "le monde";         // Crée une autre chaîne contenant "Bonjour tout le monde"
```

- Dans les programmes qui manipulent intensivement des chaînes de caractères, la perte de temps et d'espace mémoire qui en résulte peut devenir gênante.
- La solution consiste à utiliser des objets de la classe `StringBuffer` ou `StringBuilder`, qui offrent un moyen de construire et de modifier une chaîne de caractères de manière beaucoup plus efficace.
- On utilise des méthodes telles que `append()`, `insert()` et `reverse()`, qui ne sont pas disponibles dans la classe `String`, pour construire une chaîne temporaire. Une fois cette dernière construite, on obtient une instance de la classe `String` en appelant la méthode `toString()`.
- Exemple : concaténation de chaînes en utilisant un objet de la classe `StringBuilder`.

```
String ch;
StringBuilder builder = new StringBuilder("Bonjour ");  // Créer une chaîne temporaire
builder.append("tout ");  // append() permet de faire des ajouts à la fin de la chaîne
builder.append("le monde");
ch = builder.toString();  // Convertir la chaîne temporaire en String
```

Farida Ait-Ammar
Nicolas Leduc

27

Classes `StringBuffer` et `StringBuilder`

- Pour manipuler des chaînes de caractères, les objets de type `String` doivent toujours être utilisés, sauf si les objets de type `StringBuilder` ou `StringBuffer` offrent un avantage au niveau de la **simplification du code** ou de la **performance**, comme lors de la concaténation d'un grand nombre de chaînes, qui consomme trop de ressources.

- Étapes à suivre pour travailler avec une chaîne temporaire de type `StringBuilder` ou `StringBuffer` :
  1. Créer la chaîne temporaire
     ```
     StringBuilder builder = new StringBuilder();
                    OU
     StringBuffer buffer = new StringBuffer();
     ```
  2. Apporter les modifications à la chaîne temporaire `builder` ou `buffer`
  3. Convertir finalement la chaîne temporaire en `String` avec la méthode `toString()`
     ```
     String chaine = builder.toString();
     String chaine = buffer.toString();
     ```

Farida Ait-Ammar
Nicolas Leduc

28

Classe `StringBuffer` vs `StringBuilder`

- Les classes `StringBuffer` et `StringBuilder` sont équivalentes. Elles offrent les mêmes méthodes.
- La principale différence entre ces deux classes est que les objets de type `StringBuffer` sont synchronisés (*thread safe*), alors que ceux de type `StringBuilder` ne sont pas synchronisés.
- Un objet synchronisé ne permet qu'à un seul processus à la fois d'y accéder. Par exemple, si un programme utilise plusieurs processus simultanément, un processus ne peut pas modifier un objet synchronisé pendant qu'un autre processus est en train de lire ce même objet. Ce mécanisme permet d'éviter que des objets se retrouvent dans un état d'incohérence.
- La classe `StringBuilder` offre une meilleure performance que la classe `StirngBuffer`. Les méthodes de la classe `StringBuilder` sont plus rapides que celles de la classe `StringBuffer` puisqu'elles ne sont pas synchronisées, mais elles ne sont pas *thread safe*.
- Par défaut, on doit privilégier `StringBuilder`. Toutefois, si la variable concernée est en concurrence d'accès (accédée par plusieurs processus simultanément), alors on doit utiliser `StringBuffer`.
- Tout comme la classe `String`, les classes `StringBuffer` et `StringBuilder` se trouvent dans le package `java.lang` et ne requièrent aucun `import`.

Farida Ait-Ammar
Nicolas Leduc

9.2.2 Constructeur

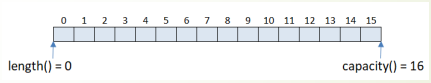
30 Classe `StringBuilder`

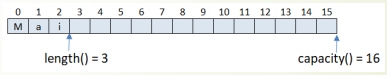
- Les objets de type `StringBuilder` sont similaires aux objets de type `String`, sauf qu'ils peuvent être modifiés.
- À l'interne, ces objets sont traités comme des tableaux de longueurs variables, qui contiennent une séquence de caractères. À tout moment, la longueur et le contenu de la séquence peuvent être modifiés par des appels de méthodes.
- Longueur et capacité :
  - Un objet `StringBuilder` a une longueur tout comme un objet `String`, mais également une capacité.
  - La **longueur** correspond au nombre de caractères contenus dans la chaîne qu'il encapsule. Elle est donnée par la méthode `length()`, comme pour un objet `String`.
  - La **capacité** est le nombre d'espaces mémoire qui ont été alloués pour stocker les caractères. Autrement dit, c'est le nombre de caractères maximum qu'un objet peut contenir. Elle est donnée par la méthode `capacity()`. Sa valeur est toujours plus grande ou égale à `length()`.
  - Certaines opérations, comme `append()` et `insert()`, peuvent faire augmenter la longueur de la séquence de caractères dans l'objet `StringBuilder` de sorte que la longueur devient supérieure à la capacité courante. Lorsque cela se produit, la capacité est automatiquement augmentée par l'allocation d'une nouvelle zone de mémoire.
- Exemple :

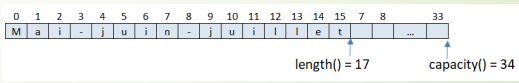
```
// Construit une chaîne de caractères vide de 16 espaces.
StringBuilder builder = new StringBuilder();

// Ajoute 3 caractères à la chaîne. La longueur devient 3.
// La capacité est toujours 16.
builder.append("Mai");

// Ajoute 14 caractères à la chaîne. La longueur devient 17.
// La capacité s'est élargie, elle devient 34 (2*16+2)
builder.append("-juin-juillet-");
```







Farida Ait-Ammar
Nicolas Leduc

31 Constructeurs de `StringBuilder`

Constructeur	Exemple
`StringBuilder()` Crée une chaîne `StringBuilder` vide d'une capacité maximale de 16 caractères.	`StringBuilder builder = new StringBuilder();`

`StringBuilder(String str)`
Crée une chaîne `StringBuilder` contenant la séquence de caractères de `str` plus 16 espaces mémoire vides.
	`String ch = "Mai";`
	`StringBuilder sb1 = new StringBuilder(ch);`
	`StringBuilder sb2 = new StringBuilder("Bonjour");`
	
	`sb1` contient `Mai`
	`sb1` a une capacité de 3 + 16 espaces mémoire
	`sb2` contient `Bonjour`
	`sb2` a une capacité de 7 + 16 espaces mémoire.

`StringBuilder(int capacity)` Crée une chaîne `StringBuilder` vide avec une capacité initiale spécifiée par `capacity`.	`StringBuilder builder = new StringBuilder(50);` `builder` a une capacité de 50 espaces mémoire

Farida Ait-Ammar
Nicolas Leduc

9.2.3 Transformation de chaîne

32

Méthodes de `StringBuilder` (1/4)

- Les méthodes suivantes de la classe `StringBuilder` fonctionnent comme celles de la classe `String` :
  - `length()`, `charAt()`, `getChars()`, `indexOf()`, `lastIndexOf()`, `substring()` et `compareTo()`.
- Autres méthodes inexistantes dans la classe `String` :

Méthode	Exemple

`String toString()`
Retourne un objet `String` qui représente la séquence des caractères contenus dans le `StringBuilder`.
	`String ch = "";`
	`StringBuilder builder = new StringBuilder("ab");`
	`builder.append('-').append("cd").append('-').append("ef");`
	`// builder contient ab-cd-ef`
	`ch = builder.toString(); // Convertit builder en String`

`StringBuilder append(valeur)`
Convertit *`valeur`* en chaîne, puis ajoute les caractères de cette chaîne à la fin du `StringBuilder`.
Cette méthode est surchargée, ainsi l'argument `valeur` peut être de type: `String`, `char`, `int`, `long`, `double`, `float`, `boolean`, `char[]` ou `Object`.
`StringBuilder append(String str)`
`StringBuilder append(char c)`
`StringBuilder append(int i)`
…
Note : On ne peut pas utiliser l'opérateur `+` pour la concaténation, comme pour les `String`. La méthode `append()` est plus performante que l'opérateur `+`.
	`String ch = "abc", resultat = "";`
	`char car = '$';`
	`int nb = 12;`
	`StringBuilder builder = new StringBuilder();`
	`builder.append(ch);  // builder contient abc`
	`builder.append(car);  // builder contient abc$`
	`builder.append(nb);  // builder contient abc$12`
	`resultat = builder.toString(); // Convertit builder en String`

Farida Ait-Ammar
Nicolas Leduc

33

Méthodes de `StringBuilder` (2/4)

Méthode	Exemple

`StringBuilder insert(int offset, valeur)`
Convertit *`valeur`* en chaîne, puis insère les caractères de cette chaîne à la position `offset` de ce `StringBuilder`.
Cette méthode est surchargée, ainsi l'argument `valeur` peut être de type: `String`, `char`, `int`, `long`, `double`, `float`, `boolean`, `char[]` ou `Object`.
`StringBuilder insert(int offset, String str)`
`StringBuilder insert(int offset, boolean b)`
`StringBuilder insert(int offset, char c)`
…
	`String cours = "ZF5MO";`
	`StringBuilder builder = new StringBuilder(cours);`
	`builder.insert(3, '-');  // builder contient ZF5-MO`
	`builder.insert(0, "420-"); // builder contient 420-ZF5-MO`
	`cours = builder.toString();// Convertit builder en String`

`StringBuilder replace(int start, int end, String str)`
Remplace la sous-séquence de caractères comprise entre l'indice `start` et `end-1` par la séquence `str` dans ce `StringBuilder`.
• Si l'indice `end-1` n'existe pas, ce sont tous les caractères compris entre l'indice `start` et le dernier indice qui seront remplacés.
• La méthode lance une exception `StringIndexOutOfBoundsException` si `start` est négatif, supérieur à `length()` ou supérieur à `end`.
	`String cours = "420-ZF5-MO";`
	`StringBuilder builder = new StringBuilder(cours);`
	`builder.replace(4, 7, "AG4"); // Remplace ZF5 par AG4`
	`// builder contient 420-AG4-MO`
	`cours = builder.toString();// Convertit builder en String`

Farida Ait-Ammar
Nicolas Leduc

34 Méthodes de `StringBuilder` (3/4)

Méthode	Exemple

`void setCharAt(int index, char ch)`
Modifie le caractère qui se trouve à l'indice `index` dans ce `StringBuilder` par le caractère *ch*.
La méthode lance une exception `IndexOutOfBoundsException` si `index` est négatif ou supérieur ou égal à `length()`.
	`String cours = "420-ZF5";`
	`StringBuilder builder = new StringBuilder(cours);`
	`builder.setCharAt(3, ' ');`
	`// Remplace '-' par ' '. builder contient 420 ZF5`
	`cours = builder.toString(); // Convertit en String`

`StringBuilder delete(int start, int end)`
Supprime la sous-séquence de caractères comprise entre l'indice `start` et `end-1` dans ce `StringBuilder`.
• Si l'indice `end-1` n'existe pas, elle supprime la séquence jusqu'à la fin.
• Si `start` est égal à `end`, aucune modification n'est apportée.
• La méthode lance une exception `StringIndexOutOfBounds` si `start` est négatif, supérieur à `length()` ou supérieur à `end`.
	`String telephone = "1-450-223-0567";`
	`StringBuilder builder = new StringBuilder(telephone);`
	`builder.delete(0, 2);`
	`// Supprime 1-. builder contient 450-223-0567`
	`telephone = builder.toString(); // Convertit en String`

`StringBuilder deleteCharAt(int index)`
Supprime le caractère situé à l'indice `index` de ce `StringBuilder`.
La méthode lance une exception `StringIndexOutOfBoundsException` si `index` est négatif ou supérieur ou égal à `length()`.
	`String cours = "420-ZF5";`
	`StringBuilder builder = new StringBuilder(cours);`
	`builder.deleteCharAt(3);`
	`// Supprime le caractère -. builder contient 420ZF5`
	`cours = builder.toString(); // Convertit en String`

Farida Ait-Ammar
Nicolas Leduc

35 Méthodes de `StringBuilder` (4/4)

Méthode	Exemple

`StringBuilder reverse()`
Inverse la séquence de caractères dans ce `StringBuilder`.
	`String cours = "420-ZF5";`
	`StringBuilder builder = new StringBuilder(cours);`
	`builder.reverse();  // builder contient 5FZ-024`
	`cours = builder.toString(); // Convertit en String`

`void setLength(int newLength)`
Modifie la longueur de la séquence de caractères.
• Si `newLength` est inférieur à `length()`, les derniers caractères de la séquence sont tronqués.
• Si `newLength` est supérieur à `length()`, des caractères nuls (`\0`) sont ajoutés à la fin de la séquence de caractères.
	`String cours = "420-ZF5";`
	`StringBuilder builder = new StringBuilder(cours);`
	`int longueur = builder.length();`
	`builder.setLength(longueur + 10);`
	`// Ajoute 10 caractères \0 à la fin`
	`cours = builder.toString(); // Convertit en String`
	`System.out.println(cours.length()); // Affiche 17`

Farida Ait-Ammar
Nicolas Leduc

9.2.4 Comparaison

36

Égalité de `StringBuilder` - Problème

- Pour vérifier si deux `StringBuilder` sont égaux, il n'est pas possible d'utiliser l'opérateur `==`, car ce sont des objets. On pourrait penser utiliser la méthode `equals()`, comme pour les `String`, mais celle-ci n'est pas implémentée pour les `StringBuilder`. Ainsi appeler `equals()` équivaut à utiliser l'opérateur `==` dans ce cas.
- Pour comparer correctement les `StringBuilder`, il faut, en fait, comparer leurs représentations en chaînes de caractères en faisant appel à `toString()`.
- De plus, il n'est pas possible d'utiliser `Arrays.equals()` pour comparer des tableaux de `StringBuilder`, car cette méthode utilise `equals()` pour vérifier l'égalité de chacun des éléments des tableaux. Encore une fois, il est nécessaire de faire nous-même la comparaison manuellement.
- Afin de faire correctement le travail, il y a plusieurs cas à traiter, par exemple, il faut s'assurer que les objets ne sont pas `null` avant d'appeler `toString()`, il faut s'assurer que les deux tableaux ont la même taille, etc.
- La page suivante contient une implémentation de deux méthodes qui traitent les différents cas possibles pour la comparaison de `StringBuilder` et de `StringBuilder[]`.

Farida Ait-Ammar
Nicolas Leduc

37

Égalité de `StringBuilder` - Méthodes

- Méthode qui vérifie l'égalité de deux `StringBuilder`.

```
boolean equalsStringBuilder(StringBuilder sb1, StringBuilder sb2) {
    boolean egal = false;
    if (sb1 == sb2) {
        egal = true;
    } else if (sb1 != null && sb2 != null) {
        egal = sb1.toString().equals(sb2.toString());
    }
    return egal;
}
```

- Méthode qui vérifie l'égalité de deux tableaux de `StringBuilder`. Elle fait appel à la méthode précédente.

```
boolean equalsStringBuilderArray(StringBuilder[] sba1, StringBuilder[] sba2) {
    boolean egal = false;
    if (sba1 == sba2) {
        egal = true;
    } else if (sba1 != null && sba2 != null) {
        egal = sba1.length == sba2.length;
        int i = 0;
        while (egal && i < sba1.length) {
            egal = egal && equalsStringBuilder(sba1[i], sba2[i]);
            i++;
        }
    }
    return egal;
}
```

Farida Ait-Ammar
Nicolas Leduc

9.2.5 Exemple

38

Classe `StringBuilder` – Exemple palindrome

- Une chaîne de caractères est un palindrome si l'ordre des lettres reste le même qu'on la lise de gauche à droite ou de droite à gauche. Par exemple, *Laval* est un palindrome. La méthode `estPalindrome()`, ci-dessous, reçoit en paramètre une chaîne de caractères et retourne `true` si cette chaîne est un palindrome ou `false` dans le cas contraire.

```
public static boolean estPalindrome(String ch) {
    StringBuilder builder = new StringBuilder(ch);
    builder.reverse();
    String inverse = builder.toString();
    return ch.equalsIgnoreCase(inverse);
}
```

- Voici un exemple d'appel de la méthode `estPalindrome()`

```
public static void main(String[] args) {
    Scanner clavier = new Scanner(System.in);
    String chaine = "";
    System.out.print("Entrez un mot: ");
    chaine = clavier.next();
    if (estPalindrome(chaine)) {
        System.out.println(chaine + " est un palindrome");
    } else {
        System.out.println(chaine + " n'est pas un palindrome");
    }
}
```

Terminal :

```
Entrez un mot: Laval
Laval est un palindrome

Entrez un mot: Allo
Allo n'est pas un palindrome
```

Farida Ait-Ammar
Nicolas Leduc

39

Classe `StringBuilder` – Exemple assurance maladie

- Le numéro d'assurance maladie est unique. Il comprend les renseignements suivants :
  - Trois premières lettres du nom de famille
  - Première lettre du prénom
  - Deux derniers chiffres de l'année de naissance
  - Mois de naissance auquel on additionne 50 pour indiquer le sexe féminin
  - Jour de naissance
  - Code administratif de deux chiffres à l'usage de la RAMQ
  - Exemples :
  - Félix Bouchard, de sexe masculin (M), né le 1994/02/14 avec un code administratif de 19 aura le numéro d'assurance maladie : *BOUF 9402 1419*
  - France Boucher, de sexe féminin (F), née le 1994/02/14 avec un code administratif de 20 aura le numéro d'assurance maladie : *BOUF 9452 1420*
- On veut écrire une méthode qui reçoit, en paramètre, une chaîne de caractères contenant les informations sur un individu. Le traitement de cette méthode consiste à construire un numéro d'assurance maladie à partir des informations contenues dans la chaîne reçue. La nouvelle chaîne est retournée et celle originale n'est pas modifiée.
- La chaîne de caractères contient les informations séparées par le caractère `:` . Le format de la chaîne est : `prénom:nom:année/mois/jour:codeAdministratif:sexe`
  - Exemples :
  - Chaîne de caractères : `Félix:Bouchard:1994/02/14:19:M`
  - Numéro d'assurance maladie : `BOUF 9402 1419`
  - Chaîne de caractères : `France:Boucher:1994/02/14:20:F`
  - Numéro d'assurance maladie : `BOUF 9452 1420`

Farida Ait-Ammar
Nicolas Leduc

40 Classe `StringBuilder` – Exemple assurance maladie

```
public static String construireNumeroMaladie(String chaineInfo) {
     // Format de chaineInfo : prénom:nom:année/mois/jour:codeAdministratif:sexe
     int moisNaissance = 0;
     String[] tabInfos = chaineInfo.split(":");                 // Décomposer chaineInfo avec le délimiteur :
          // tabInfos[] = {prénom, nom, date de naissance (année/mois/jour), code administratif, sexe}
     String[] tabNaissance = tabInfos[2].split("/");            // Décomposer la date de naissance avec le délimiteur /
          // tabNaissance[] = {année, mois, jour}
     if (tabInfos[4].equalsIgnoreCase("F")) {                   // Ajouter 50 au mois si l'individu est de sexe F
          moisNaissance = Integer.parseInt(tabNaissance[1]);    // Convertir le mois de naissance en un nombre
          moisNaissance += 50;                                  // pour ajouter 50
          tabNaissance[1] = String.valueOf(moisNaissance);      // Convertir mois en String dans tabNaissance[1]
     }
     // Construire le numéro d'assurance maladie
     StringBuilder numeroMaladie = new StringBuilder(tabInfos[1].substring(0, 3).toUpperCase()); // Nom de famille
     numeroMaladie.append(tabInfos[0].substring(0, 1).toUpperCase());                            // Prénom
     numeroMaladie.append(" ").append(tabNaissance[0].substring(2));                             // Année
     numeroMaladie.append(tabNaissance[1]);                                                      // Mois
     numeroMaladie.append(" ").append(tabNaissance[2]);                                          // Jour
     numeroMaladie.append(tabInfos[3]);                                                          // Code admin
     return numeroMaladie.toString();                           // Convertir numeroMaladie en String et le retourner
}
```

- Exemple d'appel de la méthode `construireNumeroMaladie()`

```
public static void main(String[] args) {
     String chaine1 = "Félix:Bouchard:1994/02/14:19:M";
     String numero1 = construireNumeroMaladie(chaine1);         // numero1 contient BOUF 9402 1419
     String chaine2 = "France:Boucher:1994/02/14:20:F";
     String numero2 = construireNumeroMaladie(chaine2);         // numero2 contient BOUF 9452 1420
     System.out.println(numero1);
     System.out.println(numero2);
}
```

Farida Ait-Ammar
Nicolas Leduc

41 Classe `StringBuilder` – Exemple enlever espaces

- La méthode `enleverEspaces()` reçoit, en paramètre, une chaîne de caractères. Le traitement de cette méthode consiste à enlever les espaces en trop dans la chaîne de caractères. Une seule espace est gardée entre les mots. Elle retourne la chaîne résultant de ce traitement.

```
public static String enleverEspaces(String chaine) {
     // Enlever les espaces de début et de fin de chaîne avec strip().
     // Récupérer chaque mot dans une case d'un tableau avec split().
     String[] tableMots = chaine.strip().split(" ");
     StringBuilder builder = new StringBuilder("");
     for (int i = 0; i < tableMots.length; ++i) {
          if (tableMots[i].length() != 0) {  // Si un mot est présent
               builder.append(tableMots[i]); // On ajoute le mot
               builder.append(' ');          // suivi d'une espace
          }
     }
     if (builder.length() != 0) {  // Enlever l'espace de trop si au moins un mot est présent
          builder.deleteCharAt(builder.length() - 1);
     }
     return builder.toString();
}
```

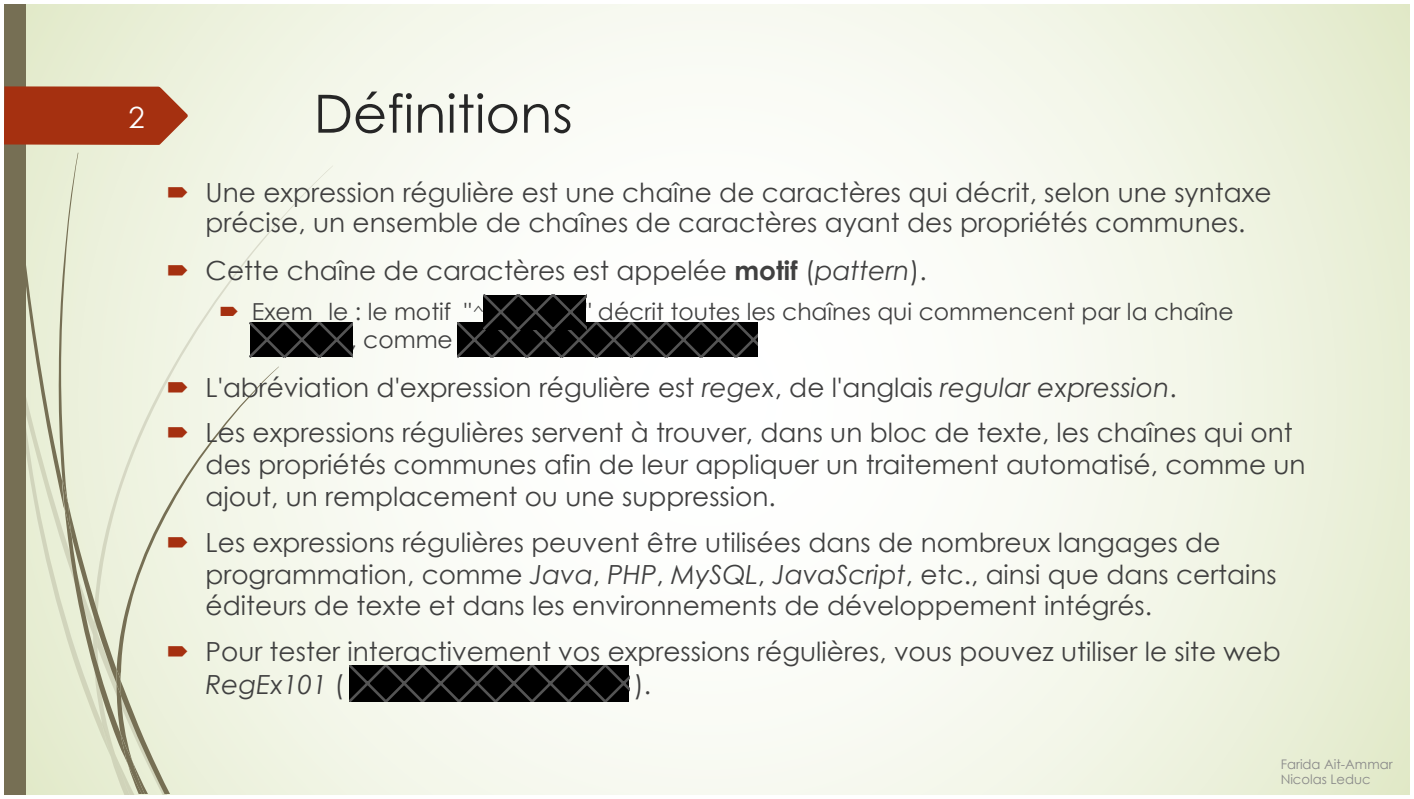
- Exemple d'appel de la méthode `enleverEspaces()`

```
public static void main(String[] args) {
     String phrase = "   Le langage    de   programmation     Java  ";
     String phraseSansEspaces = enleverEspaces(phrase);
     // phraseSansEspaces contient "Le langage de programmation Java"
     System.out.println(phraseSansEspaces);
}
```

Farida Ait-Ammar
Nicolas Leduc

10. Expression régulière

10.1 Principe de base



3

Principes de base

- Un caractère se représente lui-même dans un motif
  - Exemple : le motif `Laval` représente le mot *Laval*
- Les expressions régulières sont sensibles à la casse
  - Exemple : le motif `Laval` ne détectera pas le mot *laval*
- Le motif est appliqué au texte de gauche à droite
- Il est possible d'ajouter des caractères spéciaux à un motif afin qu'il représente un ensemble de chaînes de caractères
  - Exemple : `\ [ ] { } ^ $ - + * ? | ( ) .`
- Le point (`.`) remplace n'importe quel caractère (un seul)
- La barre oblique inverse (`\`) sert de caractère d'échappement. Elle précède le caractère spécial que l'on veut utiliser de façon littérale.
- **Il est plus facile d'écrire un motif complexe que de le relire!**

Farida Ait-Ammar
Nicolas Leduc

4

Principes de base - Exemples

Motif	Chaîne	Trouvé	Commentaire
chien	chien	V	chien
abcd	xyzabcdefgabcde	V	xyzabcdefgabcde : 2 correspondances
a.c	abc	V	abc
a.c	boa cobra	V	boa cobra
a.c	xyza9cdef	V	xyza9cdef
a.c	sacoche	F	Pas de caractère entre a et c
3.14	03@14159	V	03@14159
3\.14	03@14159	F	Ici le point est utilisé de façon littérale à cause du \
3\.14	03.14159	V	03.14159
ch..s	Les chiens et les chats.	V	Les chiens et les chats. : 2 points -> 2 caractères

Farida Ait-Ammar
Nicolas Leduc

10.2 Début et fin

5

Symboles de début et de fin

Symbole	Description	Exemple
^	Ce qui suit doit se trouver au début de la chaîne. `^` doit être le premier caractère du motif	`^La pomme` reconnaît les chaînes qui débutent par *La pomme*
$	Ce qui précède doit se trouver en fin de chaîne. `$` doit être le dernier caractère du motif	`rouge\.$` reconnaît les chaînes qui se terminent par *rouge.*
\b	Correspond aux débuts ou aux fins de mots	`poire\b` correspond à la chaîne *La poire est* , mais non à la chaîne *Les poires sont*
\B	Correspond à tout sauf aux débuts et aux fins de mots	`poire\B` correspond à la chaîne *Les poires sont* , mais non à la chaîne *La poire est*

Farida Ait-Ammar
Nicolas Leduc

6

Symboles de début et de fin - Exemples

Motif	Chaîne	Trouvé	Commentaire
^Laval	Laval est une île.	V	Laval est une île.
^Laval$	Laval est une île.	F	Laval ne se trouve pas en fin de ligne
^Laval	L'île de Laval est…	F	Laval ne se trouve pas en début de chaîne
^Montréal$	Montréal	V	Montréal
Montréal\.$	Je vis à Montréal.	V	Je vis à Montréal.
te\b	Les carottes sont cuites.	F	e n'est pas la dernière lettre de carottes ni de cuites
te.\b	Les carottes sont cuites.	V	Les carottes sont cuites.
\bch...\b	chien chatte chine niche	V	chien chatte chine niche
\bch...\b	Le chat ronronne.	V	Le chat ronronne.
\bch.\B.\B.\b	Le chat ronronne.	F	Pas de fin de mot entre les caractères

Farida Ait-Ammar
Nicolas Leduc

10.3 Classe de caractères

7

Classes de caractères

Symbole	Description	Exemple
`[ ]`	Correspond à **un seul** des caractères parmi l'ensemble inscrit entre les crochets	`[aeiouy]` reconnaît une voyelle. Il y a quatre correspondances dans la chaîne *chat et chien*
`[ - ]`	Permet d'inclure tous les caractères entre les bornes de part et d'autre du tiret (-).	`[a-fwz]` est équivalent à `[abcdefwz]` alors que `[a-zA-Z]` correspond à toutes les lettres minuscules ou majuscules.
`[^ ]`	Correspond à tous les caractères sauf ceux entre crochets. `^` doit apparaître immédiatement après le crochet ouvrant.	`[^aeiouy]` reconnaît tout sauf les voyelles. Dans la chaîne *chat et chien* , il y a neuf correspondances, car les espaces sont aussi attrapées.

Farida Ait-Ammar
Nicolas Leduc

8

Classes de caractères - Équivalences

Méta-caractère	Classe équivalente	Description
`.`		Unique caractère quelconque sauf saut de ligne
`\d`	`[0-9]`	N'importe quel chiffre de 0 à 9 (*d -> digit*)
`\D`	`[^0-9]`	Pas un chiffre
`\w`	`[a-zA-Z_0-9]`	Caractère utilisable dans un mot (*w -> word*)
`\W`	`[^a-zA-Z_0-9]`	Pas un caractère de mot (non mot)
`\s`	`[ \t\n\r\f\x0b]`	Espace blanche, incluant le retour de chariot
`\S`	`[^ \t\n\r\f\x0b]`	Pas une espace blanche
`\p{javaLowerCase}`	`[a-zàâäèéêëîïôùûüÿçœ]`	Lettre minuscule incluant accent
`\p{javaUpperCase}`	`[A-ZÀÂÄÈÉÊËÎÏÔÙÛÜŸÇŒ]`	Lettre majuscule incluant accent
`\p{javaLetter}`	`[a-zàâäèéêëîïôùûüÿçœ A-ZÀÂÄÈÉÊËÎÏÔÙÛÜŸÇŒ]`	Minuscule ou majuscule incluant accent
`\p{Alpha}`	`[a-zA-Z]`	Minuscule ou majuscule sans accent
`\p{javaDigit}`	`[0-9]`	Chiffre de 0 à 9
`\p{javaLetterOrDigit}`	`[a-zàâäèéêëîïôùûüÿçœ A-ZÀÂÄÈÉÊËÎÏÔÙÛÜŸÇŒ0-9]`	Minuscule ou majuscule incluant accent ou chiffre
`\p{javaWhitespace}`	`[ \t\n\r\f]`	Espace blanche, incluant le retour de chariot

Farida Ait-Ammar
Nicolas Leduc

9

Classes de caractères - Exemples

Motif	Chaîne	Trouvé	Commentaire
m[aei]t	mit mot miette mettre	V	mit mot miette mettre
[0-9]$	Tout à 5$	F	Le chiffre doit apparaître à la fin de la chaîne
[0-9]\$	2 cafés à 5$ + taxes	V	2 cafés à 5$ + taxes
\d!	4! plus 3! égale 30	V	4! plus 3! égale 30
\w\w\w\w\w	chat chien éléphants	V	chat chien éléphants
\w\s[A-Z]\w\w	Guy et Jean mangent.	V	Guy et Jean mangent.
[aeiou][^aeiou][aeiou]	La table est basse.	V	La table est basse.
[^aeiou][aeiou][^aeiou]	La table est basse.	V	La table est basse.
[0-9][-+*/][0-9]=	16+4=20	V	16+4=20
[0-9][-+*/][0-9]=	8*11=88	F	Un seul chiffre doit apparaître entre * et =

Farida Ait-Ammar
Nicolas Leduc

10.4 Quantificateur

10

Quantificateurs

- Un quantificateur permet de spécifier le **nombre d'occurrences** de l'élément du motif immédiatement à sa gauche.

Symbole	Description	Exemple
`{n}`	Exactement `n` occurrences	`\b[a-zA-Z]{5}\b` : mots contenant exactement cinq lettres.
`{n,}`	Minimum de `n` occurrences	`[a-zA-Z]\w{4,}` : noms de variables ayant au moins cinq caractères.
`{n,m}`	Au moins `n` occurrences, mais sans dépasser `m` occurrences	`[a-z]{4,10}` : mots contenant entre quatre et dix lettres minuscules.
`?`	Élément optionnel : zéro ou une occurrence. Équivalent à `{0,1}`	`tes?\b` : mots se terminant par *te* ou par *tes* . Le *s* est optionnel.
`*`	Zéro occurrence ou plus. Équivalent à `{0,}`	`0*[0-9]` : chiffre précédé de n'importe quel nombre de zéros.
`+`	Au moins une occurrence. Équivalent à `{1,}`	`\w+` : mots comportant au moins un caractère alphanumérique.

Farida Ait-Ammar
Nicolas Leduc

11

Quantificateurs - Exemples

Motif	Chaîne	Trouvé	Commentaire
a?	"" (chaîne vide)	V	a apparait zéro fois
a*	"" (chaîne vide)	V	a apparait zéro fois
a+	"" (chaîne vide)	F	a doit apparaître au moins une fois
a.?c	sacoche boa cobra	V	sacoche boa cobra
a.*c	sacoche	V	sacoche : maximise la correspondance (glouton)
a.+c	sac à dos	F	aucun caractère entre a et c
a.+c	cache cache	V	cache cache : maximise la correspondance (glouton)
z{2}	manger une pizza au zoo	V	manger une pizza au zoo
[0-9]{3,}	12 3456 789	V	12 3456 789
[0-9]{3,5}	12 3456789 123 4567 89	V	12 3456789 123 4567 89

Farida Ait-Ammar
Nicolas Leduc

10.5 Groupe

12

OU logique et groupes

- Le symbole `|` (barre verticale) indique une option (*ou* logique). L'expression régulière correspond au motif à gauche du `|` ou à celui à droite.
  - Exemple : `La pomme|fraise rouge` trouve *La pomme OU fraise rouge* .
- Pour obtenir le résultat voulu, nous devons utiliser les groupes délimités par des parenthèses `()`
  - Exemple : `La (pomme|fraise) rouge` trouve *La pomme rouge OU La fraise rouge* .
- Il est aussi possible d'affecter un opérateur de répétition à un groupe complet.
- Les groupes sont numérotés de gauche à droite selon l'ordre de leur parenthèse ouvrante. Le groupe zéro contient toujours l'expression entière même si aucun groupe n'est défini.
- Lors de l'application de l'expression régulière sur une chaîne, les sous-chaînes vérifiant les motifs définis par chacun des groupes seront conservées en mémoire (capturées). Ces sous-chaines sont référencées par `$n` où `n` est le numéro de groupe entre un et neuf. Ces références peuvent être utilisées dans les chaînes de remplacement. (voir les exemples)
  - NB : Si un opérateur de répétition est affecté à un groupe, seule la dernière occurrence sera capturée

Farida Ait-Ammar
Nicolas Leduc

13

OU logique et groupes - Exemples

Motif	Chaîne	Trouvé	Commentaire
Le chat|chien noir	Un chien noir mange.	V	Un chien noir mange.
Le (chat|chien) noir	Un chien noir mange.	F	Trouve *Le chien noir*
Le ch(at|ien) noir	Le chat noir dort.	V	Le chat noir dort. - $1:at
Le (beau)?chat	Le chat noir dort.	V	Le chat noir dort. - $1:null – beau est optionnel
(\d{3}) (\d{3}) (\d{3})	NAS: 125 467 873	V	NAS: 125 467 873 - $1:125 - $2:467 - $3:873
(\d){3} (\d){3} (\d){3}	NAS: 125 467 873	V	NAS: 125 467 873 - $1:5 - $2:7 - $3:3
([0-9]+([-+*/])[0-9]+)=	52*67=3484	V	52*67=3484 - $1:52*67 - $2:*
([0-9]+)([-+*/])([0-9]+)=	52*67=3484	V	52*67=3484 - $1:52 - $2:* - $3:67
(0[1-9]|1[0-2]):00 ([AP]M)	Heure: 08:00 AM	V	Heure: 08:00 AM - $1:08 - $2:AM
e(.*)r	Le petit chat noir dort.	V	Le petit chat noir dort. - $1: petit chat noir do

Farida Ait-Ammar
Nicolas Leduc

14

Tester les expressions régulières

- Il est important de tester nos expressions régulières à l'aide de tests unitaires contenant des chaînes valides ainsi que des chaînes invalides. En effet, il est relativement facile d'écrire une expression régulière qui détecte les chaînes valides, mais c'est plus difficile d'en écrire une qui refuse celles invalides.
- Exemple
  - Écrire une expression régulière afin de valider un code postal dans le format : `A1A 1A1`
  - Le motif suivant permet d'accepter tous les codes postaux valides : `[A-Z0-9 ]{7}`
  - Il permet aussi de refuser des chaînes qui ne sont pas des codes postaux, comme `La pomme`
  - Toutefois, des codes postaux invalides sont acceptés par ce motif, comme `H2 L4KS`
- Vous devez donc vous assurer que votre motif discrimine bien les chaînes valides par rapport à celles invalides en testant avec des chaînes limites, soit qui contiennent de petites erreurs pour quelques caractères. En fait, il faut se demander s'il est possible d'écrire une chaîne qui sera acceptée, mais qui ne correspond pas au format voulu.

Farida Ait-Ammar
Nicolas Leduc

10.6 Expression régulière en *Java*

16

Barre oblique inverse en *Java* (\)

- Dans une chaîne de caractères *Java*, la **barre oblique inverse** (\) a une signification particulière. En effet, elle sert de **caractère d'échappement** et permet d'écrire des caractères spéciaux (ex : `\n`, `\u00E9`).
- Lorsqu'une barre oblique inverse apparaît dans nos motifs, il faut la doubler (****) pour éviter que *Java* ne tente de l'interpréter.
  - Si on veut utiliser `\w`, on doit écrire `\\w` en *Java*
  - Si `\` doit apparaître dans notre motif, on écrit `\\` dans l'expression régulière et `\\\\` en *Java* (chacun des deux `\` a été doublé).
- **Une bonne pratique est d'écrire le véritable motif dans un commentaire.**

Farida Ait-Ammar
Nicolas Leduc

17

Méthodes de la classe `String` qui supportent les expressions régulières

- Il existe quatre méthodes de la classe `String` qui supportent les expressions régulières :
  - `matches()` : vérifie si la chaîne entière correspond au motif
  - `split()` : sépare la chaîne en utilisant le motif comme séparateur
  - `replaceFirst()` : remplace la première occurrence du motif
  - `replaceAll()` : remplace toutes les occurrences du motif
- Note : la méthode `replace()` ne supporte pas les expressions régulières.

Farida Ait-Ammar
Nicolas Leduc

18

Méthode `matches()`

- `public boolean matches(String regex)`
  - Retourne `true` si, et seulement si, cette chaîne correspond à l'expression régulière donnée.
- Attention : `matches()` ne vérifie pas si le motif est contenu dans la chaîne, mais plutôt si la chaîne entière correspond au motif :
  - `"chien".matches("[Cc]hien")` retourne `true`
  - `"Le chien noir.".matches("[Cc]hien")` retourne `false`
- Exemple 1 : Vérifier si une chaîne est l'un des mots : `Vrai`, `vrai`, `Oui`, `oui`
  - Le motif est `[Vv]rai|[Oo]ui`

```
public static void main(String[] args) {
    String str = "Oui";
    System.out.print(str.matches("[Vv]rai|[Oo]ui")); // Affiche true
}
```

- Exemple 2 : Vérifier si une chaîne **contient** l'un des mots : `Vrai`, `vrai`, `Oui`, `oui`
  - Le motif est "`.*([Vv]rai|[Oo]ui).*`"
  - Il faut ajouter `.*` avant et après le motif afin de faire correspondre le reste de la chaîne.

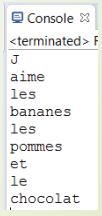
Farida Ait-Ammar
Nicolas Leduc

19

Méthode `split()`

- `public String[] split(String regex)`
  - Permet de scinder une chaîne en plusieurs sous-chaînes grâce à un délimiteur défini par `regex`
  - Retourne un tableau de `String`
    - S'il n'y a aucune correspondance, le tableau résultant ne contient qu'un seul élément, à savoir la chaîne
    - Les chaînes vides de fin ne sont pas incluses dans le tableau
- Exemple : Séparer tous les mots d'une phrase dans un tableau.
  - Le délimiteur est un ou plusieurs caractères qui ne sont pas dans un mot : `\W+`

```
public static void main(String[] args) {
    String regex = "\\W+";
    String phrase = "J'aime les bananes, les pommes et le chocolat. ";
    String[] mots = phrase.split(regex);
    for (int i = 0; i < mots.length; i++) {
            System.out.println(mots[i]);
    }
}
```



Farida Ait-Ammar
Nicolas Leduc

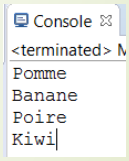
20

Méthodes `replaceFirst()` et `replaceAll()`

- `public String replaceFirst(String regex, String replacement)`
  - Remplace la première occurrence de l'expression régulière (`regex`) dans cette chaîne par la chaîne de remplacement (`replacement`).
- `public String replaceAll(String regex, String replacement)`
  - Remplace toutes les occurrences de l'expression régulière (`regex`) dans cette chaîne par la chaîne de remplacement (`replacement`).
- La chaîne de remplacement peut contenir des références aux différents groupes de l'expression régulière en utilisant la notation `$n`

- Exemple : Remplacer les espaces par des sauts de ligne (`\n`)

```
public static void main(String[] args) {
    String phrase = "Pomme  Banane\t   Poire  Kiwi";
    String phraseTransformee = phrase.replaceAll("\\s+", "\n");
    System.out.print(phraseTransformee);
}
```



Farida Ait-Ammar
Nicolas Leduc

21

Exemple – Numéro de téléphone

- Nous voulons être en mesure de reconnaître les numéros de téléphone dans différents formats et de les transformer dans un format normalisé.
  - Exemples de numéros acceptés : `4505551234`, `514-555-1234`, `438 555 1234`, `(450) 555-1234`
  - Format normalisé : `(450)555-1234`

- Expression régulière : `\(?(\d{3})\)?[- ]?(\d{3})[- ]?(\d{4})`
- Code *Java* :

```
public static void main(String[] args) {
    String tel = "(450)-661 7070";
    String telNorm = "";
    if (tel.matches("\\(?(\\d{3})\\)?[- ]?(\\d{3})[- ]?(\\d{4})")) {
        telNorm = tel.replaceFirst(
            "\\(?(\\d{3})\\)?[- ]?(\\d{3})[- ]?(\\d{4})", "($1)$2-$3");
    }
    System.out.println(telNorm); // Affiche (450)661-7070
}
```

Farida Ait-Ammar
Nicolas Leduc

11. Exception

11.1 Capturer une exception (`try/catch`)

2

Exceptions - Définition

- Une exception est une erreur qui se produit lorsqu'une situation anormale est rencontrée durant l'exécution d'un programme. Si elle n'est pas traitée, elle provoque l'arrêt brusque (interruption) du programme, ce qui n'est pas recommandé.
- Exemple : Dans la méthode `charPourcent()` ci-dessous, l'instruction `String mot = ph.strip().split(" ")[idxMot];` peut provoquer une exception de type `ArrayIndexOutOfBoundsException` si `idxMot` dépasse le nombre de mots dans la chaîne. Si l'exception se produit, comme elle n'est pas traitée, elle remonte jusqu'à la méthode `main()` et provoque l'interruption du programme.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    System.out.print("Entrez un nombre et une phrase : ");
    int pct = charPourcent(cl.nextInt(), cl.nextLine());
    System.out.println("Caractères : " + pct + "% de la chaine");
}

public static int charPourcent(int idxMot, String ph) {
    int pctCh;
    pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
    String mot = ph.strip().split(" ")[idxMot];
    System.out.println("Mot #" + idxMot + " : " + mot);
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```

```
PROBLÈMES 65  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  GITLENS

hexabit@moo 420-NDC %  /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-17.0.10.jdk/Contents/Home/bin/java @/var/folders/
_n/nh2jnvl52kj24_3mrg8718fm0000gn/T/cp_8mjqhrhilcopyfaxakv67rvwt.argfile _enseignant_.exceptions.PhraseP02
Entrez un nombre et une phrase : 3 Le chat
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 2
        at _enseignant_.exceptions.PhraseP02.charPourcent(PhraseP02.java:16)
        at _enseignant_.exceptions.PhraseP02.main(PhraseP02.java:9)
hexabit@moo 420-NDC %
```

Provoque une exception lorsque `idxMot` >= nombre de mots

Le message de l'exception est affiché dans le terminal. Il donne des informations sur l'exception dans l'ordre suivant :

- Nom de l'exception : `ArrayIndexOutOfBoundsException` et un message descriptif `Index 3 out of bounds for length 2`.
- Ligne qui a généré l'exception : ici, c'est la ligne 16 de la méthode `charPourcent()` de la classe `PhraseP02`.
- Ligne qui a appelé cette méthode : ici, c'est la ligne 9 de la méthode `main()`.

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

3

Capturer une exception dans un bloc `try/catch`

- On peut capturer (ou gérer) une exception pour éviter qu'elle ne se propage jusqu'à la méthode `main()`, provoquant ainsi l'interruption du programme.
- Pour capturer une exception, on procède comme suit :
  - On imbrique, dans un bloc `try`, les instructions susceptibles de provoquer l'exception, par exemple, un accès hors des limites d'un tableau possible.
  - On imbrique le code qui va traiter l'exception dans le bloc `catch`.
  - S'il y a du code que l'on veut exécuter dans tous les cas, exception levée ou non, on l'imbrique dans le bloc `finally`. Ce bloc optionnel est présenté plus loin.
- Voici la forme la plus simple d'un bloc `try/catch` :

```
// début du programme
try {
      // code qui peut déclencher une exception
} catch (ClasseException nomObjetException) {
      // traitement pour ce type d'exception
}
// suite du programme
```

- Fonctionnement :
  - Si une **exception est lancée** dans le bloc `try`, alors :
    - le programme **abandonne** l'exécution du **code restant** dans le bloc `try`
    - le programme exécute le code du bloc `catch`
  - Si **aucune** exception n'est lancée dans le bloc `try`, le programme **ignore** le bloc `catch`

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

4

Capturer une exception dans un bloc `try/catch`

- On reprend l'exemple précédent d'analyse d'une phrase en y ajoutant la gestion de l'exception d'accès hors limites

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    System.out.print("Entrez un nombre et une phrase : ");
    int pct = charPourcent(cl.nextInt(), cl.nextLine());
    System.out.println("Caractères : " + pct + "% de la chaine");
}
public static int charPourcent(int idxMot, String ph) {
    int pctCh;
    pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
    try {
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    } catch (ArrayIndexOutOfBoundsException ex) {
        System.out.println("Mot introuvable!");
    }
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```

Affichage au terminal	Explication

`Entrez un nombre et une phrase : 3 Le chat`
`Mot introuvable!`
`--- FIN : charPourcent() ---`
`Caractères : 75% de la chaine`
	L'instruction suivante, du bloc `try`, lance une exception :
	`String mot = ph.strip().split(" ")[idxMot];`
	Le programme ignore l'instruction qui suit dans le bloc `try`, soit :
	`System.out.println("Mot #" + idxMot + " : " + mot);`
	Le programme exécute les instructions du bloc `catch`, puisque l'exception est de type `ArrayIndexOutOfBoundsException`.

`Entrez un nombre et une phrase : 1 Le chat noir`
`Mot #1 : chat`
`--- FIN : charPourcent() ---`
`Caractères : 76% de la chaine`
	Aucune exception n'est lancée dans le bloc `try`. Le bloc `catch` est alors ignoré.

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

5 Paramètres du bloc `catch`

- Le bloc `catch` contient un seul paramètre :

  `catch (ClasseException` `nomObjetException`**)**

  - **`ClasseException`** : Renseigne sur la classe de l'exception lancée.
  - `nomObjetException` : Objet de type **`ClasseException`**, qui contient des informations sur la nature de l'exception. Pour extraire les informations contenues dans l'objet `nomObjetException`, il est possible d'utiliser les méthodes suivantes :
    - `toString()` : **retourne** le nom de l'exception et un message d'erreur.
    - `getMessage()` : **retourne** une description détaillée du message d'erreur, si elle existe.
    - `getClass().getName()` : **retourne** le nom de la classe de l'objet `nomObjetException`.
    - `printStackTrace()` : **affiche** l'exception et la trace d'exécution de l'objet dans la pile d'appels. Utilise `System.err` pour l'affichage au lieu de `System.out`

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

6 Paramètres du bloc `catch` - Exemple

- On reprend l'exemple d'analyse d'une phrase en y appliquant les méthodes vues dans la diapositive précédente.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    System.out.print("Entrez un nombre et une phrase : ");
    int pct = charPourcent(cl.nextInt(), cl.nextLine());
    System.out.println("Caractères : " + pct + "% de la chaine");
}

public static int charPourcent(int idxMot, String ph) {
    int pctCh;
    pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
    try {
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    } catch (ArrayIndexOutOfBoundsException ex) {
        System.out.println("Mot introuvable!");
        System.out.println("ex.getMessage() : " + ex.getMessage());
        System.out.println("ex.getClass().getName() : " + ex.getClass().getName());
        System.out.println("ex.printStackTrace() : ");
        ex.printStackTrace();  // C'est la fonction qui se charge de l'affichage
    }
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

7

Paramètres du bloc `catch` – Exemple (suite)

- Affichage au terminal :

```
Entrez un nombre et une phrase : 3 Le chat
Mot introuvable!
ex.getMessage() : Index 3 out of bounds for length 2
ex.getClass().getName() : java.lang.ArrayIndexOutOfBoundsException
ex.printStackTrace() :
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 2
        at _enseignant_.exceptions.PhraseP06.charPourcent(PhraseP06.java:17)
        at _enseignant_.exceptions.PhraseP06.main(PhraseP06.java:9)
--- FIN : charPourcent() ---
Caractères : 75% de la chaine
```

- Description du message d'erreur
- Nom de la classe de l'exception
- Nom de l'exception et trace de l'objet dans la pile d'exécution
  - le nom et le message associés à l'exception sont donnés en premier
  - ils sont suivis par des références aux différentes lignes de code, en ordre inverse de l'appel des méthodes qui ont généré l'exception

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

11.2 Capturer plusieurs exceptions

8

Capturer plusieurs exceptions

- Il est possible de capturer plusieurs exceptions, dans un bloc `try`, et de les gérer différemment, en utilisant plusieurs blocs `catch`. On utilise **un bloc `catch` spécifique pour chaque type d'exception.**
- Les blocs `catch` sont **examinés dans l'ordre où ils apparaissent dans le code**. **Le premier** qui correspond au type d'exception à traiter **est exécuté** et les **autres blocs sont ignorés**.
- L'ordre des blocs est donc important. Il faut traiter, en premier, les exceptions les plus précises (sous-classes) avant celles plus générales (super-classes). Un message d'erreur est émis par le compilateur dans le cas contraire. Un exemple est présenté plus loin.
- Voici le modèle :

```
// début du programme
try {
   // code qui peut déclencher des exceptions
} catch (ClasseException1 nomObjet1) {
   // traitement pour le type d'exception 1
} catch (ClasseException2 nomObjet2) {
   // traitement pour le type d'exception 2
} catch (ClasseException3 nomObjet3) {
   // traitement pour le type d'exception 3
}
// suite du programme
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

9

Capturer plusieurs exceptions - Exemple

- La division est susceptible de lever une exception de type `ArithmeticException` si le diviseur vaut zéro, soit une division par zéro.
- On reprend l'exemple précédent en ajoutant la gestion de l'exception de type `ArithmeticException`.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    System.out.print("Entrez un nombre et une phrase : ");
    int pct = charPourcent(cl.nextInt(), cl.nextLine());
    System.out.println("Caractères : " + pct + "% de la chaine");
}

public static int charPourcent(int idxMot, String ph) {
    int pctCh = -1;
    try {
        pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    } catch (ArithmeticException ex) {
        pctCh = 0; // On retourne une valeur de 0 pour indiquer le problème
    } catch (ArrayIndexOutOfBoundsException ex) {
        System.out.println("Mot introuvable!");
    }
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```

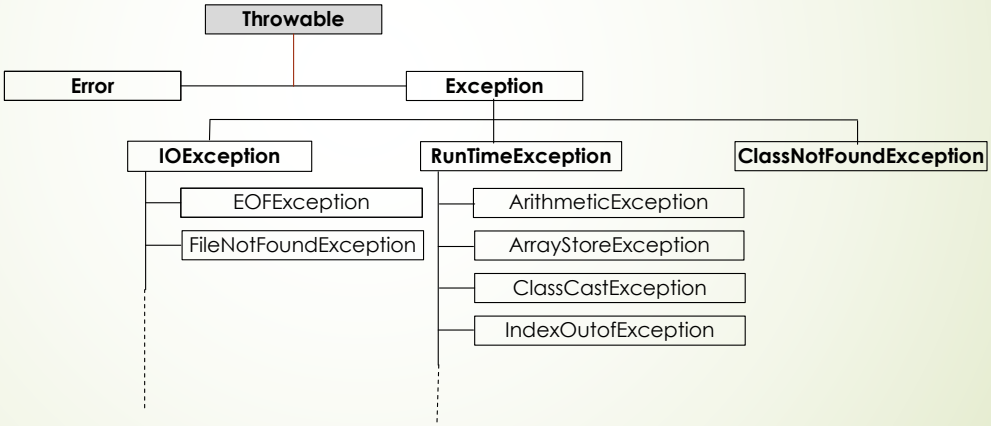
```
Entrez un nombre et une phrase : 3
--- FIN : charPourcent() ---
Caractères : 0% de la chaine
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

10

Ordre d'interception des exceptions hiérarchisées

- Les exceptions se présentent sous forme d'une hiérarchie. Toutes les classes héritent directement ou indirectement de la classe **`Throwable`** du package `java.lang`.
- Lorsqu'il y a une hiérarchie entre les exceptions à intercepter, il est important de gérer les exceptions dans l'ordre inverse de la hiérarchie, c'est-à-dire qu'il faut traiter les sous-classes avant de traiter les classes parents.
- *Rappel* : tout objet de la sous-classe est aussi considéré comme un objet du type de la classe parent.

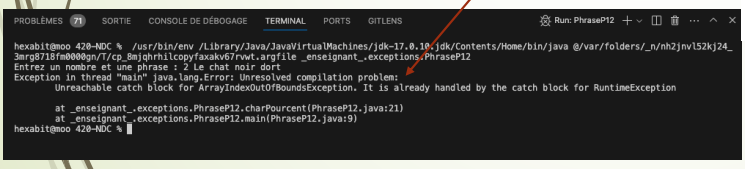


Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

11 Ordre d'interception des exceptions hiérarchisées

- On reprend l'exemple précédent en ajoutant la gestion de l'exception de type `RuntimeException` avant l'exception `ArrayIndexOutOfBoundsException`

```
public static int charPourcent(int idxMot, String ph) {
    int pctCh = -1;
    try {
        pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    } catch (RuntimeException ex) {
        pctCh = 0;
    } catch (ArrayIndexOutOfBoundsException ex) {
        System.out.println("Mot introuvable!");
    }
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```



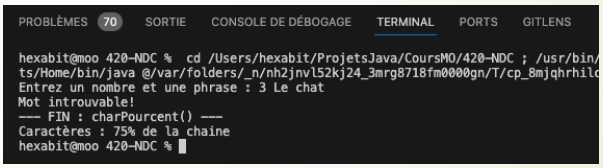
- Un message d'erreur de compilation, indiquant que l'exception `ArrayIndexOutOfBoundsException` est déjà interceptée, est affiché au terminal.
- **`catch`**`(`**`RuntimeException`** `ex)` est chargé d'intercepter les exceptions de la classe `RuntimeException` impliquant que les exceptions des sous-classes sont aussi interceptées, par exemple, `ArrayIndexOutOfBoundsException`.
- Ainsi `ArrayIndexOutOfBoundsException` est interceptée par
  `catch``(`**`RuntimeException`** `ex)`
  mais jamais par
  `catch``(`**`ArrayIndexOutOfBoundsException`** `ex)`.

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

12 Ordre d'interception des exceptions hiérarchisées

- Lorsqu'il faut intercepter des exceptions des sous-classes d'une classe d'exception, il est important de placer la classe parent à la fin de la liste du bloc `catch`.
- On reprend l'exemple précédent, mais cette fois l'exception `ArrayIndexOutOfBoundsException` est interceptée puisqu'elle est placée avant l'exception `RuntimeException`.

```
public static int charPourcent(int idxMot, String ph) {
    int pctCh = -1;
    try {
        pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    } catch (ArrayIndexOutOfBoundsException ex) {
        System.out.println("Mot introuvable!");
    } catch (RuntimeException ex) {
        pctCh = 0;
    }
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```



Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

11.3 Bloc `finally`

13

Bloc `finally`

- Le code du bloc `finally` est facultatif. Toutefois, s'il est présent, il s'exécute en tout temps, qu'une exception ait été interceptée ou non.
- Voici le modèle :

```
try {
     // code qui peut déclencher des exceptions
} catch (ClasseException1 nomObjet1) {
     // traitement pour le type d'exception 1
} catch (ClasseException2 nomObjet2) {
     // traitement pour le type d'exception 2
} finally {
    // traitement qui sera toujours exécuté,
    // qu'une exception ait été déclenchée dans le bloc try ou non,
    // qu'une exception ait été déclenchée dans le bloc try et ait été capturée dans un bloc catch,
    // qu'une exception ait été déclenchée dans le bloc try, mais qu'elle n'ait pas été capturée
    // par un bloc catch. Dans ce dernier cas, l'exception est renvoyée à la méthode appelante.
}
```

- Il est recommandé d'utiliser un bloc `finally` dès que l'on doit fermer une ressource, comme un fichier.

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

14

Bloc `finally` - Exemple

- On reprend l'exemple précédent en mettant le message `--- FIN : charPourcent() ---` dans le bloc `finally`, afin qu'il s'affiche en tout temps.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    System.out.print("Entrez un nombre et une phrase : ");
    int pct = charPourcent(cl.nextInt(), cl.nextLine());
    System.out.println("Caractères : " + pct + "% de la chaine");
}

public static int charPourcent(int idxMot, String ph) {
    int pctCh = -1;
    try {
        pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    } catch (ArrayIndexOutOfBoundsException ex) {
        System.out.println("Mot introuvable!");
    } finally {
        System.out.println("--- FIN : charPourcent() ---");
    }
    return pctCh;
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

11.4 Exception vérifiée (`throws`)

15

Exceptions vérifiées et non vérifiées

- En *Java*, les exceptions sont regroupées en deux catégories :
  - Exceptions **implicites** ou **non vérifiées** (*unchecked*)
    - Il n'est pas nécessaire d'ajouter un `throws` (voir diapositive suivante) à la signature de la méthode si on ne veut pas gérer ce type d'exception dans celle-ci. Ces exceptions peuvent être tout de même capturées dans un bloc `try/catch`, mais il n'est pas obligatoire de le faire. Par exemple, il n'est pas nécessaire d'attraper une exception de type `ArithmeticException`.
      - Uniquement les exceptions dérivant des classes **`Error`** et **`RunTimeException`** sont des exceptions non vérifiées.
  - Exceptions **explicites** ou **vérifiées** (*checked*)
    - Lorsque les exceptions levées dans une méthode sont des exceptions **vérifiées**, il faut obligatoirement les attraper dans un bloc `try/catch` ou signaler au compilateur qu'elles seront propagées à la méthode appelante sans traitement (`throws`), sinon il y a aura une erreur de compilation. Vous verrez de telles exceptions lorsque nous parlerons de la manipulation des fichiers.
- Les exceptions vérifiées ou non fonctionnent de la même façon. La différence réside dans la manière de propager l'une ou l'autre des catégories d'exceptions.

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

16

Propager les exceptions avec le mot clé `throws`

- En *Java*, il est possible de déclarer les classes d'exceptions qui doivent être propagées en utilisant le mot clé **`throws`** suivi de la liste des classes d'exceptions qui sont propagées.
- Ceci est fait entre la liste des paramètres et l'accolade ouvrante de la méthode. Seules les exceptions vérifiées doivent être déclarées dans l'en-tête de la méthode.
- `throws` permet de déléguer la responsabilité du traitement des exceptions à la méthode appelante.
- Plusieurs classes d'exceptions peuvent être indiquées en séparant leurs noms par des virgules.
- Voici un modèle :

```
<modificateurs> <type> <identificateur>(<paramètres>) throws <liste d'exceptions> {
    // code de la méthode
}
```

- Exemple :

```
public void  gererArticles() throws IOException {
    // code de la méthode
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

17

Propager les exceptions avec le mot clé `throws`

- Exemple :

```
public void gererArticles() throws IOException{
    traiterArticles();
    // suite du code
}

public void traiterArticles() throws IOException{
    int nbArticles = lireArticles();
    // suite du code
}

public int lireArticles() throws IOException {
    // Le code inséré ici peut déclencher une exception vérifiée de type IOException
    // Comme elle n'est pas traitée dans un bloc try/catch, il faut indiquer au
    // compilateur qu'elle sera propagée à la méthode appelante en utilisant throws.
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

11.5 Lancer une exception (`throw`)

18

Lancer une exception avec l'instruction `throw`

- En *Java*, on peut choisir de capturer une exception, mais on peut aussi choisir de la lancer, par exemple pour fournir une description qui est mieux adaptée au contexte.
- Pour lancer une exception, il faut :
  - trouver la classe d'exception appropriée
  - créer un objet à partir de cette classe en indiquant un message, s'il est différent de celui par défaut
  - lancer l'objet créé en utilisant l'instruction **`throw`**
- Exemples d'utilisation :

```
throw new ClasseException();
 ou
throw new ClasseException("message");
 ou
ClasseException objException = new ClasseException();
throw objException;
 ou
ClasseException objException = new ClasseException("message");
throw objException;
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

19

Lancer une exception avec `throw` - Exemple

- Voici un modèle du lancement d'une exception avec `throw` :

```
if (condition qui cause l'exception) {
    // Lancer l'exception
    throw new ClasseException();
    // OU throw new ClasseException("message");
    System.out.println("Ce texte ne sera jamais affiché");
}
```

- L'exécution s'arrête immédiatement après l'instruction `throw`, donc avant d'atteindre l'instruction suivante (`System.out.println()`).

- Exemple :

```
public static int charPourcent(int idxMot, String ph) throws IllegalArgumentException {
    int pctCh;
    pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
    if (idxMot >= ph.strip().split(" ").length) {
        IllegalArgumentException objEx = new IllegalArgumentException("Index mot invalide!");
        throw objEx;
        // throw new IllegalArgumentException("Index mot invalide!"); // Équivalent
    } else {
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    }
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

20

Lancer une exception avec `throw` - Exemple

- L'appel de la méthode `charPourcent()` se fera de la manière suivante pour l'exemple précédent :

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    System.out.print("Entrez un nombre et une phrase : ");
    try {
        int pct = charPourcent(cl.nextInt(), cl.nextLine());
        System.out.println("Caractères : " + pct + "% de la chaine");
    } catch (IllegalArgumentException ex) {
        System.out.println(ex);
    }
}
```

- Affichage au terminal :

```
Entrez un nombre et une phrase : 3 Le chat
java.lang.IllegalArgumentException: Index mot invalide!
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

21

Relancer une exception

- Il est possible d'intercepter une exception et de la relancer dans un bloc `catch`. Cela se produit lorsque :
  - Il faut changer le type (classe) d'exception
  - Le bloc `catch` n'est pas en mesure de traiter l'exception en question
  - On désire un traitement plus approfondi par l'une des méthodes appelantes
- Cette façon de faire permet de traiter différemment l'exception d'origine.
- Pratique si l'on veut continuer à traiter l'exception interceptée dans la méthode appelante.

- Modèle pour relancer une exception :

```
try {
   // Code qui peut déclencher une exception
} catch (TypeException nomObjet){
   throw new AutreException("Un autre message : " + nomObjet.getMessage());
   // On relance l'exception à la méthode appelante
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

22

Relancer une exception - Exemple

- Dans cet exemple, la méthode `charPourcent()` intercepte l'exception `ArrayIndexOutOfBoundsException`, puis elle la relance dans le `catch` en utilisant un nouveau type, soit `IllegalArgumentException`, et un nouveau message. C'est cette dernière exception que le programme principal doit capturer.

```
public static void main(String[] args) {
    Scanner cl = new Scanner(System.in);
    System.out.print("Entrez un nombre et une phrase : ");
    try {
        int pct = charPourcent(cl.nextInt(), cl.nextLine());
        System.out.println("Caractères : " + pct + "% de la chaine");
    } catch (IllegalArgumentException ex) {
        System.out.println(ex);
    }
}

public static int charPourcent(int idxMot, String ph) throws IllegalArgumentException {
    int pctCh;
    pctCh = 100 * ph.replaceAll("\s", "").length() / ph.length();
    try {
        String mot = ph.strip().split(" ")[idxMot];
        System.out.println("Mot #" + idxMot + " : " + mot);
    } catch (ArrayIndexOutOfBoundsException ex) {
        throw new IllegalArgumentException("Index mot invalide!");
    }
    System.out.println("--- FIN : charPourcent() ---");
    return pctCh;
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

11.6 Créer une exception

23

Créer sa propre classe d'exception

- Dans certains cas, notre code pourrait rencontrer un problème qui ne soit pas décrit d'une manière adéquate par les classes d'exception prédéfinies. Ainsi, il est possible de créer notre propre classe d'exception :
  - La nouvelle classe doit hériter de la classe `Exception` ou de l'une de ses sous-classes, par exemple `IOException`. Il est recommandé d'inclure le mot *`Exception`* dans le nom de la nouvelle classe.
  - Déclarez un constructeur par défaut, qui ne prend aucun paramètre et qui donne un message d'exception prédéfini, ainsi qu'un constructeur qui reçoit un message détaillé.
  - Notre classe d'exception n'a pas à faire de traitements particuliers, mais le fait qu'elle existe dans le système des types nous permet de produire du code qui génère des exceptions plus faciles à comprendre.
- Voici un modèle :

```
public class NomDeNotreException extends Exception {
   // Constructeur par défaut
   public NomDeNotreException() {
      // Le message sera affiché par la super-classe
      super("Message prédéfini.");
   }

   // Constructeur qui reçoit le message détaillé
   public NomDeNotreException(String message) {
      // Le message sera affiché par la super-classe
      super(message);
   }
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

24

Créer sa propre classe d'exception

- Il est à présent possible de lancer notre propre type d'exception.
- Les méthodes pouvant lancer cette exception doivent inclure une clause `throws NomDeNotreException` dans leur en-tête. Cela précise au compilateur que cette méthode pourra lancer cette exception et que toute méthode qui l'appelle devra la prendre en compte (traitement ou propagation).

```
typeRetour nomMéthode(paramètres) throws NomDeNotreException {
   if (condition qui cause notre exception) {
      throw new NomDeNotreException();
      // OU throw new NomDeNotreException("message");
   }
}
```

- On peut aussi capturer notre type d'exception :

```
try {
   // Code qui peut déclencher notre propre exception
} catch (NomDeNotreException nomObjet) {
   // Traitement pour ce type d'exception
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

25

Créer sa propre classe d'exception - Exemple

- Construction de notre classe d'exception qui permet d'indiquer que la longueur d'un rectangle dépasse la limite.

```
public class LongueurInvalideException extends Exception {
    public LongueurInvalideException() {
        super("La longueur dépasse la limite permise.");
    }

    public LongueurInvalideException(String message) {
        super(message);
    }
}
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

26

Créer sa propre classe d'exception - Exemple

- Méthode qui lance l'exception

```
public static void calculerAireRectangle(int nb1, int nb2) throws LongueurInvalideException {
    int rep = 0;
    System.out.print("(Longueur " + nb1 + ")*(Largeur " + nb2 + ") = ");
    if (nb1 > 20) {
        throw new LongueurInvalideException();
    } else if (nb1 < 0) {
        throw new LongueurInvalideException("Longueur négative.");
    } else {
        rep = nb1 * nb2;
        System.out.println(rep);
    }
}

public static void main(String[] args) {
    try {
        calculerAireRectangle(15, 10);
        calculerAireRectangle(-5, 10);
    } catch (LongueurInvalideException errLong) {
        System.out.println(errLong);
    }
}
```

- Affichage au terminal lors de l'exécution du programme

```
(Longueur 15)*(Largeur 10) = 150
(Longueur -5)*(Largeur 10) = _enseignant_.exceptions.LongueurInvalideException: Longueur négative.
```

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

11.7 Conseil d'utilisation

27

Conseils sur l'utilisation des exceptions

- La gestion des exceptions ne doit pas remplacer un **test simple**. Il faut plus de temps pour lancer et intercepter une exception que pour effectuer un simple test (`if`).
- Il est parfois raisonnable d'encapsuler une tâche entière dans un bloc `try` plutôt que d'encapsuler chaque instruction dans son propre bloc `try`. Si l'une des instructions est impossible à exécuter, il est souvent préférable d'abandonner le reste de la tâche.
- Il est préférable d'utiliser les exceptions fournies par *Java* lorsque l'une de ces exceptions répond au besoin plutôt que de définir sa propre exception.
- Il est fortement suggéré de lancer et d'attraper des exceptions les plus précises possibles. Par exemple, si on s'attend à une division par zéro, il faut attraper seulement une `ArithmeticException` et non la classe parent, par exemple `Exception` . En effet, en attrapant une classe trop générique d'exception, on devrait traiter toutes les exceptions possibles de cette classe et de ses sous-classes, ce qui n'est pas souhaitable.

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

28

Conseils sur l'utilisation des exceptions

- Si un bloc `catch` est vide (aucune instruction entre les accolades), l'exception capturée est ignorée. Une telle utilisation de l'instruction `try/catch` n'est pas une bonne pratique, car il est préférable de toujours réaliser un traitement adapté lors de la capture d'une exception. Un bloc `catch` vide est très dangereux, car si une exception survient, elle sera passée sous silence et le programme continuera de fonctionner, ce qui peut engendrer des bogues difficiles à identifier.
- Il faut éviter d'insérer du code qui risque de lancer une exception dans un bloc `finally`.
- Il est préférable de séparer le traitement normal et le traitement d'erreur. Dans un `catch` seul le traitement et la correction de l'erreur doivent apparaître.

```
try{
   calculerAireRectangle(15,10);
   calculerAireRectangle(25,10);
}
```

Traitement normal

```
catch(LongueurInvalideException errLon) {
   System.out.println(errLon);
}
```

Traitement des erreurs

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

29

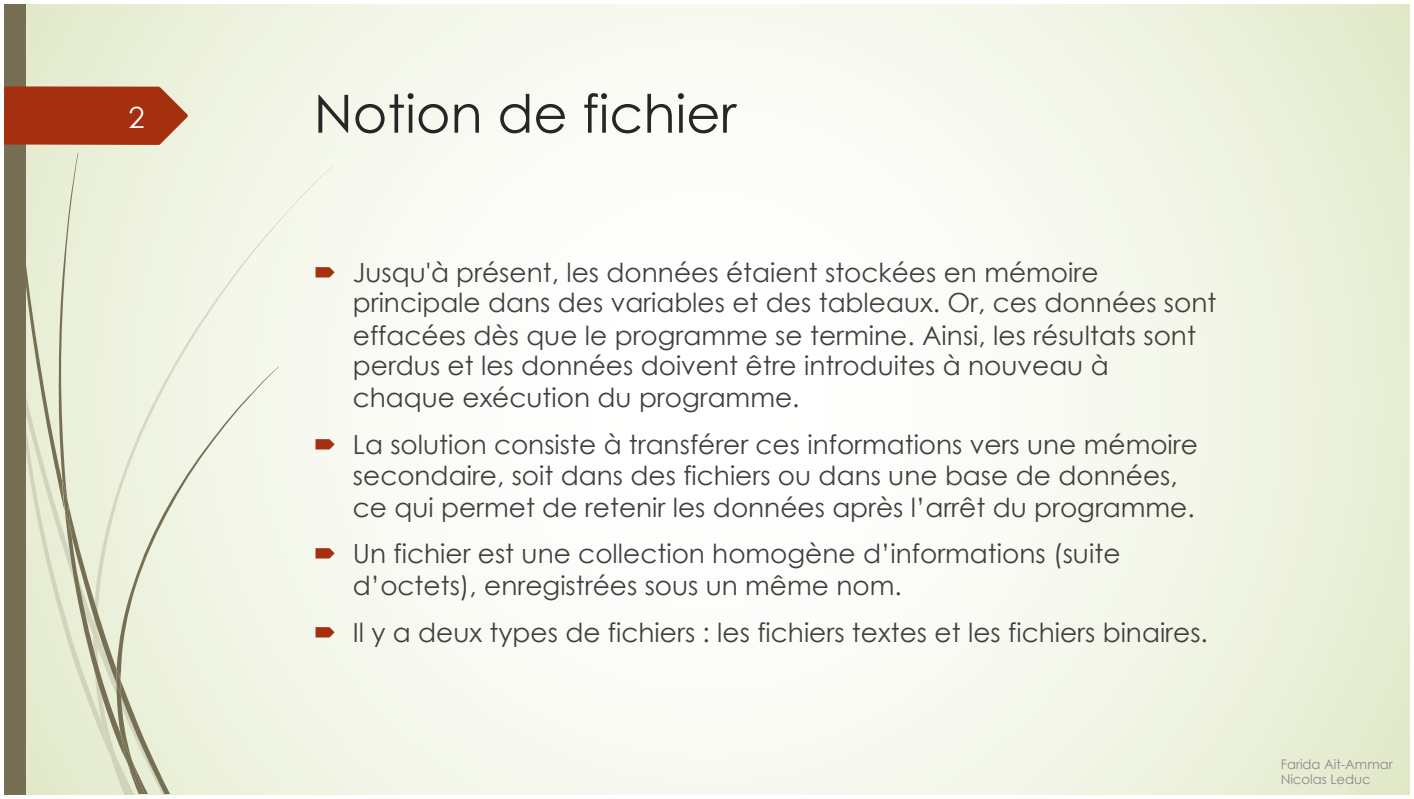
Sources

- [redacted]
- Document de Sotira Kyprianou : 9 – Gestion des exceptions(420-ZD4-MO)

Farida Ait-Ammar
Nicolas Leduc
Ouiza Ouyed

12. Flux et fichier

12.1 Type de fichier



3

Fichier texte

- Un fichier texte est un fichier dont le contenu est sous forme d'une suite de caractères, habituellement alphanumériques, découpée en lignes de longueurs variables.
- Avantages
  - Il peut être créé, lu et modifié par n'importe quel éditeur de texte
  - Habituellement, un humain peut facilement le comprendre et le modifier
- Inconvénients
  - Il est plus volumineux qu'un fichier binaire, par exemple, pour stocker des valeurs numériques
  - L'encodage des caractères peut causer des problèmes si tous les utilisateurs n'utilisent pas la même table de caractères (ex : *ANSI*, *ISO-8859*, *UTF-8*)
- Il existe plusieurs formats standards de fichiers textes, comme *CSV*, *JSON*, *HTML/XML*.

Farida Ait-Ammar
Nicolas Leduc

4

Fichier texte - Formats

- *CSV* : *Comma-Separated Values*
  - Chaque ligne du fichier correspond à une ligne d'un tableau et les colonnes (cellules) sont habituellement séparées par des virgules, mais d'autres séparateurs peuvent être utilisés.
  - Il est utile pour importer/exporter des données d'*Excel* ou de bases de données.
- *JSON* : *JavaScript Object Notation*
  - Format dérivé de la notation des objets en *JavaScript*. Il permet de représenter des listes ordonnées ou des objets en utilisant des associations clé : donnée.
  - Il est facile à manipuler, car des librairies sont disponibles dans la plupart des langages de programmation. On l'utilise, entre autres, pour les fichiers de configuration ou pour stocker des instances d'objets.
- *HTML/XML* : *eXtensible Markup Language*
  - Il est structuré à l'aide de balises et on dit qu'il est extensible, car l'usager peut définir ses propres balises selon ses besoins. On peut aussi associer des propriétés à chaque balise ce qui permet de représenter des structures de données plus complexes.
  - L'exemple le plus connu est le *HTML*, mais des fichiers *XML* sont aussi utilisés pour représenter des graphes ou pour stocker la configuration de certains logiciels.

Farida Ait-Ammar
Nicolas Leduc

5

Fichier texte - Exemples

CSV

2022-04-11 10:18:30 70 octets Tout le reste ANSI PC

```
Ark Nova;69,95;9;true
Piece of pie;19,79;7;false
1830;79,95;8;true
```

JSON

2022-04-11 12:38:33 18 339 octets Tout le reste ANSI PC

```
[
        {
                "Nom":"Ark Nova",
                "Prix":69.95,
                "Note":9,
                "Complet":true
        },
        {
                "Nom":"Piece of pie",
                "Prix":19.79,
                "Note":7,
                "Complet":false
        },
        {
                "Nom":"1830",
                "Prix":79.95,
                "Note":8,
                "Complet":true
        }
]
```

XML

2022-04-11 12:38:33 18 339 octets Tout le reste ANSI PC

```
<?xml version="1.0" encoding="UTF-8"?>
<jeux>
        <fichejeu>
                <nom>Ark Nova</nom>
                <prix>69.95</prix>
                <note>9</note>
                <complet>true</complet>
        </fichejeu>
        <fichejeu>
                <nom>Piece of pie</nom>
                <prix>19.79</prix>
                <note>7</note>
                <complet>false</complet>
        </fichejeu>
        <fichejeu>
                <nom>1830</nom>
                <prix>79.95</prix>
                <note>8</note>
                <complet>true</complet>
        </fichejeu>
</jeux>
```

Farida Ait-Ammar
Nicolas Leduc

6

Fichier binaire

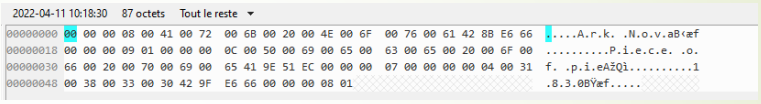
- Tout fichier qui ne contient pas uniquement des caractères intelligibles, donc qui n'est pas un fichier texte, est un fichier binaire. Toutefois, on peut retrouver des chaînes de caractères à l'intérieur des fichiers binaires, mais il est presque impossible de les modifier sans risquer d'endommager les fichiers.
- Les fichiers binaires servent à enregistrer des images, de la musique, des documents, des données compressées, etc. On s'en sert aussi pour sauvegarder le code compilé ou la version exécutable d'un logiciel lorsque l'on fait de la programmation.
- Avantages
  - Taille réduite par rapport au fichier texte équivalent
  - Meilleure performance lors de l'écriture/lecture, car les données contenues peuvent être mises directement en mémoire sans avoir besoin d'être transformées
- Inconvénients
  - Difficile à comprendre et à modifier sans outils spécialisés
  - Format propriétaire dont la structure du contenu n'est connue que de l'entreprise qui a créé le fichier
- Il existe de nombreux formats de fichiers binaires. Certains sont définis par des normes, comme *JPEG*, *WAV*, *MP3*, *ZIP*, etc. alors que d'autres sont des formats spécifiques à certaines applications, comme *XLS*, *WPD*, *PPT*, etc.

Farida Ait-Ammar
Nicolas Leduc

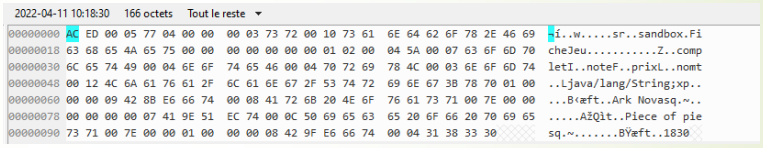
7

Fichier binaire - Exemples

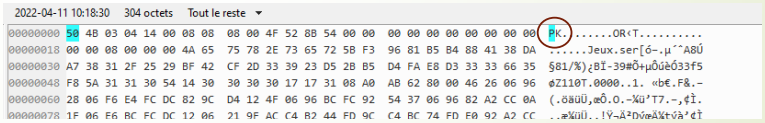
- Binaire



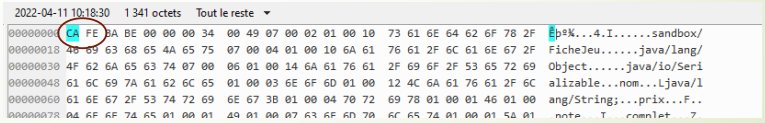
- Objet *Java*



- *Zip*



- Classe *Java*



Farida Ait-Ammar
Nicolas Leduc

8

Attention aux *endian* (boutisme)

- En informatique, les nombres encodés en `int` utilisent trente-deux bits soit quatre octets de huit bits. Chaque adresse mémoire permet de stocker un octet, donc on peut placer notre `int`, par exemple `012345AB`, de deux façons différentes :
  - L'octet le moins significatif à l'adresse ayant l'index le plus petit(-boutisme) (*little-endian*)

	[0]	[1]	[2]	[3]	
XX	AB	45	23	01	XX

  - L'octet le moins significatif à l'adresse ayant l'index le plus gros(-boutisme) (*big-endian*)

	[0]	[1]	[2]	[3]	
XX	01	23	45	AB	XX

- Comme dans l'œuvre de *Jonathan Swift*, *Les Voyages de Gulliver*, il y a un débat sur le boutisme, mais il n'y a pas de consensus. Les différents systèmes et logiciels utilisent une convention ou l'autre. Par exemple, *Intel* utilise le petit-boutisme alors que *Motorola* utilise le gros-boutisme.
- Tant que l'on reste sur le même système, la convention est uniforme et ne cause pas de problème. Par contre, lorsque l'on crée des fichiers binaires destinés à être partagés, il faut être conscient que le boutisme peut être différent et il faut en tenir compte. Par exemple, les fichiers codés en *UTF-16* débutent par `FFFE` ou `FEFF` afin de détecter le boutisme.
- Dans notre cas, étant donné que les fichiers sont écrits et lus par les librairies *Java*, le problème ne se pose pas. En effet, *Java* utilise la convention gros-boutisme sur toutes les plateformes.

Farida Ait-Ammar
Nicolas Leduc

12.2 Manipulation de flux

9

Flux *Java*

- En *Java*, l'accès aux ressources externes (fichiers, page web, etc) se fait à l'aide de flux (*Stream*). Les flux sont de deux types de base :
  - Flux de lecture (`*InputStream`) : Le flux contient des données en mémoire secondaire. Le programme va transférer ces données en mémoire principale pour les traiter ⇨ Flux en lecture.
  - Flux d'écriture (`*OutputStream`) : Les données existent en mémoire principale. Le programme va les sauvegarder dans un flux en mémoire secondaire afin de pouvoir y accéder lors d'une exécution subséquente ⇨ Flux en écriture.
- Les flux de base ne permettent que de manipuler des données binaires (`byte`). Pour faciliter la tâche du programmeur, *Java* propose d'autres types de flux, qui peuvent être combinés. Par exemple, il existe un flux pour les données texte, un pour les données compressées, un flux tampon, etc.

Farida Ait-Ammar
Nicolas Leduc

10

Manipulation des flux

- L'utilisation d'un flux par un programme comporte trois étapes :
  1. **Ouverture du flux** : Instancier la ou les classes qui correspondent au flux que l'on désire manipuler. On doit toujours s'assurer que l'ouverture a bien été réalisée et traiter les exceptions dans le cas contraire.
  2. **Lecture / écriture dans le flux** : Permet de lire ou d'écrire des données à l'aide des méthodes disponibles dans le flux instancié. Une lecture/écriture est toujours effectuée à la position courante du pointeur de lecture/écriture du flux. Par la suite, le pointeur avance automatiquement du nombre d'octets lus/écrits.
  3. **Fermeture du flux** : Un flux doit toujours être fermé afin de s'assurer que l'ensemble des informations ont été traitées. Dans le cas contraire, des pertes d'informations sont possibles. La fermeture permet aussi de libérer la ressource afin que d'autres logiciels puissent y accéder.

Farida Ait-Ammar
Nicolas Leduc

11

Ouverture de fichiers

- Pour accéder aux fichiers sur le disque de votre ordinateur à partir de votre programme, il faut créer un flux de type fichier (`File*`) dans la direction correspondant à l'opération voulue, soit lecture ou écriture. De plus, il faut indiquer le nom du fichier comme paramètre du constructeur.
  - Écriture : `FileOutputStream(String name)`
    - **ATTENTION** : L'ouverture d'un fichier en écriture efface complètement son contenu, s'il existe, sinon un nouveau fichier est créé.
  - Lecture : `FileInputStream(String name)`
- Si le fichier est introuvable, le constructeur peut lancer une exception de type `FileNotFoundException`, qu'il faut attraper à l'aide d'un bloc `try/catch` :

```
String nf = "Test.txt";
try {
    FileInputStream ficIn = new FileInputStream(nf);
    // Traitements
} catch (FileNotFoundException ex) {
    System.out.println("Fichier " + nf + " introuvable.");
}
```

Farida Ait-Ammar
Nicolas Leduc

12

Combiner les flux

- Le `File[IO]Stream` ne permet que d'écrire ou de lire des données binaires (`byte`) dans les fichiers. Il serait fastidieux de transformer toutes nos données en `byte` lors d'interactions avec les fichiers. C'est pour cette raison que d'autres flux existent, dont voici les principaux :
  - `OutputStreamWriter` et `BufferedWriter` : Écriture de fichiers textes
  - `InputStreamReader` et `BufferedReader` : Lecture de fichiers textes
  - `Data[IO]Stream` : Manipulation de fichiers binaires
  - `Object[IO]Stream` : Manipulation de fichiers objets de *Java*
  - `Zip[IO]Stream` et `GZIP[IO]Stream` : Manipulation de fichiers compressés
  - `Buffered[IO]Stream` : Permet d'utiliser un tampon afin d'accélérer l'accès
- Nous verrons les particularités de chacun de ces flux par la suite

Farida Ait-Ammar
Nicolas Leduc

13

Combiner les flux

- En général, les flux de la page précédente acceptent un flux comme paramètre de leur constructeur. Il est donc possible de les instancier en leur passant le `File[IO]Stream` lié à notre fichier ou d'autres flux afin de créer, par exemple, un fichier texte compressé.

```
// Ouverture d'un fichier binaire en lecture
String nf = "Test.bin";
DataInputStream ficBinIn = null;
try {
    // Le DataInputStream accédera à notre fichier en lecture
    ficBinIn = new DataInputStream(new FileInputStream(nf));
    // Traitements
} catch (FileNotFoundException ex) {
    System.out.println("Fichier " + nf + " introuvable.");
}
```

Farida Ait-Ammar
Nicolas Leduc

14

Fermeture de fichiers

- On doit fermer le flux indépendamment du succès ou de l'échec de l'opération de lecture ou d'écriture. Un fichier qui n'est pas fermé correctement peut causer toutes sortes de problèmes : perte de données, perte du fichier, erreurs intermittentes dans les programmes, etc.
- La méthode `close()` permet de fermer le fichier et de libérer les ressources du système d'exploitation qui y sont associées. On appelle la méthode `close()` sur le dernier flux instancié.
- La méthode `close()` lance une exception de type `IOException` si une erreur quelconque d'entrée/sortie survient. Si la gestion des exceptions est réalisée dans le programme, la fermeture du flux doit se faire dans le bloc `finally`. En effet, ce bloc est toujours exécuté, qu'il y ait eu une exception ou non.

```
String nf = "Test.bin";
DataOutputStream ficBinOut = null;
try {
    ficBinOut = new DataOutputStream(new FileOutputStream(nf)); // Ouverture
    // Traitements
} catch (FileNotFoundException ex) {
    System.out.println("Fichier " + nf + " introuvable.");
} finally {
    try {
        if (ficBinOut != null) {
            ficBinOut.close(); // Fermeture du flux s'il a été ouvert
        }
    } catch (IOException exc) {
        System.out.println("Problème de fermeture du fichier : " + nf);
    }
}
```

Farida Ait-Ammar
Nicolas Leduc

15

Fermeture automatique du flux

- *Java* propose une structure baptisée *try-with-resources*, qui prend automatiquement en charge la fermeture des fichiers.
- Les fichiers sont déclarés à l'intérieur des parenthèses qui suivent le mot clé `try`. Les instructions sont séparées par des points virgules.
  - `try (déclaration des fichiers) { … }`
- Il n'est ainsi plus nécessaire d'appeler la méthode `close()` dans le bloc `finally`, car *Java* s'occupe automatiquement de la fermeture du fichier lors de la sortie normale ou anormale du bloc `try`.

```
String nf = "Test.bin";
try (DataOutputStream ficBinOut = new DataOutputStream(new FileOutputStream(nf))) {
    // Traitements
} catch (FileNotFoundException ex) {
    System.out.println("Fichier " + nf + " introuvable.");
}
```

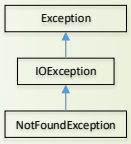
Farida Ait-Ammar
Nicolas Leduc

16

Gabarit – Manipulation de fichiers

```
import java.io.*;

public static void main(String[] args) {
    String nf = <nom du fichier>;
    <type de flux> fluxFic = null;        // Déclaration du flux
    try {
        // Instanciation / ouverture du flux
        // Traitements
    } catch (FileNotFoundException ex) { // Erreur d'ouverture
        System.out.println("Fichier " + nf + " introuvable.");
    } catch (IOException ex) {           // Erreur de lecture / écriture
        System.out.print(ex);
    } finally {
        try {
            if (fluxFic != null) {
                fluxFic.close();         // Fermeture du flux
            }
        } catch (IOException exc) {      // Erreur de fermeture
            System.out.println("Problème de fermeture du fichier : " + nf);
        }
    }
}
```



Farida Ait-Ammar
Nicolas Leduc

12.3 Fichier texte

17

Classe `FicheJeu`

FicheJeu
String nom;
float prix;
int note;
boolean complet;

- Classe – Utilisée dans tous les exemples de manipulation des fichiers

```
public class FicheJeu {
     public FicheJeu(String nom, float prix, int note, boolean complet) {
          this.nom     = nom;     // Nom du jeu
          this.prix    = prix;    // Coût d'achat
          this.note    = note;    // Évaluation du jeu [1;10]
          this.complet = complet; // Jeu sans pièce manquante
     }

     public String   getNom()     { return nom; }
     public float    getPrix()    { return prix; }
     public int      getNote()    { return note; }
     public boolean  getComplet() { return complet; }

     private String  nom;
     private float   prix;
     private int     note;
     private boolean complet;
}
```

- Instance – Données utilisées lors de l'écriture des fichiers

```
FicheJeu[] listeJeux = new FicheJeu[] {
     new FicheJeu("Ark Nova",     69.95f, 9, true),
     new FicheJeu("Piece of pie", 19.79f, 7, false),
     new FicheJeu("1830",         79.95f, 8, true)
};
```

Farida Ait-Ammar
Nicolas Leduc

18

Classes de flux – Fichier texte

- Le flux de base, `File[IO]Stream`, ne traite que les données binaires et n'est donc pas adapté pour la manipulation de fichiers textes. Nous allons l'encapsuler dans l'une des classes de flux suivantes :
  - `InputStreamReader` : Interprète le flux comme une série de caractères selon l'encodage spécifié. Permet de lire le flux un caractère à la fois.
  - `BufferedReader` : Accepte un `Reader` dans son constructeur et permet de lire le flux une ligne à la fois. Il sert aussi de tampon afin d'accélérer les accès au fichier en les combinant.
  - `OutputStreamWriter` : Permet d'écrire les caractères sur le flux en respectant l'encodage spécifié. Accepte des caractères ou des chaînes de caractères.
  - `BufferedWriter` : Ajoute la fonctionnalité pour faire des sauts de ligne. Il sert aussi de tampon afin d'accélérer les accès au fichier en les combinant.
  - `FileReader` : Équivaut à combiner `InputStreamReader` et `FileInputStream`
  - `FileWriter` : Équivaut à combiner `OutputStreamWriter` et `FileOutputStream`

Farida Ait-Ammar
Nicolas Leduc

19

Méthodes – Fichier texte

- La classe `BufferedWriter`, qui permet d'écrire du texte dans un fichier, offre, entre autres, deux méthodes d'écriture :
  - `write()` : Permet d'écrire un caractère ou une chaîne de caractères
    - Similaire au `System.out.print()`
  - `newLine()` : Permet d'insérer un saut de ligne qui respecte les conventions
- Pour la lecture, la classe `BufferedReader` possède la méthode :
  - `readLine()` : Permet de lire une ligne de texte jusqu'au caractère de fin de ligne. Elle retourne une chaîne contenant les caractères de la ligne, à l'exception du caractère de fin de ligne. Si cette méthode rencontre la fin du fichier, un pointeur `null` est retourné. On l'utilise habituellement de cette façon :

```
String uneLigne;
while ( (uneLigne = ficTextIn.readLine()) != null ) { … }
```

Farida Ait-Ammar
Nicolas Leduc

FicheJeu

```
String nom;
float prix;
int note;
boolean complet;
```

20

Exemple – Écriture texte

- Dans cet exemple, on utilise le `String.format()` dans le `write()` afin d'écrire les informations dans le format voulu. On se sert du point-virgule comme séparateur afin de produire un fichier au format *CSV*.

```
String nf = "Jeux.csv";
BufferedWriter ficTextOut =  null;
try {
    ficTextOut = new BufferedWriter(new OutputStreamWriter(new FileOutputStream(nf)));
    // ficTextOut = new BufferedWriter(new FileWriter(nf)); // Équivalent à la ligne précédente
    for (FicheJeu unJeu : listeJeux) {
        ficTextOut.write(String.format("%s;%.2f;%d;%b",
            unJeu.getNom(), unJeu.getPrix(), unJeu.getNote(), unJeu.getComplet() ));
        ficTextOut.newLine();
    }
} catch (...) {
```

2022-04-11 10:18:30 70 octets Tout le reste ANSI PC

```
Ark Nova;69,95;9;true
Piece of pie;19,79;7;false
1830;79,95;8;true
```

Farida Ait-Ammar
Nicolas Leduc

21

Exemple – Lecture texte

```
FicheJeu
String nom;
float prix;
int note;
boolean complet;
```

- Dans cet exemple, on lit une ligne à la fois, on la découpe en utilisant le point-virgule comme délimiteur et on construit les objets `FicheJeu` correspondants en prenant soin de faire les conversions de données nécessaires. En effet, on doit convertir certaines chaînes en valeurs.

```
String nf = "Jeux.csv";
FicheJeu[] listeLue = new FicheJeu[3];
BufferedReader ficTextIn = null;
try {
    ficTextIn = new BufferedReader(new InputStreamReader(new FileInputStream(nf)));
    // ficTextIn = new BufferedReader(new FileReader(nf)); // Équivalent à la ligne précédente
    String uneLigne;
    int idxLigne = 0;
    while ( (uneLigne = ficTextIn.readLine()) != null ) {  // Tant qu'il reste des lignes à lire
        String[] elems = uneLigne.split(";");              // Découpe en utilisant le point-virgule
        listeLue[idxLigne++] = new FicheJeu(elems[0],      // Nom du jeu sans conversion
            Float.parseFloat(elems[1].replace(',', '.')), // Prix : virgule remplacée par point
            Integer.parseInt(elems[2]),                    // Conversion de note
            Boolean.parseBoolean(elems[3]));               // Conversion de complet
    }
} catch (...) {
```

Farida Ait-Ammar
Nicolas Leduc

12.4 Fichier binaire

22

Traitements – Fichier binaire

- En ce qui concerne les fichiers binaires, nous verrons d'abord comment écrire manuellement chacune des données dans notre propre format.
- Pour écrire un fichier binaire, nous devons utiliser la classe `DataOutputStream`, qui contient une méthode `write*()` pour chaque type de base *Java*. Toutes ces méthodes utilisent la convention gros-boutisme.
  - `writeBoolean()`, `writeByte()`, `writeChar()`, `writeDouble()`, `writeFloat()`, `writeInt()`, `writeLong()`, `writeShort()` : Permet d'écrire une seule donnée du type spécifié.
  - `writeChars()` : Permet d'écrire une séquence de caractères. (Attention : voir ci-dessous)
- Pour lire les données binaires contenues dans le fichier, nous instancions la classe `DataInputStream`, qui contient les méthodes analogues au `DataOutputStream`, et qui respecte la convention gros-boutisme.
  - `readBoolean()`, `readByte()`, `readChar()`, `readDouble()`, `readFloat()`, `readInt()`, `readLong()`, `readShort()` : Permet de lire une seule donnée du type spécifié.
  - Il n'existe pas de méthode `readChars()` pour lire une chaîne de caractères, car *Java* n'utilise pas de marqueur de fin de chaîne lorsqu'il l'écrit. Pour lire une chaîne, on doit alors connaître sa taille au préalable. Il faut soit enregistrer la taille de la chaîne dans le fichier ou forcer toutes les chaînes à avoir la même taille.
  - `available()` : Détermine s'il reste des données à lire dans le fichier. Retourne un estimé du nombre de données disponibles. S'il retourne zéro, la fin du fichier a été atteinte.

Farida Ait-Ammar
Nicolas Leduc

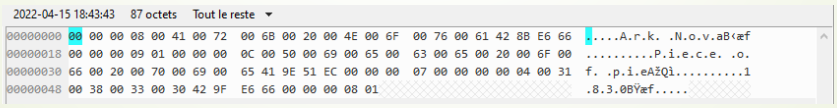
FicheJeu
String nom;
float prix;
int note;
boolean complet;

23

Exemple – Écriture binaire v1

- Dans cet exemple, on écrit les données de la classe `FicheJeu` les unes à la suite des autres. Pour le nom, on sauvegarde d'abord la taille de la chaîne avant d'enregistrer les caractères qui la composent. Lors de la lecture, il faudra lire les données dans l'ordre d'écriture.

```
String nf = "Jeux.bin";
DataOutputStream ficBinOut = null;
try {
    ficBinOut = new DataOutputStream(new FileOutputStream(nf));
    for (FicheJeu unJeu : listeJeux) {
        ficBinOut.writeInt(unJeu.getNom().length());    // Taille de la chaîne    (4 octets)
        ficBinOut.writeChars(unJeu.getNom());           // Nom du jeu en UTF-16   (2 octets/char)
        ficBinOut.writeFloat(unJeu.getPrix());          // Prix en float          (4 octets)
        ficBinOut.writeInt(unJeu.getNote());            // Note en int            (4 octets)
        ficBinOut.writeBoolean(unJeu.getComplet());     // Complet en boolean     (1 octet)
    }
catch (...) {
```



Farida Ait-Ammar
Nicolas Leduc

FicheJeu
String nom;
float prix;
int note;
boolean complet;

24

Exemple – Lecture binaire v1

- Pour lire le fichier binaire, nous devons connaître son format. Comme c'est nous qui l'avons écrit, il suffit de lire les données dans le même ordre qu'elles ont été écrites. Pour lire la chaîne, nous récupérons d'abord sa taille, puis nous lisons chacun des caractères dans une boucle `for`.

```
String nf = "Jeux.bin";
FicheJeu[] listeLue = new FicheJeu[3];
DataInputStream ficBinIn = null;
try {
    ficBinIn = new DataInputStream(new FileInputStream(nf));
    int idxLigne = 0;
    while (ficBinIn.available() > 0) {                  // Détermine s'il reste des données à lire
        int nbChars = ficBinIn.readInt();               // Lis le nombre de caractères (4 octets)
        StringBuilder nom = new StringBuilder();
        for (int c = 0; c < nbChars; ++c) {             // Ajoute chacun des caractères
            nom.append(ficBinIn.readChar());            // lus dans une chaîne (2 octets/char)
        }
        listeLue[idxLigne++] = new FicheJeu(nom.toString(),// Nom du jeu
            ficBinIn.readFloat(),                       // Prix du jeu          (float : 4 octets)
            ficBinIn.readInt(),                         // Note                 (int : 4 octets)
            ficBinIn.readBoolean());                    // Complet              (boolean : 1 octet)
    }
catch (...) {
```

Farida Ait-Ammar
Nicolas Leduc

FicheJeu
String nom;
float prix;
int note;
boolean complet;

25

Exemple – Écriture binaire v2

- La différence avec l'exemple précédent est qu'ici on enregistre le nombre de `FicheJeu` contenu dans le tableau au début du fichier. L'opération de lecture pourra ainsi se faire à l'aide d'une boucle `for` au lieu d'une boucle `while`.

```
String nf = "Jeux.bin";
DataOutputStream ficBinOut = null;
try {
    ficBinOut = new DataOutputStream(new FileOutputStream(nf));
    ficBinOut.writeInt(listeJeux.length);              // Nombre de FicheJeu     (4 octets)
    for (FicheJeu unJeu : listeJeux) {
        ficBinOut.writeInt(unJeu.getNom().length());   // Taille de la chaîne    (4 octets)
        ficBinOut.writeChars(unJeu.getNom());          // Nom du jeu en UTF-16   (2 octets/char)
        ficBinOut.writeFloat(unJeu.getPrix());         // Prix en float          (4 octets)
        ficBinOut.writeInt(unJeu.getNote());           // Note en int            (4 octets)
        ficBinOut.writeBoolean(unJeu.getComplet());    // Complet en boolean     (1 octet)
    }
} catch (...) {
```

2022-04-15 18:43:43 91 octets Tout le reste

```
00000000  00 00 00 03 00 00 00 08  00 41 00 72 00 6B 00 20  00 4E 00 6F 00 76 00 61  ........A.r.k. .N.o.v.a
00000018  42 88 E6 66 00 00 00 09  01 00 00 00 0C 00 50 00  69 00 65 00 63 00 65 00  B‹æf..........P.i.e.c.e.
00000030  20 00 6F 00 66 00 20 00  70 00 69 00 65 41 9E 51  EC 00 00 00 07 00 00 00   .o.f. .p.i.eAžQì.......
00000048  00 04 00 31 00 38 00 33  00 30 42 9F E6 66 00 00  00 08 01                 ...1.8.3.0BŸæf.....
```

Farida Ait-Ammar
Nicolas Leduc

FicheJeu
String nom;
float prix;
int note;
boolean complet;

26

Exemple – Lecture binaire v2

- Lors de la lecture, on récupère d'abord le nombre de `FicheJeu` contenu dans le fichier. Il est alors possible d'initialiser un tableau de la bonne taille et de lire chacune des instances présentes dans le fichier à l'aide d'une boucle `for`.

```
String nf = "Jeux.bin";
FicheJeu[] listeLue = null;
DataInputStream ficBinIn = null;
try {
    ficBinIn = new DataInputStream(new FileInputStream(nf));
    int nbJeux = ficBinIn.readInt();                   // Lecture du nombre de FicheJeu (int : 4 octets)
    listeLue = new FicheJeu[nbJeux];                   // Initialisation du tableau de la taille lue
    for (int i = 0; i < nbJeux; ++i) {
        int nbChars = ficBinIn.readInt();              // Lecture de la taille du nom    (int : 4 octets)
        StringBuilder nom = new StringBuilder();
        for (int c = 0; c < nbChars; ++c) {
            nom.append(ficBinIn.readChar());           // Lecture de chaque caractère    (2 octets/char)
        }
        listeLue[i] = new FicheJeu(nom.toString(),     // Nom du jeu
            ficBinIn.readFloat(),                      // Prix du jeu                    (float : 4 octets)
            ficBinIn.readInt(),                        // Note                           (int : 4 octets)
            ficBinIn.readBoolean());                   // Complet                        (boolean : 1 octet)
    }
} catch (...) {
```

Farida Ait-Ammar
Nicolas Leduc

12.5 Fichier objet *Java*

27

Fichier objet *Java* - `Serializable`

- Écrire ou lire des objets complexes dans un fichier peut devenir fastidieux si nous devons tout gérer manuellement. Heureusement, *Java* propose les classes `ObjectOutputStream` et `ObjectInputStream` afin d'écrire et de lire des objets directement. Celles-ci implémentent un protocole spécifique qui enregistre suffisamment d'informations afin de valider les objets contenus dans le fichier.
- Pour utiliser ces classes de flux, nos objets doivent implanter l'interface `Serializable`. Pour ce faire, il suffit simplement de l'ajouter après le nom de notre classe, car elle ne contient aucune méthode associée. Nous devons toutefois ajouter la variable membre suivante, qui permet de savoir si l'objet dans le fichier correspond à la bonne version de la classe :

```
private static final long serialVersionUID = 1L;
```

- Avantages
  - Facilite l'écriture d'objets sérialisables dans un fichier
  - Permet de gérer automatiquement les structures complexes, comme les tableaux ou les objets à l'intérieur d'autres objets
- Désavantages
  - Difficile de partager les fichiers générés avec d'autres applications
  - Protocole complexe à décoder

Farida Ait-Ammar
Nicolas Leduc

28

FicheJeu
String nom;
float prix;
int note;
boolean complet;

Classe `FicheJeu` sérialisable

- Ajout de l'interface `Serializable` afin de pouvoir écrire `FicheJeu` dans un fichier objet :

```
public class FicheJeu implements Serializable {
    public FicheJeu(String nom, float prix, int note, boolean complet) {
        this.nom     = nom;      // Nom du jeu
        this.prix    = prix;     // Coût d'achat
        this.note    = note;     // Évaluation du jeu [1;10]
        this.complet = complet;  // Jeu sans pièce manquante
    }

    public String   getNom()     { return nom; }
    public float    getPrix()    { return prix; }
    public int      getNote()    { return note; }
    public boolean  getComplet() { return complet; }

    private String  nom;
    private float   prix;
    private int     note;
    private boolean complet;
    private static final long serialVersionUID = 1L;
}
```

Farida Ait-Ammar
Nicolas Leduc

29

Traitements – Fichier objet *Java*

- La classe `ObjectOutputStream` offre les mêmes méthodes d'écriture que `DataOutputStream`, mais enregistre les informations différemment. De plus, la méthode suivante s'ajoute :
  - `writeObject()` : Permet d'écrire un objet sérialisable dans le fichier. Si l'objet reçu en paramètre n'implémente pas l'interface `Serializable`, une exception `NotSerializableException` est lancée.
- Pour lire un fichier objet, la classe `ObjectInputStream` contient les mêmes méthodes que `DataInputStream`, en plus de la méthode suivante :
  - `readObject()` : Celle-ci retourne un objet générique qu'il faut convertir explicitement (*cast*) en l'objet attendu. Elle peut lancer l'exception `ClassNotFoundException`, qu'il faut obligatoirement attraper et gérer.
- **Attention** : Les classes `Object[IO]Stream` et `Data[IO]Stream` ne sont pas interchangeables, même si elles contiennent essentiellement les mêmes méthodes. En effet, la structure du fichier obtenu est très différente!

Farida Ait-Ammar
Nicolas Leduc

30

Exemple – Écriture objet v1

FicheJeu
`String nom;`
`float prix;`
`int note;`
`boolean complet;`

- Dans cet exemple, on sauvegarde d'abord le nombre d'objets contenus dans le tableau avant d'écrire chacun des objets directement.

```
String nf = "Jeux.ser";
ObjectOutputStream ficObjOut = null;
try {
    ficObjOut = new ObjectOutputStream(new FileOutputStream(nf));
    ficObjOut.writeInt(listeJeux.length);  // Taille du tableau
    for (FicheJeu unJeu : listeJeux) {
        ficObjOut.writeObject(unJeu);      // Écriture de chaque objet FicheJeu
    }
} catch (NotSerializableException ex) {
    System.out.print(ex);
} catch (...) {
```

2022-04-16 09:41:01 166 octets Tout le reste

```
00000000 AC ED 00 05 77 04 00 00  00 03 73 72 00 10 73 61  6E 64 62 6F 78 2E 46 69  ¬í..w.....sr..sandbox.Fi
00000018 63 68 65 4A 65 75 00 00  00 00 00 00 00 01 02 00  04 5A 00 07 63 6F 6D 70  cheJeu..........Z..comp
00000030 6C 65 74 49 00 04 6E 6F  74 65 46 00 04 70 72 69  78 4C 00 03 6E 6F 6D 74  letI..noteF..prixL..nomt
00000048 00 12 4C 6A 61 76 61 2F  6C 61 6E 67 2F 53 74 72  69 6E 67 3B 78 70 01 00  ..Ljava/lang/String;xp..
00000060 00 00 09 42 8B E6 66 74  00 08 41 72 6B 20 4E 6F  76 61 73 71 00 7E 00 00  ...B‹æft..Ark Novasq.~..
00000078 00 00 00 00 07 41 9E 51  EC 74 00 0C 50 69 65 63  65 20 6F 66 20 70 69 65  .....AžQìt..Piece of pie
00000090 73 71 00 7E 00 00 01 00  00 00 08 42 9F E6 66 74  00 04 31 38 33 30        sq.~.......BŸæft..1830
```

Farida Ait-Ammar
Nicolas Leduc

FicheJeu
String nom;
float prix;
int note;
boolean complet;

31

Exemple – Lecture objet v1

- On doit lire d'abord le nombre d'objets contenus dans le fichier afin d'instancier un tableau de la bonne taille. On lit ensuite chaque objet directement à l'aide d'une boucle `for`. On doit faire la conversion explicite (*cast*) de chaque objet lu en `FicheJeu`.

```
String nf = "Jeux.ser";
FicheJeu[] listeLue = null;
ObjectInputStream ficObjIn = null;
try {
    ficObjIn = new ObjectInputStream(new FileInputStream(nf));
    int nbObj = ficObjIn.readInt();                      // Lecture du nombre d'objets
    listeLue = new FicheJeu[nbObj];                      // Allocation du tableau
    for (int i = 0; i < nbObj; ++i) {                    // Lecture de chaque objet
        listeLue[i] = (FicheJeu) ficObjIn.readObject(); // Conversion explicite nécessaire
    }
} catch (ClassNotFoundException ex) {
    System.out.print(ex);
} catch (...) {
```

Farida Ait-Ammar
Nicolas Leduc

FicheJeu
String nom;
float prix;
int note;
boolean complet;

32

Exemple – Écriture objet v2

- Comme un tableau *Java* est un objet sérialisable, on peut l'écrire directement, ainsi que tout son contenu, en une seule commande.

```
String nf = "Jeux.ser";
ObjectOutputStream ficObjOut = null;
try {
    ficObjOut = new ObjectOutputStream(new FileOutputStream(nf));
    ficObjOut.writeObject(listeJeux);  // Écriture du tableau avec tous les objets FicheJeu
} catch (NotSerializableException ex) {
    System.out.print(ex);
} catch (...) {
```

2022-04-16 09:41:01 200 octets Tout le reste

```
00000000 AC ED 00 05 75 72 00 13  5B 4C 73 61 6E 64 62 6F  78 2E 46 69 63 68 65 4A  ¬í..ur..[Lsandbox.FicheJ
00000018 65 75 3B C5 E5 CC 98 9A  80 4D 3A 02 00 00 78 70  00 00 00 03 73 72 00 10  eu;ÅåÌ˜š€M:...xp....sr..
00000030 73 61 6E 64 62 6F 78 2E  46 69 63 68 65 4A 65 75  00 00 00 00 00 00 00 01  sandbox.FicheJeu........
00000048 02 00 04 5A 00 07 63 6F  6D 70 6C 65 74 49 00 04  6E 6F 74 65 46 00 04 70  ...Z..completI..noteF..p
00000060 72 69 78 4C 00 03 6E 6F  6D 74 00 12 4C 6A 61 76  61 2F 6C 61 6E 67 2F 53  rixL..nomt..Ljava/lang/S
00000078 74 72 69 6E 67 3B 78 70  01 00 00 00 09 42 8B E6  66 74 00 08 41 72 6B 20  tring;xp.....B‹æft..Ark
00000090 4E 6F 76 61 73 71 00 7E  00 02 00 00 00 00 07 41  9E 51 EC 74 00 0C 50 69  Novasq.~.......AžQìt..Pi
000000A8 65 63 65 20 6F 66 20 70  69 65 73 71 00 7E 00 02  01 00 00 00 08 42 9F E6  ece of piesq.~.......BŸæ
000000C0 66 74 00 04 31 38 33 30                                                    ft..1830
```

Farida Ait-Ammar
Nicolas Leduc

33

Exemple – Lecture objet v2

```
FicheJeu
String nom;
float prix;
int note;
boolean complet;
```

- Lors de la lecture, il suffit de faire la bonne conversion explicite, soit un tableau de `FicheJeu`. Le tableau ainsi obtenu est identique à celui qui avait été écrit.

```
String nf = "Jeux.ser";
FicheJeu[] listeLue = null;
ObjectInputStream ficObjIn = null;
try {
    ficObjIn = new ObjectInputStream(new FileInputStream(nf));
    listeLue = (FicheJeu[]) ficObjIn.readObject(); // Lecture et conversion explicite
} catch (ClassNotFoundException ex) {
    System.out.print(ex);
} catch (...) {
```

Farida Ait-Ammar
Nicolas Leduc

12.6 Fichier compressé

34

Fichiers compressés

- Il est facile de produire des fichiers compressés à l'aide de *GZip* ou de *Zip* en utilisant les flux *Java*. En effet, il suffit d'encapsuler le `File[IO]Stream` avec le flux correspondant à la méthode de compression voulue. On encapsule enfin ce nouveau flux avec celui que nous utilisons normalement pour écrire ou lire les données.

- Exemple :
  - Écriture d'un fichier binaire :
    ```
    DataOutputStream ficBinOut = new DataOutputStream(new FileOutputStream(nf));
    ```
  - Écriture d'un fichier binaire compressé avec *Gzip* :
    ```
    DataOutputStream ficGZOut = new DataOutputStream(new GZIPOutputStream(new FileOutputStream(nf)));
    ```

Farida Ait-Ammar
Nicolas Leduc

```
FicheJeu
String nom;
float prix;
int note;
boolean complet;
```

35

Exemple – Écriture et lecture - *GZip*

- En format *GZip*, le fichier compressé ne contient qu'un seul fichier. Pour inclure plusieurs fichiers, il faut utiliser l'utilitaire `tar`, mais ceci dépasse le cadre du cours. Les deux exemples suivants permettent d'écrire et de lire un seul fichier d'objets compressé en *GZip*.
- Écriture

```
String nf = "Jeux.ser.gz";
ObjectOutputStream ficGZOut = null;
try {
     ficGZOut = new ObjectOutputStream(new GZIPOutputStream(new FileOutputStream(nf)));
     ficGZOut.writeObject(listeJeux);
} catch (...) {
```

- Lecture

```
String nf = "Jeux.ser.gz";
FicheJeu[] listeLue = null;
ObjectInputStream ficGZIn = null;
try {
     ficGZIn = new ObjectInputStream(new GZIPInputStream(new FileInputStream(nf)));
     listeLue = (FicheJeu[]) ficGZIn.readObject();
} catch (...) {
```

Farida Ait-Ammar
Nicolas Leduc

```
FicheJeu
String nom;
float prix;
int note;
boolean complet;
```

36

Exemple – Écriture et lecture - *Zip*

- Le format *Zip* permet de rassembler plusieurs fichiers dans un seul fichier compressé. Il est alors nécessaire d'indiquer les noms des fichiers inclus dans l'archive. L'écriture d'un fichier *Zip* est donc un peu plus complexe que pour le format *GZip*.
- Écriture

```
String nf = "Jeux.zip";
ObjectOutputStream ficZipOut = null;
try {
     ZipOutputStream zos = new ZipOutputStream(new FileOutputStream(nf));                // Création du Zip
     zos.putNextEntry(new ZipEntry(nf.replace("zip", "ser")));        // Ajout du nom du fichier dans le Zip
     ficZipOut = new ObjectOutputStream(zos);
     ficZipOut.writeObject(listeJeux);
} catch (...) {
```

- Lecture

```
String nf = "Jeux.zip";
FicheJeu[] listeLue = null;
ObjectInputStream ficZipIn = null;
try {
     ZipInputStream zis = new ZipInputStream(new FileInputStream(nf));                   // Ouverture du Zip
     zis.getNextEntry();                                    // Extraction d'un fichier contenu dans le Zip
     ficZipIn = new ObjectInputStream(zis);
     listeLue = (FicheJeu[]) ficZipIn.readObject();
} catch (...) {
```

Farida Ait-Ammar
Nicolas Leduc

37

Sources

- [redacted]
- Document de Sotira Kyprianou : 8 - Fichiers textes (420-ZD4-MO)

Farida Ait-Ammar
Nicolas Leduc

A. Guide d'installation

A.1 *Java JDK* 25

A.1.1 Validation de l'installation

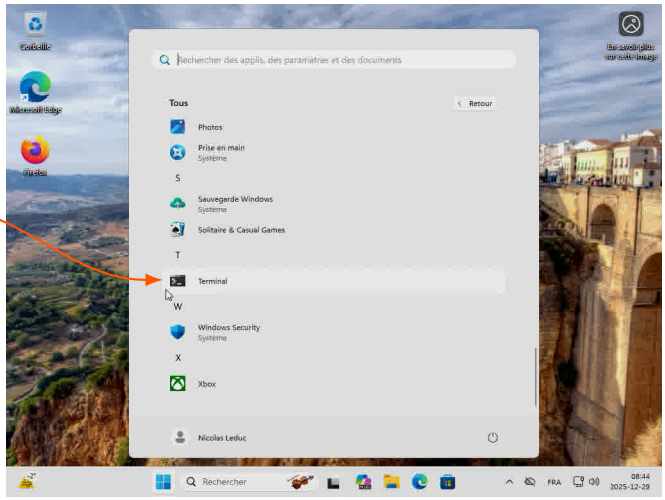
Afin de valider qu'un *Java JDK* 25 ou plus récent est installé sur votre ordinateur, utilisez la procédure suivante :

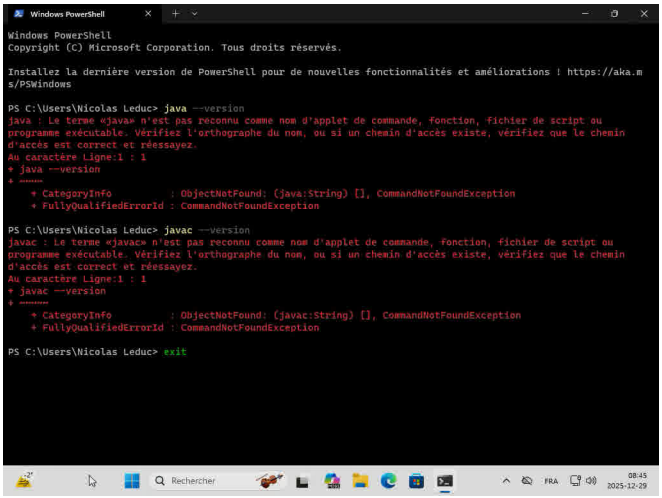
- Ouvrez le terminal (*Windows* -> *Tous* -> *Terminal*)
- Sur la ligne de commande inscrivez :

```
$ java --version
$ javac --version
```

- Observez le résultat puis fermez la fenêtre en tapant :

```
$ exit
```

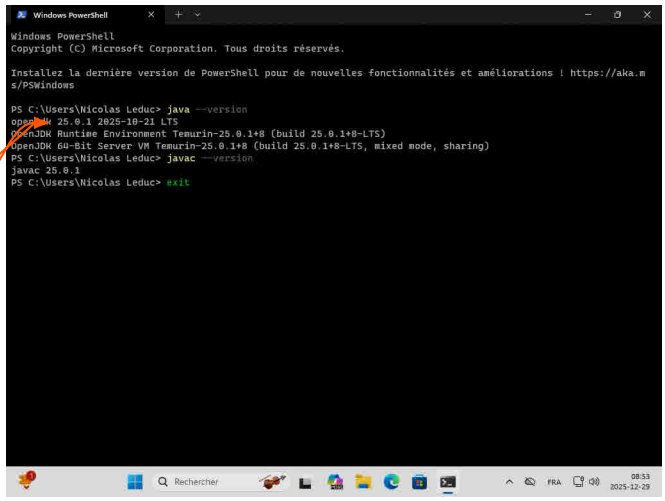




Si on vous indique que la commande n'est pas reconnue, il faut installer le *JDK*, tel qu'indiqué dans les pages suivantes.

Si un *JDK* est installé, le numéro de version est alors affiché.

Si le numéro débute par 25 ou plus, vous avez une version adéquate du *Java JDK*. Dans le cas contraire, il faut installer le *JDK*, tel qu'indiqué dans les pages suivantes.

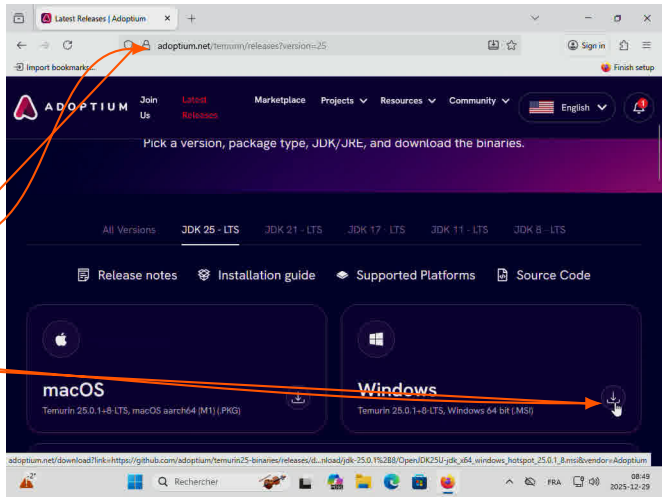


A.1.2 Installation du *JDK*

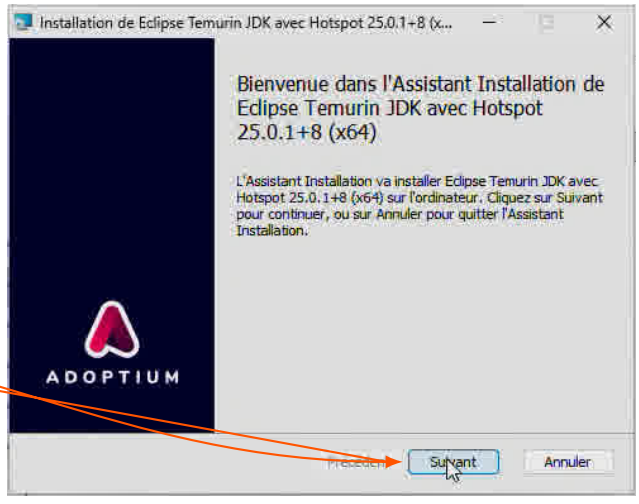
Pour installer le *Java JDK* 25 sur votre ordinateur, exécutez la procédure suivante :

- Ouvrez un navigateur web et collez le lien ci-après dans la barre d'adresse :

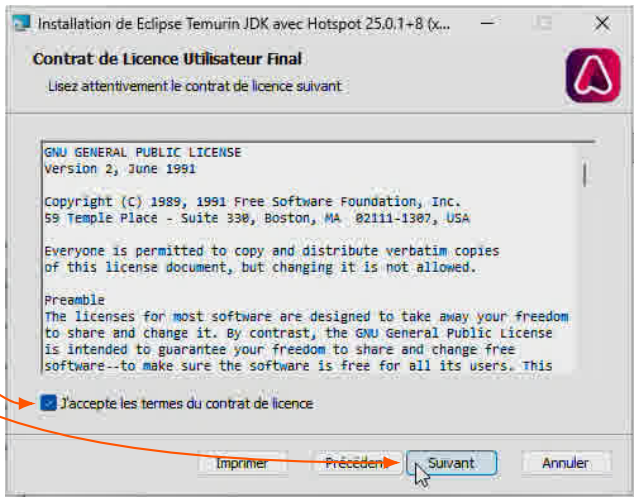
  [redacted]
- Cliquez sur le bouton pour télécharger la version correspondante à votre système d'exploitation, habituellement *Windows*
- Le *JDK* ainsi téléchargé se trouve normalement dans le dossier *Téléchargements*



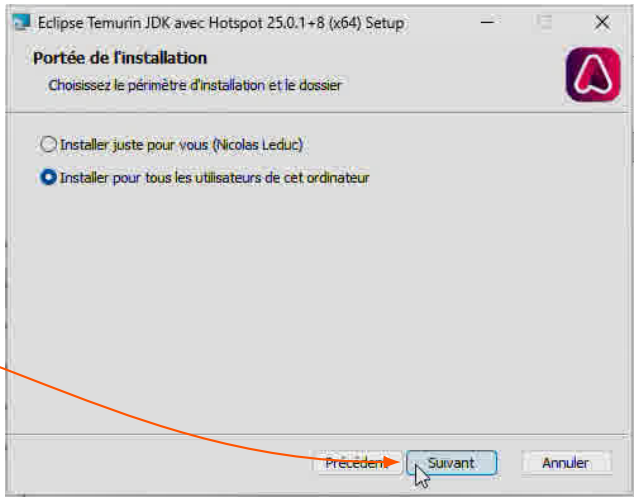
- Ouvrez un explorateur de fichiers, puis sélectionnez le dossier *Téléchargements* et double cliquez sur le fichier que vous venez de télécharger, soit `OpenJDK*`, afin de démarrer l'*Assistant Installation* du *JDK*
- Cliquez sur le bouton *Suivant*



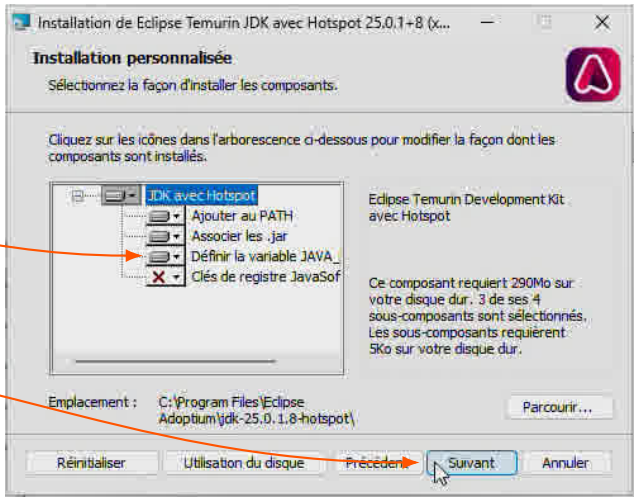
- Acceptez le *Contrat de Licence*
- Cliquez sur le bouton *Suivant*



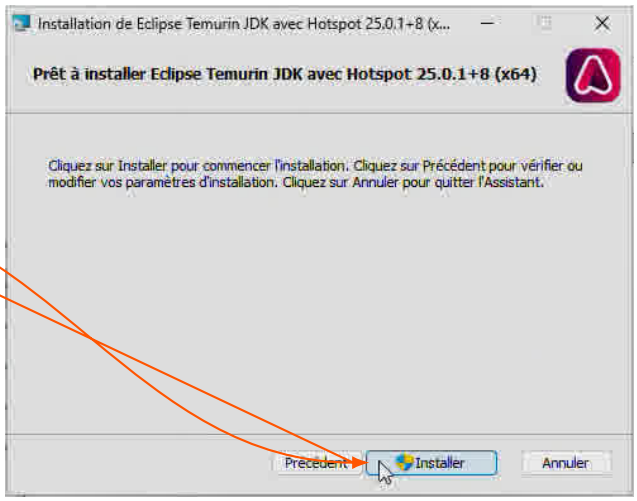
- Faites l'installation pour tous les utilisateurs de l'ordinateur (option par défaut)
- Cliquez sur le bouton *Suivant*



- Activez l'installation pour *Définir la variable* `JAVA_HOME`
- Conservez les deux options d'installation par défaut, soit *Ajouter au* `PATH` et *Associer les* `.jar`
- Gardez l'emplacement d'installation proposé
- Cliquez sur le bouton *Suivant*

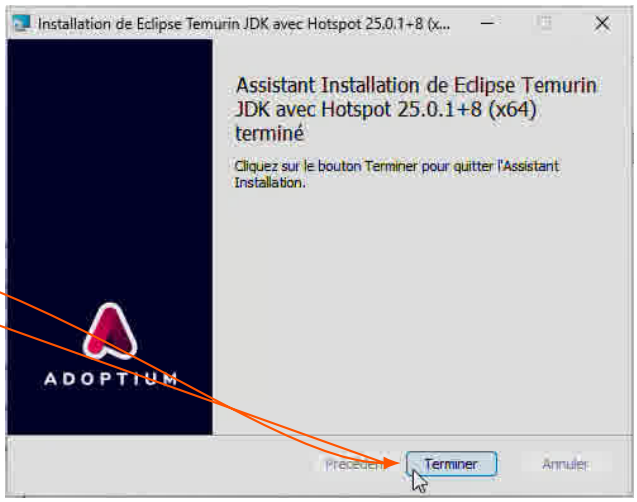


- Cliquez maintenant sur le bouton *Installer*
- Si on vous demande *Voulez-vous autoriser cette application à apporter des modifications à votre appareil ?* , cliquez sur *Oui*



- Après un peu d'attente, l'installation est terminée
- Cliquez enfin sur le bouton *Terminer*

Il est conseillé de refaire la procédure de la section A.1.1 afin de confirmer que le *Java JDK* est correctement installé.

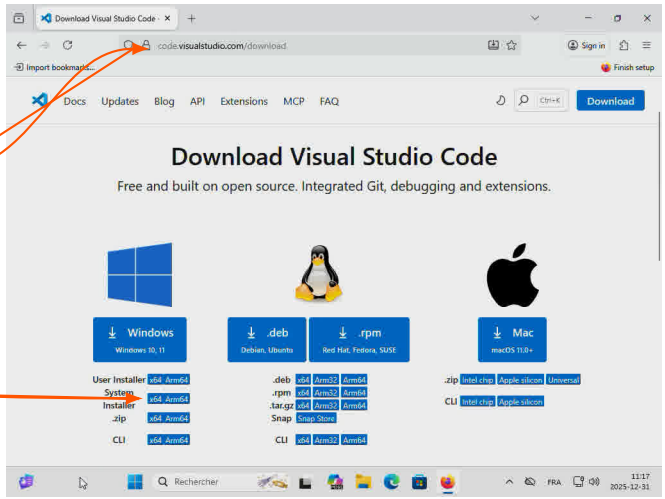


A.2 *Visual Studio Code*

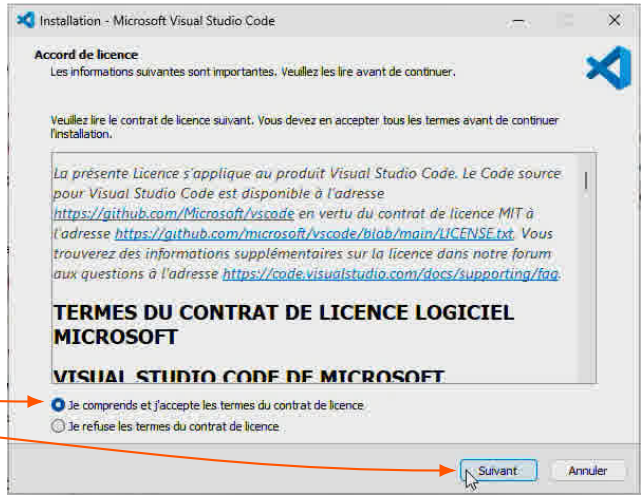
Pour installer *Visual Studio Code* sur votre ordinateur, exécutez la procédure suivante :

- Ouvrez un navigateur web et collez le lien ci-après dans la barre d'adresse :

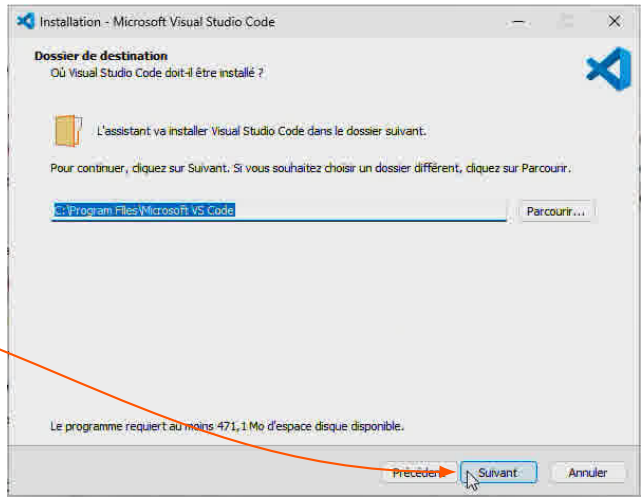
  [redacted]
- Vous pouvez l'installer uniquement dans votre compte d'utilisateur (*User Installer*) ou pour tous les utilisateurs (*System Installer*). Cliquez sur le bouton afin de télécharger la version pour tous les utilisateurs qui correspond à votre système d'exploitation, habituellement *Windows x64*
- L'installateur de *Visual Studio Code* ainsi téléchargé se trouve normalement dans le dossier *Téléchargements*



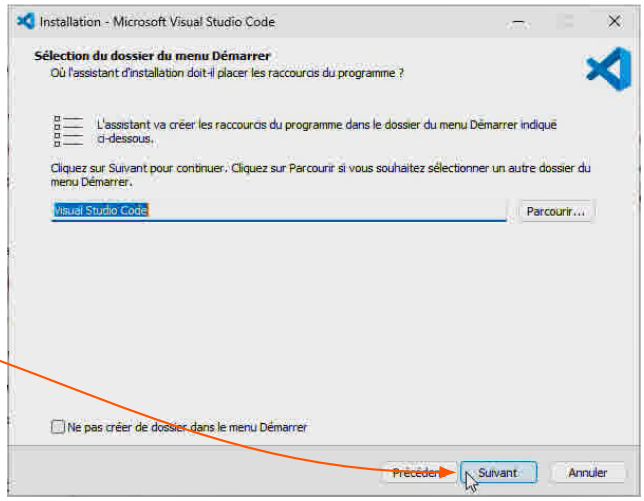
- Ouvrez un explorateur de fichiers, puis sélectionnez le dossier *Téléchargements* et double cliquez sur le fichier que vous venez de télécharger, soit `VSCodeSetup*` , afin de démarrer l'installation de *Visual Studio Code*
- Si on vous demande *Voulez-vous autoriser cette application à apporter des modifications à votre appareil ?* , cliquez sur *Oui*
- Acceptez le *Contrat de Licence*
- Cliquez sur le bouton *Suivant*



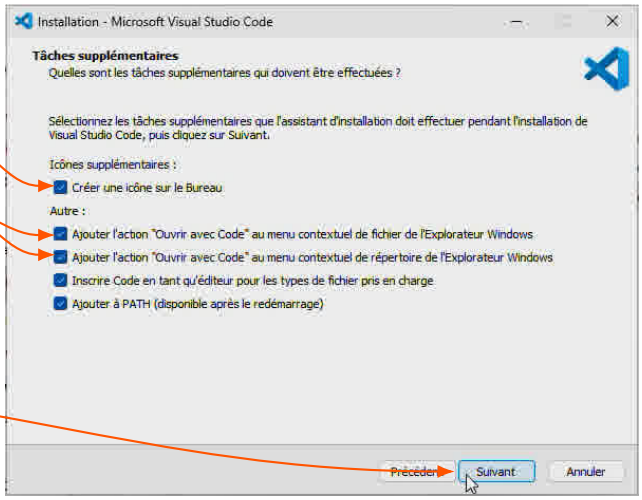
- Gardez l'emplacement d'installation proposé
- Cliquez sur le bouton *Suivant*



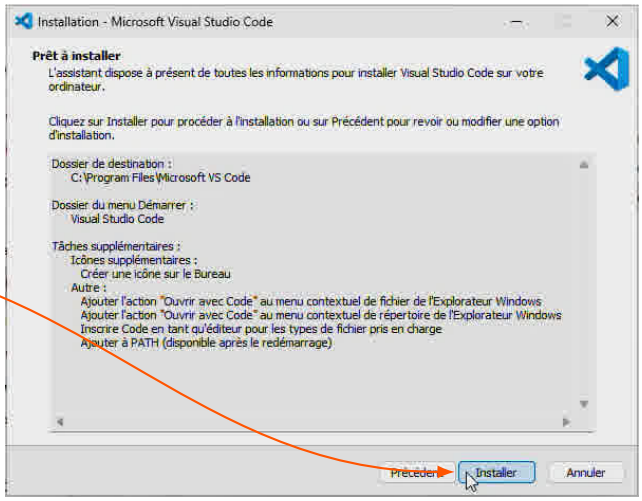
- Conservez le nom par défaut pour le dossier du menu *Windows*
- Cliquez sur le bouton *Suivant*



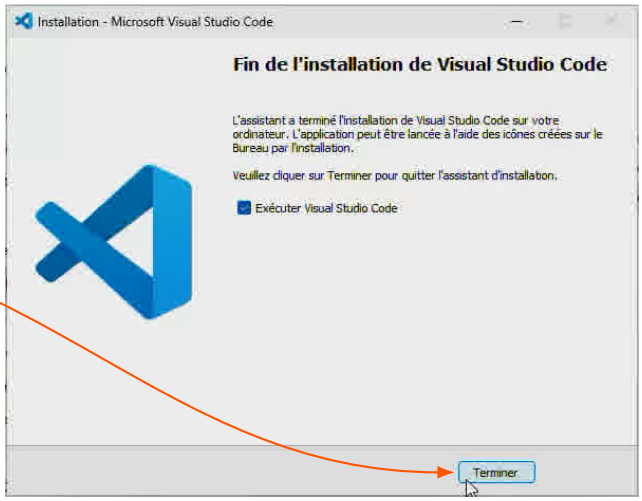
- Vous pouvez ajouter l'icône de *VSCode* sur le bureau
- Vous pouvez ajouter l'option *Ouvrir avec Code* dans le menu contextuel (clic droit) des fichiers et des dossiers
- Les deux options cochées par défaut permettent d'ouvrir certains types de fichiers directement dans *VSCode* lorsque l'on double clic sur leur nom et d'avoir accès à *VSCode* à partir du *Terminal*
- Cliquez sur le bouton *Suivant*



- Cliquez maintenant sur le bouton *Installer*

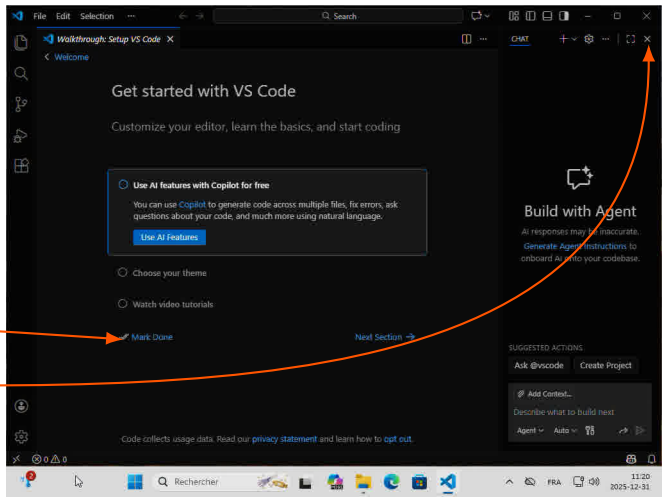


- Après un peu d'attente, l'installation est terminée
- Cliquez enfin sur le bouton *Terminer*
- Le programme d'installation va alors se fermer et *Visual Studio Code* démarrera automatiquement

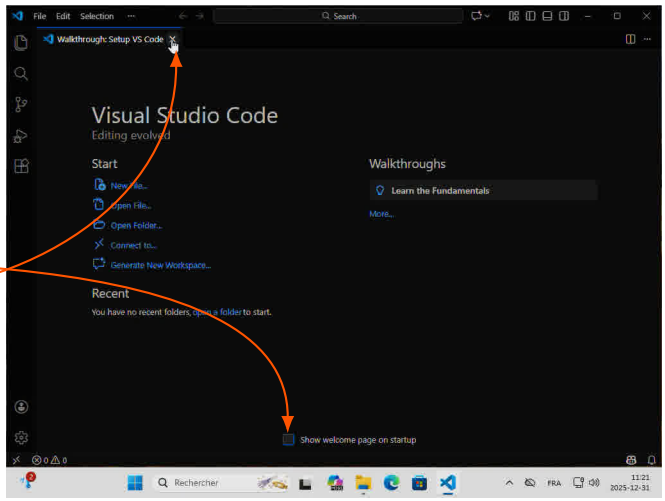


Lors du premier démarrage de *Visual Studio Code*, plusieurs onglets sont ouverts pour vous offrir différentes options de configuration. Dans ces notes de cours, les exemples utilisent la configuration par défaut.

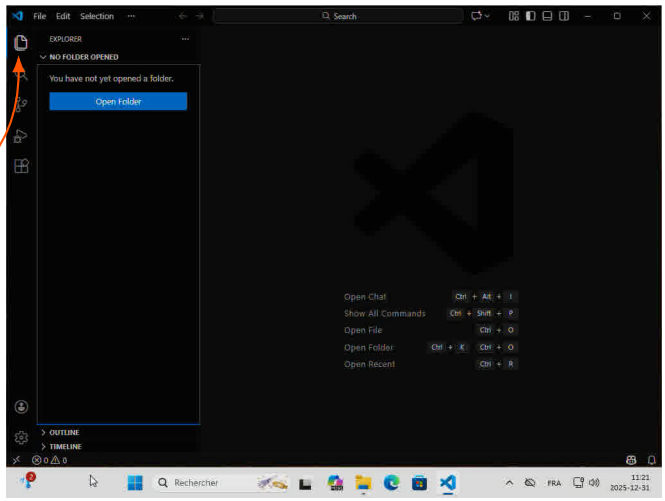
- Fermez l'onglet de configuration en cliquant sur *Mark Done*
- Fermez le panneau d'Intelligence Artificielle, soit *CHAT* , en cliquant sur le *X*



- Vous pouvez maintenant désactiver l'écran d'accueil en décochant la case
- Fermez ensuite son onglet en cliquant sur le *X*



- Il ne reste qu'à ouvrir le panneau de l'explorateur de fichiers, soit *EXPLORER* , en cliquant sur l'icône de la feuille
- Vous pouvez ensuite installer les différentes extensions nécessaires pour vos cours en vous référant à la section suivante

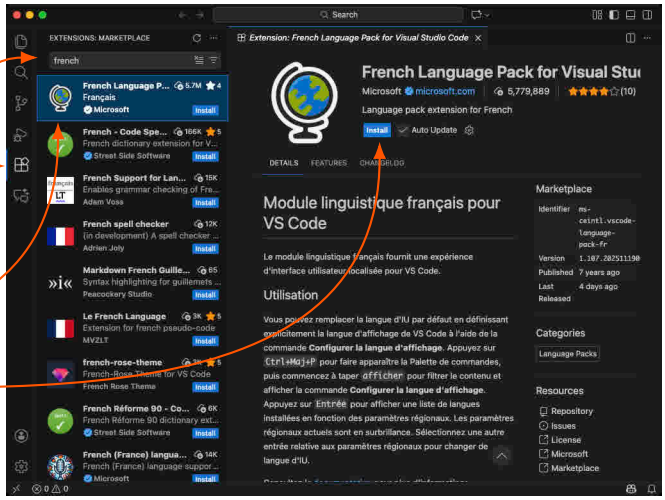


A.3 *Visual Studio Code* : Extensions

A.3.1 *French Language Pack*

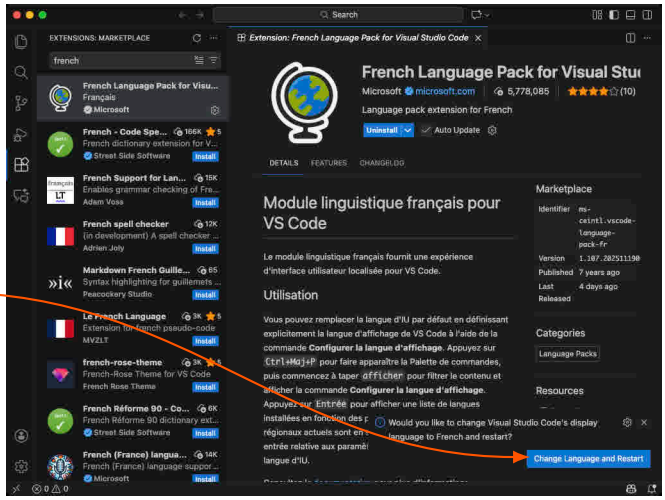
Réalisez les étapes suivantes pour installer le module linguistique français dans *Visual Studio Code* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `french` dans la zone de recherche
- Cliquez sur *French Language Pack for Visual Studio Code*
- Cliquez ensuite sur *Install*



À la suite de l'installation du *French Language Pack for Visual Studio Code*, vous devez redémarrer *Visual Studio Code* pour obtenir l'interface en français :

- Cliquez sur le bouton *Change Language and Restart*
- Votre interface est maintenant en français

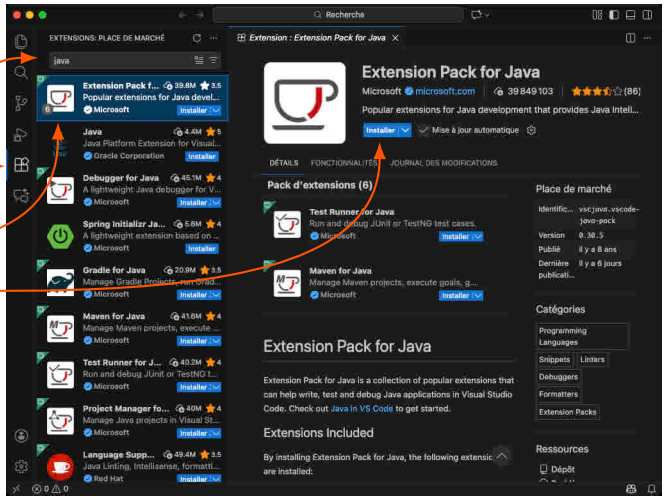


A.3.2 *Extension Pack for Java*

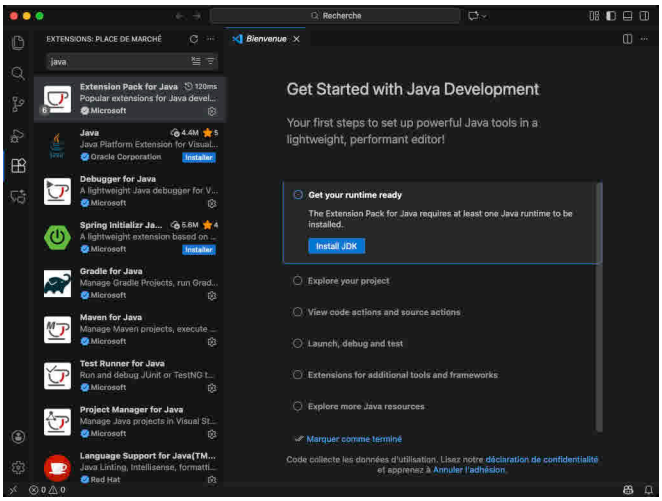
Réalisez les étapes suivantes pour installer le pack d'extensions pour *Java* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `java` dans la zone de recherche
- Cliquez sur *Extension Pack for Java*
- Cliquez ensuite sur *Installer*

Le pack contient les six extensions suivantes : *Language Support for Java(TM) by Red Hat*, *Debugger for Java*, *Test Runner for Java*, *Maven for Java*, *Gradle for Java* et *Project Manager for Java*.



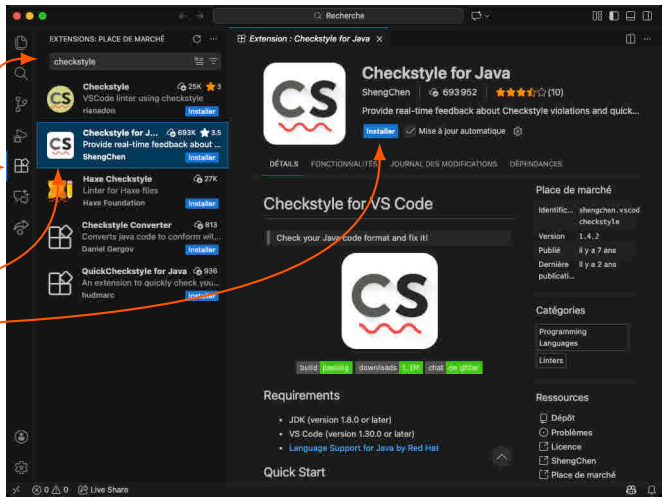
Après l'installation du pack d'extensions pour *Java*, *Visual Studio Code* ouvre une page contenant des suggestions, comme sur la capture d'écran à droite. Vous pouvez ignorer celle-ci, car vous avez normalement déjà installé le *JDK* de *Java* et les suggestions suivantes ne sont pas pertinentes dans le cadre du cours. Vous pouvez ainsi installer la prochaine extension sans crainte.



A.3.3 *Checkstyle for Java*

Réalisez les étapes suivantes pour installer *Checkstyle for Java* :

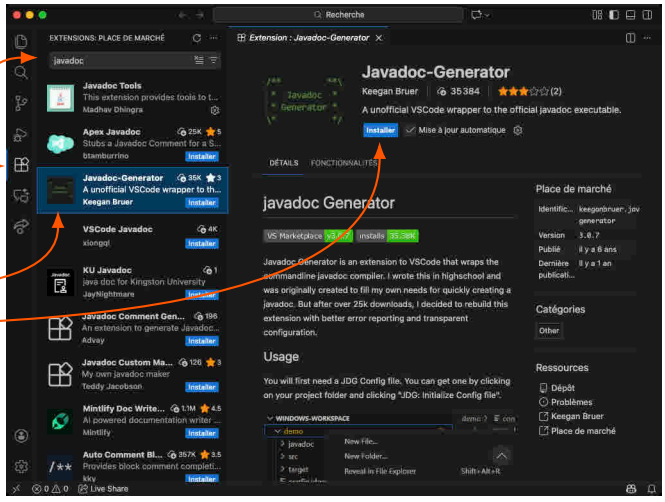
- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `checkstyle` dans la zone de recherche
- Cliquez sur *Checkstyle for Java*
- Cliquez ensuite sur *Installer*
- Si on vous demande si vous faites confiance à l'éditeur, cliquez sur *Approuver l'éditeur installer*



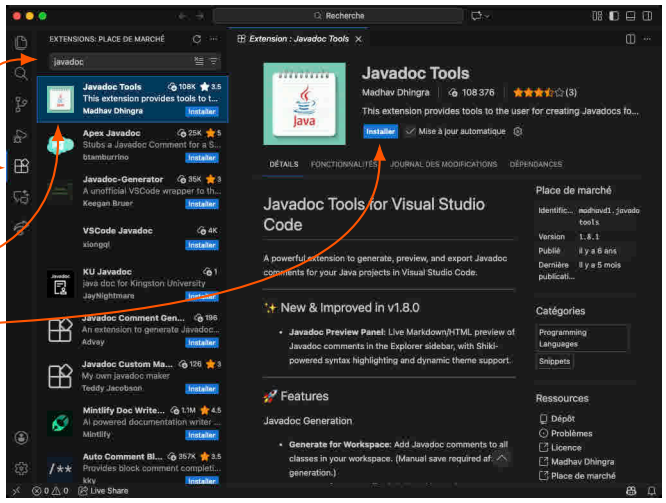
A.3.4 *Javadoc-Generator*

Réalisez les étapes suivantes pour installer *Javadoc-Generator* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `javadoc` dans la zone de recherche
- Cliquez sur *Javadoc-Generator*
- Cliquez ensuite sur *Installer*
- Si on vous demande si vous faites confiance à l'éditeur, cliquez sur *Approuver l'éditeur installer*



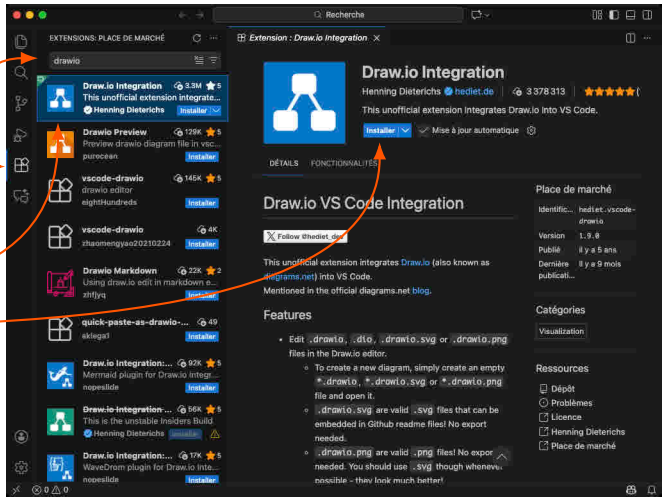
A.3.5 *Javadoc Tools*



Réalisez les étapes suivantes pour installer *Javadoc Tools* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `javadoc` dans la zone de recherche
- Cliquez sur *Javadoc Tools*
- Cliquez ensuite sur *Installer*
- Si on vous demande si vous faites confiance à l'éditeur, cliquez sur *Approuver l'éditeur installer*

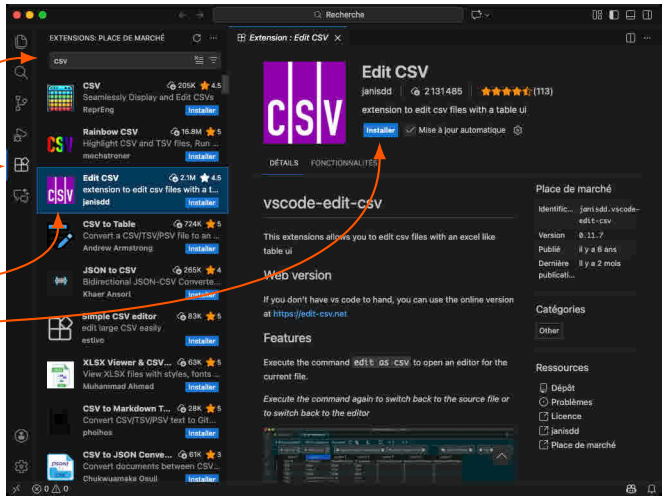
A.3.6 *Draw.io Integration*



Réalisez les étapes suivantes pour installer *Draw.io Integration* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `drawio` dans la zone de recherche
- Cliquez sur *Draw.io Integration*
- Cliquez ensuite sur *Installer*
- Si on vous demande si vous faites confiance à l'éditeur, cliquez sur *Approuver l'éditeur installer*

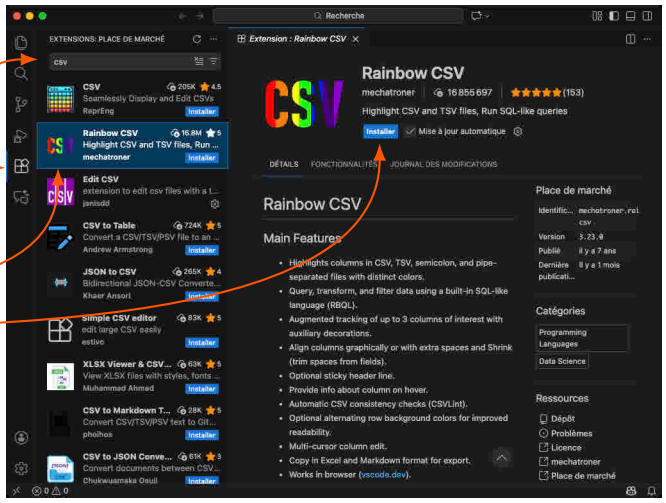
A.3.7 *Edit CSV*



Réalisez les étapes suivantes pour installer *Edit CSV* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `csv` dans la zone de recherche
- Cliquez sur *Edit CSV*
- Cliquez ensuite sur *Installer*
- Si on vous demande si vous faites confiance à l'éditeur, cliquez sur *Approuver l'éditeur installer*

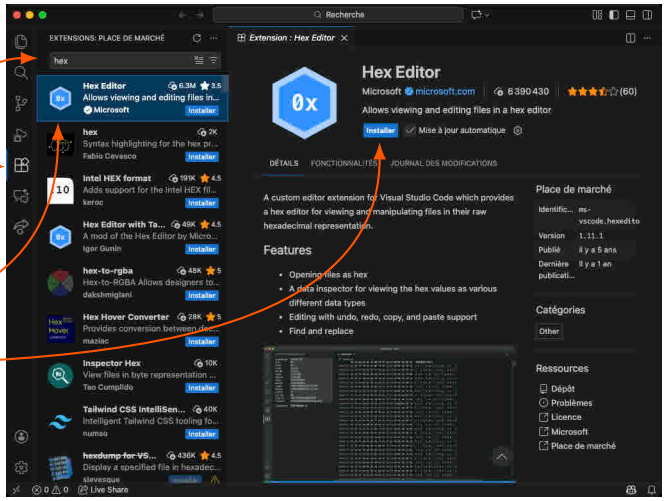
A.3.8 *Rainbow CSV*



Réalisez les étapes suivantes pour installer *Rainbow CSV* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `csv` dans la zone de recherche
- Cliquez sur *Rainbow CSV*
- Cliquez ensuite sur *Installer*
- Si on vous demande si vous faites confiance à l'éditeur, cliquez sur *Approuver l'éditeur installer*

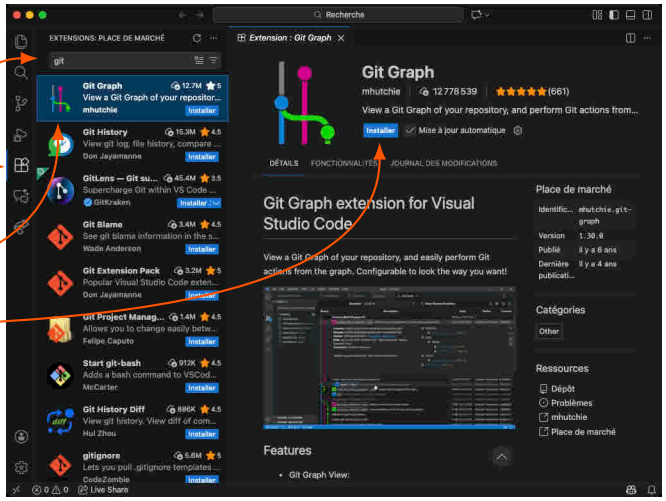
A.3.9 *Hex Editor*



Réalisez les étapes suivantes pour installer *Hex Editor* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `hex` dans la zone de recherche
- Cliquez sur *Hex Editor*
- Cliquez ensuite sur *Installer*

A.3.10 *Git Graph*



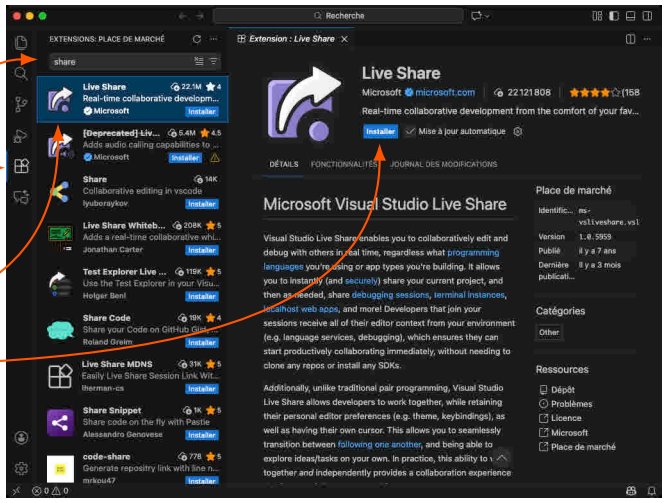
Réalisez les étapes suivantes pour installer *Git Graph* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `git` dans la zone de recherche
- Cliquez sur *Git Graph*
- Cliquez ensuite sur *Installer*
- Si on vous demande si vous faites confiance à l'éditeur, cliquez sur *Approuver l'éditeur installer*

A.3.11 *Live Share*

Réalisez les étapes suivantes pour installer *Live Share* :

- Ouvrez *Visual Studio Code*
- Cliquez sur le bouton pour gérer les extensions
- Écrivez `share` dans la zone de recherche
- Cliquez sur *Live Share*
- Cliquez ensuite sur *Installer*



A.4 *Git*

A.4.1 Validation de l'installation

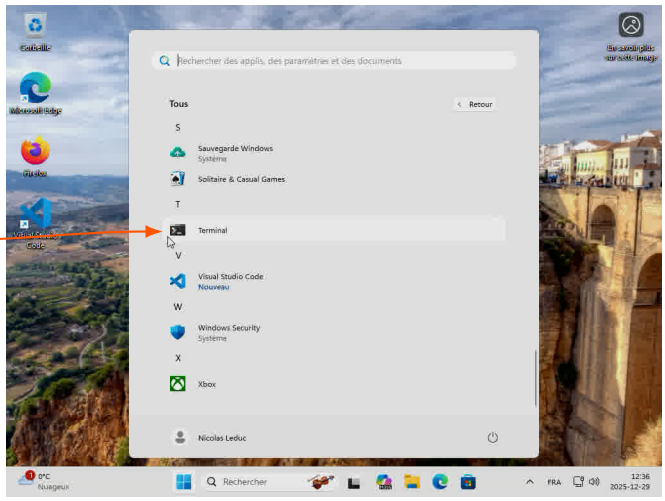
Afin de valider que les outils en ligne de commande de *Git* sont installés sur votre ordinateur, vous pouvez utiliser la procédure suivante :

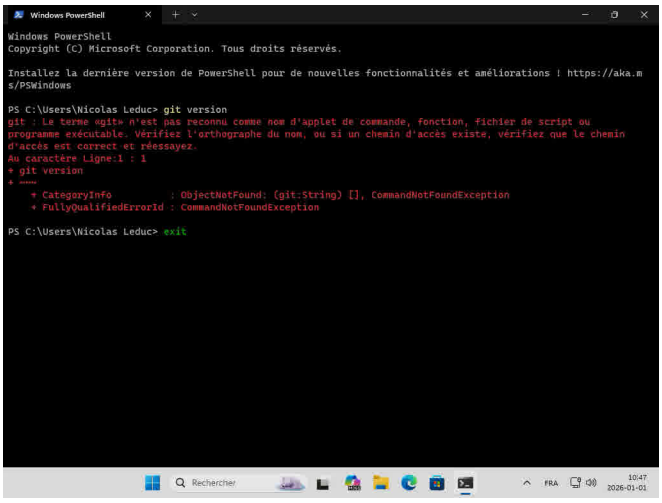
- Ouvrez le terminal (*Windows -> Tous -> Terminal*)
- Sur la ligne de commande inscrivez :

```
$ git version
```

- Observez le résultat puis fermez la fenêtre en tapant :

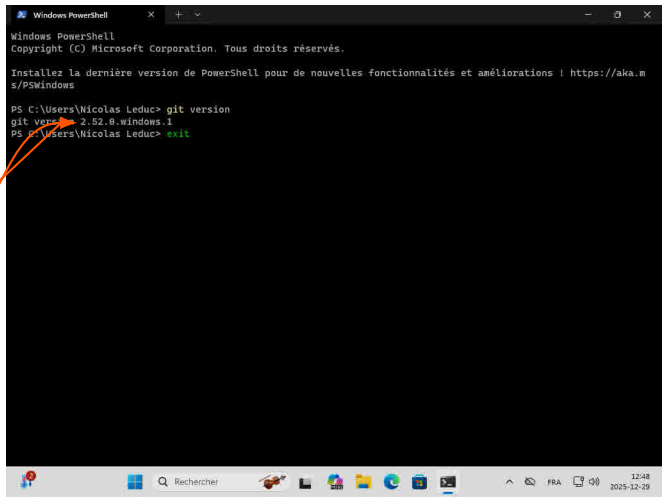
```
$ exit
```





Si on vous indique que la commande n'est pas reconnue, il faut installer *Git*, tel qu'indiqué dans les pages suivantes.

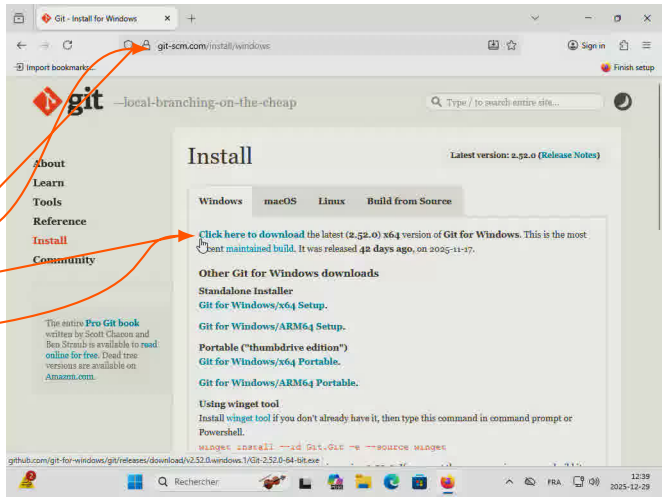
Si *Git* est installé, le numéro de version est alors affiché.



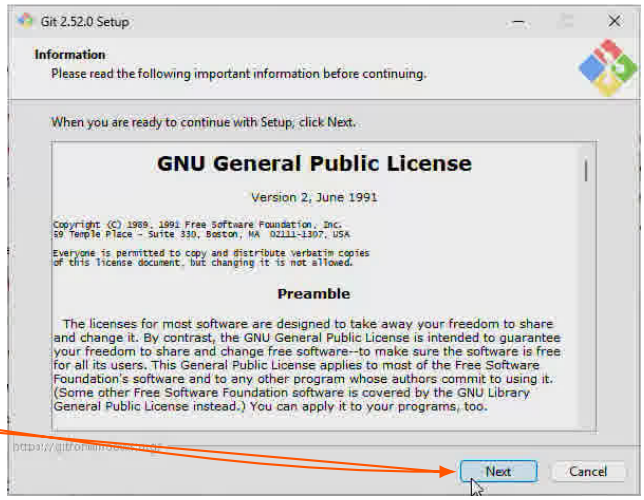
A.4.2 Installation de *Git*

Pour installer les outils *Git* en ligne de commande sur votre ordinateur, exécutez la procédure suivante :

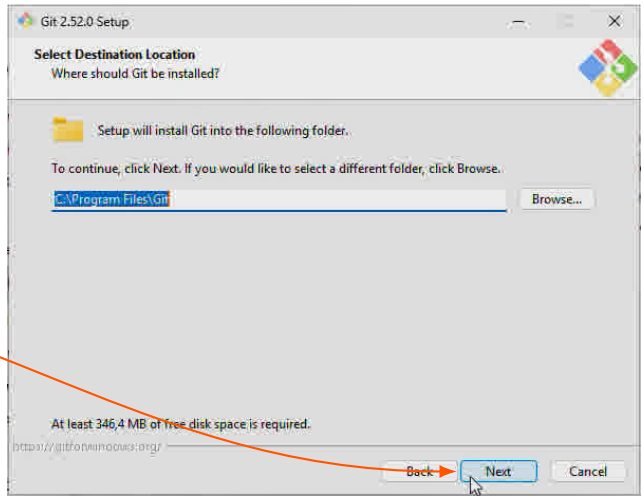
- Ouvrez un navigateur web et collez le lien ci-après dans la barre d'adresse :
  [redacted]
- Cliquez sur le bouton pour télécharger la version correspondante à votre système d'exploitation, habituellement *Windows*
- L'installateur *Git* ainsi téléchargé se trouve normalement dans le dossier *Téléchargements*



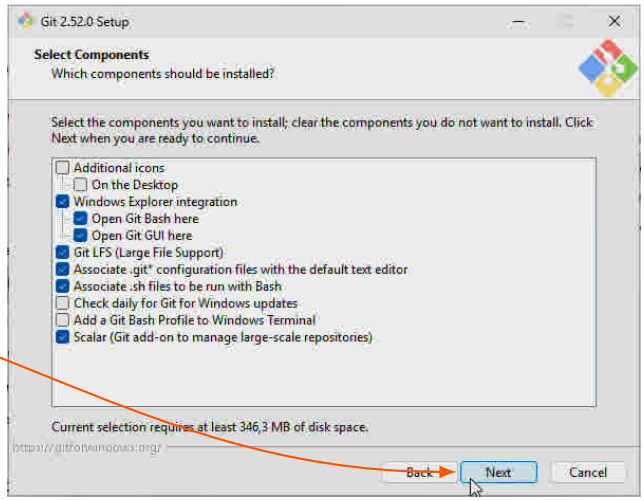
- Ouvrez un explorateur de fichiers, puis sélectionnez le dossier *Téléchargements* et double cliquez sur le fichier que vous venez de télécharger, soit `Git*` , afin de démarrer l'installation de *Git*
- Si on vous demande *Voulez-vous autoriser cette application à apporter des modifications à votre appareil ?* , cliquez sur *Oui*
- On vous présente la licence *GPL*
- Cliquez sur le bouton *Next*



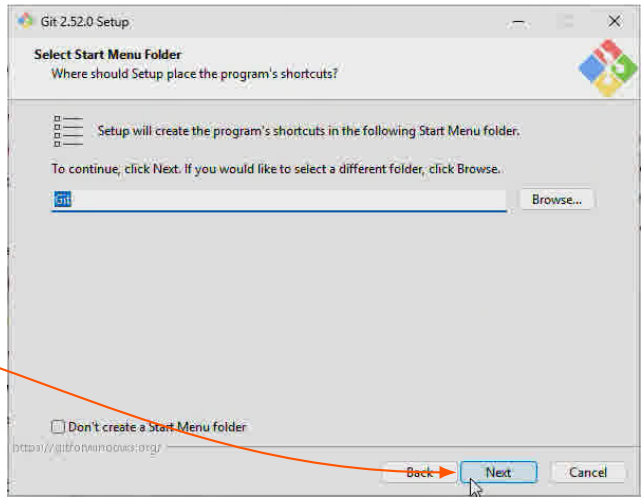
- Conservez l'emplacement d'installation par défaut
- Cliquez sur le bouton *Next*



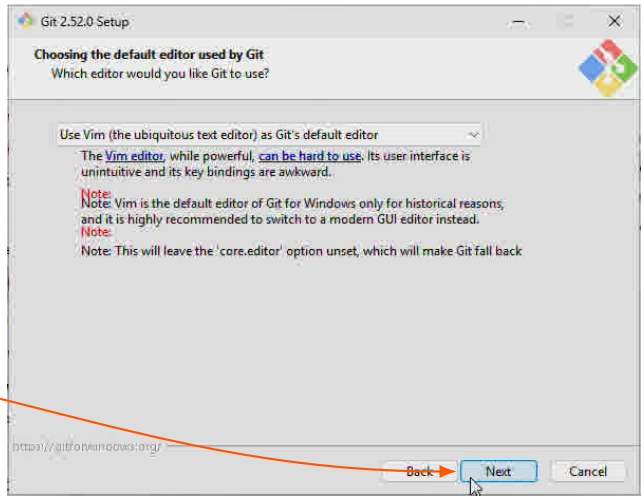
- Les éléments sélectionnés par défaut sont adéquats
- Cliquez sur le bouton *Next*



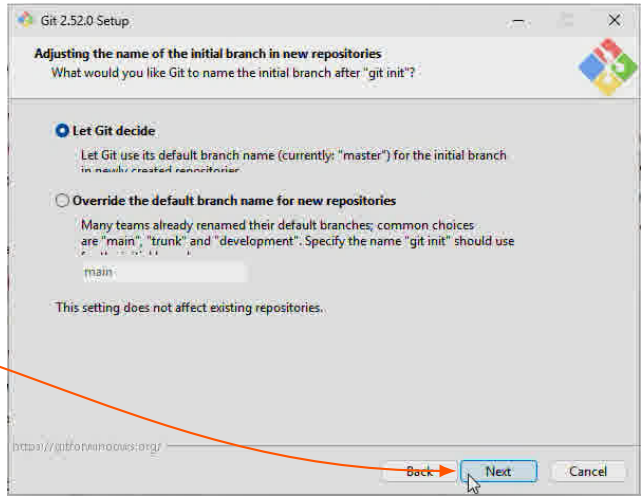
- Conservez le nom par défaut pour le dossier du menu *Windows*
- Cliquez sur le bouton *Next*



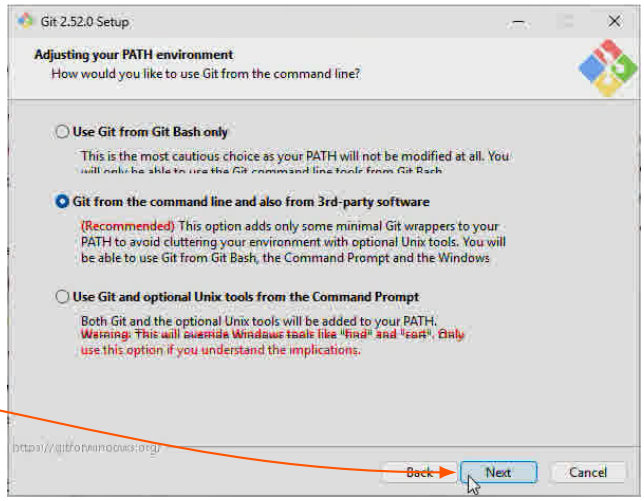
- Vous pouvez maintenant sélectionner l'éditeur de texte qu'utilisera *Git*, par exemple pour inscrire les messages des *Commit* en ligne de commade
- Conservez l'éditeur par défaut, vous pourrez le modifier par la suite
- Cliquez sur le bouton *Next*



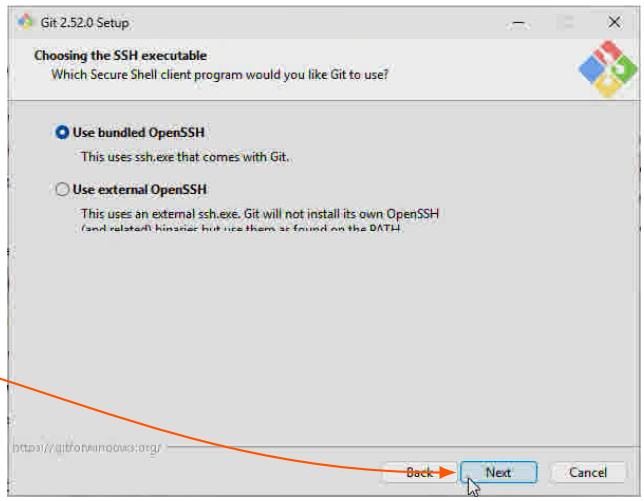
- On peut spécifier le nom de la branche principale
- Conservez le nom par défaut de *Git*
- Cliquez sur le bouton *Next*



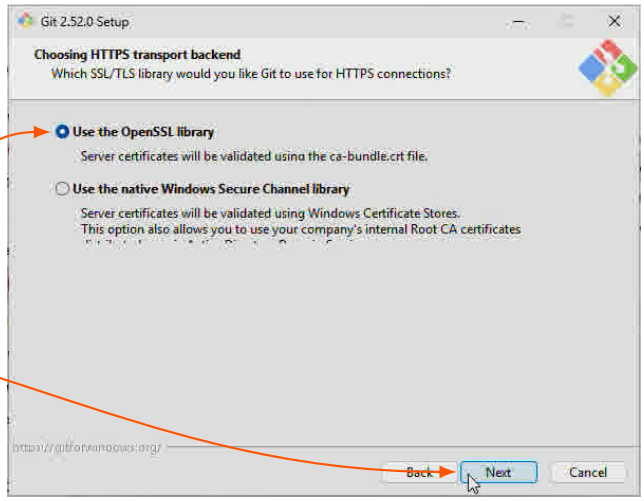
- On peut ensuite spécifier comment la variable d'environnement `PATH` doit être modifiée pour nous donner accès à *Git* en ligne de commande. La troisième option nous donne accès à des outils *Unix*, en plus de *Git*, mais peut causer des conflits
- Conservez la seconde option, soit celle par défaut
- Cliquez sur le bouton *Next*



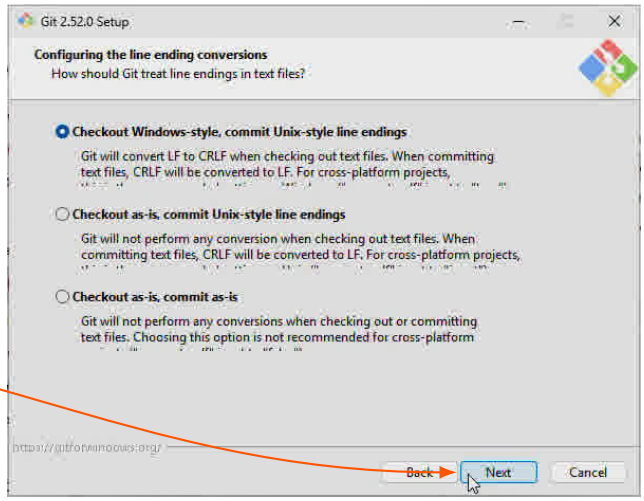
- Il est maintenant possible de choisir le client *SSH* que *Git* utilisera pour se connecter aux serveurs
- Utilisez le client *OpenSSH* inclus avec *Git* (défaut)
- Cliquez sur le bouton *Next*



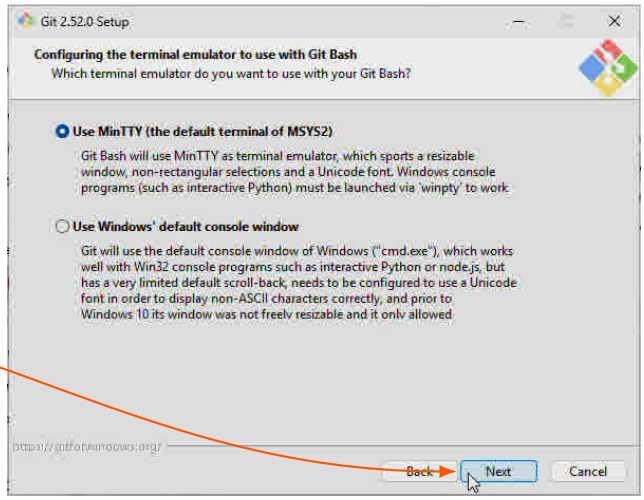
- Il faut ensuite spécifier la bibliothèque *HTTPS* à utiliser pour se connecter aux serveurs *Git*
- Nous utiliserons la bibliothèque *OpenSSL*
- Cliquez sur le bouton *Next*



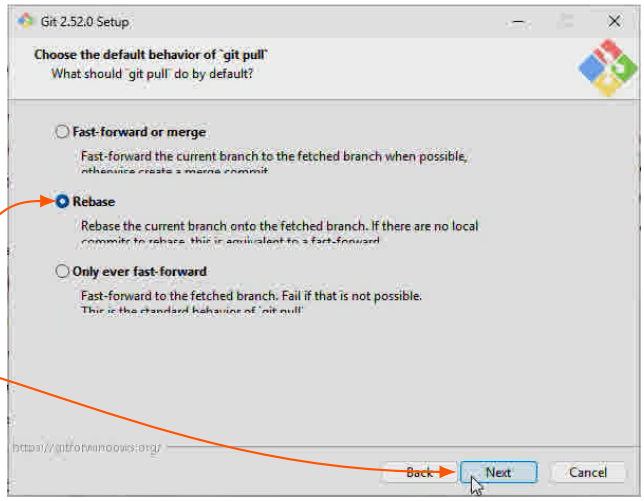
- Comme les fins de lignes en *Windows* et en *Unix* sont différentes, *Git* veut savoir comment les gérer
- Conservez la gestion des fins de lignes par défaut afin de faciliter le partage de votre dépôt
- Cliquez sur le bouton *Next*



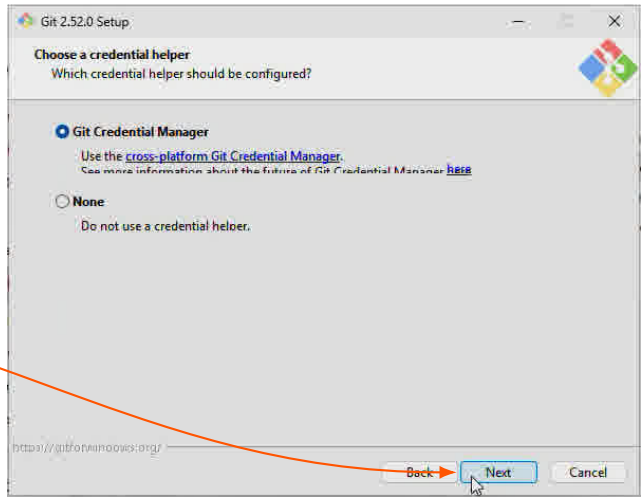
- Sélectionnez le type de terminal qu'utilisera *Git Bash*
- Le choix par défaut, soit *MinTTY*, est correct
- Cliquez sur le bouton *Next*



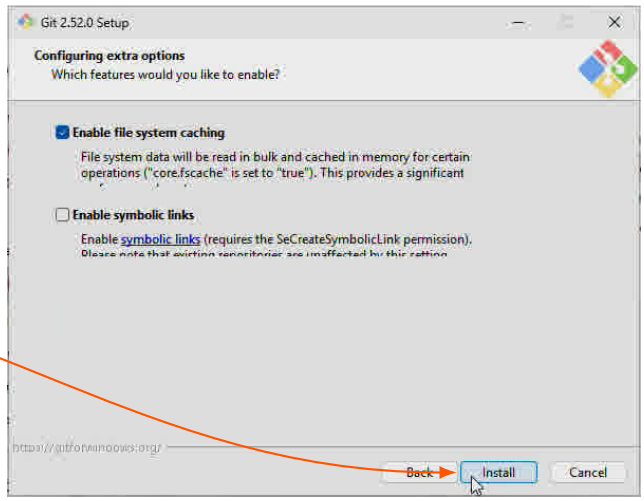
- Lors d'un `git pull` , nous voulons faire un *rebase* au lieu d'un *merge*, tel que proposé par défaut
- Sélectionnez l'option *Rebase*
- Cliquez sur le bouton *Next*



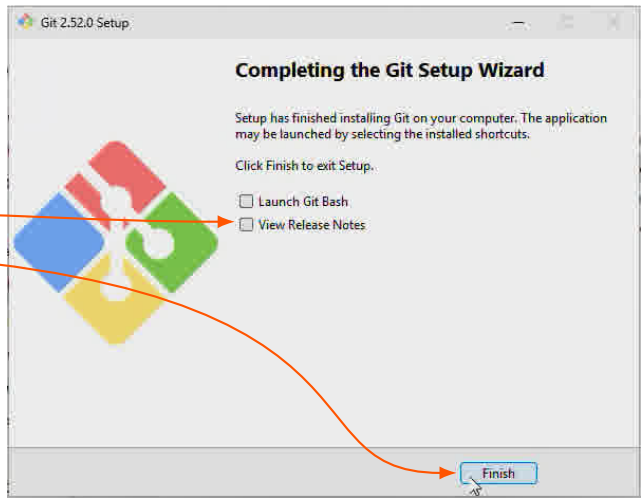
- On utilise le gestionnaire de mots de passe de *Git*, soit l'option par défaut
- Cliquez sur le bouton *Next*



- Les options sélectionnées par défaut sont adéquates
- Cliquez sur le bouton *Next*



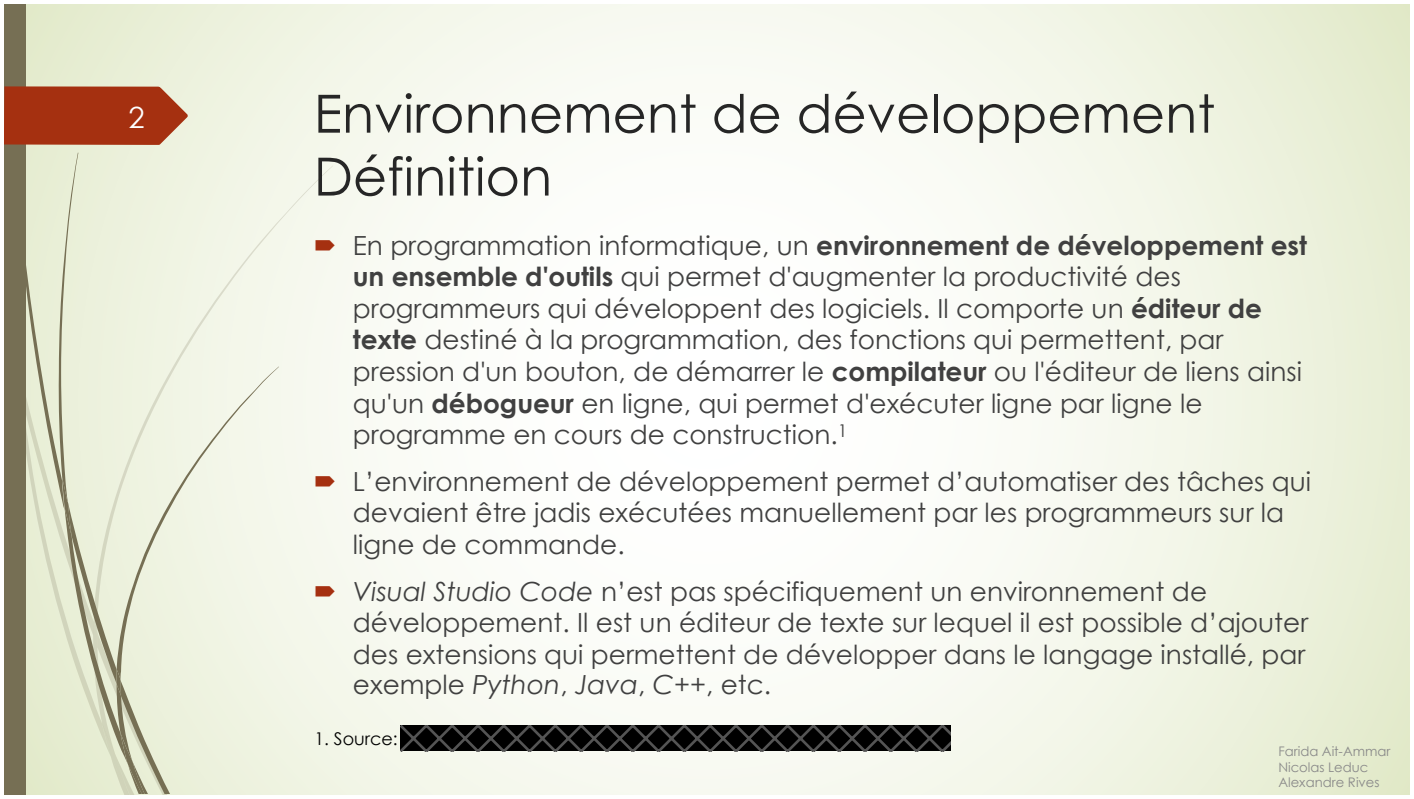
- Après un peu d'attente, l'installation est terminée
- Décochez la case pour voir les notes de version
- Cliquez enfin sur le bouton *Finish*



Il est conseillé de refaire la procédure de la section A.4.1 afin de confirmer que *Git* est correctement installé.

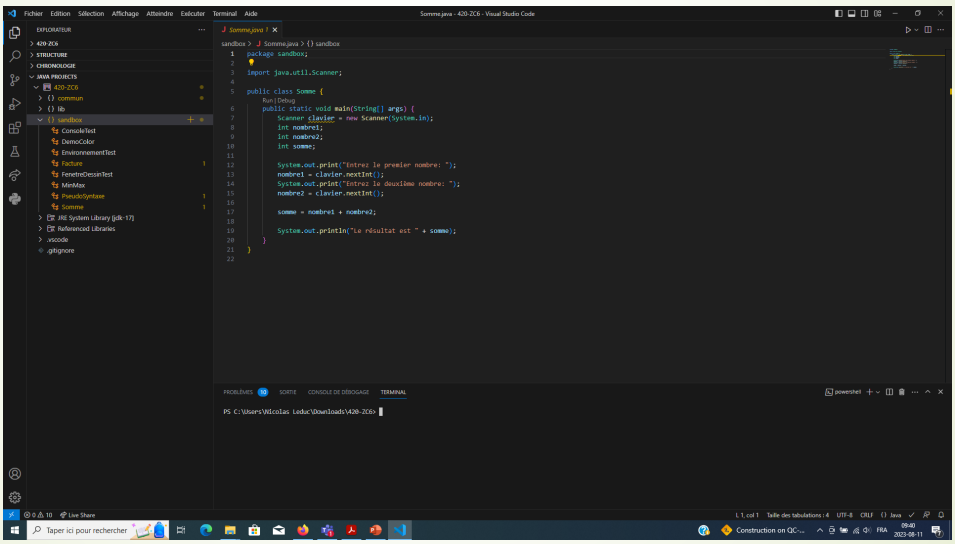
B. *Visual Studio Code*

B.1 Environnement de développement *Java*



3

Environnement de développement Exemple: *Visual Studio Code*



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

4

Développement en *Java* Prérequis

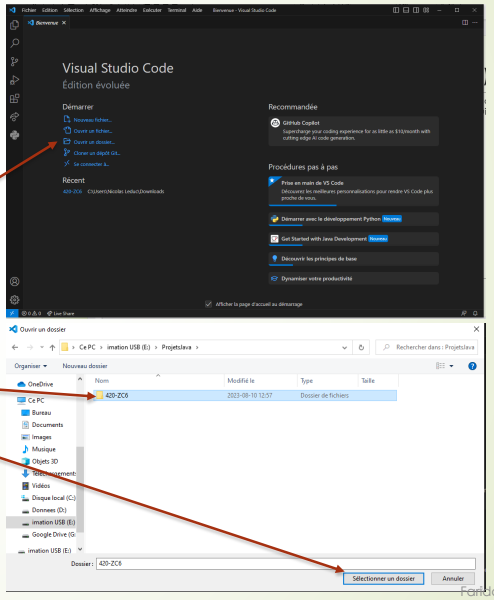
- Pour développer des applications en *Java*, vous devez installer minimalement deux logiciels sur votre ordinateur:
  - **Kit de Développement *Java*** (*Java Development Kit - JDK*)
    - Contient les outils nécessaires pour compiler les programmes *Java*, les exécuter et les tester. D'autres outils sont aussi inclus, par exemple, pour la gestion des librairies ou pour l'analyse de performance des programmes *Java*.
  - **Environnement de développement** (*Visual Studio Code* dans notre cas)
    - Interface graphique qui permet d'interagir avec les outils du kit de développement de façon relativement transparente. Il offre de plus du soutien au programmeur, entre autres, en lui faisant des suggestions lors de l'écriture des programmes.
    - *Visual Studio Code* compile automatiquement le code en cours d'écriture. Il souligne en **rouge** les lignes d'instructions qui **ne compilent pas** et en **jaune** les instructions qui compilent, mais qui sont **susceptibles de poser problème** (avertissement / *warning*).
    - Note: L'extension *Extension Pack for Java* est nécessaire pour développer en *Java*.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.2 Démarrage de *Visual Studio Code*

5 Démarrage de *VS Code* (1/3)

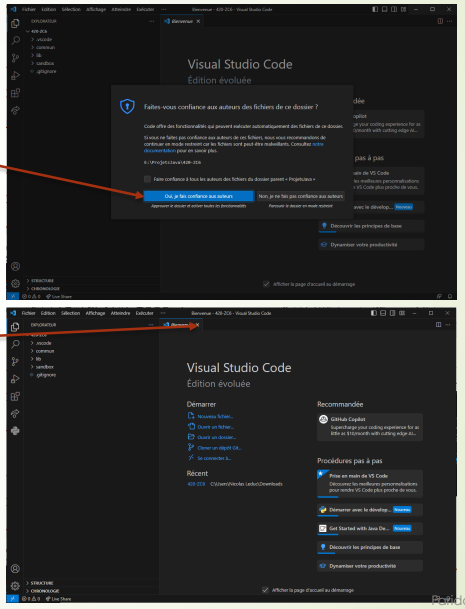
- Après avoir téléchargé l'archive de l'enseignant fournie sur *Moodle*, copiez le dossier qui y est contenu dans un répertoire nommé `ProjetsJava` .
- Démarrez ensuite *VS Code*.
- Sélectionnez *Ouvrir un dossier…*, puis naviguez jusqu'au dossier que vous venez de mettre dans `ProjetsJava` .
- Sélectionnez le dossier puis cliquez sur *Sélectionner un dossier.*



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

6 Démarrage de *VS Code* (2/3)

- Indiquez que vous faites confiance à l'auteur du code source que vous ouvrez. Si vous appuyez sur le mauvais bouton, vous serez en mode restreint et vous ne pourrez pas exécuter votre code.
- Fermez la fenêtre *Bienvenue* .

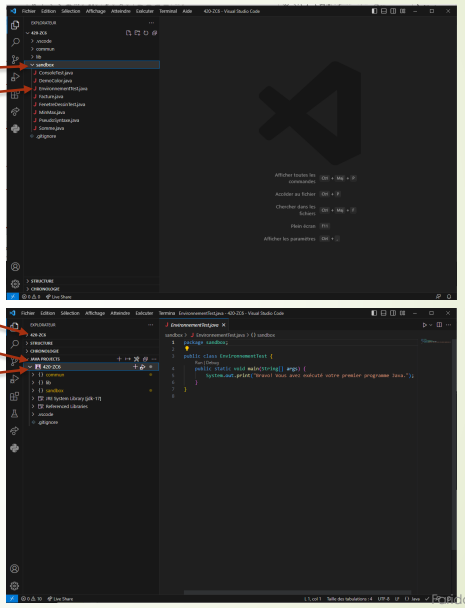


Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

7

Démarrage de *VS Code* (3/3)

- Appuyez sur `sandbox` puis sur `EnvironnementTest.java` afin que *VS Code* détecte que vous programmez en *Java* .
- Fermez l'onglet de votre dossier et ouvrez l'onglet *JAVA PROJECTS* .
- Appuyez sur le nom de votre dossier sous *JAVA PROJECTS* pour afficher l'arborescence de son contenu.
- Vous êtes maintenant prêt à travailler!

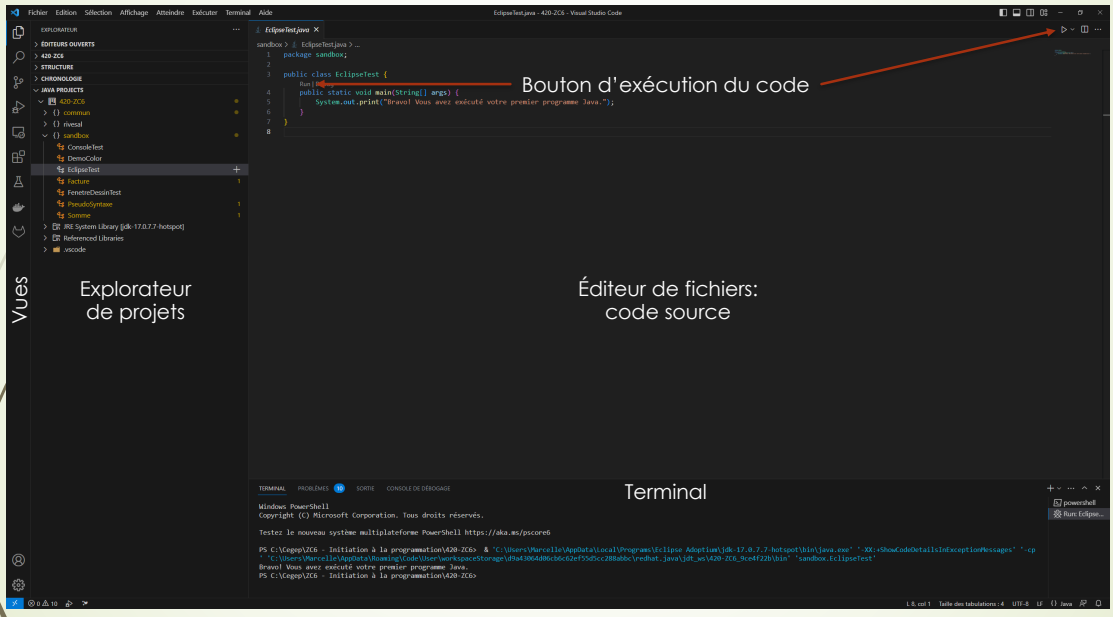


Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.3 Interface

8

VS Code - Interface



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

9

VS Code - Vues

Dans *Visual Studio Code*, le contenu du panneau de gauche est géré par les icônes placés verticalement à l'extrême gauche. Chacun permet d'avoir une vue différente sur le projet ouvert. Il est possible de naviguer librement et en tout temps entre les différentes vues ou de faire disparaître le panneau en appuyant sur l'icône de la vue active.

Icône	Raccourcis	Description
	Ctrl+Maj+E	***Explorateur***: Permet de voir la hiérarchie des fichiers inclus dans le projet ouvert ainsi que les librairies utilisées.
	Ctrl+Maj+F	***Recherche***: Permet de faire des recherches et des remplacements dans un ou plusieurs fichiers. Accepte les expressions régulières.
	Ctrl+Maj+G	***Contrôle de code source***: Permet de gérer les différentes versions du code source à l'aide de *Git*.
	Ctrl+Maj+D	***Exécuter et déboguer***: Permet d'interagir avec le débogueur. Affiche les points d'arrêts, la valeur des variables, la pile d'exécution, etc.
	Ctrl+Maj+X	***Extensions***: Permet de voir, d'installer et de configurer les extensions disponibles dans *VS Code*.
		Test: Permet de gérer et d'exécuter les tests unitaires présents dans le projet.
		Live Share: Permet de travailler en équipe sur un même fichier ou un même projet en temps réel. Une connexion au serveur *Microsoft* est requise.

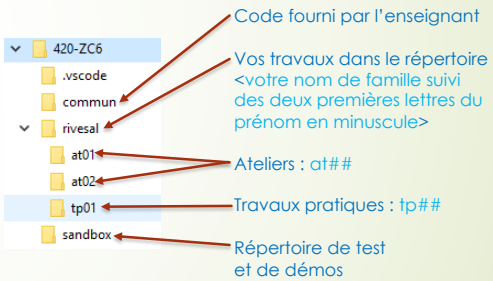
Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.4 *Package* et classe *Java*

10

Packages Java (1/2)

- En *Java*, un *package* correspond à un sous-répertoire par rapport au répertoire racine de vos fichiers source. Il permet de regrouper le code en respectant une certaine logique afin d'en faciliter la navigation.
- Le nom d'un *package* ne doit comporter que des lettres minuscules [a-z] sans accents et des chiffres, mais ne doit pas débuter par un chiffre.
- Dans le cadre de ce cours, vous devez placer vos travaux dans différents sous-répertoires (*packages*) dans votre projet.
- Pour créer un *package*, il suffit de créer le répertoire correspondant dans l'explorateur de fichiers.
- Nous vous demandons de respecter la hiérarchie et la nomenclature ci-contre afin d'éviter de mauvaises surprises lors de la correction des travaux.

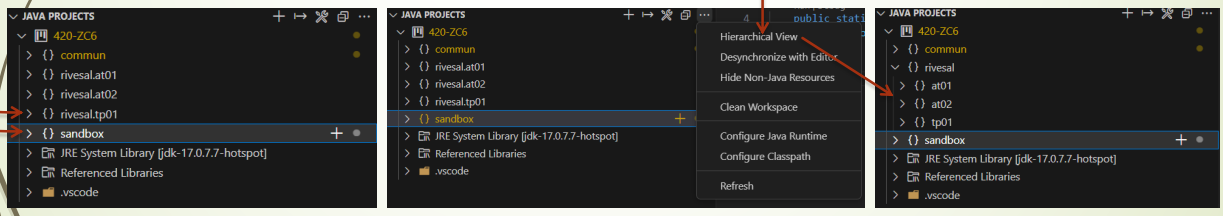


Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

11

Packages Java (2/2)

- Si vous copiez un fichier dans un des sous-répertoires (*packages*) de votre projet, il apparaîtra dans l'explorateur de *VS Code* sous `<NomPackage>`.
- Le nom du *package* correspond au chemin pour y accéder à partir du répertoire racine des sources de votre projet. Le caractère **** , habituellement utilisé pour séparer les répertoires, est remplacé par un point.
  - Exemple: `E:\ProjetsJava\420-ZC6\rivesal\tp01`
- Pour voir les *packages* sous forme hiérarchique, il faut sélectionner les trois points horizontaux près de l'onglet *JAVA PROJECTS*, puis *Hierarchical View*.



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

12

Création d'une classe *Java* (1/3)

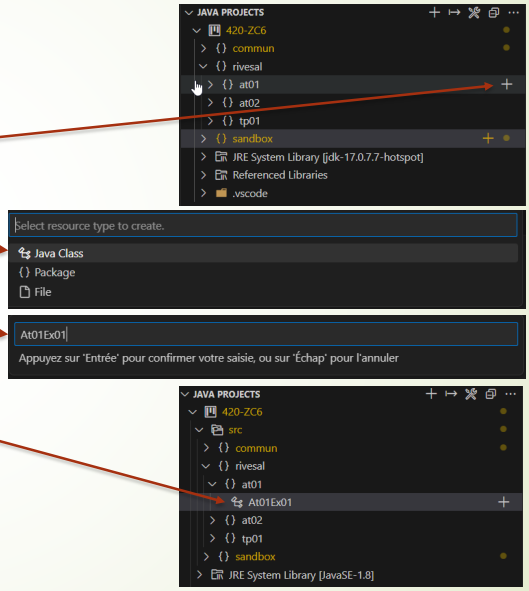
- En *Java*, si vous désirez écrire vos propres programmes, il vous faut créer des classes.
- Le concept de classe est issu de la programmation orientée objet que vous verrez dans un autre cours.
- Dans le cadre de ce cours, nous traiterons les classes comme des contenants pouvant chacun contenir un seul programme.
  - En pratique, vous devrez créer une classe distincte pour chaque exercice / programme que l'on vous demande d'écrire.
- **Attention**: Le nom du fichier *Java* et le nom de la classe qu'il contient doivent être identiques (sensible à la casse).
- **Attention**: Le sous-répertoire dans lequel se trouve le fichier *Java* et le nom du *package* doivent être identiques (**** remplacés par **.**)

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

13

Création d'une classe *Java* (2/3)

- Pour créer une classe (programme) :
  - Cliquez sur + à droite du *package* désiré
  - Un menu s'ouvre en haut de votre *VS Code*, cliquez sur *Java Class*
  - Inscrivez le nom de la classe, qui doit être significatif et commencer par une lettre majuscule, par exemple `At01Ex01` .
  - Votre classe est créée. Elle est visible à la fois dans votre dossier dans l'explorateur de fichiers *Windows* et dans l'onglet *JAVA PROJECTS*.
- **N.B. : Afin d'éviter tout problème de compilation, il est fortement recommandé de créer vos classes comme indiqué ci-dessus.**

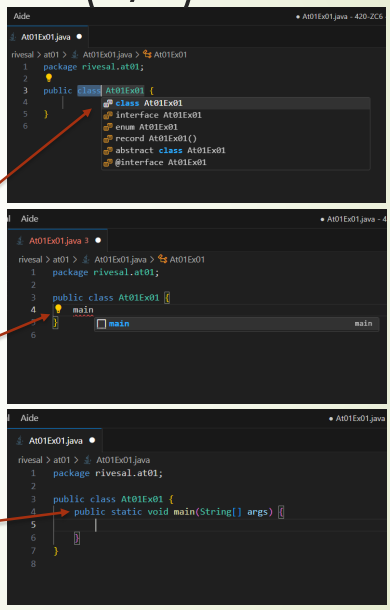


Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

14

Création d'une classe *Java* (3/3)

- Pour qu'une classe *Java* puisse être exécutée, elle doit contenir la fonction `main()` .
- Pour l'ajouter, faites *ECHAP* sur cette première fenêtre après la création de votre classe pour ne pas prendre en compte les propositions faites par *VS Code*.
- Positionnez-vous sur la ligne en dessous de `public class` et écrivez `main` puis cliquez sur la suggestion `main` qui s'affiche avec le carré.
- La ligne devrait maintenant contenir: `public static void main(String[] args)`



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.5 Exécution d'un programme

15

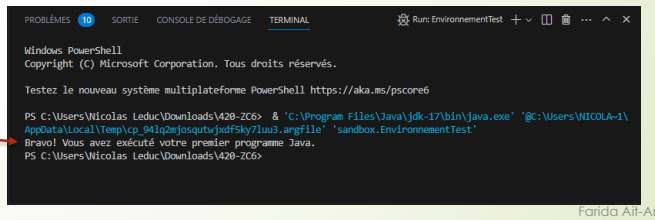
VS Code - Exécution d'un programme

Pour exécuter un programme, plusieurs possibilités s'offrent à vous :

- Cliquez sur le bouton en forme de triangle en haut à droite
- Utilisez le bouton *Run* au-dessus du `main()`
- Faites un clic droit dans la fenêtre contenant votre code pour ouvrir le menu contextuel, puis choisir l'option *Run Java*

Exécutez votre programme afin de vérifier qu'il fonctionne correctement.

- Inspectez le terminal au bas de l'écran afin de valider qu'aucune erreur ne s'est produite. Si c'est le cas, vous devez voir la sortie du programme.



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

16

VS Code - Terminal

- Le terminal permet d'interagir avec l'usager en mode texte uniquement. On peut y afficher des chaînes de caractères et y lire des données entrées au clavier.
- Le terminal s'affiche automatiquement lorsque vous exécutez votre programme.
- Si vous exécutez à nouveau votre programme avant que l'exécution précédente ne se termine, un nouveau terminal sera ouvert. La liste des terminaux ouverts apparaît alors à droite. Vous devez fermer les terminaux inutiles en appuyant sur l'icône poubelle qui apparaît à droite de chacun dans la liste afin de ne pas gaspiller de ressources et ainsi ralentir votre ordinateur.
- Si vous fermez par erreur le terminal, vous pouvez toujours l'ouvrir à nouveau en sélectionnant le menu *Affichage -> Terminal*.
- Ci-dessous, un exemple d'interaction au terminal. Le texte en bleu correspond à la ligne de commande utilisée par *VS Code* pour exécuter votre programme.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.6 Édition du code

17

VS Code - Édition du code

- *Visual Studio Code* **compile automatiquement** le code en cours d'écriture en soulignant en **rouge** les lignes d'instructions qui **ne compilent pas** (erreurs) et en **jaune** les instructions qui compilent, mais qui sont **susceptibles de poser un problème** (avertissement, ou *warning* en anglais).
- Afin d'**exécuter** votre code, il vous faut **corriger toutes les erreurs**.
- L'ensemble des erreurs et des avertissements sont aussi accessibles dans l'onglet *PROBLÈMES* au bas de l'écran.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

18

VS Code - Identification des erreurs

- Lorsque l'on écrit du code, quatre principaux types d'erreurs sont possibles:
  - **Erreurs**
    - Ce sont des erreurs graves qui empêchent le code de compiler, donc de s'exécuter. *VS Code* souligne ce type d'erreur en rouge et donne une explication de la cause ainsi que des suggestions de correction si nous plaçons la souris sur le mot fautif.
    - Exemple: erreurs d'orthographe, de syntaxe, d'incompatibilité de type, etc.
  - **Avertissements**
    - Ce sont habituellement des problèmes mineurs qui peuvent causer un mauvais fonctionnement du logiciel, mais qui ne l'empêchent pas de fonctionner. Comme pour le cas des erreurs, *VS Code* affiche une info-bulle pour vous aider à les corriger.
    - Exemple: variable inutilisée, commentaire manquant, utilisation de fonctions désuètes, etc.
  - **Information**
    - Ce type d'erreur n'est pas généré par le compilateur *Java*, alors nous l'utilisons dans le cadre du cours pour indiquer en bleu les principales normes de codage qui n'ont pas été respectées. En effet, le non respect des normes n'empêche pas le code de fonctionner, mais le rend difficile à comprendre et à modifier en plus d'augmenter la probabilité de faire des erreurs.
    - Exemple: omission des accolades dans un énoncé conditionnel, bloc de code vide, etc.
  - **Logique**
    - Votre logiciel peut s'exécuter, mais vous n'obtenez pas les résultats escomptés. Dans ce cas, il comporte des erreurs de logique que vous devrez corriger. *VS Code* n'a aucun moyen d'identifier ce type d'erreur, car elles dépendent de l'intention du programmeur. Par contre, vous pouvez utiliser le débogueur pour les trouver.
    - Exemple: utilisation de la mauvaise variable, mauvaise condition d'arrêt, nombre de boucles, etc.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

19

VS Code - Édition du code Formatage

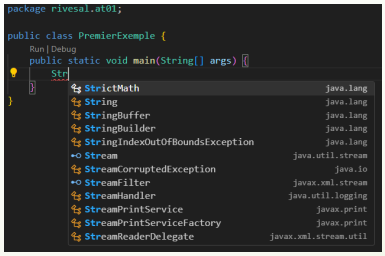
- Le formatage du code touche l'indentation, les espaces, le nombre de caractères par ligne, le nombre de lignes vides avant les blocs, la présentation des instructions `if`, `for`, etc.
- *VS Code* ne formate pas automatiquement le code en cours d'écriture si vous faites, par exemple, des erreurs d'indentation.
- Pour formater tout le code, utilisez la commande du menu contextuel (clic droit), puis *Mettre le document en forme* ou bien le raccourci clavier ***ALT+SHIFT+F***.
- Si on ne veut que formater certaines lignes de code, il faut d'abord les sélectionner, puis faire successivement les touches ***CTRL+K*** puis ***CTRL+F***.
- **Attention**: Votre code ne doit comporter aucune erreur pour que ces commandes de formatage fonctionnent correctement.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

20

VS Code - Édition du code Suggestions de code

- Pour accélérer l'édition du code et éviter les erreurs de frappe, *VS Code* peut suggérer des attributs, des méthodes ou tout simplement des classes qui pourraient convenir à la suite des caractères saisis.
- La suggestion de code se fait automatiquement avec *VS Code*, mais si vous l'avez fait disparaître, vous pouvez la réactiver avec la combinaison de touches ***CTRL+Espace***.
- Exemple: Voici les suggestions après avoir tapé les caractères `Str`. *VS Code* nous propose toutes les classes commençant par la chaîne `Str`.



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.7 Bibliothèque `FenetreDessin`

21 Utilisation de la librairie `FenetreDessin.java`

- La librairie `FenetreDessin.java` vous permet d'afficher des formes et du texte dans une fenêtre graphique. Elle permet aussi de récupérer les touches du clavier et les clics de la souris afin de produire une application interactive.
- Pour l'utiliser dans un projet, vous devez copier le fichier `FenetreDessin.java` dans le *package* `commun` de votre projet *VS Code*.
- Vous devez aussi inclure le code en gras dans votre fichier source:

```
import java.awt.Color;                                  // Pour utiliser Color
import commun.FenetreDessin;

public class FenetreDessinTest {
    public static void main(String[] args) {
        FenetreDessin fd = new FenetreDessin("Test", 640, 480);

        fd.dessineRectangle(160, 120, 480, 360); // Exemple d'utilisation
    }
}
```

Titre de la fenêtre — Largeur — Hauteur

Préfixe pour dessiner

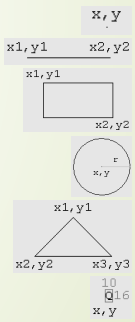
Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

22 `FenetreDessin` – méthodes

- `public void couleurCrayon(Color aCouleur)`
- `public void couleurCrayon(float aRouge, float aVert, float aBleu)`
- `public void couleurRemplissage(Color aCouleur)`
- `public void couleurRemplissage(float aRouge, float aVert, float aBleu)`
- `public void effaceFenetre()`
- `public void dessinePoint(int x, int y)`
- `public void dessineSegment(int x1, int y1, int x2, int y2)`
- `public void dessineRectangle(int x1, int y1, int x2, int y2)`
- `public void dessineRectanglePlein(int x1, int y1, int x2, int y2)`
- `public void dessineCercle(int x, int y, int r)`
- `public void dessineCerclePlein(int x, int y, int r)`
- `public void dessineTriangle(int x1, int y1, int x2, int y2, int x3, int y3)`
- `public void dessineTrianglePlein(int x1, int y1, int x2, int y2, int x3, int y3)`
- `public void dessineTexte(int x, int y, String aTexte)`
- `public int litCouleurPixel[RVB](int x, int y)`

```
import java.awt.Color;

Color.BLACK          Color.MAGENTA
Color.BLUE           Color.ORANGE
Color.CYAN           Color.PINK
Color.DARK_GRAY      Color.RED
Color.GRAY           Color.WHITE
Color.GREEN          Color.YELLOW
Color.LIGHT_GRAY
```



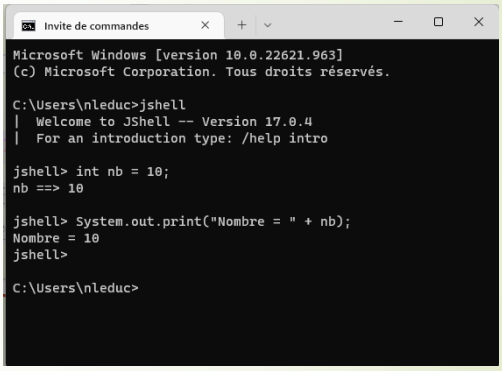
Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.8 Interpréteur interactif

23

Interpréteur interactif *Java* - `jshell`

- Parfois nous souhaitons simplement tester quelques lignes de code en *Java* afin de s'assurer qu'elles sont correctes. Il est possible de le faire à l'aide de l'interpréteur interactif `jshell`. Celui-ci nous permet d'inscrire des lignes de code et nous donne immédiatement le résultat de l'exécution.
- Pour utiliser `jshell`, vous devez premièrement lancer une *Invite de commandes*, puis inscrire `jshell` en autant que le *JDK* soit installé correctement.
- Vous inscrivez ensuite vos lignes de code *Java* et vous observez les résultats obtenus.
- Pour quitter `jshell`, il faut appuyer sur ***CTRL+D*** puis taper `exit` pour fermer l'*Invite de commandes*.



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.9 Débogueur

25

Débogueur
Définition

- Lorsqu'un programme ne donne **pas les résultats attendus** à son exécution, on dit qu'il est ***bogué***. **Déboguer** un programme consiste à **chercher**, dans le code, les **erreurs** qui sont à l'origine de son mauvais fonctionnement dans le but de les **corriger**.
- Un débogueur permet **d'exécuter un programme pas à pas** afin de vérifier son fonctionnement et **d'inspecter l'état des variables** ainsi que de la mémoire.
- *VS Code* offre un mode de débogage qui permet d'exécuter un programme de manière interactive. Après chaque pas d'exécution, il affiche le code source, l'état des variables ainsi que d'autres informations pertinentes.
- Principe de fonctionnement
  - Insérer un ou des point(s) d'arrêt au début du code que l'on souhaite inspecter.
  - Lancer le programme en mode débogage. Le programme s'arrête quand il atteint le premier point d'arrêt et *VS Code* bascule vers la vue *Exécuter et déboguer*.
  - On peut alors examiner les valeurs des variables et poursuivre l'exécution du programme pas à pas. On peut également poursuivre l'exécution du programme jusqu'au prochain point d'arrêt.
- Le débogueur permet d'éviter de polluer votre code avec des affichages au terminal dans le but de tenter d'inspecter la valeur des variables.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

26

Débogueur
Points d'arrêt

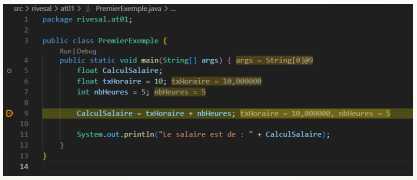
- Un point d'arrêt spécifie où l'exécution du programme doit s'arrêter pendant le débogage.
- Il est matérialisé par un disque rouge dans la marge de gauche du fichier source.
- Ajout / suppression d'un point d'arrêt
  - Positionner le pointeur de la souris sur la marge de gauche, au niveau de la ligne d'instruction qui nous intéresse, puis choisir, dans le menu contextuel (clic droit), *Ajouter un point d'arrêt*.
  - Ou simplement cliquer à cet endroit.
  - Ou placer le curseur d'édition sur la ligne visée et appuyer sur ***F9***.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

27

Débogueur - Lancement

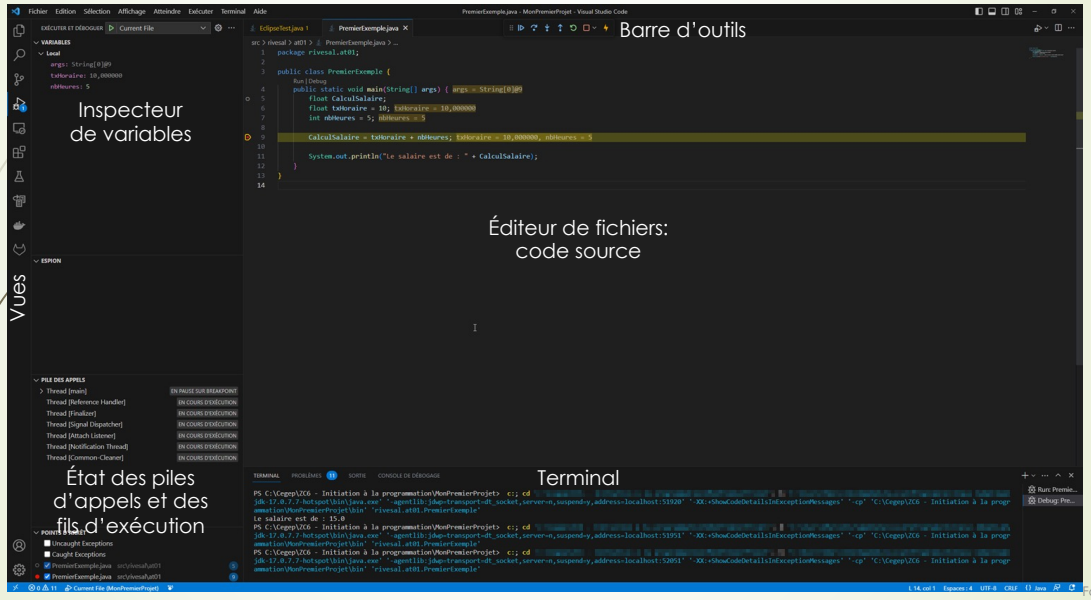
- Après avoir défini un ou plusieurs points d'arrêt, on peut lancer le débogueur de plusieurs façons:
  - Cliquez sur la flèche à droite du triangle, puis sur *Debug Java*
  - Cliquez sur *Debug* au-dessus du `main()`
  - Faites un clic droit dans la fenêtre contenant votre code pour ouvrir le menu contextuel, puis choisissez l'option *Debug Java*
- Lors du lancement du débogueur, il se peut que le **pare-feu** de *Windows* vous donne un avertissement. Dans ce cas, il faut **autoriser** la demande.
- Une fois le débogueur lancé, *VS code* s'arrête sur le premier point d'arrêt en **surlignant la ligne en jaune**.



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

28

Débogueur - Interface

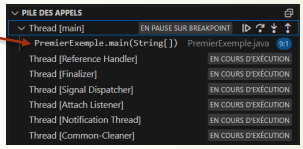


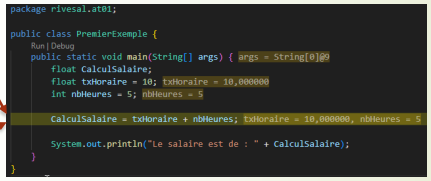
Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

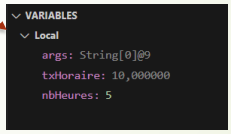
29

Débogueur - Interface

- La ***PILE DES APPELS*** montre la **hiérarchie des fonctions appelées** au moment de l'arrêt de l'exécution par le débogueur et permet de voir leurs interactions. Le seul fil d'exécution qui nous intéresse est le *Thread [main]*.
- Le **surlignage en jaune** indique la **prochaine instruction qui sera exécutée** ou celle en cours d'exécution dépendant de sa position dans la pile d'appels.
- Les valeurs courantes des variables de la méthode en cours d'exécution, **immédiatement avant l'exécution de la ligne surlignée en jaune**, sont affichées dans **l'onglet *VARIABLES***. De plus, les valeurs courantes des variables associées aux lignes en cours d'exécution ou déjà exécutées sont affichées en jaune dans le code.





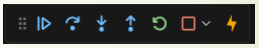


Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

30

Débogueur
Contrôle

- Pour contrôler l'exécution de votre programme dans le débogueur, vous pouvez utiliser la barre d'outils ou les raccourcis clavier.



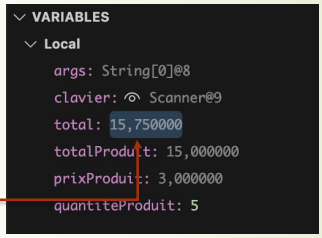
Outil	Icône	Raccourcis	Description
Continuer		*F5*	Reprend l'exécution normale du programme jusqu'au prochain point d'arrêt ou jusqu'à la fin du programme.
Pause		*F6*	Mets le programme en pause et démarre le débogueur sur la ligne en cours d'exécution.
Redémarrer		*Ctrl+Maj+F5*	Relance le programme en mode débogueur.
Arrêter		*Maj+F5*	Arrête l'exécution du programme.
Pas à pas détaillé		*F11*	Avance l'exécution d'une ligne. Si cette ligne concerne un appel de méthode, on affiche son code source et on s'arrête sur sa première ligne.
Pas à pas principal		*F10*	Avance l'exécution d'une ligne. Si cette ligne concerne un appel de méthode, on l'exécute sans entrer dans le débogueur.
Pas à pas sortant		*Maj+F11*	Termine l'exécution de la méthode en cours et s'arrête dans l'appelant de cette méthode.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

31

Débogueur
Inspecteur de variables

- Dans la vue *Exécuter et déboguer*, le programme s'arrête au premier point d'arrêt rencontré. On peut alors inspecter les variables dans l'onglet *VARIABLES* se trouvant dans la zone supérieure gauche de la fenêtre.
- Les variables dont la valeur a changé depuis l'arrêt précédent sont surlignées en bleu.
- On peut ainsi exécuter le programme pas à pas et vérifier l'exactitude des données dans l'onglet *VARIABLES*.
- Il est aussi possible de modifier la valeur des variables en double cliquant sur la valeur et en y inscrivant une nouvelle valeur. Cette dernière sera utilisée pour la suite de l'exécution.



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

32

Débogueur
Quitter

- Pour quitter le débogueur, vous devez premièrement arrêter l'exécution de votre programme en utilisant l'icône *Arrêter*.
- Ensuite, il est recommandé de revenir à la vue de l'*Explorateur*. Pour y arriver, cliquez sur l'icône se trouvant en haut à gauche ou utilisez le raccourci clavier ***CTRL+MAJ+E***.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

33

Débogueur
Exemple

- Ouvrez, dans *VS Code*, le fichier `Facture.java`, qui se trouve dans le *package* `sandbox` de votre projet *Java*. S'il n'est pas présent, vous pouvez le télécharger sur le site *Moodle* du cours.
- Placez un point d'arrêt à la ligne 18 :
```
totalProduit = prixProduit * quantiteProduit;
```
- Démarrez le débogueur et entrez un prix de 10 et une quantité de 5.
- Faites une exécution pas à pas (*F10*) et vérifiez si les valeurs des variables correspondent à celles de la trace d'exécution de la page suivante.

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

34

Trace d'exécution

- Algorithme qui calcule le coût, incluant les taxes, d'un produit

```
ALGORITHME calculFacture
CONSTANTES
      TPS = 5%
      TVQ = 9,975%
VARIABLES
      total, totalProduit, prixProduit,
      quantiteProduit  TYPE NUMÉRIQUE
DÉBUT
14    AFFICHER("Entrez le prix du produit : ")
15    LIRE(prixProduit)
16    AFFICHER("Entrez la quantité achetée : ")
17    LIRE(quantiteProduit)
18    totalProduit ← prixProduit * quantiteProduit
19    total ← totalProduit
20    total ← total + totalProduit * TPS
21    total ← total + totalProduit * TVQ
22    AFFICHER("Le total de la facture est ", total)
FIN
```

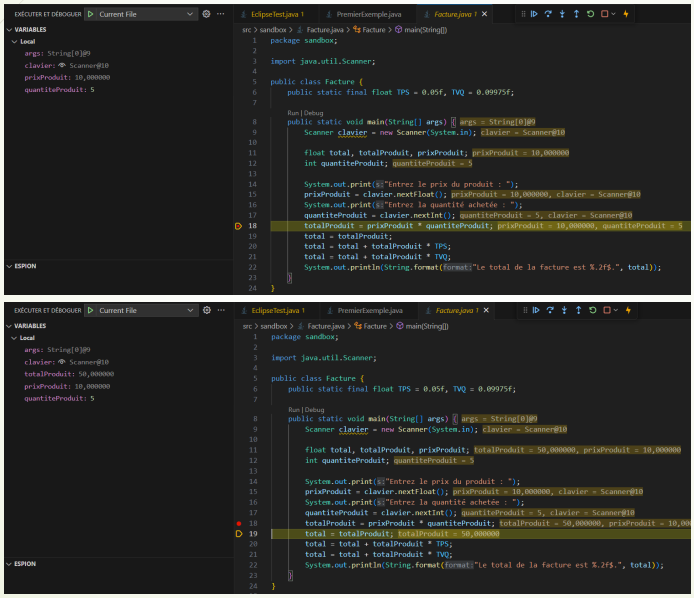
Pour cette trace on suppose que l'usager entre un prix de 10$ et une quantité de 5.

#	total	total Produit	prix Produit	quantite Produit	Terminal
14					Entrez le prix du produit :
15			10		
16					Entrez la quantité achetée :
17				5	
18		50			
19	50				
20	52,50				
21	57,49				
22					Le total de la facture est 57,49

Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

35

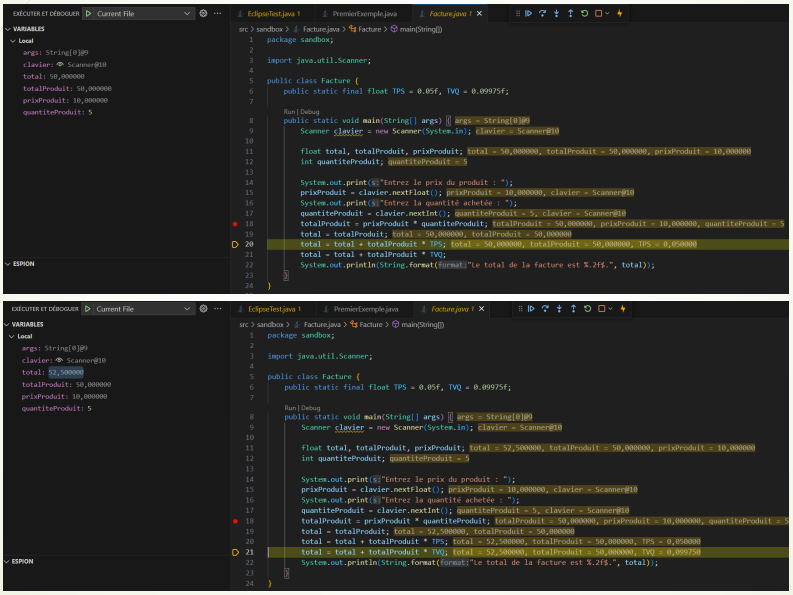
Débogueur
Exemple – Exécution (1/3)



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

36

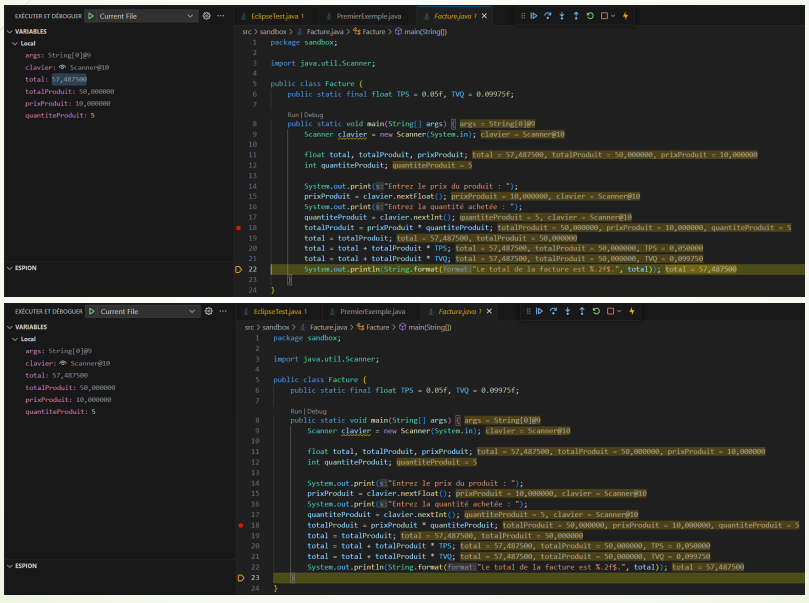
Débogueur
Exemple – Exécution (2/3)



Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

37

Débogueur
Exemple – Exécution (3/3)



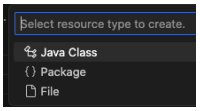
Farida Ait-Ammar
Nicolas Leduc
Alexandre Rives

B.10 Édition d'un ordinogramme (*draw.io*)

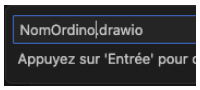
Visual Studio Code - Annexe

Édition d'ordinogrammes (*draw.io*)

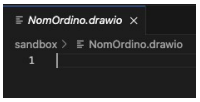
- Pour créer un nouvel ordinogramme, dans l'onglet *JAVA PROJECTS*, cliquez sur le + à droite du nom du répertoire dans lequel vous désirez l'enregistrer.



- Dans le menu qui apparaît au haut de la fenêtre de *VS Code*, sélectionnez *File*.



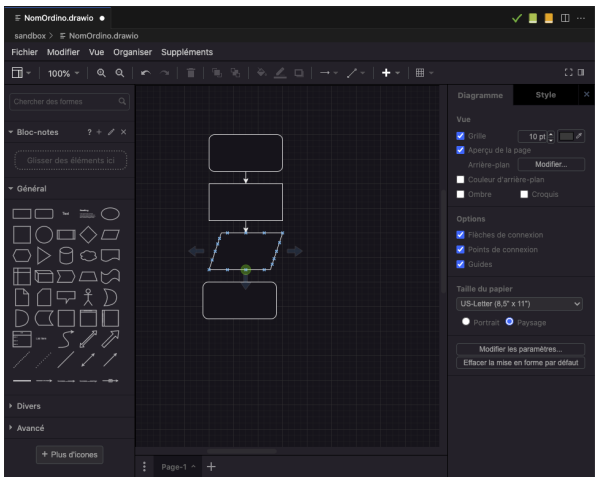
- Inscrivez ensuite le nom du fichier et ajoutez l'extension `.drawio`

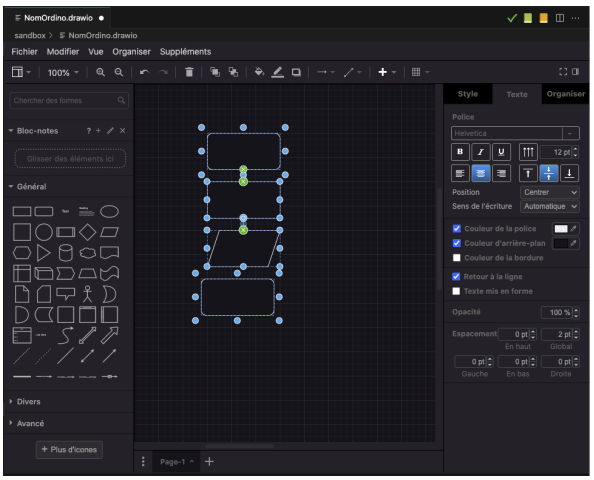


- Un onglet vide risque de s'ouvrir. Juste au dessus de la ligne *1*, sélectionnez le nom de votre fichier, puis appuyez sur la barre d'espacement afin que l'éditeur s'ouvre.

- Ouvrez l'onglet *Général* à gauche de l'éditeur pour avoir accès au formes. Vous pouvez alors les glisser-déposer dans la zone centrale pour construire votre ordinogramme.
- Double cliquez sur les formes afin d'y indiquer votre texte.
- Placez votre souris sur les bordures des formes pour faire apparaître un cercle vert qui correspond au point de départ d'une flèche. Cliquez, maintenez le bouton enfoncé et déplacez vous vers la position finale de votre flèche, puis relâchez le bouton.

ATTENTION : Si vous copiez-collez du texte dans vos formes, le format du texte sera aussi copié, ce qui est problématique lors de l'exportation des ordinogrammes. Désactivez la mise en forme du texte afin de corriger le problème :
- Sélectionnez toutes les formes de votre ordinogramme (*CTRL+A*).
- À droite de la fenêtre, sélectionnez l'onglet *Texte*.
- Décochez *Texte mis en forme*, dans le bas du volet de droite. S'il n'est pas coché, cochez-le puis décochez-le afin d'appliquer la modification à toutes les formes.
- Corrigez manuellement la position de vos textes, si nécessaire.



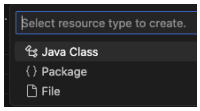


B.11 Édition d'une trace d'exécution (`.csv`)

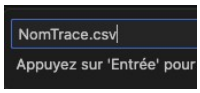
Édition de traces d'exécution (*csv*)

- Pour créer une nouvelle trace d'exécution, dans l'onglet *JAVA PROJECTS*, cliquez sur le + à droite du nom du répertoire dans lequel vous désirez l'enregistrer.

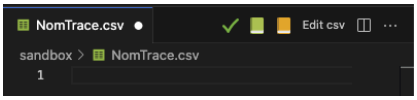
- Dans le menu qui apparaît au haut de la fenêtre de *VS Code*, sélectionnez *File*.



- Inscrivez ensuite le nom du fichier et ajoutez l'extension `.csv`

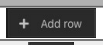
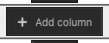
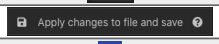


- Un onglet vide s'ouvre alors. Insérez une espace au début du fichier pour que l'éditeur *csv* détecte votre fichier, sinon *No data received* est indiqué et l'édition est impossible.

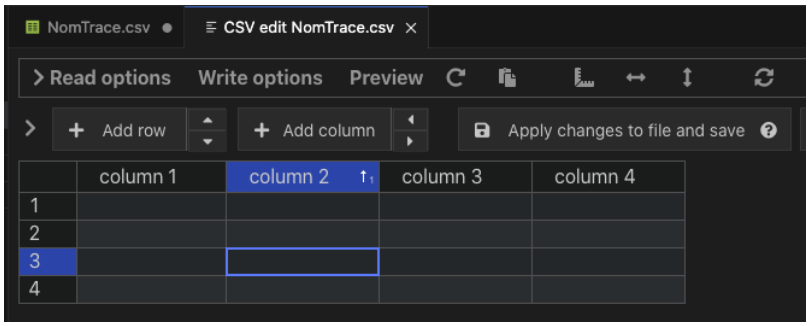


- Sélectionnez *Edit csv* en haut à droite de la fenêtre de *VS Code*.

- Vous devez sauvegarder (*CTRL+S*) le contenu de l'onglet *CSV edit* ou de celui de votre fichier *csv* afin que l'autre onglet soit aussi modifié. **ATTENTION** : Si vous fermez l'onglet *CSV edit* sans l'avoir sauvegardé, vous modifications seront perdues.

Bouton	Utilisation
	Ajoute une ligne au bas du tableau
	Ajoute une ligne au dessus de la ligne active
	Ajoute une ligne en dessous de la ligne active
	Ajoute une colonne à droite du tableau
	Ajoute une colonne à gauche de la colonne active
	Ajoute une colonne à droite de la colonne active
	Copie le contenu de l'onglet *CSV edit* vers votre fichier *csv*
	Efface la ligne ou la colonne associée

- Pour déplacer une ligne ou une colonne, cliquez d'abord sur son numéro pour la sélectionner en entier, puis placez vous dans la zone bleu à gauche dans la cellule d'entête de ligne ou de colonne afin d'obtenir l'icône d'une main ouverte. Vous pourrez alors glisser-déplacer votre ligne ou votre colonne.



C. Gestion de versions (*Git*)

C.1 Introduction à la gestion de versions

2

Gestion de versions
Définition

- Qu'est-ce que la gestion de versions?
  - La **gestion de versions** (en anglais : *version control* ou *revision control*) consiste à maintenir **l'ensemble des versions d'un ou de plusieurs fichiers** (généralement en texte) afin d'en **suivre l'évolution** au cours du temps. Essentiellement utilisée dans le domaine de la création de logiciels, elle concerne surtout la **gestion des codes source**.
  - Cette activité étant fastidieuse et relativement complexe, un appui logiciel est presque indispensable. On parle alors de système de gestion de versions (en anglais : *version control system* (*VCS*) ou *revision control system* (*RCS*)). Il existe différents logiciels de gestion de versions qui, bien qu'ayant des concepts communs, apportent chacun leur propre vocabulaire et leurs propres usages.

Source: [redacted]

Farida Ait-Ammar
Nicolas Leduc

3

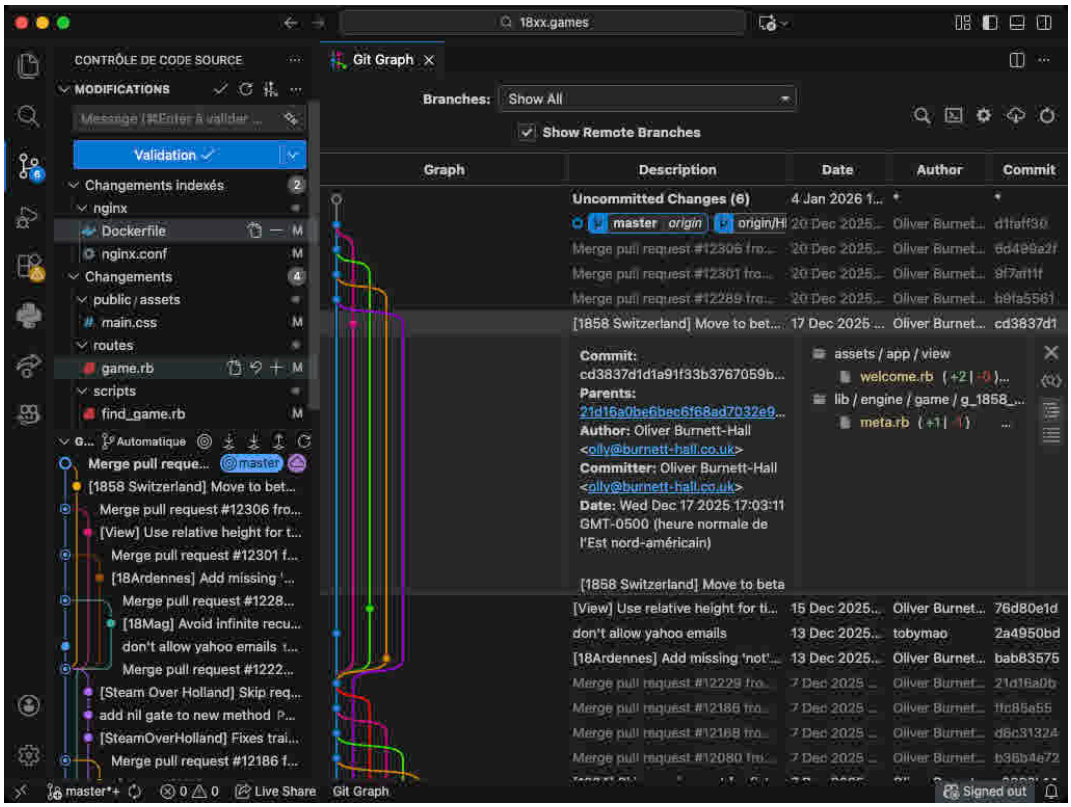
Gestion de versions
Avantages

- Ramener un fichier à un état précédent
- Ramener le projet complet à un état précédent
- Visualiser les changements au cours du temps
- Suivre l'évolution du développement du logiciel
- Simplifier le développement collaboratif de logiciels
- Identifier la source et le moment d'apparition d'un problème
- Peu de travail additionnel

Utiliser un *SGV* signifie généralement que si vous vous trompez ou que vous perdez des fichiers, vous pouvez facilement revenir à un état stable.

Farida Ait-Ammar
Nicolas Leduc

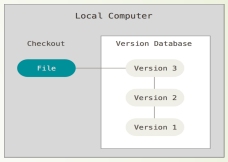
Exemple d'interface de gestion de versions dans *Visual Studio Code*

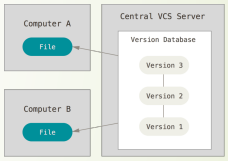


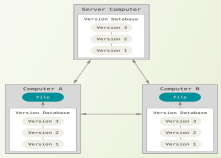
5

Gestion de versions Logiciels

- **Systèmes locaux** : aucune copie de l'historique n'est conservée sur un serveur, uniquement sur la machine locale
  - Copie manuelle de fichiers
  - *RCS*
- **Systèmes centralisés** : tous les traitements sont réalisés directement sur le serveur
  - *Concurrent Versions System (CVS)*
  - *Subversion (SVN)*
  - *Perforce*
- **Systèmes distribués** : chaque usager a une copie (clone) du dépôt dont la version maître est conservée sur un serveur
  - *Git*
  - *Mercurial*
  - *Bazaar*







Source des images : [redacted]

Farida Ait-Ammar
Nicolas Leduc

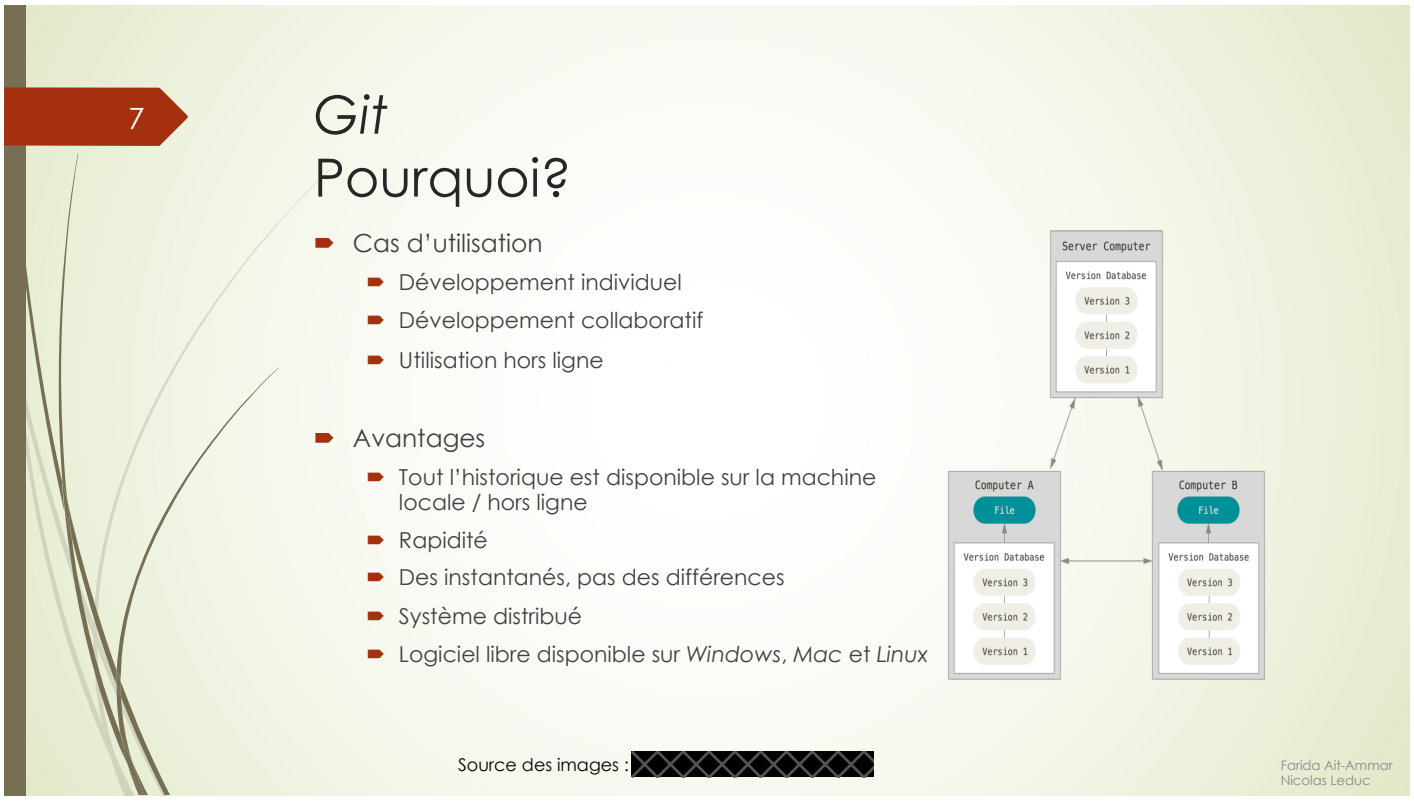
C.2 Introduction à *Git*

6

Git Historique

- En 1991, *Linus Torvalds* commence le développement du noyau de *Linux*
- Jusqu'en 2002, les versions du noyau étaient gérées à l'aide de patches et d'archives de fichiers
- En 2002, le projet fut déplacé vers un système de gestion de versions distribué, soit *BitKeeper*
- En 2005, *BitKeeper* devint un logiciel payant, donc la communauté de *Linux* dut trouver une alternative
- *Linus Torvalds* développa alors son propre outil en se basant sur les leçons apprises
- Certains des objectifs du nouveau système étaient les suivants :
  - Vitesse
  - Compacité des données
  - Conception simple
  - Support pour les développements non linéaires (milliers de branches parallèles)
  - Complètement distribué
  - Capacité à gérer efficacement des projets d'envergure, comme le noyau *Linux*

Farida Ait-Ammar
Nicolas Leduc



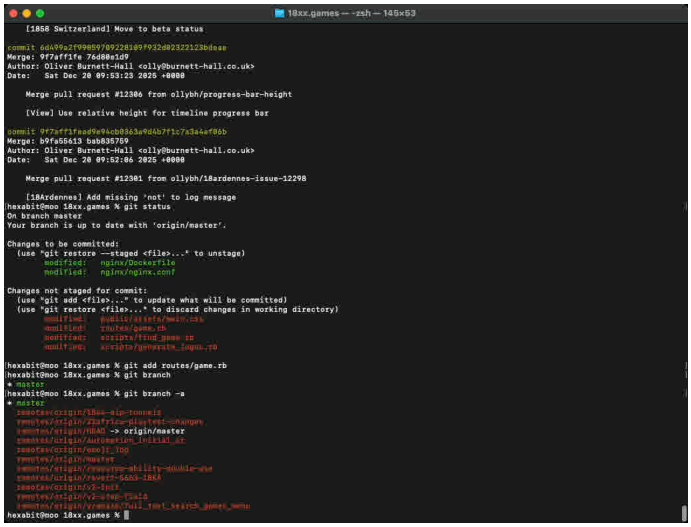
Pour utiliser *Git*, vous devez d'abord l'installer. Il est gratuit et peut être installé sur *Windows*, *Linux* et *Mac*. Selon le système d'exploitation, la procédure d'installation est différente :

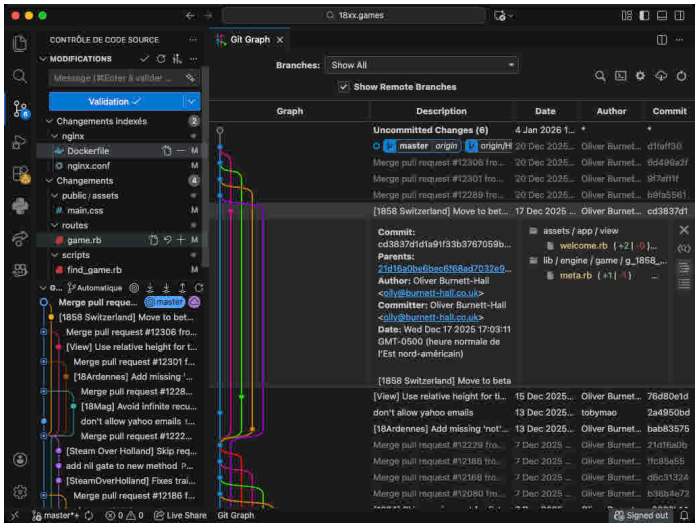
- *Windows* : Téléchargez *Git* à partir du site [redacted], puis exécutez le fichier obtenu. La procédure complète est indiquée dans l'annexe A.4 de ce document.
- *Linux* : Il est parfois installé automatiquement dans *Linux*. Si ce n'est pas le cas, vous pouvez l'installer à l'aide de la commande `apt-get install git` en tant qu'administrateur (`root`) dans un terminal. Cette commande est valide pour les systèmes basés sur *Debian*, comme *Debian*, *Ubuntu*, *Mint*, etc. Pour les autres distributions de *Linux*, consultez le site [redacted].
- *Mac* : Exécutez, dans un terminal, la commande `xcode-select --install` , qui installera *Git* et d'autres programmes, en ligne de commande, pour le développement d'applications.

Une fois installé, vous avez accès à *Git* sur la ligne de commande. Des interfaces graphiques sont toutefois disponibles :

- Directement dans l'environnement de développement, comme *VSCode*, *Eclipse*, etc.
- *Git GUI*, inclus en installant *Git* pour *Windows*
- *SmartGit* pour *Windows*, *Linux* et *Mac* ([redacted])
- *GitKraken* pour *Windows*, *Linux* et *Mac* ([redacted])
- *Sourcetree* pour *Windows* et *Mac* ([redacted])
- *Git Extensions* pour *Windows* ([redacted])

Voici une comparaison entre la ligne de commande et l'interface graphique.





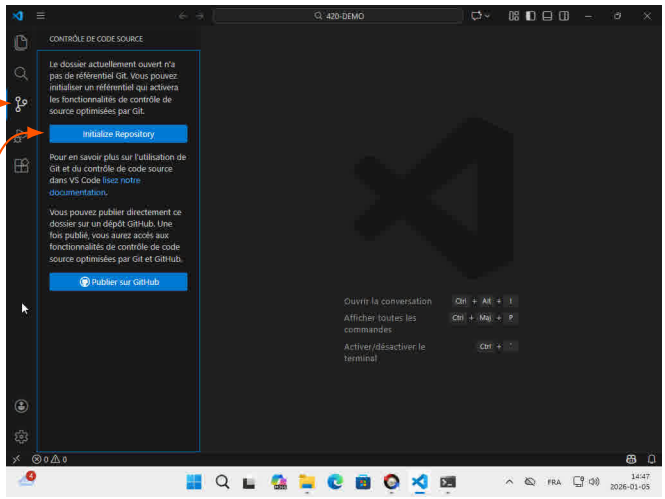
C.3 Configuration de *Git* et *Gitea*

Pour commencer à travailler avec *Git*, vous devez d'abord installer les exécutables de *Git* ainsi que l'extension *Git Graph* dans *Visual Studio Code*, qui sera utilisée dans le cadre de ce cours. Consultez le guide d'installation, à l'annexe A, pour connaître la procédure et les configurations recommandées.

Ensuite, vous devez initialiser un dépôt, c'est-à-dire demander à Git de créer une structure de dossiers dans le dossier racine de votre futur dépôt afin de conserver les différentes versions de vos fichiers ainsi que d'autres informations.

Pour initialiser ou utiliser un dépôt *Git*, exécutez la procédure suivante dans *Visual Studio Code* :

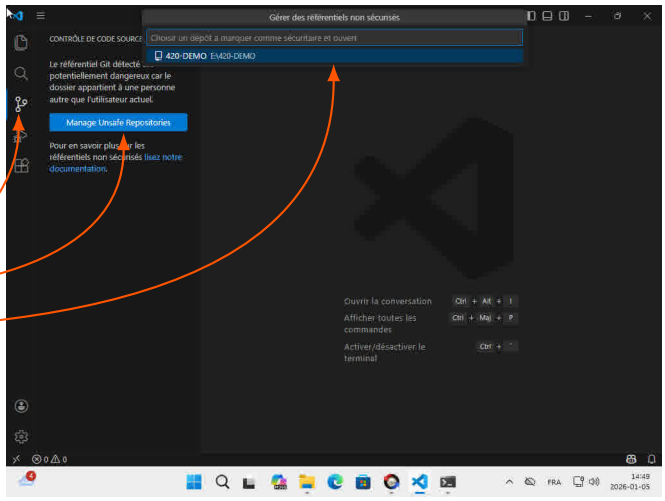
- Ouvrez le dossier racine de votre projet *Java* : *Fichier -> Ouvrir le dossier ...*
- Sélectionnez la vue du *Contrôle de code source*
- Si aucun dépôt n'est détecté, on vous propose d'en créer un en appuyant sur *Initialize Repository*
- Si un dépôt existe, soit le cas usuel, les listes des fichiers modifiés et des *Commit* sont alors affichées



IMPORTANT : Votre dépôt ne doit pas se trouver dans *One Drive*, car il pourrait devenir inutilisable.

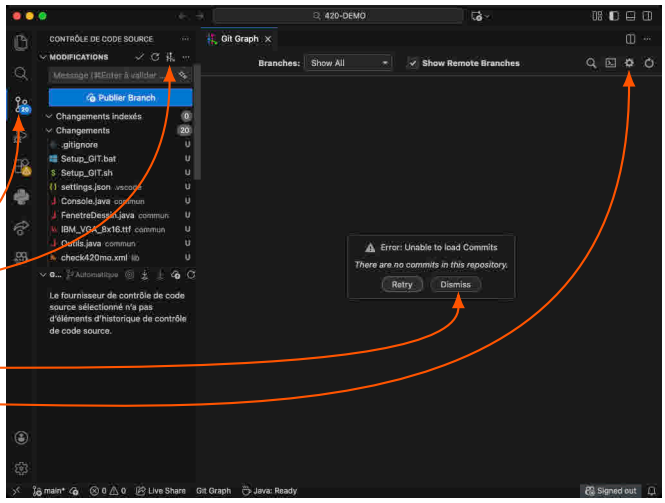
Le système de fichiers utilisé sur les clé *USB*, soit *FAT32* ou *exFAT*, n'est pas considéré sécuritaire par *Git*, Si vous tentez de créer ou de manipuler un dépôt *Git* sur une clé *USB*, vous obtenez l'erreur : *Git : fatal : detected dubious ownership in repository at ...* Pour régler le problème vous devez :

- Sélectionnez la vue du *Contrôle de code source*
- Cliquez sur *Manage Unsafe Repositories*
- Sélectionnez le dossier de votre dépôt
- Si l'option n'est pas disponible, ouvrez un terminal (*Affichage -> Terminal*) et inscrivez :
  `git config --global safe.directory "*"`

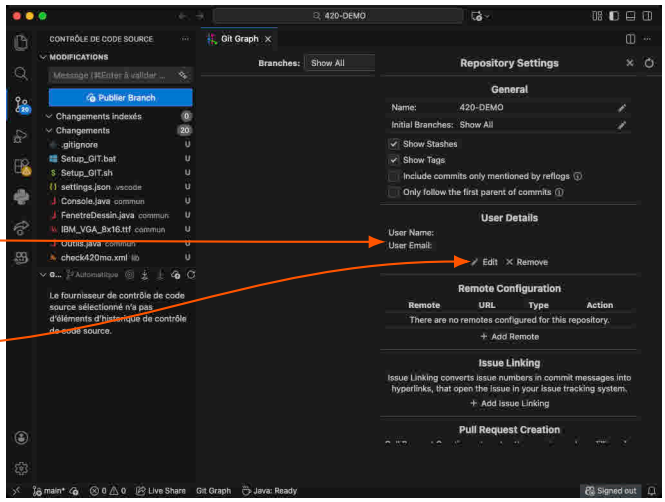


Pour être en mesure d'ajouter de nouvelles versions de fichiers dans le dépôt *Git*, il est obligatoire de fournir son nom et son courriel. Exécutez la procédure suivante une seule fois pour chaque dépôt afin d'effectuer la configuration :

- Sélectionnez la vue du *Contrôle de code source*
- Ouvrez l'onglet *Git Graph*
- Si votre dépôt est vide, une erreur apparaît et vous devez appuyer sur *Dismiss*
- Ouvrez le panneau de configuration de *Git Graph*

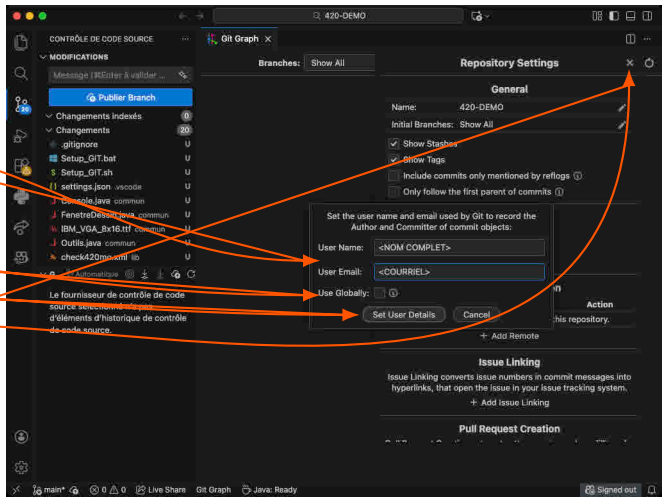


- Dans le panneau de configuration de *Git Graph*, la section *User Details* indique le nom et le courriel configurés dans *Git*
- Pour enregistrer les informations dans votre dépôt et ne pas avoir à les inscrire à chaque fois que vous utiliser un nouveau poste de travail, cliquez sur *Edit*

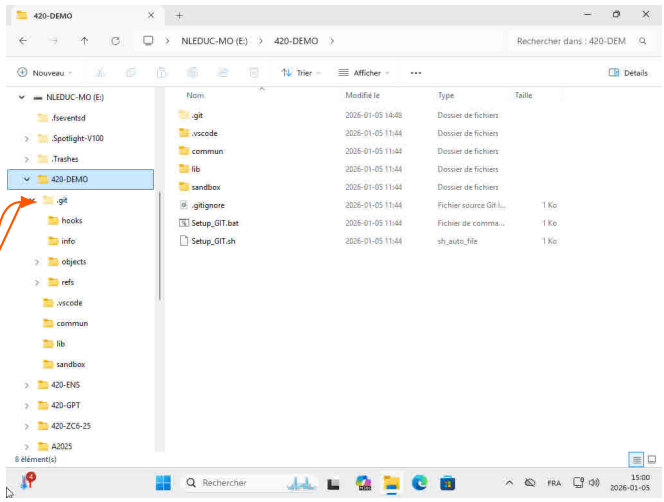


- Inscrivez votre nom complet et un courriel
- Décochez la case *Use Globally* afin que les informations soient enregistrées dans votre dépôt et non uniquement sur le poste de travail
- Cliquez sur *Set User Details*
- Fermez le panneau de configuration de *Git Graph*

Vous êtes maintenant prêt à utiliser votre dépôt *Git*.



Un dépôt *Git* est en fait un sous-dossier caché du dossier racine de votre projet *Java*. Il se nomme `.git` et contient une hiérarchie de fichiers et de dossiers. Par exemple, le dossier `objects` contient tous les fichiers, sous forme compressée, que vous avez mis dans votre dépôt *Git*. **Vous ne devez jamais modifier ou supprimer le contenu du dossier `.git`.**



12

Serveur *Git*

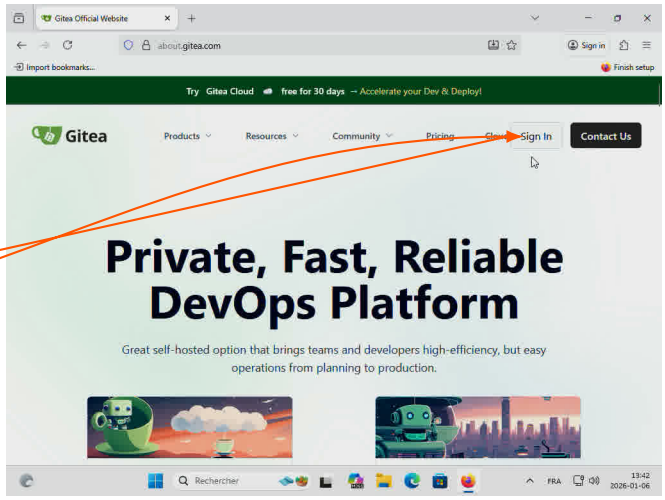
- *Git* est utile pour gérer son code source sur sa machine locale. Toutefois, dans les projets d'envergure, on veut être en mesure de partager notre code avec les autres développeurs ou avec notre enseignant.
- *Git* permet de partager l'historique de votre code via un serveur. Vous pouvez soit créer votre propre serveur ou faire appel aux services d'une entreprise qui hébergera votre dépôt *Git*, par exemple:
  - *GitHub* ([redacted])
  - *GitLab* ([redacted])
  - *BitBucket* ([redacted])
- En plus d'héberger votre code, ces entreprises vous offrent d'autres outils comme la navigation et la visualisation de vos fichiers et de leur historique en ligne, des outils de collaboration, etc.
- Plusieurs forfaits sont offerts et il est souvent possible d'obtenir un compte gratuit limité pour une utilisation non-commerciale.

Farida Ait-Ammar
Nicolas Leduc

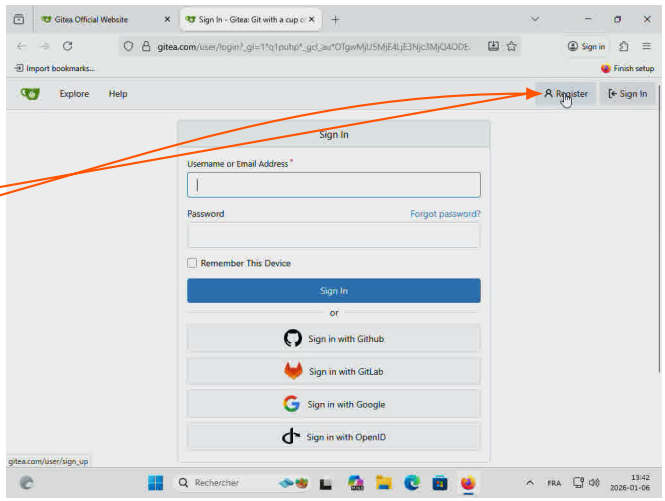
Dans le cadre de ce cours, nous utiliserons *Gitea*, disponible à l'adresse [redacted]. Vous devez vous créer un compte si vous n'en avez pas déjà un.

Pour créer un compte :

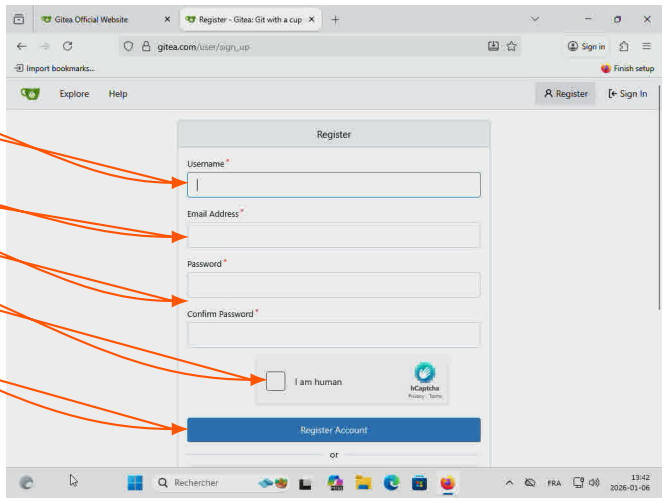
- Sélectionnez *Sign in*



- Sélectionnez *Register*

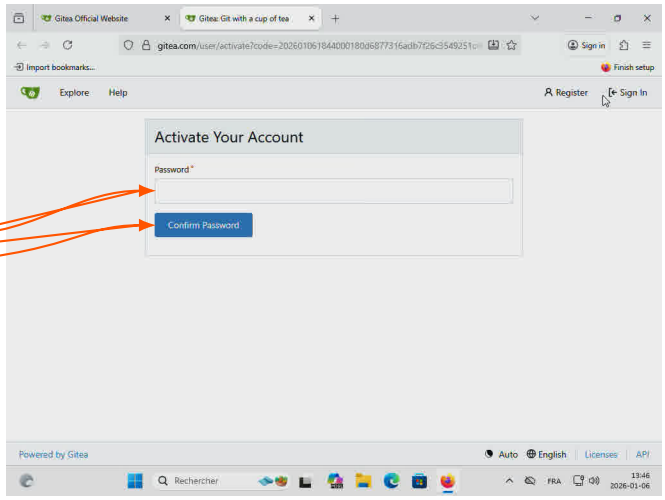


- Sélectionnez un nom d'utilisateur qui permettra à votre enseignant de vous identifier facilement
- Indiquez votre courriel personnel ou celui du *Collège*, soit ########@[redacted]
- Choisissez un mot de passe et confirmez-le
- Prouvez que vous n'êtes pas un robot!
- Cliquez sur *Register Account*



On vous propose aussi de vous créer un compte en utilisant différents sites, comme *Google* ou *GitHub*, mais ce n'est pas recommandé, car ça complique la procédure. Vous pourrez associer vos comptes par la suite.

- Cliquez sur le lien que vous recevrez par courriel
  Il est à noter que votre courriel du *Collège* est visible dans *Outlook* et non dans la messagerie [redacted]
- Entrez votre mot de passe pour confirmer votre compte
- Cliquez enfin sur *Confirm Password*

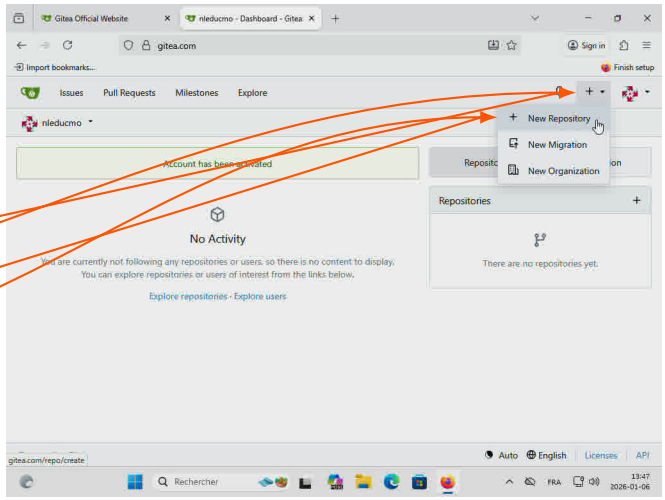


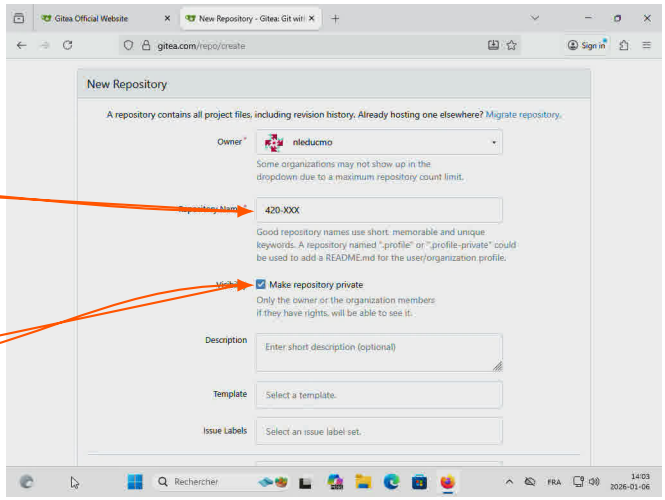
Vous avez maintenant un compte sur le serveur *Gitea*. Ne perdez pas votre mot de passe, vous en aurez besoin tout au long de la session.

Votre compte vous donne accès au serveur *Gitea*. Toutefois, il vous faut créer un dépôt *Git* sur celui-ci afin de partager votre travail avec votre enseignant.

Pour créer un dépôt *Git* sur *Gitea* :

- Cliquez sur +
- Cliquez sur + *New Repository*

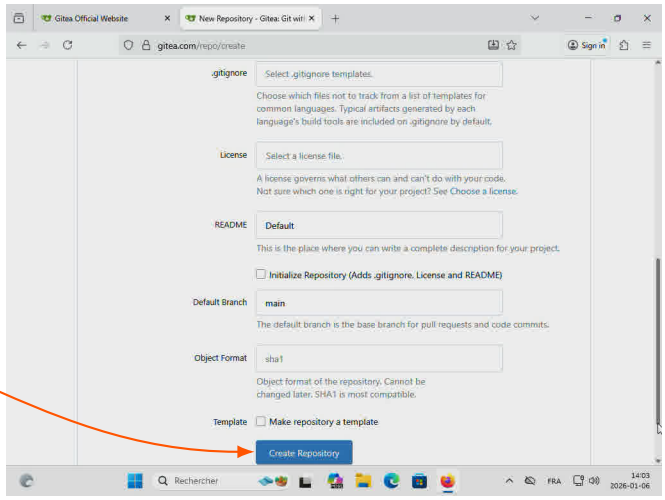




- Inscrivez le nom du dépôt, soit le sigle de votre cours, par exemple, `420-ZC6`, `420-ZF5`, etc
  Dans la suite des notes de cours, `420-XXX` ou `NOMPRO` font référence au nom de votre dépôt
- Assurez-vous que votre dépôt est privé en cochant la case *Make repository private*, sinon les autres étudiants pourront voir et copier vos travaux

- Faites défiler la page vers le bas
- Assurez-vous de ne pas initialiser le dépôt en décochant la case *Initialize Repository*, si nécessaire, sinon vous ne pourrez pas pousser votre dépôt local
- Cliquez sur *Create Repository*

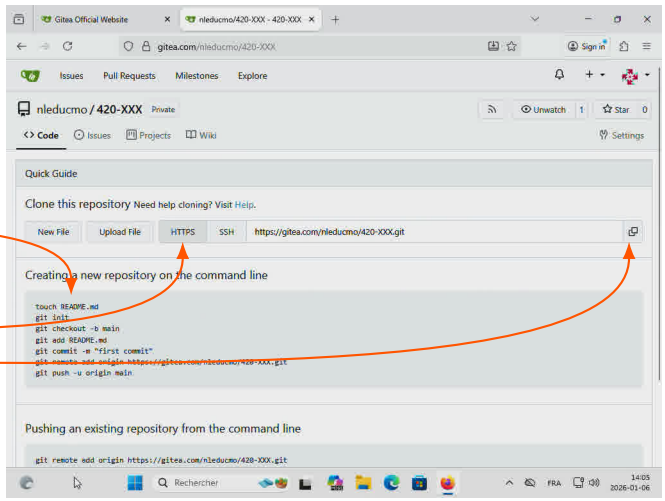
Bravo ! Vous avez un dépôt *Git* sur le serveur *Gitea*.



Vous arrivez ensuite sur la page de votre dépôt. Comme il est vide pour l'instant, on vous indique différentes façons de le configurer sur votre machine locale.

Dans notre cas, il nous faut seulement l'*URL* du dépôt :

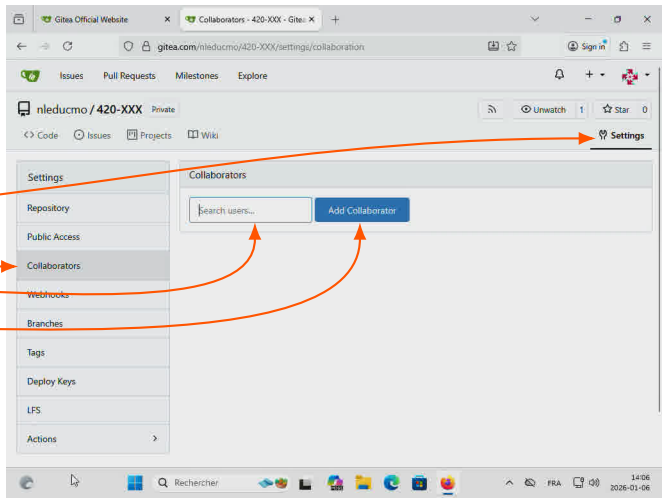
- Sélectionnez *HTTPS*
- Copiez l'adresse à droite de *HTTPS*



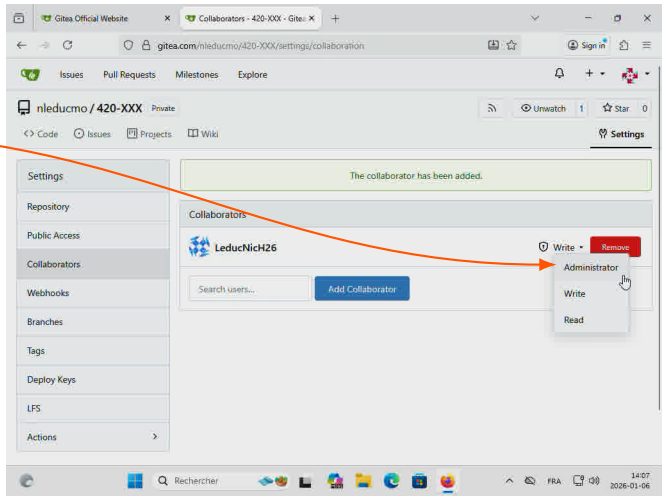
Pour que votre enseignant puisse voir votre travail, vous devez lui partager votre dépôt.

- Dans la page principale de votre dépôt, cliquez sur l'onglet *Settings*
- Cliquez *Collaborators* dans le menu à gauche
- Indiquez le nom du compte de votre enseignant
- Appuyez sur *Add Collaborator*

Votre enseignant recevra un courriel et vous verrez son nom apparaître dans la liste des collaborateurs au projet.



- Autorisez votre enseignant à modifier vos fichiers en lui donnant les permissions d'*Administrator*



Votre travail sur le site *Gitea* est maintenant terminé. Vous pourrez y retourner seulement pour voir l'état de votre dépôt. **Toute modification à partir du site *Gitea* est interdite, car elle peut rendre votre dépôt inutilisable. De plus, il est formellement défendu de supprimer un dépôt sans avoir consulté au préalable votre enseignant. Le non respect de ces règles pourrait être considéré comme du plagiat.**

20

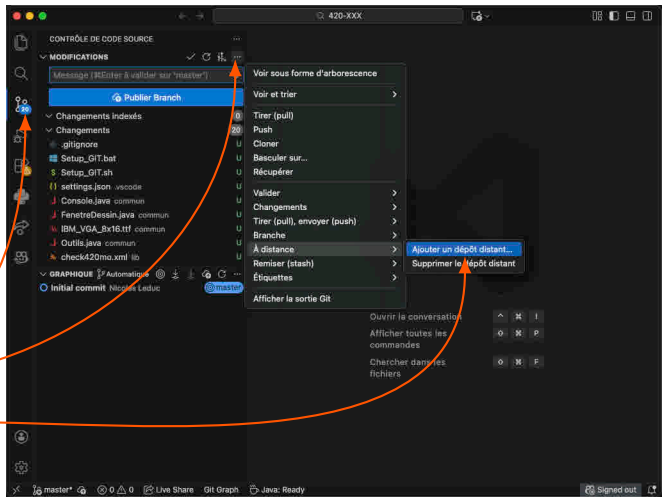
Git
Connexion à un serveur

- Pour utiliser un dépôt *Git* hébergé sur un serveur, il vous faut créer une connexion entre votre dépôt local et celui du serveur.
- Pour ce faire, vous avez deux possibilités:
  - Si le dépôt sur le serveur contient déjà des documents, il vous faudra le cloner.
    - Exemple: Vous voulez accéder à vos fichiers à partir d'un autre ordinateur.
  - Si vous voulez copier votre dépôt local vers un serveur, il vous faudra le pousser vers un dépôt vide sur le serveur.
    - Exemple: Rendre public un logiciel sur lequel vous travaillez depuis quelque temps.
- Par la suite, vous pourrez partager vos modifications sur le serveur.

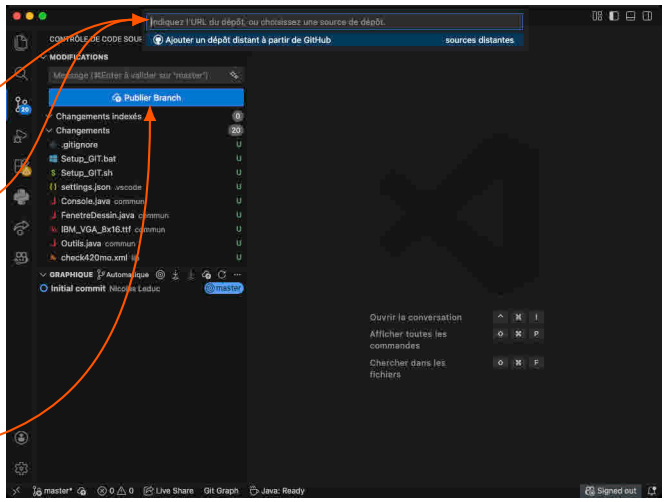
Farida Ait-Ammar
Nicolas Leduc

Après avoir créé un dépôt *Git* sur le serveur, il vous faut maintenant ajouter le lien entre celui-ci et votre dépôt local. Ce lien vous permettra d'y pousser votre travail afin que votre enseignant puisse voir votre progression. Pour connecter votre dépôt local au dépôt sur le serveur, vous devez avoir son adresse *HTTPS*, que vous avez copiée précédemment. Réalisez les étapes suivantes pour lier les dépôts :

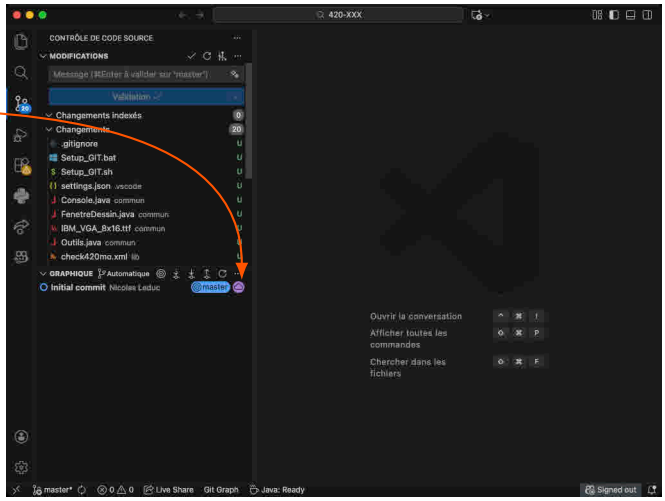
- Sélectionnez la vue du *Contrôle de code source*
- Cliquez sur ... à droite de *MODIFICATIONS*
- Sélectionnez *À distance -> Ajouter un dépôt distant...*.



- Dans la zone de texte au haut de l'écran, on vous demande *Indiquez l'URL du dépôt* , donc collez-y l'adresse *HTTPS* du serveur (*Ctrl+V*)
- Dans la même zone de texte, on vous demande ensuite *Nom du dépôt distant*, inscrivez alors `origin`
- La procédure de connexion au serveur s'enclenche ensuite et peut varier selon le système d'exploitation
  - On demande votre nom d'usager et mot de passe du serveur directement dans la zone de texte
  - Un navigateur s'ouvre, vous devez vous connecter et appuyer sur *Authorize Application*
- Cliquez enfin sur *Publier Branch*



- La procédure est un succès si un nuage apparaît dans l'onglet *GRAPHIQUE* à droite du ruban *master*



Vous êtes maintenant prêt à utiliser *Git*. Vous n'aurez pas à refaire les étapes précédentes lors de vos prochaines séances de travail, car la configuration a été sauvegardée directement dans votre dépôt *Git* sur votre clé *USB*.

Si vous n'avez pas réussi à voir le nuage apparaître, vous devez consulter votre enseignant. La commande suivante, exécutée au terminal, peut parfois régler le problème

```
git push origin master
```

C.4 Ignorer certains fichiers (`.gitignore`)

23

Git
Quels fichiers sauvegarder?

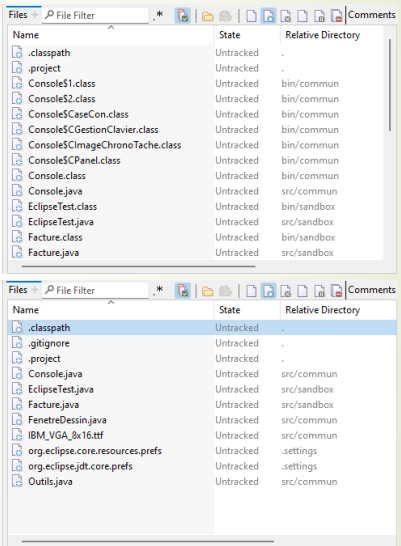
- L'utilisation d'un logiciel de gestion de versions est très utile pour garder un historique de notre travail. Toutefois, chaque instantané contient une copie de tous les fichiers modifiés depuis le dernier *commit*. Afin de limiter l'espace disque requis pour la gestion de versions, il est important de ne pas sauvegarder systématiquement tous les fichiers.
- Fichiers à sauvegarder:
  - Configuration du projet *Java* (`.vscode\settings.json`, `lib\*`)
  - Code source (`*.java`) et tests unitaires
  - Fichiers de données
  - Documentation
- Fichiers à ne pas sauvegarder:
  - Fichiers générés lors de la compilation (`*.class`)
  - Fichiers générés lors de l'exécution du programme (fichiers logs, fichiers temporaires, etc)

Farida Ait-Ammar
Nicolas Leduc

24

Git
Cacher certains fichiers

- Par défaut, *Git* nous indique tous les fichiers qui ont été modifiés, même ceux que l'on ne veut pas sauvegarder. Il est donc difficile de retrouver les fichiers pertinents à ajouter à l'historique.
- Il est possible d'indiquer à *Git* une liste de fichiers à ignorer à l'aide du fichier de configuration `.gitignore` (le nom du fichier débute par un point)
- Dans notre cas, nous allons ignorer tous les fichiers générés par le compilateur *Java* (`*.class`). Pour ce faire:
  - Créez un nouveau fichier texte (*VS Code* ou *Bloc-notes*)
  - Inscrivez `*.class` sur la première ligne
  - Sauvegardez le fichier, que vous nommerez `.gitignore` , dans le répertoire racine de votre dépôt
    - Exemple: `E:\ProjetsJava\NOMPRO\.gitignore`
- À la suite de l'opération, la liste des fichiers modifiés ne contient plus ceux générés par *le compilateur Java* (`*.class`).

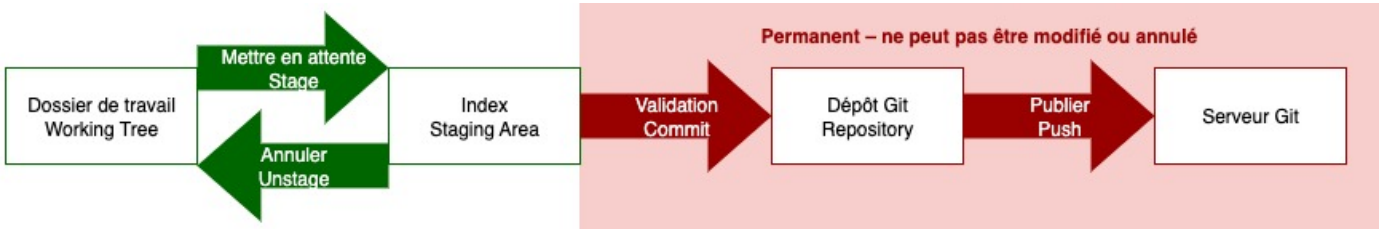


Farida Ait-Ammar
Nicolas Leduc

C.5 Sauvegarder les modifications

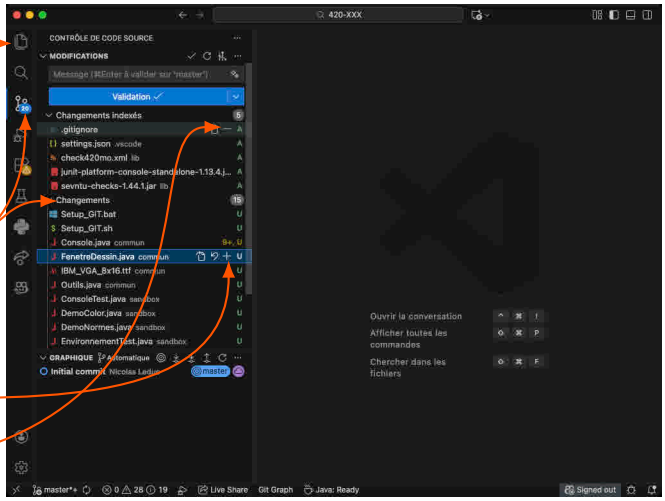
Dans *Git*, les fichiers peuvent se trouver dans différents états :

- Les fichiers actuels de votre projet se retrouvent dans le dossier de travail (*Working Tree*) sur votre disque dur ou votre clé *USB*. Lorsque vous modifiez un fichier, *Git* le détecte et l'indique à l'interface.
- Pour sauvegarder une version de votre projet, vous devez d'abord ajouter (copier) les fichiers concernés dans l'index (*Staging Area*) en les mettant en attente (*Stage*). L'index est un dossier caché de *Git* qui permet de préparer la prochaine version à sauvegarder.
- Ensuite, la validation (*Commit*) permet de transférer les fichiers de l'index (*Staging Area*) vers votre dépôt *Git* (*Repository*) local afin d'ajouter cette nouvelle version à l'historique et ainsi rendre permanent vos modifications.
- Enfin, vous transférez le nouvel historique de votre dépôt *Git* local vers le serveur *Git* en utilisant la commande publier (*Push*) afin que vos collègues puissent avoir accès à la dernière version de votre code.

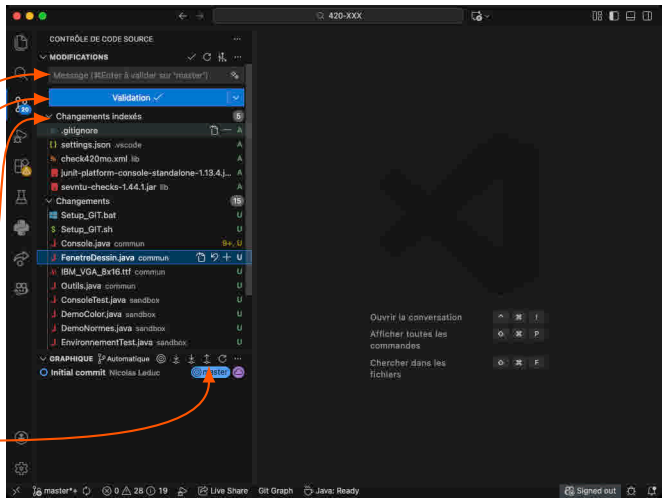


Pour sauvegarder une nouvelle version de votre programme dans *Git*, vous devez réaliser les étapes suivantes :

- Éditez ou créez des fichiers dans votre dossier de travail en utilisant l'*Explorateur* de *Visual Studio Code*
- Assurez-vous que votre code compile et testez-le
- Les fichiers ajoutés ou modifiés dans le dossier apparaissent dans la vue *Contrôle de code source* sous *Changements*, par ordre de dossier
- Ajoutez tous les fichiers associés à votre travail dans l'index, soit *Changements indexés* (*Staging Area*), en appuyant sur le + à droite du nom de chacun
- Vous pouvez retirer un fichier de l'index (*Unstage*) en appuyant sur le - à droite de son nom



- Inscrivez une description pertinente de vos changements qui sera associée à cette nouvelle version
- Appuyez sur le bouton *Validation* (*Commit*) pour ajouter vos changements à l'historique de *Git*
  À ce moment, tous les fichiers dans la section *Changements indexés* (*Staging Area*) et la description sont enregistrés de façon permanente dans votre dépôt *Git*
  ATTENTION : Cette opération ne peut pas être annulée et la description ne peut plus être modifiée !
- Appuyer sur le bouton publier (*Push*) pour envoyer vos changements sur le serveur *Git*



27

Git
Problèmes potentiels

- Si vous modifiez un fichier que vous aviez mis dans le *Staging Area*, alors vous devez refaire *Stage* sur le fichier modifié avant de faire *Commit*. En effet, lorsque vous appuyez sur ***Stage***, *Git* crée une **copie** de votre fichier. Il faut donc lui demander de faire une nouvelle copie de votre fichier avant de pouvoir le sauvegarder dans le dépôt.
- Lorsque vous appuyez sur le bouton ***Commit***, les fichiers du *Staging Area* et le message du *commit* sont enregistrés de façon **permanente** dans le dépôt. Assurez-vous que tout est parfait avant de faire votre *commit*.
- Pour éviter des problèmes, on vous conseille fortement d'ajouter les fichiers au *Staging Area* lorsque votre travail est terminé et que vous êtes prêt à sauvegarder votre nouvelle version dans votre dépôt (*Commit*).

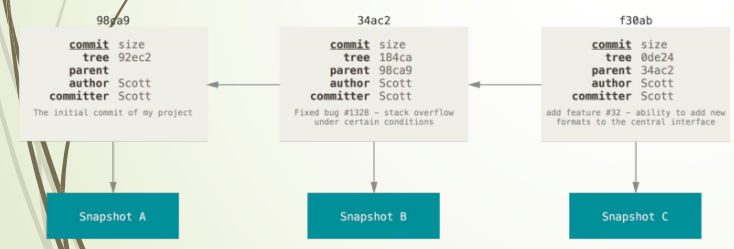
Farida Ait-Ammar
Nicolas Leduc

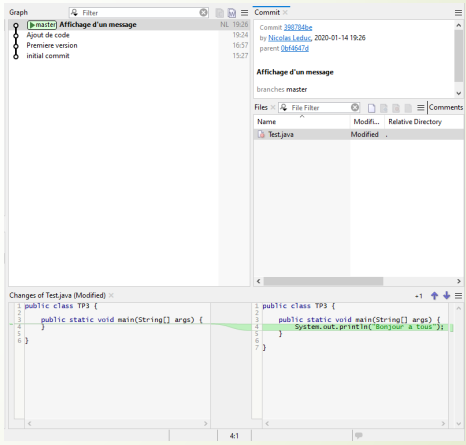
C.6 Naviguer dans l'historique

28

Git
Historique des instantanés

- Chaque *commit* crée un nouvel instantané dans l'historique de votre dépôt *Git*. Un instantané contient:
  - Informations (message, auteur, date, etc)
  - Liste de fichiers (*tree*)
  - Lien vers l'instantané précédent (*parent*)
  - Code d'identification unique

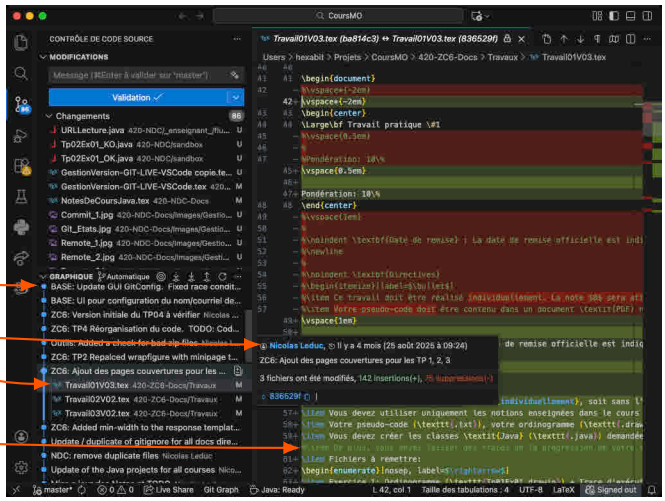




Farida Ait-Ammar
Nicolas Leduc

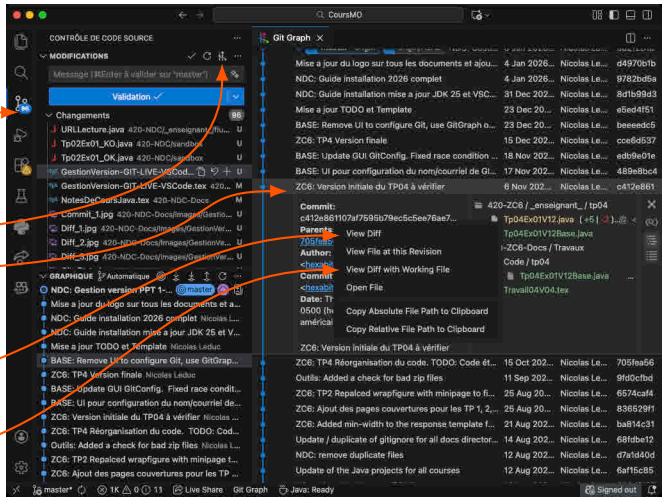
Dans *Visual Studio Code*, vous pouvez voir les informations sur un instantané en le sélectionnant dans la section *GRAPHIQUE* de la vue *Contrôle de code source*. Il est aussi possible d'accéder au graphe de l'historique en utilisant l'extension *Git Graph*. Vous pouvez ainsi consulter

- Séquence des instantanés
- Informations sur l'instantané en plaçant son curseur de souris sur celui-ci
- Fichiers concernés en cliquant sur l'instantané
- Différence entre le fichier sélectionné dans l'instantané et sa version précédente



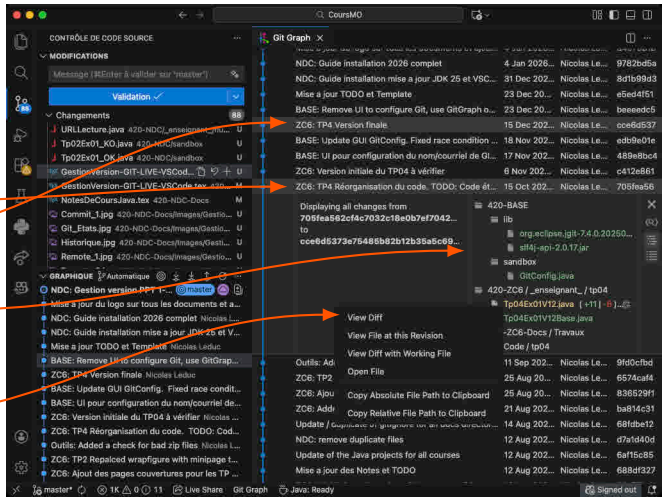
Lors du développement d'un logiciel, il arrive parfois que l'on désire comparer notre fichier courant avec une version antérieure. Le plus facile est d'utiliser l'extension *Git Graph* dans *Visual Studio Code*. Pour y arriver, vous devez :

- Sélectionnez la vue du *Contrôle de code source*
- Ouvrez l'onglet *Git Graph*
- Sélectionnez l'instantané désiré
- Cliquez avec le bouton de droit sur le fichier désiré
- Sélectionnez *View Diff* pour voir les différences entre la version de l'instantané et celle précédente
- Sélectionnez *View Diff with Working File* pour voir les différences entre l'instantané et le fichier actuel



Il est aussi possible de comparer deux versions précédentes d'un fichier, par exemple, pour identifier la source d'un bogue. Pour y arriver, dans l'onglet *Git Graph*, vous devez :

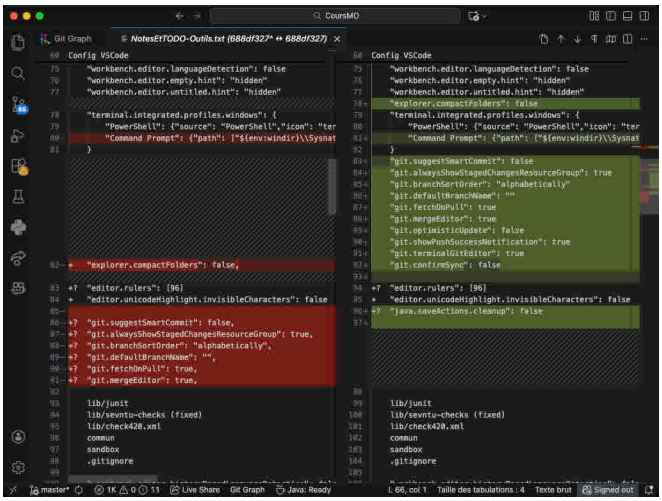
- Faites un simple clic sur un premier instantané
- Faites un *Ctrl* + clic sur un second instantané
- La boîte qui s'ouvre montre tous les fichiers qui ont été modifiés entre les deux instantanés sélectionnés
- Cliquez avec le bouton de droit sur le fichier désiré
- Sélectionnez *View Diff* pour voir les différences entre les deux versions des instantanés sélectionnés



Après avoir choisi l'option *View Diff* ou *View Diff with Working File*, un onglet en lecture seule s'ouvre pour vous montrer les différences entre les deux versions du fichier. Dans cet onglet, l'ancienne version du fichier est à gauche et la version récente est à droite. Les changements sont indiqués par différentes couleurs :

- Rouge : retraits ou modifications -> fichier de gauche
- Jaune : ajouts ou modifications -> fichier de droite

Plusieurs logiciels existent pour comparer les fichiers et il est souvent possible d'utiliser son logiciel favori avec *Git*.



C.7 Annuler les modifications

33

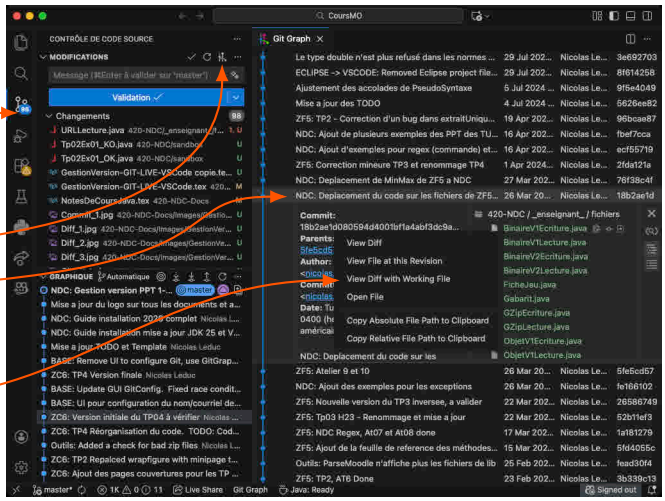
Git
Retour à une version précédente

- Lorsque la source d'un bogue a été identifiée, il peut être nécessaire de revenir à la dernière version ne comportant pas le bogue.
- Dans *Git*, il existe plusieurs méthodes pour revenir en arrière:
  - *Check Out* d'un *Commit*: remplace les fichiers de votre répertoire de travail par les fichiers de l'ancienne version et place *Git* dans un état appelé *Detached head*. Tous les *Commit* réalisés dans cet état sont perdus!
  - *Reset*: Efface les derniers *Commit* de l'historique. Tous les changements suivant le *Commit* choisi sont perdus.
  - *Revert*: Crée un nouveau *Commit* qui correspond à l'inverse du ou des *Commit* choisis. On conserve ainsi l'historique.
- Ces trois méthodes peuvent être difficiles à maîtriser et vous risquez de perdre vos modifications ou une partie de votre historique.

Farida Ait-Ammar
Nicolas Leduc

Dans le cadre du cours, si vous désirez revenir à une version précédente de l'un de vos fichiers, on vous conseille d'utiliser la procédure suivante :

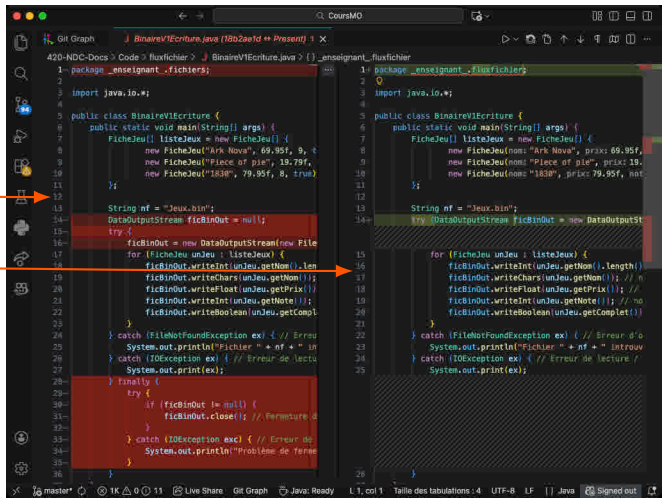
- Sélectionnez la vue du *Contrôle de code source*
- Ouvrez l'onglet *Git Graph*
- Sélectionnez l'instantané voulu
- Cliquez avec le bouton de droit sur le fichier désiré
- Sélectionnez *View Diff with Working File* pour voir les différences entre l'instantané et le fichier actuel



L'onglet qui s'ouvre montre la différence entre le fichier de l'historique et votre fichier actuel :

- Volet de gauche (*ID Commit*) : fichier provenant de l'historique de *Git*, qui est en lecture seule
- Volet de droite (*Present*)) : fichier actuel de votre dossier de travail, qui peut être modifié

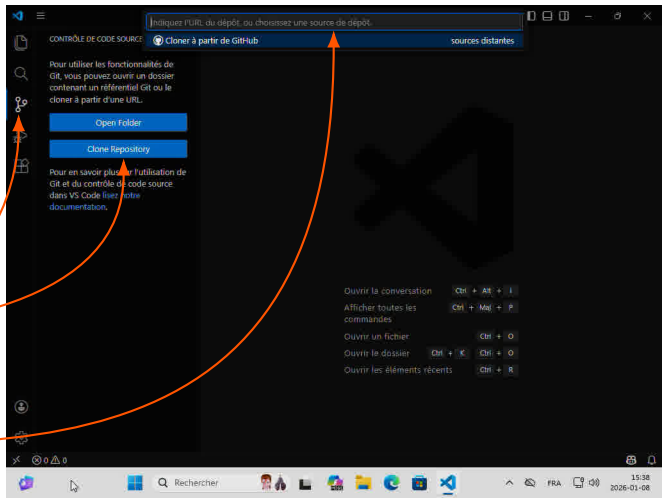
Vous pouvez ainsi copier des bouts de code vers votre fichier actuel et sauvegarder ce dernier afin de récupérer l'ancienne version de votre programme. N'oubliez pas d'ajouter cette version corrigée à votre dépôt *Git*.

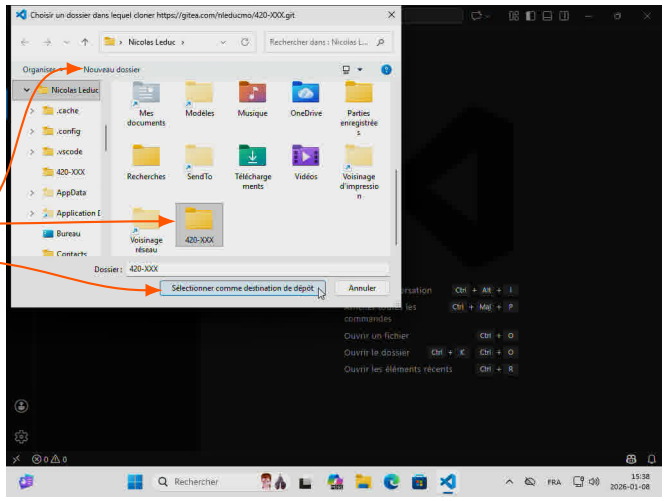


C.8 Interagir avec un serveur *Git*

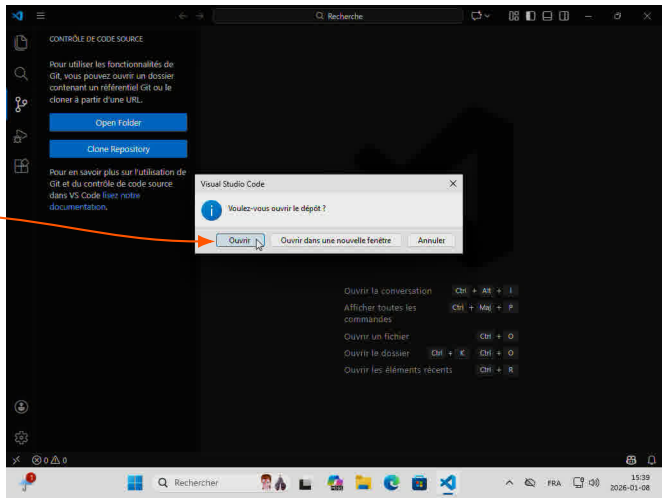
Afin de travailler avec un dépôt contenant déjà des fichiers sur le serveur, par exemple pour avoir accès à votre code sur deux ordinateurs différents, vous devez le cloner. Pour y arriver, exécutez la procédure suivante :

- Ouvrez une nouvelle fenêtre de *Visual Studio Code*, mais n'ouvrez pas de dossier
- Sélectionnez la vue du *Contrôle de code source*
- Cliquez sur le bouton *Clone Repository*
- Dans la zone de texte au haut de l'écran, on vous demande *Indiquez l'URL du dépôt*, alors collez-y l'adresse *HTTPS* du serveur



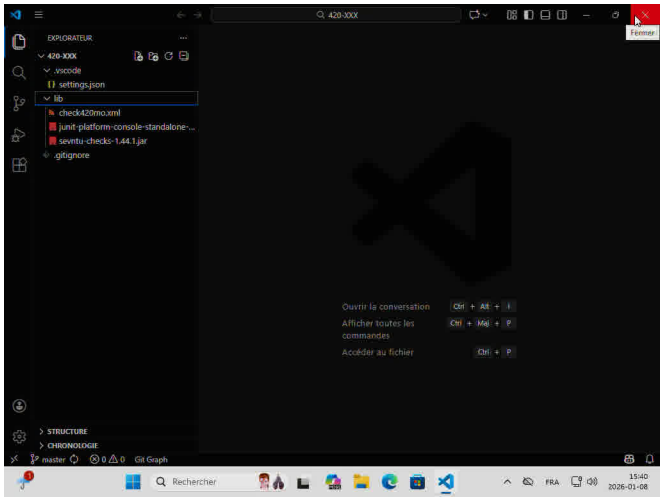


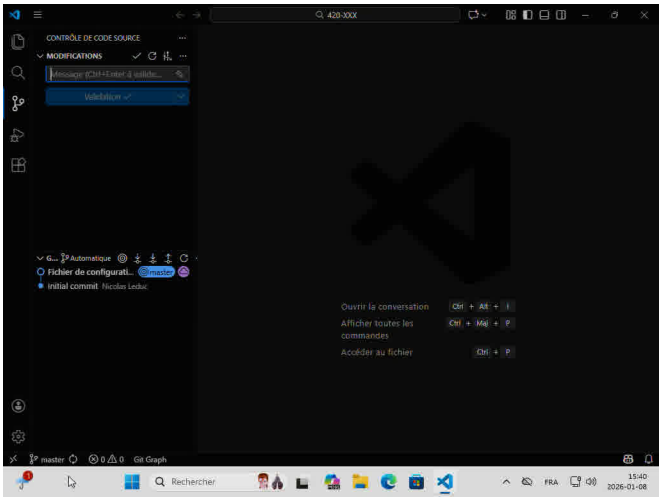
- On vous demande maintenant de sélectionner le dossier dans lequel le dépôt *Git* sera sauvegardé. Vous devez choisir un dossier vide ou en créer un nouveau
- Cliquez sur *Sélectionner comme destination de dépôt*
- La procédure de connexion au serveur s'enclenche ensuite, donc connectez-vous, puis revenez à la fenêtre de *Visual Studio Code*



- La copie de votre dépôt s'effectue et, selon sa taille, l'opération pourrait durer plusieurs secondes
- Une fois le téléchargement terminé, *Visual Studio Code* vous offre la possibilité d'ouvrir le dossier associé à votre nouveau dépôt, cliquez alors sur *Ouvrir*

Vous retrouvez ainsi l'historique de votre dépôt *Git* et les fichiers que vous avez publiés sur le serveur *Git*. Les fichiers qui ne sont pas dans votre dépôt ne sont pas copiés par cette procédure, tel qu'illustré ci-dessous. En effet, on remarque que les dossiers *commun* et *sandbox* sont absents, car ils n'avaient pas été ajoutés au dépôt *Git*.

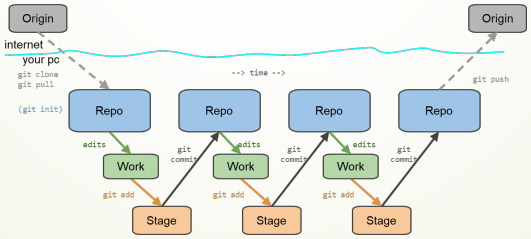


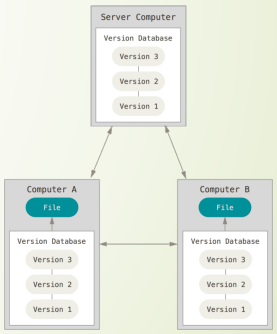


37

SmartGit et *GitLab*
Partager ses modifications

- *Git* n'est pas en constante communication avec le serveur. Il est donc possible de continuer à travailler même si vous perdez votre connexion à *Internet* ou au serveur.
- Toutefois, il est important de synchroniser régulièrement son dépôt local avec celui sur le serveur pour:
  - Télécharger la dernière version du code (*Pull*)
  - Rendre public ses modifications (*Push*)



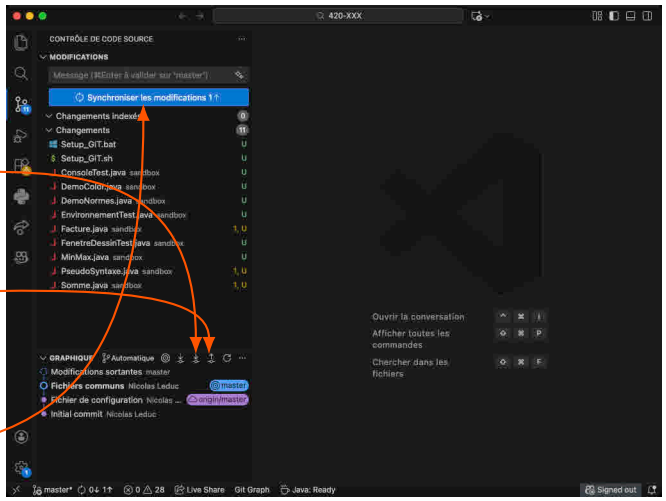


Farida Ait-Ammar
Nicolas Leduc

Une bonne habitude à prendre est de télécharger la dernière version disponible sur le serveur *Git* avant de commencer à modifier le projet afin de travailler sur le code le plus récent et éviter les conflits. Pour ce faire, cliquez sur *Tirer* (*Pull*).

À la fin d'un séance de travail, vous devez publier vos changements sur le serveur *Git*. Il est aussi correct de le faire après chaque *Commit*. Pour ce faire, cliquez sur *Push*. Si l'opération est une réussite, le nuage, représentant le serveur *Git*, devrait être sur la même ligne que le ruban *master*.

Enfin, le bouton *Synchroniser* permet de faire un *Pull* suivi d'un *Push* en une seule commande.

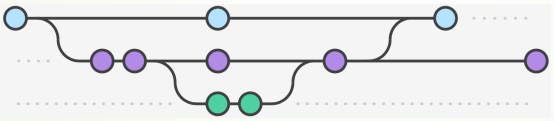


C.9 Gérer une branche

39

Git
Introduction aux branches

- Lors du développement de logiciels, il arrive que l'on ait à travailler sur plusieurs correctifs ou fonctionnalités en parallèle. Pour éviter les conflits, on peut créer des branches, ce qui permet d'isoler les modifications. Lorsque le travail est terminé, il suffit de fusionner nos modifications dans la branche principale, soit la version officielle du logiciel.
- La gestion du développement à l'aide de branches est simple et efficace avec *Git*. Il est donc recommandé de les utiliser.



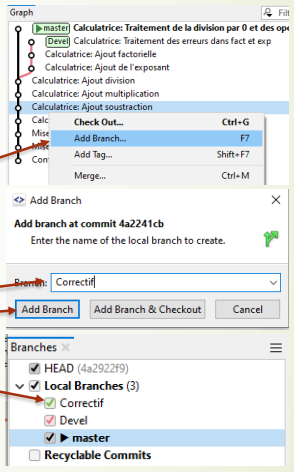
Branche principale (*master*)
Brache de développement
Branche des correctifs

Farida Ait-Ammar
Nicolas Leduc

40

Git
Création d'une branche

- Pour utiliser des branches, il faut premièrement en créer.
- Dans *SmartGit*, pour créer une branche:
  - Cliquez, avec le bouton droit, sur le *Commit*, donc la version du code, à partir duquel vous voulez créer votre branche.
  - Sélectionnez *Add Branch…*
  - Donnez un nom à votre branche (Ex: `Correctif`)
  - Cliquez sur *Add Branch*
- Votre branche apparaît ensuite dans la liste des branches

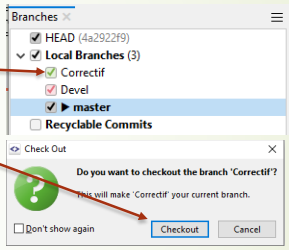


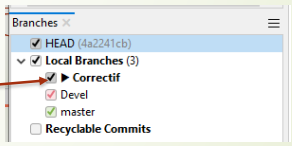
Farida Ait-Ammar
Nicolas Leduc

41

Git
Navigation entre les branches

- Pour accéder à la version du logiciel correspondant à une branche, double-cliquez sur son nom et confirmez (bouton *Checkout*).
  - **Attention**: Assurez-vous que vous n'avez aucune modification dans votre répertoire de travail, car vous risquez de les perdre
- Lors du changement de branche, tous les fichiers de votre répertoire de travail sont remplacés par ceux de la branche.
- Tous les *Commit* se font dans la branche active (en gras dans la liste)
  - *Correctif* dans cet exemple



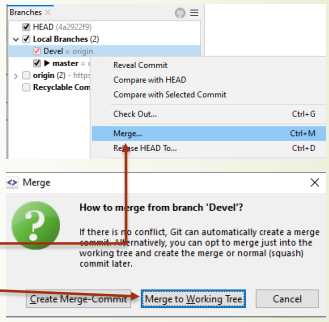


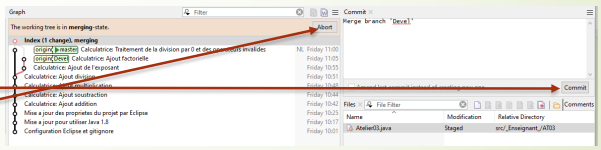
Farida Ait-Ammar
Nicolas Leduc

42

Git
Fusion des branches

- Lorsque vous êtes satisfait de la version du logiciel dans votre branche, vous pouvez l'incorporer dans une autre branche (fusion).
- Pour fusionner deux branches dans *SmartGit*:
  - Assurez-vous que le répertoire de travail ne contient aucune modification
  - Déplacez-vous dans la branche cible, soit celle qui recevra le nouveau code, en double cliquant sur son nom
  - Cliquez, avec le bouton de droite, sur le nom de la branche source
  - Sélectionnez *Merge…*
  - On vous conseille de cliquer ensuite sur *Merge to Working Tree* : De cette façon, vous pourrez inspecter le résultat de la fusion et le corriger au besoin, avant de l'ajouter au dépôt
  - Le résultat de la fusion est maintenant visible.
    Pour l'accepter, cliquez sur *Commit*
    Pour annuler l'opération, cliquez sur *Abort*



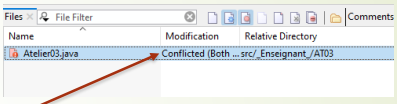


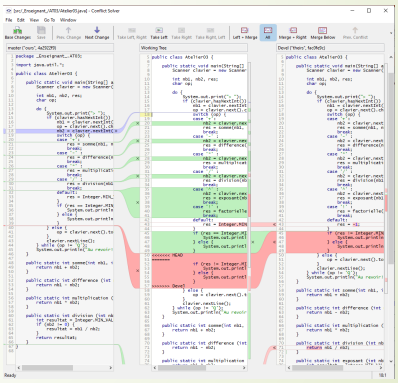
Farida Ait-Ammar
Nicolas Leduc

43

Git
Résolution de conflits

- Habituellement, lors d'une fusion, *Git* réussit à combiner automatiquement les différentes versions de vos fichiers. Toutefois, il arrive que certaines modifications entrent en conflit et que vous deviez les résoudre manuellement.
- Lorsque cela se produit, *SmartGit* indique *Conflicted* dans la colonne *Modification* de l'onglet *Files*
- Si vous double-cliquez sur le nom du fichier, vous obtenez une fenêtre contenant trois colonnes pour vous aider à résoudre le conflit.
- Dans le cadre de ce cours, si la situation se produit, on vous recommande d'annuler la fusion (*Abort*) et de consulter votre enseignant pour vous aider à résoudre le problème.



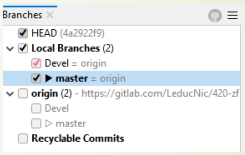


Farida Ait-Ammar
Nicolas Leduc

44

SmartGit et *GitLab*
Partager ses branches

- Lorsque vous établissez une connexion avec un serveur *Git*, par défaut, seule la branche *master* est synchronisée.
- Pour envoyer les autres branches sur le serveur, il faut les sélectionner et faire un *Push* sur chacune individuellement.
- Il faut faire de même pour télécharger les nouvelles versions du serveur à l'aide d'un *Pull*, car seule la branche active est récupérée.
- La liste des branches reliées au serveur est disponible sous *origin* dans l'onglet *Branches*



- Il n'est pas nécessaire de pousser toutes ses branches vers un serveur. Il est acceptable de garder des branches locales personnelles.

Farida Ait-Ammar
Nicolas Leduc

C.10 Ligne de commande

45

Git
Ligne de commande

- Toutes les actions que vous faites dans *SmartGit* peuvent aussi être exécutées manuellement sur la ligne de commande. En fait, *SmartGit* génère des lignes de commande qu'il exécute pour vous. Celles-ci peuvent être consultées dans l'onglet *Output* (*Window -> Output*)
- Exemples de lignes de commande:
  - Version de *Git*: `git --version`
  - Configuration du nom: `git config --global user.name "Mon Nom"`
  - Configuration du courriel: `git config --global user.email "[redacted]"`
  - Création d'un dépôt: `git init`
  - État de vos fichiers: `git status`
  - Ajout au *Staging Area*: `git add <fichier>`
  - *Commit*: `git commit`
  - Voir l'historique: `git log`
  - Téléchargement du serveur: `git pull`
  - Envoi au serveur: `git push`

Farida Ait-Ammar
Nicolas Leduc

46

Git
Ressources

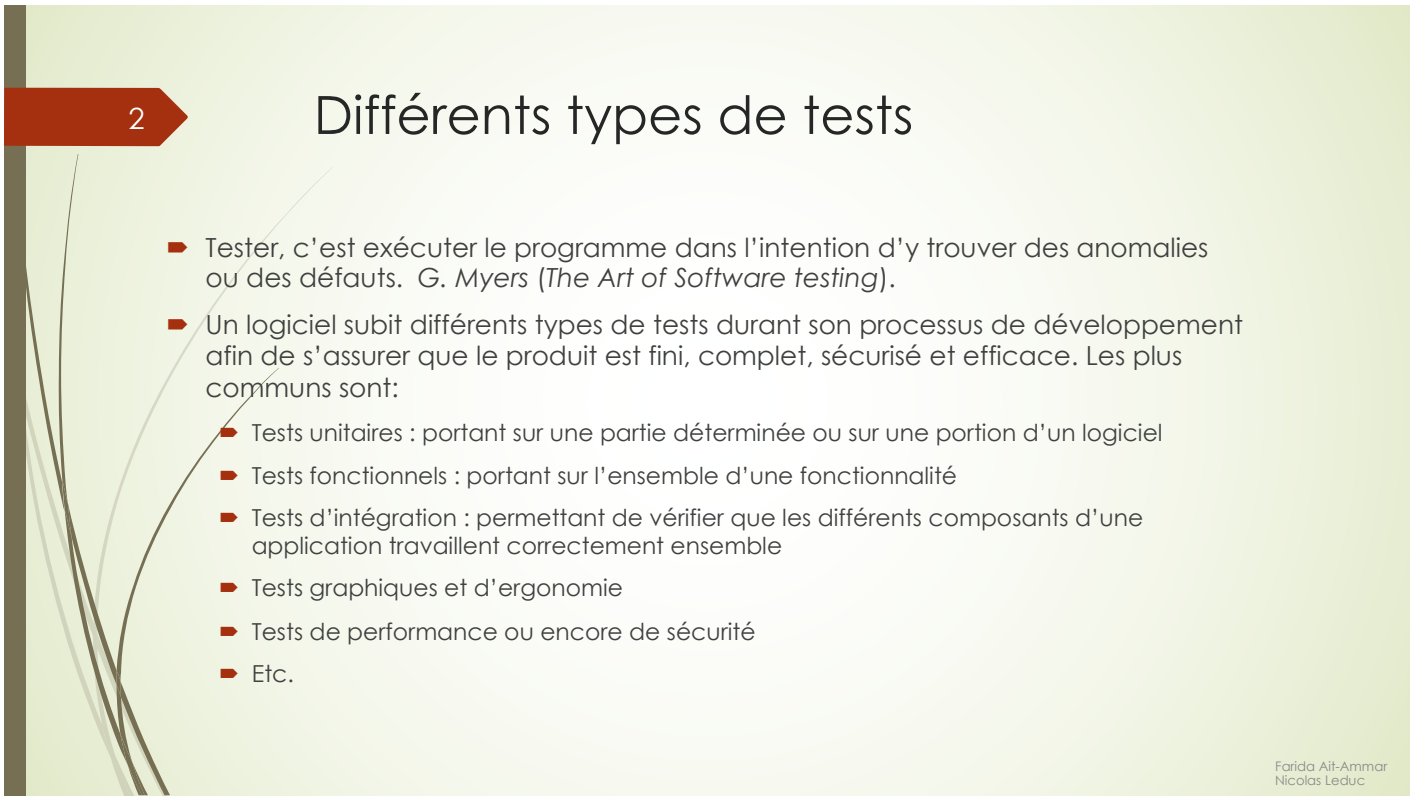
- Site officiel de *Git*: [redacted]
- Livre gratuit sur *Git*: [redacted]
- Bon tutoriel en anglais: [redacted]
- Référence visuelle de *Git*: [redacted]

Farida Ait-Ammar
Nicolas Leduc

D. Test unitaire

« Le test peut être utilisé pour démontrer la présence de problèmes, mais jamais pour prouver leur absence ! »
— Edsger DIJKSTRA

D.1 Stratégie de test



3

Stratégies de tests

- Test en boîte noire (black-box)
  - Le test de type boîte noire est un niveau élevé de test axé sur le comportement du logiciel ou de ses éléments. Le testeur ne s'intéresse pas aux mécanismes internes du logiciel ou de la fonction à tester, mais uniquement à ses entrées/sorties. Il peut être appliqué à tous les niveaux de test logiciel: test unitaire, test d'intégration, etc.
- Test en boîte blanche (white-box)
  - Technique de test qui vérifie le fonctionnement interne du système ou d'une méthode particulière. Dans cette stratégie, les tests sont basés sur la couverture d'instructions de code, de branches, de chemins ou de conditions. Il est considéré comme un test de bas niveau. Exemple: validation d'un programme à l'aide d'une trace d'exécution et du débogueur.

Farida Ait-Ammar
Nicolas Leduc

4

Tests unitaires

- Les tests unitaires sont les premiers tests à être réalisés. Ils visent à vérifier le fonctionnement correct d'une unité du logiciel, comme une classe ou une méthode, indépendamment de l'environnement qui l'entoure.
- Pour chaque classe, on teste toutes ses méthodes.
- Pour chaque méthode, on peut implémenter plusieurs tests appelés **cas de test**. On doit effectuer des tests avec des données valides, mais également avec des données invalides (ou incohérentes), dans le contexte d'utilisation de la méthode. L'ensemble de ces tests est appelé un **jeu d'essai**.
- Un test unitaire sur une méthode consiste simplement à vérifier que son appel donne le résultat attendu, tel que spécifié.
  - Exemple: l'appel de `max3(10, 2, 18)` donne 18

Farida Ait-Ammar
Nicolas Leduc

D.2 Plan de test

5

Programme à tester - `MinMax`

- On veut tester la classe `MinMax` comportant deux méthodes :
  - *`max3()`*, qui retourne le maximum entre trois entiers reçus en paramètres. Elle retourne 0 si le maximum est plus petit que 0 et retourne 100 si le maximum est plus grand que 100.
  - *`min3()`*, qui retourne le minimum entre trois entiers reçus en paramètres. Elle retourne 0 si le minimum est plus petit que 0 et retourne 100 si le minimum est plus grand que 100.
- Comme nous effectuons des tests en boîte noire, il n'est pas utile de connaître le code de ces méthodes pour les tester. En effet, seules leurs signatures et leurs descriptions fonctionnelles sont nécessaires.

```
public class MinMax {
    public static int max3(int val1, int val2, int val3) { … }
    public static int min3(int val1, int val2, int val3) { … }
}
```

- La classe `MinMax`, que nous allons maintenant tester, est disponible dans le package `sandbox` du projet *Java* de base fourni par votre enseignant sur *Moodle*. Notez que le code comporte volontairement des erreurs et que vous ne devez pas les corriger pour l'instant.

Farida Ait-Ammar
Nicolas Leduc

6

Plan de test

- Afin de valider le fonctionnement d'un programme, il faut établir un plan de test permettant de tester **toutes les situations possibles**. Toutefois, il ne s'agit pas de tester toutes les combinaisons d'entrées possibles, mais seulement celles qui sont pertinentes. En général, les tests doivent porter sur les éléments suivants :
  - vérifier que les données valides sont bien traitées et que les résultats fournis sont exacts
  - vérifier que le programme intercepte les données invalides ou incohérentes
  - vérifier les cas limites, par exemple, la dernière donnée valide ou la première donnée invalide
  - vérifier avec un mélange de données valides, invalides et de cas limites
  - vérifier le format de sortie afin de s'assurer que les résultats produits, en plus d'être exacts, apparaissent sous la forme souhaitée
- Un exemple de plan de test est fourni plus loin.

Farida Ait-Ammar
Nicolas Leduc

7

Plan de test – Exemple `MinMax.max3()`

- **Données valides**: Choisir `val1`, `val2` et `val3` dans l'intervalle [0, 100].
  - Varier la position du maximum :
    - À droite : `max3(10, 45, 89)`
    - Au milieu : `max3(20, 45, 30)`
    - À gauche : `max3(78, 13, 45)`
  - Tester les valeurs identiques : `max3(45, 45, 45)`
- **Données invalides**: Choisir `val1`, `val2` et `val3` à l'extérieur de l'intervalle [0, 100].
  - Cas où le maximum donne 100 : `max3(152, -4, 300)`
  - Cas où le maximum donne 0 : `max3(-4, -300, -7)`
- **Données limites**: Choisir des valeurs sur la limite ou à ±1.
  - Valeur 0 : `max3(0, 75 ,4)` et `max3(0, 1, 0)`
  - Valeur 100 : `max3(78, 100, 8)` et `max3(101, 25, 42)`

Farida Ait-Ammar
Nicolas Leduc

8

Jeu d'essai

- Le jeu d'essai consiste à choisir un ensemble de données qui permettent de vérifier le fonctionnement du programme conformément aux conditions définies dans le plan de test.
- Exemple de jeu d'essai pour la fonction `max3(int val1, int val2, int val3)`

	Type de test	val1	val2	val3	Résultat attendu
Données valides	3 données valides: cas où le maximum est à droite	10	45	89	89
	3 données valides: cas où le maximum est au milieu	20	45	30	45
	3 données valides. cas où le maximum est à gauche	78	13	45	78
	3 données valides: données toutes identiques	45	45	45	45
Données invalides	3 données invalides: cas où le maximum donne 100	152	-4	300	100
	3 données invalides: cas où le maximum donne 0	-4	-300	-7	0
Données limites	1 donnée limite égale à 0	0	75	4	75
	1 donnée limite égale à 1	0	1	0	1
	1 donnée limite égale à 100	78	100	8	100
	1 donnée limite égale à 101	101	25	42	100

Farida Ait-Ammar
Nicolas Leduc

D.3 Tester sans outil

9

Tester sans outils (1/2)

- Une approche rudimentaire consiste à vérifier le résultat en écrivant du code qui l'affiche dans le terminal. Le développeur peut alors s'assurer que le résultat affiché est bien celui attendu.
- Exemple
  - Test de la méthode `MinMax.max3()` avec deux cas de tests.

```
public static void main(String[] args) {
    System.out.println("Test de (78, 13, 45) : " + MinMax.max3(78, 13, 45));
    System.out.println("Test de (152, -4, 300) : " + MinMax.max3(152, -4, 300));
}
```

  - Le terminal affiche :

```
Test de (78, 13, 45) : 78
Test de (152, -4, 300) : 0
```

- L'inconvénient majeur est qu'il faut un contrôle humain pour s'assurer que le test passe. Le testeur doit comparer la valeur obtenue dans le premier test (78) avec la valeur attendue (78). Il doit faire le contrôle pour chaque résultat du test, ce qui peut être ardu si l'on a de multiples cas de tests.
- De plus, lorsque l'on modifie notre code, il faut à nouveau refaire les vérifications pour s'assurer que le code modifié est correct, ce qui peut décourager le programmeur.

Farida Ait-Ammar
Nicolas Leduc

10

Tester sans outils (2/2)

- Nous pouvons améliorer les tests précédents en déléguant à l'ordinateur l'analyse des résultats. L'affichage pourrait être `OK` si le test passe et `ERREUR` s'il ne passe pas.

```
public static void main(String[] args) {
    if (MinMax.max3(78, 13, 45) == 78) {
        System.out.println("OK        Test de (78, 13, 45)");
    } else {
        System.out.println("ERREUR    Test de (78, 13, 45)");
    }
    if (MinMax.max3(152, -4, 300) == 100) {
        System.out.println("OK        Test de (152, -4, 300)");
    } else {
        System.out.println("ERREUR    Test de (152, -4, 300)");
    }
}
```

- Le terminal affiche:

```
OK        Test de (78, 13, 45)
ERREUR    Test de (152, -4, 300)
```

- Même si la lecture du résultat du test est plus facile, cette façon de faire n'est pas pratique si l'on doit vérifier des centaines de cas de tests. En effet, si l'on veut connaitre le résultat global, on doit lire chaque ligne du terminal.
- Heureusement, il existe des outils de tests, comme *JUnit*, qui facilitent la lecture des résultats des tests en fournissant une représentation binaire globale des résultats (vert si tous les tests passent et rouge sinon), le nombre de tests exécutés et un rapport des tests en échec.

Farida Ait-Ammar
Nicolas Leduc

D.4 Bibliothèque *JUnit*

12

Bibliothèque *JUnit*

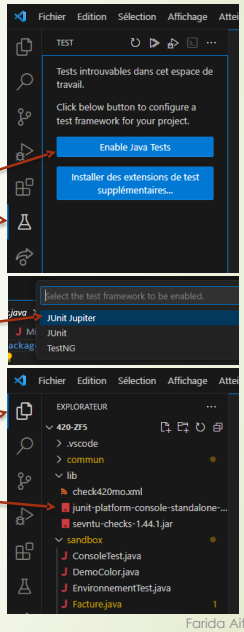
- *JUnit* est une bibliothèque (*framework*) pour le développement et l'exécution de tests unitaires automatisés en *Java*.
- *JUnit* est un logiciel libre hébergé chez *GitHub*.
  - [redacted]
- *JUnit* est intégré par défaut dans les environnements *Java* comme *Eclipse* et *NetBeans*. Pour *Visual Studio Code*, il faut installer une extension afin d'y avoir accès.

Farida Ait-Ammar
Nicolas Leduc

13

Ajout de *JUnit* dans le projet *Java*

- Pour utiliser *JUnit*, il faut que sa bibliothèque soit installée dans le projet *Java*.
  - Sélectionnez la vue *Test*
  - Cliquez sur le bouton *Enable Java Tests*
    S'il est absent, c'est que *JUnit* est déjà installé dans votre projet
  - Sélectionnez *JUnit Jupiter*
  - Sélectionnez la vue *Explorateur* et vous verrez que la librairie *JUnit* a été ajoutée à votre projet dans le dossier *lib*



Farida Ait-Ammar
Nicolas Leduc

D.5 Création d'un test

14

Création d'une classe de test

- Une classe de test est créée de la même manière que n'importe quelle autre classe *Java*. Ce sont les méthodes qu'elle contient qui en feront une classe de test reconnue par *JUnit*.
- Gabarit d'une classe de test :

```
package <nompackage>;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class <NomClasse>Test {
    // Méthodes de test
}
```

- En général, les classes de test sont contenues dans un *package* différent de celui qui contient les fichiers sources, soit un *package* `test`. Cependant, pour faciliter la remise des travaux de ce cours, nous mettrons la classe de test dans le même *package* que la classe à tester.
- Cet `import` permet d'utiliser les méthodes de la classe `Assertions`. Note: Le mot clé `static` permet de faire appel à des méthodes statiques sans préciser le nom de la classe qui définit ces méthodes. Par exemple, on peut faire appel à la méthode `fail()` avec l'instruction `fail("Test non implémenté");` au lieu de `Assertions.fail("Test non implémenté");`
- Cet `import` permet d'utiliser l'annotation `@Test`. Les méthodes de test ainsi annotées seront identifiées par le lanceur de tests, qui sera en charge de les exécuter.
- Les tests portant sur `NomClasse` sont regroupés dans une classe nommée `NomClasse`**`Test`**. Cette convention permet de faciliter l'identification des classes de tests. Par exemple, si la classe à tester est `MinMax`, la classe de test se nommera `MinMax`**`Test`**.

Farida Ait-Ammar
Nicolas Leduc

15

Création d'une méthode de test

- En général, chaque cas de test doit être mis dans une méthode séparée. Voici le gabarit que nous vous proposons :

```
@Test
void max3V01() {
    fail("Test non implémenté");
}
```

- Pour chaque cas de test de la classe `MinMax`, on implémente une méthode dans la classe `MinMaxTest`. Plusieurs conventions sont possibles afin de nommer nos méthodes de test (voir page suivante). Dans le cadre du cours, le nom de la méthode de test sera, en général, `<nomMéthode>`*`D00`*, où :
  - `<nomMéthode>` : nom de la méthode à tester
  - *`D`* : indique le type de données utilisées dans le test, soit valides (`V`), invalides (`I`) ou limites (`L`)
  - *`00`* : numéro du test entre `01` et `99` pour chaque méthode et type de données
  - Par exemple, pour le premier test sur des données valides de la méthode `max3()`, le nom de la méthode de test est `max3V01()`
- Chaque méthode de test doit être annotée avec la balise `@Test`, sinon elle sera ignorée par *JUnit*.
- On vous suggère de mettre l'instruction `fail("Test non implémenté");` au départ dans vos méthodes afin de faire échouer un test non écrit. N'oubliez pas de l'enlever lorsque votre test sera prêt.
- Notez qu'une classe comportant une méthode annotée avec la balise `@Test` devient automatiquement une classe de test. Toutefois, pour que cela fonctionne, il faut que la bibliothèque *JUnit* soit intégrée au projet et que l'`import` pour utiliser les annotations `@Test` soit présente.

Farida Ait-Ammar
Nicolas Leduc

16

Nomenclature des méthodes de test

- Il existe différentes conventions pour nommer les méthodes de test. L'idée générale est d'indiquer clairement le but du test dans le nom de la méthode. Cependant, le plus important est le respect de la convention et non celle choisie. Voici quelques conventions les plus populaires:
  - NomMethode_CasTraite_ResultatAttendu
    - On nomme la méthode testée, on décrit le cas traité et on indique le résultat attendu
    - `max3_MaximumSuperieurA100_Retourne100()`
  - FonctionnaliteTestee
    - Le nom de la méthode est une phrase qui décrit la fonctionnalité testée
    - `leMaximumEst100SiUneValeurEstSuperieureA100()`
  - Doit_ResultatAttendu_Quand_CasTraite
    - On ne nomme pas la méthode testée, mais on indique le cas traité et le résultat attendu
    - `doit_Retourner100_Quand_MaximumSuperieurA100()`
  - Quand_CasTraite_Attendu_Comportement
    - Ressemble à la nomenclature précédente, mais on inverse les informations
    - `quand_MaximumSuperieurA100_Attendu_Retourner100()`
- Dans le cadre du cours, nous utilisons une convention plus simple, car il est souvent difficile pour les étudiants de bien choisir le nom des méthodes.

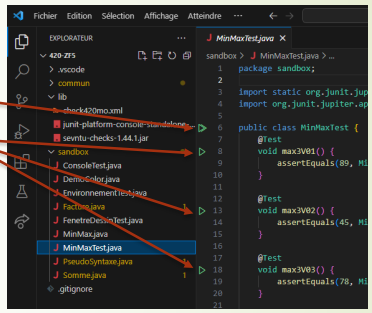
Farida Ait-Ammar
Nicolas Leduc

D.6 Exécution d'un test

17

Exécuter les tests

- Pour exécuter les tests, il suffit de cliquer sur la flèche verte dans la marge de l'éditeur de fichiers :
  - celle à gauche du nom de la classe exécute tous les tests de la classe
  - celle à gauche du nom d'une méthode exécute uniquement le test de la méthode associée
- Si vous avez déjà exécuté vos tests, les flèches seront remplacées par des X rouges ou des crochets verts. Il est possible de cliquer sur ces derniers afin d'exécuter à nouveau le ou les tests.
- Pendant l'exécution des tests, un symbole de chronomètre jaune apparaît. S'il reste affiché plus que quelques secondes, vérifiez s'il n'y a pas un message vous demandant une confirmation. Il pourrait apparaitre, par exemple, si votre code ne compile pas.
- Il se peut que le pare-feu de *Windows* vous demande d'autoriser l'accès à *OpenJDK*. Dans ce cas, acceptez la demande.

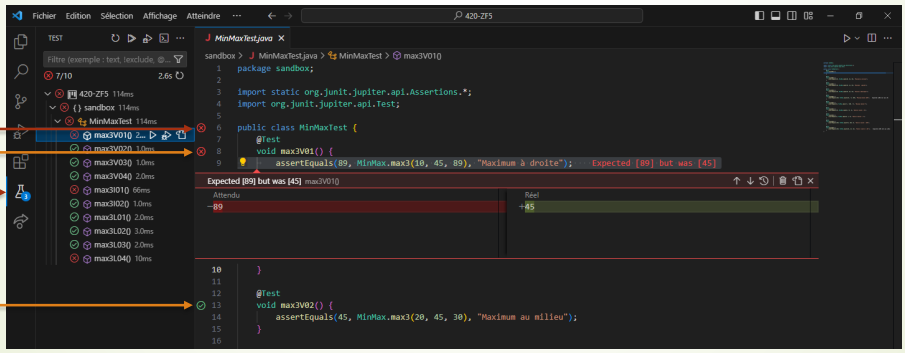


Farida Ait-Ammar
Nicolas Leduc

18

Résultats des tests

- Suite à l'exécution des tests, les résultats apparaissent dans la marge de l'éditeur. Il est aussi possible d'ouvrir la vue *Test* pour avoir un résumé des résultats.
- Les crochets verts indiquent les tests qui ont réussis, alors que les X rouges indiquent ceux qui ont échoués.
- Le symbole apparaissant à côté du nom de la classe donne le résultat global. Un crochet vert indique que tous les tests ont réussis, sinon un X rouge est affiché.

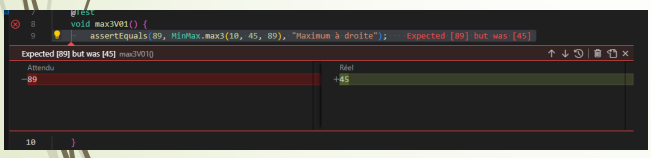


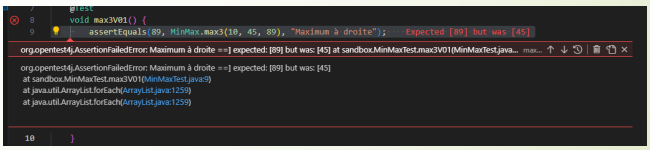
19

Rapport d'erreur

- Pour chaque test qui échoue, il est possible de voir le rapport d'erreur. En cliquant sur le texte en rouge à droite de la ligne de code du test qui a échoué, une fenêtre apparaît. Elle nous indique, à gauche, le résultat attendu et, à droite, la valeur retournée par la méthode (réel).
- Il est possible de voir la trace d'exécution ou de passer au prochain test en échec en cliquant sur les flèches en haut à droite de la fenêtre. Dans cette même section de la fenêtre, la poubelle permet d'effacer les résultats des tests et le X ferme la fenêtre.
- Le message inscrit dans les `assert*()` , ainsi que la valeur réelle et attendue, sont indiqués sur la première ligne de la trace d'exécution du test qui a échoué. Par exemple,

```
org.opentest4j.AssertionFailedError: Maximum à droite ==] expected: [89] but was: [45]
```





D.7 Méthode de test

20

Méthode de test

- Signature d'une méthode de test
  - annotée avec la balise `@Test`
  - il n'est pas nécessaire d'indiquer la visibilité, sinon mettre `public`
  - type de retour `void`
  - nom quelconque, dans le cadre du cours `<nomMéthode>D00` (voir p15)
  - aucun paramètre
- Contenu d'une méthode de test
  - Dans une méthode de test, on utilise des instructions `assert*()`, qui sont les plus petites unités de test. Ces dernières sont des affirmations (*assertion* en anglais) dont le résultat de l'exécution indique un succès ou un échec.
- Exemple :

```
@Test
void max3I01() {
  assertEquals(100, MinMax.max3(152, -4, 300), "Maximum donne 100");
}
```

Farida Ait-Ammar
Nicolas Leduc

21

Méthodes `assertEquals()` (1/2)

- Pour affirmer que la valeur **attendue** (*expected*) et que la valeur **obtenue** (*actual*), par exemple à la suite de l'appel d'une fonction, sont égales, on utilise la méthode `assertEquals()`. Celle-ci a de nombreuses méthodes surchargées pour différents types de données.
- Pour comparer deux valeurs entières (`byte`, `short`, `int` ou `long`) ou deux caractères, on utilise une des deux méthodes suivantes :

```
public static void assertEquals(int expected, int actual)
public static void assertEquals(int expected, int actual, String message)
```

- Dans la deuxième version de la méthode `assertEquals()`, le dernier paramètre contient le message qui sera affiché en cas d'échec du test.
- Exemples :

```
// Comparer 2 entiers
int nb1 = 20, nb2 = 10 + 10;
assertEquals(nb1, nb2);                          // Ce test passe
assertEquals(20, nb2, "Comparer 20 et 10+10");   // Ce test passe
// Comparer 2 caractères
char car1 = 'z', car2 = 122;
assertEquals(car1, car2);                        // Ce test passe
```

Farida Ait-Ammar
Nicolas Leduc

22 Méthodes `assertEquals()` (2/2)

- Pour comparer deux valeurs réelles (`float` ou `double`), on utilise les méthodes suivantes :

```
public static void assertEquals(float expected, float actual, float delta)
public static void assertEquals(float expected, float actual, float delta, String message)
```

- Le paramètre `delta` est la marge d'erreur. Voir la comparaison de deux nombres à virgule flottante en annexe.
- Exemple :

```
final float DELTA = 0.0001f;
assertEquals(Math.PI, 3.1415, DELTA);    // Ce test passe (sans DELTA, le test échoue)
```

- Pour comparer deux objets, comme deux chaînes de caractères, on utilise les méthodes ci-dessous. La comparaison est basée sur la méthode `equals()` définie dans la classe à laquelle appartiennent les deux objets.

```
public static void assertEquals(Object expected, Object actual)
public static void assertEquals(Object expected, Object actual, String message)
```

- Exemple :

```
String motMaj = "Bonjour".toUpperCase();
assertEquals("BONJOUR", motMaj);         // Ce test passe
```

- **Attention**: pour comparer deux **tableaux**, il faut utiliser la méthode **`assertArrayEquals()`**.

Farida Ait-Ammar
Nicolas Leduc

26 Les instructions `assert*()`

Instruction	Description	Exemple
assertEquals(attendu, obtenu)	Vérifie l'égalité de deux valeurs de type primitif (simple) ou d'objets. Dans le cas des objets, cette méthode retourne `true` si `attendu.equals(obtenu) == true`	assertEquals(100, MinMax.max3(40, 70, 100));
assertNotEquals(attendu, obtenu)	Vérifie l'inégalité de deux objets. Cette méthode retourne `true` si `atendu.equals(obtenu) == false`	assertNotEquals(70, MinMax.max3(40, 70, 100));
assertArrayEquals(attendu[], obtenu[])	Teste l'égalité de deux tableaux, qui peuvent être unidimensionnels ou multidimensionnels, en profondeur (données) et non en surface (adresses)	int[] tabTrie = {50,70,80}; int[] tabNotes = {50,80,70}; assertArrayEquals(tabTrie, triCroissant(tabNotes));
assertTrue(condition)	Vérifie qu'une condition ou une variable booléenne est vraie	assertTrue(n > 0);
assertFalse(condition)	Vérifie qu'une condition ou une variable booléenne est fausse	assertFalse(n <= 5);
assertNull(objet)	Vérifie que l'objet fourni en paramètre est `null`	int[] table; assertNull(table);
assertNotNull(objet)	Vérifie que l'objet fourni en paramètre n'est pas `null`	String chaine; assertNotNull(chaine);

assertSame(objet1, objet2)	Vérifie que les deux objets fournis en paramètres font référence à la même entité (adresse mémoire).	assertSame(obj1, obj2);
		est équivalente à
		assertTrue(obj1 == obj2);

assertNotSame(objet1, objet2)	Vérifie que les deux objets fournis en paramètres ne font pas référence à la même entité (adresse mémoire).	assertNotSame(obj1, obj2);
		est équivalente à
		assertFalse(obj1 == obj2);

fail(String)	Fait échouer la méthode de test	fail("Test non implémenté");

- Chaque méthode `assert*()` existe en deux versions : avec ou sans un dernier paramètre contenant le message qui sera affiché en cas d'échec du test.

Farida Ait-Ammar
Nicolas Leduc

D.8 Test de procédure ou de fonction

23 Tester une procédure

- Vous devez tester une procédure qui affiche un rectangle au terminal. Les dimensions doivent être entre 1 et 15 inclusivement, sinon elle affiche un message d'erreur. La signature de la procédure est la suivante :

```
public static void afficheRect(int longueur, int hauteur)
```

  - Il est impossible de valider des affichages au terminal avec *JUnit*. Comme la procédure ne retourne rien et que ses paramètres sont passés par valeur, rien ne pourra être testé avec *JUnit*.
- Vous devez tester une procédure qui permet de trier un tableau d'entiers. Les éléments du tableau doivent être entre 1 et 15 inclusivement, sinon elle n'effectue pas le tri. Le deuxième paramètre de la procédure indique le nombre d'éléments à trier. Sa signature est la suivante :

```
public static void trier(int[] valeurs, int nombre)
```

  - Ici, même si la procédure n'a pas de valeur de retour, on peut tester son paramètre de type tableau, car sa modification à l'intérieur de la méthode est visible dans le programme principal.

```
@Test
void trierV01() {
    int[] vecO = {2, 3, 1, -1, -1}; // Tableau de test pour l'appel (obtenu)
    int[] vecA = {1, 2, 3, -1, -1}; // Tableau attendu après l'appel
    Demo.trier(vecO, 3);     // Appel de la procédure: aucun retour à tester
    assertArrayEquals(vecA, vecO, "Erreur de tri"); // Test du tableau
}
```

Farida Ait-Ammar
Nicolas Leduc

24 Tester une fonction

- Vous devez tester une fonction qui calcule et retourne l'aire d'un rectangle. Les dimensions doivent être entre 1 et 15 inclusivement, sinon elle retourne -1. La signature de la fonction est la suivante :

```
public static float aireRect(float longueur, float hauteur)
```

  - Les paramètres étant passés par valeur, il n'est pas utile de les tester. Par conséquent, on ne doit tester que la valeur de retour.

```
@Test
void aireRectV01() {
    assertEquals(12.5f, Demo.aireRect(10, 1.25f), 0.01f, "Erreur aire"); // Test retour
}
```

- Vous devez tester une fonction qui reçoit une liste de salaires et une liste de pourcentages de bonus associés. Elle ajuste les salaires selon les bonus et retourne le montant total ajouté en bonus. Si une valeur négative est présente, les salaires ne sont pas modifiés et la fonction retourne -1. Sa signature est :

```
public static float ajuste(float[] salaires, float[] bonus)
```

  - Dans ce cas, nous devons tester les paramètres de type tableau ainsi que la valeur de retour. On ne doit pas oublier de valider que le tableau des bonus n'a pas été modifié.

```
@Test
void ajusteV01() {
    float[] salO = {500, 1000, 250};       // Tableau salaires passé à la fonction (obtenu)
    float[] salA = {540, 1020, 262.5f};    // Tableau salaires attendu
    float[] bonO = {0.08f, 0.02f, 0.05f};  // Tableau bonus passé à la fonction (obtenu)
    float[] bonA = bonO.clone();  // Tableau bonus ne devrait pas être modifié (attendu)
    assertEquals(72.5f, Demo.ajuste(salO, bonO), 0.01f, "Total bonus"); // Test retour
    assertArrayEquals(salA, salO, 0.01f, "Erreur salaires");     // Test tableau salaires
    assertArrayEquals(bonA, bonO, 0.01f, "Bonus modifié");       // Test tableau bonus
}
```

Farida Ait-Ammar
Nicolas Leduc

27

Remarques sur les données invalides

- Lorsque l'on parle de données invalides, il s'agit des données passées en paramètres à la fonction à tester, qui sont invalides dans le contexte d'utilisation de la fonction.
- En effet, en *Java*, il n'existe pas de valeur invalide pour les types de données simples. Par exemple, n'importe quel nombre entier qui ne dépasse pas la taille d'un `int` peut être stocké dans un `int`. Ainsi, une fonction qui reçoit un `int` en paramètre doit s'attendre à recevoir n'importe quelle valeur.
- Toutefois, si on écrit une fonction qui traite des notes d'étudiants, on s'attend à recevoir des valeurs entre 0 et 100, même si le type `int` peut stocker des valeurs beaucoup plus grandes ou même négatives. La fonction doit donc être en mesure de traiter ces valeurs inattendues et pourra indiquer les problèmes à l'aide de sa valeur de retour.
- Un test avec des données invalide consiste donc à vérifier que la valeur de retour de la fonction est conforme à ce qui a été spécifié dans le cas où des données invalides, selon les spécifications de la fonction, seraient passées en paramètres.
- Exemple: Pour le calcul de l'aire d'un carré, la fonction retourne -1 si la longueur du côté est négative afin d'indiquer une erreur dans les données

```
assertEquals(-1, aireCarre(-10), "Valeur négative"); // Ce test passe (valide)
assertEquals(25, aireCarre(-10), "Valeur négative"); // Test invalide: retour -1
```

- Le deuxième cas correspond à un test qui est invalide, car la valeur attendue indiquée (25) n'est pas conforme aux spécifications (-1). **Vous ne devez pas écrire des tests invalides!!!**, mais plutôt des tests faisant intervenir des données invalides dans le contexte d'utilisation de la fonction.

Farida Ait-Ammar
Nicolas Leduc

25

Exemple pour la méthode `max3()`

- Voici un exemple de méthodes de test utilisant le jeu d'essai établi précédemment pour tester la méthode `MinMax.max3()`. La première méthode est présentée correctement. Les autres tests ont toutefois été placées sur une seule ligne afin d'économiser de l'espace.

```
@Test
void max3V01() {
    assertEquals(89, MinMax.max3(10, 45, 89), "Maximum à droite");
}
@Test void max3V02() { assertEquals(45, MinMax.max3(20, 45, 30), "Maximum au milieu"); }
@Test void max3V03() { assertEquals(78, MinMax.max3(78, 13, 45), "Maximum à gauche"); }
@Test void max3V04() { assertEquals(45, MinMax.max3(45, 45, 45), "Données identiques"); }

@Test void max3I01() { assertEquals(100, MinMax.max3(152, -4, 300), "Maximum donne 100"); }
@Test void max3I02() { assertEquals(0, MinMax.max3(-4, -300, -7), "Maximum donne 0"); }

@Test void max3L01() { assertEquals(75, MinMax.max3(0, 75, 4), "Donnée limite = 0"); }
@Test void max3L02() { assertEquals(1, MinMax.max3(0, 1, 0), "Donnée limite = 1"); }
@Test void max3L03() { assertEquals(100, MinMax.max3(78, 100, 8), "Donnée limite = 100"); }
@Test void max3L04() { assertEquals(100, MinMax.max3(101, 25, 42), "Donnée limite = 101"); }
```

Farida Ait-Ammar
Nicolas Leduc

D.9 Cycle de vie d'un test

29

Cycle de vie d'un test unitaire

- Le cycle de vie d'un test unitaire est défini à travers les annotations.
- Une annotation, aussi appelée balise, est un marqueur particulier, introduit par le caractère `@`, qui permet d'ajouter une métadonnée à un élément du programme.
- Les annotations sont disponibles à partir de *Java 1.5* et permettent de spécifier une information au compilateur.
- Les tests unitaires, depuis la version 4 de *JUnit*, incluent l'utilisation des annotations pour les tests.
- Les annotations ont été modifiées dans la version 5 de *JUnit*. Dans le cadre du cours, nous utilisons cette dernière version.

Soumaya Medini

30

Cycle de vie d'un test unitaire

- Liste des méthodes que l'on peut définir dans une classe de test *Junit*:
  - Méthode statique annotée avec `@BeforeAll`
  - Méthode statique annotée avec `@AfterAll`
  - Constructeur de la classe de test
  - Méthode annotée avec `@BeforeEach`
  - Méthode annotée avec `@AfterEach`
  - Méthodes de tests annotées avec `@Test`
- Si d'autres méthodes sont incluses dans la classe de test, elles seront ignorées par *JUnit* et n'auront pas d'influence sur le résultat des tests.
  - Exemple : Une méthode sans annotation `@Test` ne sera pas prise en compte.

Soumaya Medini

31

Annotations – `@(Before|After)All`

- `@BeforeAll` (remplace `@BeforeClass` de *JUnit 4*) permet d'indiquer une méthode qui sera exécutée avant l'ensemble du jeu de tests.
- `@AfterAll` (remplace `@AfterClass` de *JUnit 4*) permet d'indiquer une méthode qui sera exécutée après l'ensemble du jeu de tests.
- Les méthodes avec les annotations `@BeforeAll` et `@AfterAll` doivent être des méthodes statiques sinon la machine *Java* lancera une exception `JUnitException` lors de l'exécution des tests unitaires. En effet, elles sont appelées alors que l'instance de la classe de test n'existe pas.
- Ces méthodes peuvent être utiles afin de réaliser des opérations, comme la connexion à une base de données ou l'ouverture d'un fichier qui sera utilisé pour l'ensemble des tests de la classe.

Soumaya Medini

32

Annotations – `@(Before|After)All`

- La signature de la méthode `@BeforeAll` est la suivante :

```
@BeforeAll
static void setUpBeforeClass() {
    // Initialisation
}
```

- La signature de la méthode `@AfterAll` est la suivante :

```
@AfterAll
static void tearDownAfterClass() {
    // Nettoyage
}
```

- Ces deux méthodes doivent obligatoirement être statiques, mais leurs noms peuvent être modifiés.

Soumaya Medini

33

@(Before|After)All - Exemple

```
class DummyTest {
    @BeforeAll
    static void setUpBeforeClass() {
        System.out.println("----- BeforeAll -----");
    }

    @AfterAll
    static void tearDownAfterClass() {
        System.out.println("----- AfterAll -----");
    }

    @Test
    void test1() {
        System.out.println("Test #1");
    }

    @Test
    void test2() {
        System.out.println("Test #2");
    }
}
```

```
----- BeforeAll -----
Test #1
Test #2
----- AfterAll -----
```

Soumaya Medini

34

Annotations – @(Before|After)Each

- `@BeforeEach` permet d'indiquer une méthode qui sera exécutée avant chaque méthode annotée par `@Test`.
- `@AfterEach` permet d'indiquer une méthode qui sera exécutée après chaque méthode annotée par `@Test`.
- Ces méthodes sont exécutées sur une instance de la classe de test, donc elles n'ont pas besoin d'être statiques.
- Ces méthodes peuvent servir à réinitialiser des objets ou des tableaux de données qui sont utilisés dans chacun des tests. De cette façon on s'assure que les données sont uniformes pour tous les tests sans avoir à en faire une copie dans chaque méthode de test.

Soumaya Medini

35

Annotations – `@(Before|After)Each`

- La signature de la méthode `@BeforeEach` est la suivante :

```
@BeforeEach
void setUp() {
    // Initialisation de chaque test
}
```

- La signature de la méthode `@AfterEach` est la suivante :

```
@AfterEach
void tearDown() {
    // Nettoyage après chaque test
}
```

- Les noms de ces deux méthodes peuvent être modifiés.

Soumaya Medini

36

`@(Before|After)Each` - Exemple

```
class DummyTest {
    @BeforeEach
    void setUp() {
        System.out.println("----- BeforeEach -----");
    }

    @AfterEach
    void tearDown() {
        System.out.println("----- AfterEach -----");
    }

    @Test
    void test1() {
        System.out.println("Test #1");
    }

    @Test
    void test2() {
        System.out.println("Test #2");
    }
}
```

```
----- BeforeEach -----
Test #1
----- AfterEach -----
----- BeforeEach -----
Test #2
----- AfterEach -----
```

Soumaya Medini

D.10 Tester un tableau

38

Tester les objets *Java* - Problématique

- Lorsque l'on écrit des tests unitaires, il arrive régulièrement que l'on doive comparer des objets comme des tableaux.
- Considérons les tableaux `tE` et `tA` déclarés comme suit:

```
int[] tE = {11, 14, 15};
int[] tA = {11, 14, 15};
```

- Si nous comparons les deux tableaux en utilisant le test suivant:

```
assertEquals(tE, tA, "tE == tA?"); // Échec
```

- Ce test échoue, car la méthode `assertEquals()` compare les références (adresse mémoire) des tableaux `tE` et `tA` et non leur contenu.

```
static void assertEquals(Object expected, Object actual, String message) {
     if (expected == null && actual == null) {
          return;
     }
     if (expected != null && expected.equals(actual)) { // Compare les adresses
          return;
      }
      failNotEquals(expected, actual, message);
}
```

Soumaya Medini

39

Tester les objets *Java* - Solution

- On doit utiliser la méthode `assertArrayEquals()` pour vérifier l'égalité des tableaux. Il existe une version pour les tableaux unidimensionnels de chacun des types de base *Java* ainsi que pour ceux d'objets. Exemples :

```
static void assertArrayEquals(char[] expected, char[] actual, String message)
static void assertArrayEquals(int[] expected, int[] actual, String message)
static void assertArrayEquals(float[] expected, float[] actual, float delta, String message)
static void assertArrayEquals(Object[] expected, Object[] actual, String message)
```

- Ces méthodes vérifient l'égalité des tableaux en comparant leurs propriétés, comme leurs tailles, ainsi que les valeurs de tous les éléments qu'ils contiennent. Ces méthodes sont récursives, ce qui leur permet de comparer des tableaux à plusieurs dimensions.

```
private static void assertArrayEquals(int[] expected, int[] actual, Deque<Integer> indexes, Object messageOrSupplier) {
     if (expected == actual) {
          return;
     }
     assertArraysNotNull(expected, actual, indexes, messageOrSupplier);
     assertArraysHaveSameLength(expected.length, actual.length, indexes, messageOrSupplier); // Compare taille

     for (int i = 0; i < expected.length; i++) {
          if (expected[i] != actual[i]) {                          // Comparaison de chacun des éléments
               failArraysNotEqual(expected[i], actual[i], nullSafeIndexes(indexes, i), messageOrSupplier);
          }
     }
}
```

Soumaya Medini

40

Tester les objets *Java* - Exemple

- Si nous reprenons nos deux tableaux précédents, on doit vérifier leur égalité à l'aide de la méthode `assertArrayEquals()`. Le code ci-dessous montre, de plus, comment comparer deux tableaux bidimensionnels.

```
@Test
void compareTableaux1D() {
    int[] t1 = { 11, 14, 15 };
    int[] t2 = { 11, 14, 15 };
    assertArrayEquals(t1, t2, "t1 = t2?"); // Le test passe
}

@Test
void compareTableaux2D() {
    int[][] t3 = { {6, 4, 5}, {2, 3, 1} };
    int[][] t4 = { {6, 4, 5}, {2, 3, 1} };
    assertArrayEquals(t3, t4, "t3 = t4?"); // Le test passe
}
```

Soumaya Medini

41

Comparer des listes

- À la place des tableaux, si nous avons des listes qui implémentent l'interface `Iterable<T>`, comme `ArrayList<E>`, et que nous voulons les comparer, nous devons utiliser la méthode `assertIterableEquals()`, qui réalise une comparaison profonde. Profondément égal signifie que le nombre et l'ordre des éléments de la liste doivent être identiques et que les éléments itérés doivent aussi être égaux.
- Exemples :

```
Iterable<Integer> liste1 = new ArrayList<Integer>(Arrays.asList(1,2,3,4));
Iterable<Integer> liste2 = new ArrayList<Integer>(Arrays.asList(1,2,3,4));
Iterable<Integer> liste3 = new ArrayList<Integer>(Arrays.asList(1,2,3));
Iterable<Integer> liste4 = new ArrayList<Integer>(Arrays.asList(1,2,4,3));

assertIterableEquals(liste1, liste2, "Listes égales");      // Le test passe
assertIterableEquals(liste1, liste3, "Longueur différente");// Échec
assertIterableEquals(liste1, liste4, "Ordre différent");    // Échec
```

Soumaya Medini

D.11 Tester une classe

42

Tester nos propres objets

- Jusqu'à maintenant, nous avons fait des tests sur des objets contenus dans les librairies *Java*, comme les tableaux et les listes. Les méthodes de vérification de l'égalité de ces objets sont définies par *Java*.
- Si nous créons nos propres objets, par défaut, seule l'égalité des références est testée, car *Java* n'est pas en mesure de savoir comment vérifier correctement l'égalité de ces nouveaux objets.
- Pour écrire des tests unitaires qui comparent l'égalité de nos propres objets, nous devons alors définir nous-mêmes les méthodes de vérification de l'égalité de nos nouveaux objets créés.

Soumaya Medini

43

Tester nos propres objets - Classe

- Classe `Personne` utilisée dans les exemples suivants

```
public class Personne {
    public Personne(String nom, String prenom, int age) {
        this.nom    = nom;
        this.prenom = prenom;
        this.age    = age;
    }

    public String getNom()                  { return nom; }
    public String getPrenom()               { return prenom; }
    public int    getAge()                  { return age; }

    public void setNom(String nom)          { this.nom    = nom; }
    public void setPrenom(String prenom)    { this.prenom = prenom; }
    public void setAge(int age)             { this.age    = age; }

    private String nom;
    private String prenom;
    private int    age;
}
```

Soumaya Medini

44

Tester nos propres objets - Méthode

- On commence par créer deux objets identiques de type `Personne` afin de vérifier leur égalité:

```
Personne pers1 = new Personne("Soumaya", "Medini", 25);
Personne pers2 = new Personne("Soumaya", "Medini", 25);
```

- Ensuite, on doit comparer l'égalité des deux objets en vérifiant l'égalité de toutes leurs propriétés. Dans ce cas-ci, nous devons vérifier l'égalité des prénoms, des noms et des âges des deux objets de type `Personne`.

```
assertEquals(pers1.getPrenom(), pers2.getPrenom());
assertEquals(pers1.getNom(), pers2.getNom());
assertEquals(pers1.getAge(), pers2.getAge());
```

Soumaya Medini

45

Tester nos propres objets - Problème

- La méthode précédente doit être utilisée pour vérifier l'égalité de nos objets, car si on teste l'égalité de `pers1` et `pers2` directement, le test échouera. En effet, nos deux objets sont à des adresses différentes en mémoire.
- Ainsi, à chaque fois que nous avons besoin de comparer deux personnes, nous devons écrire des assertions pour toutes les propriétés de l'objet `Personne`, soit `prenom`, `nom` et `age`.
- Dans les applications logicielles, on peut avoir des objets avec plusieurs dizaines de propriétés, ce qui implique autant d'assertions à coder.
- Les tests seront alors fastidieux et très longs à écrire. En effet, pour chaque test unitaire d'un nouvel objet, on doit réécrire les assertions pour toutes les propriétés de l'objet.
- Est-il possible de faire mieux ?

Soumaya Medini

46

Surcharge de méthodes

- Une classe enfant peut surcharger (redéfinir) une méthode héritée en faveur de sa propre implémentation.
- La nouvelle méthode doit avoir la même signature que celle du parent, mais elle contient habituellement un code différent.
- Si la méthode à redéfinir de la super classe est `public`, la méthode surchargée dans la sous classe doit être aussi `public`.
- Le type d'objet qui exécute la méthode détermine la version de la méthode qui est invoquée, soit celle du parent ou de l'enfant.

Surcharger = Redéfinir = *Override*

Soumaya Medini

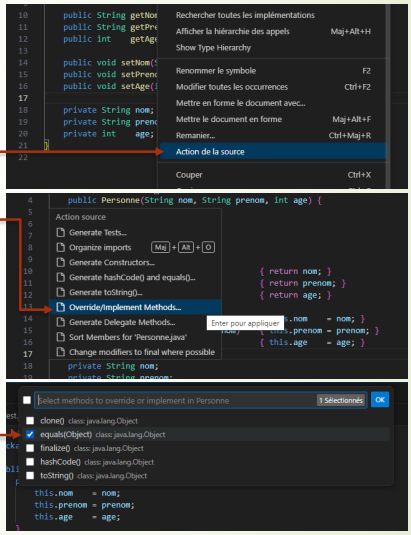
47

Surcharge de méthodes - Procédure

- Si on veut, par exemple, ajouter la surcharge de la méthode `equals()`, on peut le faire manuellement ou avec l'aide de *Visual Studio Code*.
- Pour l'ajouter avec *Visual Studio Code*
  - Cliquez sur le bouton droit à l'endroit où vous désirez ajouter la méthode dans votre classe
  - Dans le menu contextuel, sélectionnez *Action de la source*
  - Ensuite, cliquez sur *Override/Implement Methods…*
  - Choisir la méthode *equals(Object)*
  - Cliquer sur *OK*
- Une méthode `equals()` surchargée vide sera ajoutée à la classe comme suit :

```
@Override
public boolean equals(Object obj) {
    // TODO Auto-generated method stub
    return super.equals(obj);
}
```

- On pourra alors implémenter notre propre méthode d'égalité.



Soumaya Medini

48

Tester nos propres objets - Solution

- La solution consiste à déléguer à l'objet la tâche de vérifier l'égalité en surchargeant la méthode `equals()`. De cette façon, c'est l'auteur de la classe qui est en charge de faire toutes les vérifications nécessaires afin de s'assurer que la méthode `equals()` retourne un résultat correct.
- On doit utiliser la même signature de la méthode `equals()` que celle de l'objet parent. De plus, on doit ajouter l'annotation `@Override` à notre nouvelle méthode `equals()` afin que le compilateur s'assure que l'on surcharge bien une méthode héritée.
- Exemple pour la classe `Personne` :

```
@Override
public boolean equals(Object obj) {
     boolean egal = false;
     if (this == obj) {                                        // Est-ce nous-même?
          egal = true;
     } else if (obj != null &&                                 // L'autre ne doit pas être null
               this.getClass() == obj.getClass()) {            // Même classe?
          Personne pers = (Personne) obj;                      // Conversion explicite
          egal = Objects.equals(nom, pers.nom)                 // Comparaison de chaque champ
                    && Objects.equals(prenom, pers.prenom)
                    && Objects.equals(age, pers.age);
     }
     return egal;
}
```

- Une fois la méthode `equals()` implémentée, il est possible de faire le `assertEquals()` directement sur les objets `pers1` et `pers2` dans notre test unitaire :

```
assertEquals(pers1, pers2); // Le test passe, car il utilise notre equals()
```

Soumaya Medini

D.12 Exercice : Test de classe

49

Exercice

- Considérez la classe suivante qui gère une liste de produits :

```
public class Produits {
     private int nbProduits;
     private int[] tabPrix;
     private int[] tabQte;

     public Produits(int nbProduits, int[] tabPrix, int[] tabQte) {
          this.nbProduits = nbProduits;
          this.tabPrix = new int[5];
          this.tabQte = new int[5];
          for (int i = 0; i < nbProduits; ++i) {
               this.tabPrix[i] = tabPrix[i];
               this.tabQte[i] = tabQte[i];
          }
     }

     public boolean ajouterProduit(int prix, int qte) {
          boolean ajout = true;
          if (nbProduits < tabPrix.length) {
               tabPrix[nbProduits] = prix;
               tabQte[nbProduits] = qte;
               this.nbProduits++;
          } else {
               ajout = false;
          }
          return ajout;
     }
}
```

Soumaya Medini

50

Exercice

- La classe `Produits` gère les produits d'un magasin. Elle contient la méthode `ajouterProduit()`, qui permet d'ajouter la quantité et le prix d'un nouveau produit.
- Commencez par surcharger la méthode `equals()`. Pour comparer des objets, vous devez utiliser `Objects.equals()` alors que pour comparer des tableaux, la méthode `Arrays.equals()` doit être invoquée.
- Créez la classe de tests unitaires et coder la méthode annotée `@BeforeEach` afin d'initialiser deux instances de `Produits` (`p1` et `p2`).
- Ajoutez un jeu de tests comprenant trois tests unitaires:
  1. Un test qui compare les deux instances de `Produits` (`p1` et `p2`). Ce test doit être valide.
  2. Un test qui compare les deux instances de `Produits` après l'ajout d'un nouveau produit au premier objet (`p1`). Ce test doit échouer, car on a changé les valeurs des attributs de `p1`.
  3. Un test qui compare les deux instances de `Produits` après l'ajout du même nouveau produit au deuxième objet (`p2`). Ce test doit échouer, car on a changé les valeurs des attributs de `p2`.

Soumaya Medini

51

Solution – `equals()`

```
@Override
public boolean equals(Object obj) {
    boolean egal = false;
    if (this == obj) {
        egal = true;
    } else if (obj != null &&
            this.getClass() == obj.getClass()) {
        Produits produits = (Produits) obj;
        egal = Objects.equals(nbProduits, produits.nbProduits)
                && Arrays.equals(tabPrix, produits.tabPrix)
                && Arrays.equals(tabQte, produits.tabQte);
    }
    return egal;
}
```

Soumaya Medini

52

Solution – Tests unitaires

```
class ProduitsTest {
     Produits p1;
     Produits p2;

     @BeforeEach
     void setUp() {
          int[] tabPrix1 = { 5, 10, 15, 20, -1 };
          int[] tabQte1 = { 100, 40, 30, 200, -1 };
          p1 = new Produits(4, tabPrix1, tabQte1);
          p2 = new Produits(4, tabPrix1.clone(), tabQte1.clone());
     }
     @Test
     void ajouterProduitV01() {
          assertEquals(p1, p2, "Mêmes listes");          // Valide
     }
     @Test
     void ajouterProduitX01() {
          p1.ajouterProduit(2, 40);
//        assertEquals(p1, p2, "Modification de p1");    // Invalide
     }
     @Test
     void ajouterProduitX02() {
          p2.ajouterProduit(2, 40);
//        assertEquals(p1, p2, "Modification de p2");    // Invalide
          // Le setUp() a remis les valeurs par défaut dans p1
     }
}
```

Soumaya Medini

53

Conclusion

- Comme l'a dit *Edsger Dijkstra* :

  « Le test peut être utilisé pour démontrer la présence de problèmes, mais jamais pour prouver leur absence ! »

- Il est important de toujours garder cela à l'esprit, et de ne surtout pas conclure qu'un programme est correct parce qu'il passe tous les tests que l'on a pensé écrire.

Soumaya Medini

54

Ressources

- [redacted]
- [redacted]

Soumaya Medini

D.13 Complément : Comparaison de nombres en virgule flottante

56

Comparaison de deux nombres à virgule flottante (réels)

- Lors des calculs et des opérations de conversion, des erreurs d'arrondi mineures peuvent être introduites dans des nombres de types `float` ou `double`. C'est pourquoi il n'est pas conseillé de simplement s'appuyer sur l'opérateur d'égalité (==) pour comparer les nombres à virgule flottante. Voici la preuve:

```
public static void main(String[] args) {
      float reel1 = 0;
      for (int i = 1; i <= 11; i++) { reel1 += .1f; }
      float reel2 = 0.1f * 11;
      System.out.println("f1 = " + reel1);
      System.out.println("f2 = " + reel2);
      if (reel1 == reel2) {
           System.out.println("Les deux nombres réels sont égaux");
      } else {
           System.out.println("Les deux nombres réels ne sont pas égaux");
      }
 }
```

- Affichage au terminal:

```
f1 = 1.1000001
f2 = 1.1
Les deux nombres réels ne sont pas égaux
```

Farida Ait-Ammar
Nicolas Leduc

57

Comparaison de deux nombres à virgule flottante (réels)

- Une solution consiste à choisir une différence entre les deux valeurs que nous pouvons tolérer. Cette différence de valeur convenue est appelée **seuil** ou **delta**.
- Par exemple, pour savoir si deux nombres en virgule flottante (`reel1` et `reel2`) sont égaux, nous pouvons utiliser la méthode `Math.abs()` pour calculer la différence en valeur absolue entre les deux nombres et comparer cette dernière à une valeur de seuil, comme `0.001`. Si la différence est plus petite que le seuil, nous pouvons affirmer que `reel1` et `reel2` sont égaux.

```
Math.abs(reel1 - reel2) < 0.001;
```

- On reprend le programme précédent en utilisant la nouvelle formule de comparaison.

```
public static final float DELTA = 0.001f;
public static void main(String[] args) {
      float reel1 = 0;
      for (int i = 1; i <= 11; i++) { reel1 += .1f; }
      float reel2 = 0.1f * 11;
      System.out.println("f1 = " + reel1);
      System.out.println("f2 = " + reel2);
      if (Math.abs(reel1 - reel2) < DELTA) {
            System.out.println("Les deux nombres réels sont égaux");
      } else {
            System.out.println("Les deux nombres réels ne sont pas égaux");
      }
}
```

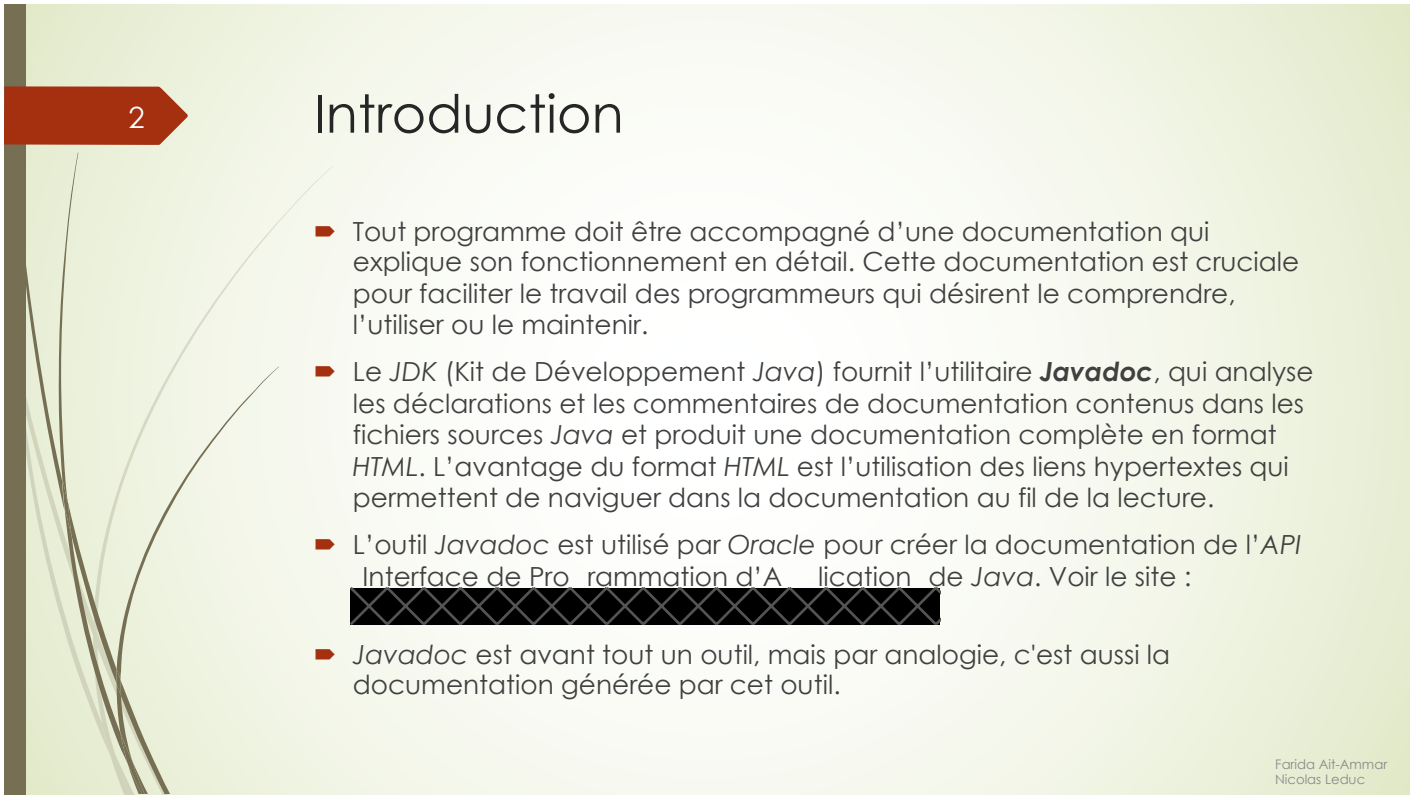
- Affichage au terminal:

```
f1 = 1.1000001
f2 = 1.1
Les deux nombres réels sont égaux
```

Farida Ait-Ammar
Nicolas Leduc

E. Documentation *Javadoc*

E.1 Introduction

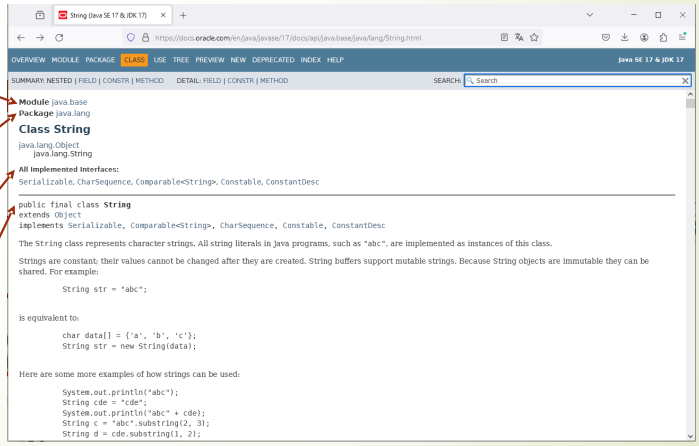


3

Introduction

- Exem le de la documentation de l'*API Java SE* édition 17.

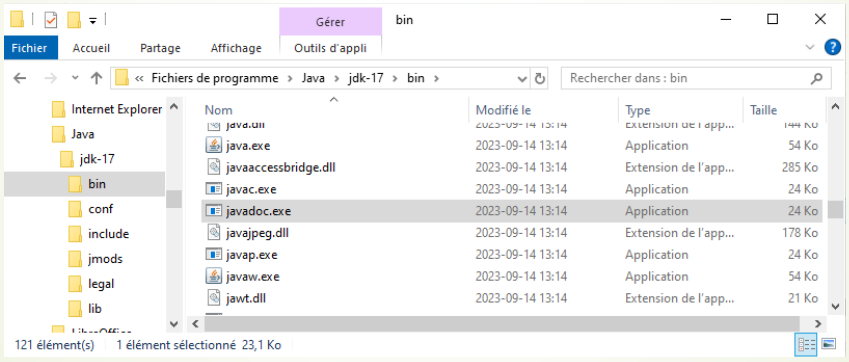
- Module dans lequel la classe actuelle se trouve. Si on clique sur son nom, on voit tous les *packages* associés au module.
- *Package* qui contient la classe actuelle. Si on clique sur son nom, on voit toutes les classes contenues dans le *package*.
- On nous indique les classes parents, les classes dérivées connues et les interfaces implémentées selon le cas.
- Affiche la documentation de la classe ou de l'interface sélectionnée. Par exemple, on voit ici celle de la classe `String`.



4

Où trouver l'outil *Javadoc*?

- L'outil *Javadoc* est inclus dans le *JDK* (Kit de Développement *Java*).
- Il se trouve dans le répertoire `bin` du dossier `jdk`.
- Exemple : `C:\Program Files\Java\jdk-17\bin`



E.2 Commentaire *Javadoc*

5

Types de commentaires en *Java*

- Blocs de commentaires
  - Servent à expliquer l'utilité et les particularités des fichiers, des méthodes, des structures de données et des algorithmes. Ils sont précédés d'une ligne blanche, et sont présentés comme suit :

```
/*
 * Ceci est un bloc de commentaires.
 * Il peut comporter plusieurs lignes, chacune commençant par une
 * étoile entourée de deux espaces pour la lisibilité de l'ensemble.
 */
```

- Commentaires sur une seule ligne
  - Commentaires courts qui peuvent être placés sur une seule ligne indentée au niveau du code. Un commentaire en ligne devrait être précédé d'une ligne blanche.

```
// Traiter la condition.
if (condition) {
    ...
}
```

- Commentaires de fin de ligne
  - Ils peuvent prendre deux formes, soit `/* ... */` ou `// ...`

```
x = 100;   // Expliquer pourquoi ici.  Préférable
x = 300;   /* Expliquer pourquoi ici. */
```

  - La forme `// ...` est aussi utilisée pour masquer une partie du code en cours de développement. Par exemple :

```
// for (int i = 0; i < limite; i++) {
//     s = s * 12; // Commentaire
// }
```

- Commentaires ***Javadoc*** (`/**`) : les seuls utilisés pour générer la documentation

```
/**
 * Commentaire Javadoc
 */
```

Farida Ait-Ammar
Nicolas Leduc

6

Commentaires *Javadoc* (1/3)

- Pour générer la documentation des programmes avec l'outil *Javadoc*, il faut ajouter des commentaires dans les fichiers sources selon un format spécifique. Ces commentaires sont appelés commentaires *Javadoc*.
- Les commentaires *Javadoc* doivent être placés entre `/**` et `*/` . Ces commentaires doivent précéder une déclaration d'interface, de classe, d'attribut, de constructeur ou de méthode.
- Un commentaire *Javadoc* est composé de deux parties : une description de l'élément à documenter suivie d'un bloc d'étiquettes (*tags*) *Javadoc*.

```
/**
 * Description de l'élément sur une ou plusieurs lignes
 *
 * @param nomParametre Explication du paramètre
 * @return Explication de la valeur de retour
 */
```

Farida Ait-Ammar
Nicolas Leduc

7

Commentaires *Javadoc* (2/3)

- La description doit apparaître au début du commentaire. La première phrase de cette description est utilisée par *Javadoc* comme résumé.
- Les étiquettes sont des mots-clés qui commencent par le caractère `@` . Elles doivent être en début de ligne et regroupées ensemble. Un texte qui suit cet ensemble d'étiquettes est ignoré. Ces étiquettes permettent de fournir des informations plus précises sur des composants particuliers de l'élément à documenter, comme l'auteur (`@author`), les paramètres (`@param`), la valeur de retour (`@return`), etc.
- Une ligne de commentaire vide doit être insérée entre la description et la liste des étiquettes.
- Les paragraphes doivent être séparés par la balise *HTML* `<p>` .

Farida Ait-Ammar
Nicolas Leduc

8

Commentaires *Javadoc* (3/3)

- Documentation d'une classe : on doit indiquer au moins une description et son auteur.

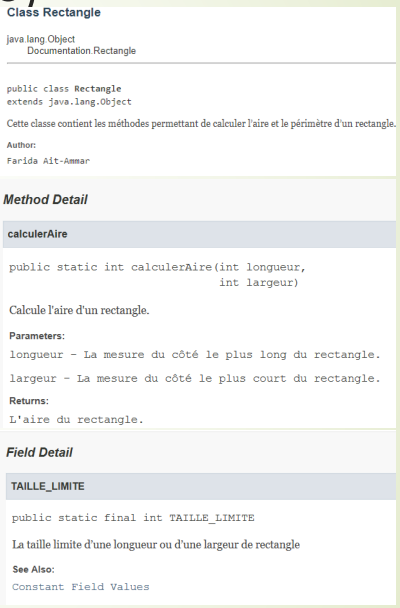
```
/**
 * Cette classe contient les méthodes permettant de calculer l'aire et
 * le périmètre d'un rectangle.
 *
 * @author Farida Ait-Ammar
 */
public class Rectangle { ... }
```

- Documentation d'une méthode : on doit spécifier ce qu'elle réalise, ses paramètres et le résultat retourné, s'il y a lieu.

```
/**
 * Calcule l'aire d'un rectangle.
 *
 * @param longueur La mesure du côté le plus long du rectangle.
 * @param largeur  La mesure du côté le plus court du rectangle.
 * @return L'aire du rectangle.
 */
public static int calculerAire(int longueur, int largeur) { ... }
```

- Documentation d'un attribut : on doit spécifier une description (pas d'étiquettes).

```
/**
 * La taille limite d'une longueur ou d'une largeur de rectangle
 */
public static final int TAILLE_LIMITE = 4000;
```



Class Rectangle

java.lang.Object
  Documentation.Rectangle

public class **Rectangle**
extends java.lang.Object

Cette classe contient les méthodes permettant de calculer l'aire et le périmètre d'un rectangle.

Author:
Farida Ait-Ammar

Method Detail

calculerAire

public static int calculerAire(int longueur,
int largeur)

Calcule l'aire d'un rectangle.

Parameters:
longueur - La mesure du côté le plus long du rectangle.
largeur - La mesure du côté le plus court du rectangle.

Returns:
L'aire du rectangle.

Field Detail

TAILLE_LIMITE

public static final int TAILLE_LIMITE

La taille limite d'une longueur ou d'une largeur de rectangle

See Also:
Constant Field Values

Farida Ait-Ammar
Nicolas Leduc

9

Balises *HTML*

- On peut utiliser les balises *HTML* dans la documentation *Javadoc* pour enrichir le formatage de la documentation. Cette dernière est mise en forme grâce à une feuille de style *CSS* générée par l'outil *Javadoc*.

Exemple

```
/**
 * Cette fonction calcule la somme des entiers positifs
 * inférieurs à nbMax.
 * <p>
 * Elle retourne -1 dans les cas suivants :
 * </p>
 * <ul>
 * <li>Si nbMax est inférieur à 0.</li>
 * <li>Si la somme dépasse Integer.MAX_VALUE.</li>
 * </ul>
 *
 * @param nbMax Le terme le plus grand de la somme.
 * @return La somme des entiers positifs inférieurs à nbMax.
 */
public static int somme(int nbMax) {
      ...
}
```

Documentation produite

Method Detail

somme

```
public static int somme(int nbMax)
```

Cette fonction calcule la somme des entiers positifs inférieurs à nbMax.

Elle retourne -1 dans les cas suivants:

- Si nbMax est inférieur à 0.
- Si la somme dépasse Integer.MAX_VALUE.

Parameters:

`nbMax - le terme le plus grand de la somme.`

Returns:

`la somme des entiers positifs inférieurs à nbMax.`

Farida Ait-Ammar
Nicolas Leduc

E.3 Étiquette *Javadoc*

10

Étiquettes définies dans *Javadoc*

- Il existe deux types d'étiquettes
  - Étiquette de bloc (*block tag*) notée **`@tag`**
  - Étiquette en ligne (*inline tag*) notée **`{@tag}`**
- Une étiquette de bloc doit apparaître en début de ligne, après le caractère `*` et d'éventuels caractères blancs. Le texte qui suit une étiquette de bloc peut s'étendre sur plusieurs lignes. Les étiquettes de bloc doivent toutes être regroupées après la description. Exemples : `@author`, `@param`, `@return`.
- Une étiquette en ligne peut être utilisée n'importe où dans le commentaire. Exemples : `{@value}`, `{@literal}`, `{@code}`.
- Note : les étiquettes sont sensibles à la casse.

Farida Ait-Ammar
Nicolas Leduc

11 Étiquettes prédéfinies dans *Javadoc* les plus utilisées (1/4)

Étiquette *Javadoc*	Signification	Entité	Exemples
`@author` texte-noms	Précise l'auteur (ou les auteurs) d'une entité.	Classe et interface	`@author Farida Ait-Ammar` `@author Nicolas Leduc`
`@version` texte-version	Spécifie le numéro de version d'une entité.	Classe et interface	`@version 1.1`
`@since` texte	Précise la version depuis laquelle cette entité existe.	Toutes	`@since 1.0`
`@param` *nom-parametre* description	Décrit un paramètre d'une méthode. Doit être suivi du nom du paramètre et d'une description. Son type sera inclus automatiquement dans la *Javadoc*.	Méthode et constructeur	`@param nbMax Le terme le plus grand de la somme.`
`@return` description	Fournit une description de la valeur de retour d'une méthode. Aucun nom de variable locale ne doit apparaître à la suite de cette étiquette.	Méthode avec retour	`@return La somme des entiers positifs inférieurs à nbMax`
`@deprecated` texte	Spécifie qu'une entité est obsolète : elle ne devrait plus être utilisée même si elle fonctionne toujours. Il est recommandé de préciser depuis quelle version l'élément est obsolète et de fournir la solution de remplacement, si elle existe, ainsi qu'un lien vers une entité de substitution. Note : *Visual Studio Code* barre l'appel d'une méthode obsolète dans le fichier source	Toutes	`@deprecated Depuis la version 1.3, remplacée par {@link #toString()}` ~~`afficher`~~`()`

Farida Ait-Ammar
Nicolas Leduc

12 Étiquettes prédéfinies dans *Javadoc* les plus utilisées (2/4)

Étiquette *Javadoc*	Signification	Entité	Exemples
`@throws` *nom-exception* description **`@exception`** *nom-exception* description Les étiquettes `@exception` et `@throws` sont équivalentes. Cependant, il est recommandé d'utiliser `@throws`.	Spécifie quelle exception peut être levée par une méthode et fournit une courte description des raisons de la levée de cette dernière. Il faut utiliser autant d'étiquettes `@throws` qu'il y a d'exceptions.	Méthode	`@throws java.io.FileNotFoundException Le fichier n'existe pas`

`@see` package
`@see` package.Class
`@see` Class
`@see` #attribut
`@see` Class#attribut
`@see` #method(Type,Type,...)
`@see` Class#method(Type,Type,...)
`@see` package.Class#method(Type,Type,...)
`@see` ...
`@see` "texte"
Note : le nom d'un attribut ou d'une méthode doit être précédé du caractère `#`.
	Fait une référence (lien hypertexte) à une autre entité incluse dans une documentation de type *Javadoc* ou vers une *URL* (*Uniform Resource Locator*).
	
	Cette étiquette crée une section « *See Also* » dans la documentation.
		Toutes	`@see #somme(int)`
			Se traduit, dans la page *HTML*, par un lien vers la méthode `somme(int)` de la classe en cours.
			
			`@see at02.Atelier02#expMax(int, int)`
			Se traduit, dans la page *HTML*, par un lien vers la méthode `expMax(int, int)` de la classe `Atelier02` du package `at02`.

Farida Ait-Ammar
Nicolas Leduc

13 Étiquettes prédéfinies dans *Javadoc* les plus utilisées (3/4)

Étiquette *Javadoc*	Signification	Entité	Exemples

{**@value**}
{**@value** package.classe#attribut_static}
	Affiche la valeur d'un attribut `static`. Lorsque l'étiquette `{@value}` est utilisée sans arguments avec un attribut `static`, l'étiquette est remplacée par la valeur de cet attribut.	Attribut `static`	`/**`
			` * Longueur ou largeur maximale ({@value}).`
			` */`
			`public static final int TAILLE_LIMITE = 4000;`
			
			La documentation sera comme suit :
			Longueur ou largeur maximale (4000).

{**@literal** texte}	Affiche un texte intégralement sans qu'il ne soit interprété comme du *HTML*. Pour afficher du code, utilisez la balise `{@code}`	Toutes	`{@literal <li> contenu </li>}` La documentation sera comme suit : * contenu *

{**@code** texte}	Utilisé pour afficher du code dans la documentation. Affiche un texte dans des balises `<code> ... </code>`, qui ne sera pas interprété comme du *HTML*. On peut, par exemple, utiliser les caractères < et > au lieu des entités *HTML* `<` et `>`
	
	L'étiquette `{@code` texte`}` est équivalente à `<code>{@literal` texte`}</code>`
		Toutes	`{@code <li> contenu </li> }`
			L'affichage sera selon le style défini pour la balise `code`.
			`<li> contenu </li>`

Farida Ait-Ammar
Nicolas Leduc

14 Étiquettes prédéfinies dans *Javadoc* les plus utilisées (4/4)

Étiquette *Javadoc*	Signification	Entité	Exemples
{**@link** package.class#member texte-de-lien} Note : cette étiquette accepte la même syntaxe que `@see`.	Insère un lien en ligne avec un texte de lien de notre choix, qui pointe vers un élément de la documentation. Le mode de fonctionnement de cette étiquette est similaire à `@see`. La différence est que l'étiquette `@see` crée un lien dans la section « *See also* », alors que `{@link}` crée un lien à n'importe quel endroit dans la documentation.	Toutes	`Utilisez la méthode {@link #somme(int) somme} pour calculer la somme des premiers entiers positifs.` Produit la documentation suivante : *Utilisez la méthode **somme** pour calculer la somme des premiers entiers positifs.* Note : La **somme** est un lien vers la méthode `somme(int)`

Farida Ait-Ammar
Nicolas Leduc

E.4 Exemple de documentation

15

Gabarit de commentaires *Javadoc* dans *Visual Studio Code*

- *Visual Studio Code* contient un gabarit de commentaires *Javadoc*. Pour l'activer, on doit positionner le curseur avant la définition d'un élément puis taper la suite de caractères **/** suivi de la touche *Entrée***. Le gabarit se développe afin d'inclure les étiquettes *Javadoc* minimales appropriées à l'élément à documenter.
- Si l'on inscrit `/**` suivi de *Entrée* avant la déclaration d'une classe, le bloc de commentaires suivant sera généré :

```
/**
 *
 */
public class Rectangle {
```

  - On pourra alors inscrire la description de la classe et ajouter les étiquettes utiles, par exemple, `@author` ou `@version` pour spécifier respectivement le ou les auteurs ainsi que le numéro de version.
- Si l'on inscrit `/**` suivi de *Entrée* avant la déclaration de la méthode `calculerAire()`, le bloc de commentaires suivant sera généré :

```
/**
 *
 * @param longueur
 * @param largeur
 * @return
 */
public static int calculerAire(int longueur, int largeur) {
```

Farida Ait-Ammar
Nicolas Leduc

16

Exemple de documentation - Classe

- Documentation d'une classe

```
/**
 * Cette classe contient les méthodes permettant de calculer
 * {@link #aire(int, int) l'aire} et {@link #perimetre(int, int) le périmètre}
 * d'un rectangle.
 *
 * @author Farida Ait-Ammar
 * @author Nicolas Leduc
 * @version 1.1
 * @see Carre
 * @see Cercle
 */
public class Rectangle { … }
```

public class **Rectangle**
extends java.lang.Object

Cette classe contient les méthodes permettant de calculer `l'aire` et `le périmètre` d'un rectangle.

Version:

`1.1`

Author:

`Farida Ait-Ammar, Nicolas Leduc`

See Also:

`Carre, Cercle`

Farida Ait-Ammar
Nicolas Leduc

17

Exemple de documentation - Attribut

- Documentation d'un attribut

```
/**
 * La taille limite d'une longueur ou d'une largeur de rectangle ({@value})
 */
public static final int TAILLE_LIMITE = 4000;
```

TAILLE_LIMITE

```
public static final int TAILLE_LIMITE
```

La taille limite d'une longueur ou d'une largeur de rectangle (4000)

See Also:

Constant Field Values

Farida Ait-Ammar
Nicolas Leduc

18

Exemple de documentation - Méthode

- Documentation d'une méthode

```
/**
 * Calcule l'aire d'un rectangle.
 *
 * @param longueur La mesure du côté le plus long du rectangle.
 * @param largeur  La mesure du côté le plus court du rectangle.
 * @return L'aire du rectangle.
 * @since 1.1
 * @see Carre#calculerAire(int, int)
 * @see Cercle#calculerAire(int, int)
 */
public static int calculerAire(int longueur, int largeur) {
```

calculerAire

```
public static int calculerAire(int longueur,
                               int largeur)
```

Calcule l'aire d'un rectangle.

Parameters:

longueur - La mesure du côté le plus long du rectangle.

largeur - La mesure du côté le plus court du rectangle.

Returns:

L'aire du rectangle.

Since:

1.1

See Also:

Carre.calculerAire(int, int), Cercle.calculerAire(int, int)

Farida Ait-Ammar
Nicolas Leduc

19

Exemple de documentation - Obsolescence

- Documentation d'une méthode obsolète

```
/**
 * Calcule l'aire d'un rectangle.
 *
 * @param longueur La mesure du côté le plus long du rectangle.
 * @param largeur  La mesure du côté le plus court du rectangle.
 * @return L'aire du rectangle.
 * @deprecated Depuis la version 1.1, elle est remplacée par
 *             {@link #calculerAire(int, int)}
 */
@Deprecated
public static int aire(int longueur, int largeur) {
```

- Note : l'annotation `@Deprecated`, qui précède la déclaration de la méthode, garantit que tous les compilateurs émettront des avertissements lorsque le code utilise cet élément du programme.

aire

```
@Deprecated
public static int aire(int longueur,
                       int largeur)
```

Deprecated. *Depuis la version 1.1, elle est remplacée par* `calculerAire(int, int)`

Calcule l'aire d'un rectangle.

Parameters:

`longueur` - La mesure du côté le plus long du rectangle.

`largeur` - La mesure du côté le plus court du rectangle.

Returns:

L'aire du rectangle.

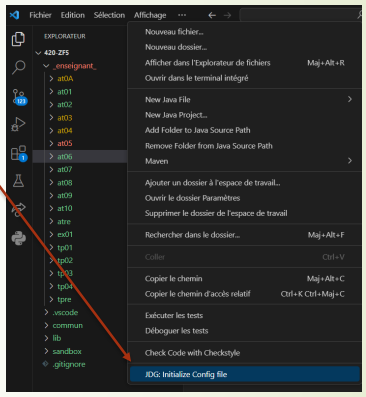
Farida Ait-Ammar
Nicolas Leduc

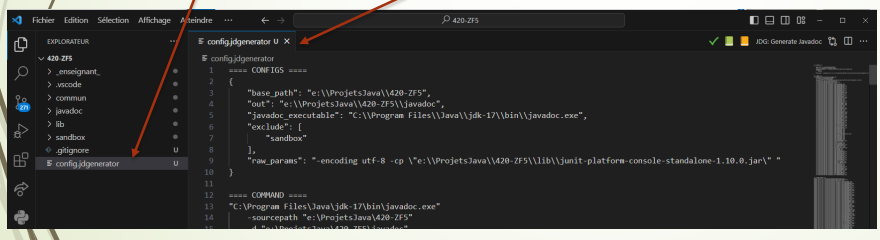
E.5 Génération de la documentation

20

Générer la documentation *Javadoc* avec *Visual Studio Code + Javadoc-Generator*

- Pour générer la documentation *Javadoc* avec *Visual Studio Code*, il faut d'abord installer l'extension *Javadoc-Generator*.
- Ensuite, dans l'*EXPLORATEUR*, cliquez avec le bouton droit en dessous du dernier fichier, puis sélectionnez *JDG: Initialize Config file* dans le menu contextuel.
- Le fichier `config.jdgenerator` devrait maintenant apparaître dans la racine de votre projet et s'ouvrir dans un éditeur.





Farida Ait-Ammar
Nicolas Leduc

21

Générer la documentation *Javadoc* avec *Visual Studio Code + Javadoc-Generator*

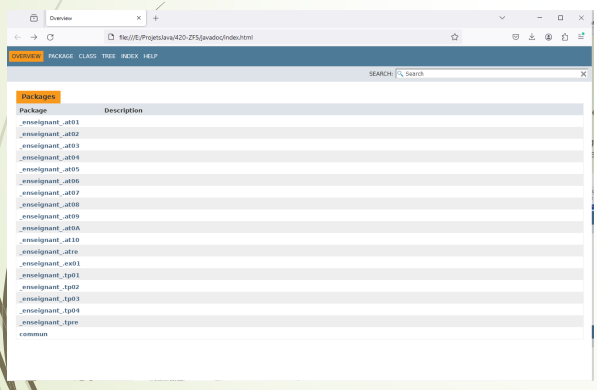
- Le fichier `config.jdgenerator` contient trois sections :
  - `==== CONFIGS ====` : Contient la configuration du générateur de la *Javadoc*, sous forme de structure *JSON*. Par défaut, plusieurs champs sont déjà remplis.
  - `==== COMMAND ====` : Indique la commande qui sera exécutée pour produire la Javadoc. Cette section est générée automatiquement à chaque sauvegarde du fichier de configuration. Toute modification manuelle est alors écrasée.
  - `==== JAVADOC OUTPUT ====` : Contient le résultat de l'exécution de la commande inscrite dans la section précédente. Cette section est écrasée à chaque exécution de la génération de la *Javadoc*. Il est important de consulter cette section pour voir les erreurs et avertissements concernant la *Javadoc*.
- Dans la section `==== CONFIGS ====`, les champs disponibles sont :
  - `"base_path"` : Répertoire de base de votre projet *Java*, par exemple : `"e:\\ProjetsJava\\420-ZF5"`
  - `"out"` : Répertoire dans lequel sera enregistré la *Javadoc*, soit tous les fichiers *HTML* et *CSS* de la documentation, par exemple : `"e:\\ProjetsJava\\420-ZF5\\javadoc"`
  - `"javadoc_executable"` : Emplacement du fichier `javadoc.exe`, par exemple : `"C:\\Program Files\\Java\\jdk-17\\bin\\javadoc.exe"`
  - `"exclude"` : Liste de noms de répertoires ou de fichiers pour lesquels on ne désire pas produire la *Javadoc*. La liste est délimitée par des crochets et les différentes chaînes sont séparées par des virgules, par exemple : `["sandbox"]`
  - `"raw_params"` : Permet d'ajouter des paramètres à la ligne de commande pour, entre autres, spécifier l'encodage des fichiers sources, inclure certaines librairies, etc. Ils seront ajoutés, sans modifications, à la ligne de commande. Dans notre cas, il faut indiquer l'encodage et inclure la librairie *Junit*, par exemple : `"-encoding utf-8 -cp \"e:\\ProjetsJava\\420-ZF5\\lib\\junit-platform-console-standalone-1.10.0.jar\" "`

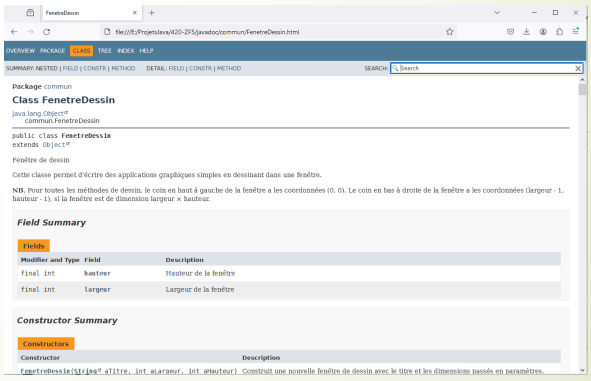
Farida Ait-Ammar
Nicolas Leduc

22

Générer la documentation *Javadoc* avec *Visual Studio Code + Javadoc-Generator*

- Après avoir sauvegardé le fichier de configuration, il suffit d'appuyer sur le bouton *JDG: Generate Javadoc* en haut à droite dans *Visual Studio Code*.
- Dans l'explorateur de fichiers de *Windows*, on peut alors ouvrir le fichier `index.html` qui se trouve dans le répertoire de sortie spécifié dans la configuration. On peut ainsi consulter la *Javadoc* générée dans un navigateur web.





Farida Ait-Ammar
Nicolas Leduc

F. Normes de codage

Normes de programmation pour *Java*

Table des matières

1 Nomenclature ---- 2

1.1 Nom de constante ---- 2
1.2 Nom de variable ---- 2
1.3 Nom de méthode ---- 2
1.4 Nom de classe ---- 3
1.5 Nom de package ---- 3

2 Commentaires ---- 3

2.1 Commentaires de programmation ---- 3
2.2 Commentaires de documentation ---- 4

3 Espaces ---- 4

3.1 Espaces blancs ---- 4
3.2 Lignes blanches ---- 5

4 Indentation ---- 5

4.1 Longueur des lignes de code ---- 6
4.2 Découpage des lignes ---- 6

5 Déclarations ---- 6

6 Utilisation des variables booléennes dans une condition ---- 7

7 Valeurs de retour ---- 7

8 Instructions ---- 7

8.1 Instructions simples ---- 7
8.2 Instructions composées ---- 7
8.3 Instructions `if`, `if-else`, `if else-if else` ---- 7
8.4 Instruction `switch` ---- 8
8.5 Instruction `while` ---- 8
8.6 Instruction `do…while` ---- 8
8.7 Instruction `for` ---- 8

9 Références ---- 9

10 Annexe : Mise en page du code source avec *Visual Studio Code* ---- 9

11 Annexe : Structures interdites ---- 10

Les règles suivantes constituent les normes de programmation qui devront être respectées dans le cadre du cours. Ces règles visent à améliorer la lisibilité et la clarté du code en utilisant un style uniforme. Elles permettent également de faciliter la modification du code et d'éviter les erreurs en lien avec la syntaxe du langage de programmation.

Tous les travaux remis par les étudiants doivent respecter cette norme.

Ces règles reprennent le standard de code *Java* présenté dans le document « Code Conventions for the Java Programming Language »

1 Nomenclature

Certaines règles de nommage ne sont pas imposées par le langage, mais sont traditionnellement respectées. Ce sont des conventions qui ont pour objectif de rendre les programmes plus aisément compréhensibles, car plus lisibles. Elles donnent des informations sur la fonction de l'identificateur. Ainsi, selon la nomenclature utilisée, on peut facilement reconnaître s'il s'agit d'une variable, d'une constante, d'une méthode ou d'une classe.

1.1 Nom de constante

Le nom d'une constante doit être entièrement en majuscules. Les mots composés doivent être séparés par un caractère de soulignement «_».

Exemples de déclaration de constantes :

```
public static final float LARGEUR_MIN = 4.5f;
public static final int MAXIMUM = 999;
```

1.2 Nom de variable

Le nom d'une variable, incluant les paramètres de fonctions, doit être en lettres minuscules, sauf s'il est composé de plusieurs mots. Dans ce cas, le premier mot commence par une lettre minuscule et tous les mots suivants débutent par une majuscule.

Exemples de déclarations de variables:

```
String nomEtudiant;
float salaire;
float prixVenteMinimum;
```

Le choix du nom doit être un court mnémonique et indiquer au lecteur l'utilité de la variable.

Le nom d'une variable avec un caractère unique comme `i`, `j`, `k` doit être évité; sauf pour une variable temporaire à usage local. Les noms communs pour de telles variables sont

`i`, `j`, `k`, `m` et `n` pour les entiers
`c`, `d` et `e` pour les caractères

Le nom d'une variable ne doit pas commencer par un blanc souligné «_», un symbole dollar «$» ou un nombre. N'utilisez que les caractères `[a-z]`, `[A-Z]` et `[0-9]`. **Vous ne devez pas utiliser les lettres accentuées.**

1.3 Nom de méthode

Le nom d'une méthode devrait commencer par un verbe qui décrit son rôle. Il doit commencer par une lettre minuscule. S'il est composé de plusieurs mots, tous les mots suivants doivent commencer par une majuscule.

Exemples de noms à utiliser :

```
afficher()
calculerMoyenne()
modifierNoteFinale()
```

1.4 Nom de classe

Le nom d'une classe doit être un nom dont la première lettre est en majuscule. S'il est composé de plusieurs mots, tous les mots suivants débutent par une majuscule.

Essayez de choisir des noms de classes simples et descriptifs. Utilisez des mots entiers plutôt que des acronymes ou des abréviations, à moins que celles-ci ne soient habituellement utilisées comme *URL*, *HTML*, etc.

Exemples de noms de classes: `Facture`, `PremierProgramme`

1.5 Nom de package

Le nom d'un package (sous-répertoire) doit être entièrement en minuscules.

2 Commentaires

2.1 Commentaires de programmation

Les commentaires servent à donner des explications sur le code ou sur une implémentation particulière. Ils sont également utilisés pour masquer une partie du code en cours de développement.

Ils peuvent être mis sous la forme de l'un des styles suivants :

- **Blocs de commentaires** : Ils servent à expliquer l'utilité et les particularités des fichiers, des méthodes, des structures de données et des algorithmes. Ils sont précédés d'une ligne blanche, et sont présentés comme suit :

  ```
  /*
   * Ceci est un bloc de commentaires.
   * Il peut comporter plusieurs lignes, chacune commençant par une
   * étoile entourée de deux espaces pour la lisibilité de l'ensemble.
   */
  ```

  Au début de chaque fichier, on insère habituellement un bloc de commentaires indiquant des renseignements concernant le fichier dont le nom du fichier, une description de son contenu, le nom du ou des auteurs, la date de création, les dates de mises à jour avec une courte description des changements apportés, etc.

- **Commentaire sur une seule ligne :** Des commentaires courts peuvent être placés sur une seule ligne indentée au niveau du code. Un commentaire en ligne devrait être précédé d'une ligne blanche.

  ```
  if (condition) {

      /* Traiter la condition. */
      ...

      // Autre exemple de commentaire
      ...
  }
  ```

- **Commentaires de fin de ligne :** Ils peuvent prendre deux formes, soit `/* … */` ou `// …` La forme `// …` est aussi utilisée pour masquer une partie du code en cours de développement. Par exemple :

```
if (nombre == 0) {      // Traiter le cas zéro.
    x = 200;
} else if (nombre < 0) {
    x = 100;            // Expliquer pourquoi ici.
} else {
    x = 300;            /* Expliquer pourquoi ici. */
}

//for (int i = 0; i < limite; i++) {
//    s = s * 12;       // Commentaire
//    System.out.println(s);
//}
```

2.2 Commentaires de documentation

Les commentaires de documentation, aussi appelés les commentaires *Javadoc*, sont spécifiques à *Java* et sont délimités par `/** … */`. Les commentaires *Javadoc* peuvent être extraits vers des fichiers au format *HTML* grâce à l'utilitaire `javadoc`. Ils peuvent aussi être lus par l'environnement de développement afin de fournir de l'aide au programmeur. Ces commentaires devraient apparaitre avant chaque déclaration.

- **Pour une classe** : Indiquez au moins une description et son auteur.

```
/**
 * Cette classe contient les méthodes permettant de calculer l'aire et le
 * périmètre d'un rectangle.
 *
 * @author Boulanger Claude
 */
public class Rectangle {
    ...
}
```

- **Pour une méthode :** Spécifiez ce qu'elle réalise, ses paramètres et le résultat retourné, s'il y a lieu.

```
/**
 * Calcule l'aire d'un rectangle.
 *
 * @param longueur La mesure du côté le plus long du rectangle.
 * @param largeur La mesure du côté le plus court du rectangle.
 *
 * @return L'aire du rectangle.
 */
public static int calculerAire(int longueur, int largeur) {
    ...
}
```

3 Espaces

3.1 Espaces blancs

- Tous les opérateurs binaires, à l'exception du point « **.** », doivent être précédés et suivis d'une espace.

```
y = a + b;
```

- Il ne faut pas laisser d'espace entre les opérateurs unaires tels que la négation « ! », le moins unaire «−», l'incrément «++» et le décrément «−−» et leurs opérandes.
```
i++;
j = -1;
```
- Il ne faut pas laisser d'espace avant le caractère point-virgule « ; » qui termine une instruction.
```
System.out.print("Entrez la note");
```
- Il faut laisser une espace après chaque virgule dans les listes d'arguments.
```
maximum(a, b, c);
```
- Un mot clé devrait être séparé par une espace de la parenthèse qui le suit.
```
while (note > 100) {
    ...
}
```
- Une espace ne doit pas être insérée entre un nom de méthode et sa parenthèse ouvrante.
```
public void caculerSalaire() {
    ...
}
```
- Les clauses d'une instruction `for` doivent être séparées par des espaces.
```
for (int i = 0; i < limite; i++) {
    ...
}
```
- On évite généralement de mettre des espaces avant et après les crochets ouvrants et fermant des tableaux.
```
taux = tableTaux[5];
```
- La conversion explicite (*cast*) doit être suivie d'une espace.
```
prixReduit = (float) (prix * 0.75);
i = (byte) nombre;
```

3.2 Lignes blanches

Une ligne blanche doit toujours être insérée dans les circonstances suivantes :

- entre les méthodes ;
- entre les déclarations de variables locales d'une méthode et sa première instruction ;
- avant un commentaire en bloc ou en ligne (voir section 2.2) ;
- entre les sections logiques à l'intérieur d'une méthode pour en améliorer la lisibilité.

4 Indentation

Les instructions incluses dans un bloc délimité par des accolades « { … } » doivent être mises en retrait vers la droite d'un niveau d'indentation afin de mettre en évidence la structure du programme. L'unité d'indentation (retrait) est constituée de quatre espaces, soit la valeur par défaut dans *Visual Studio Code*.

Premier retrait
Deuxième retrait
Troisième retrait

```
import java.util.Scanner;

public class Nombre {
    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);
        int nombre;
        System.out.print("Entrez un nombre: ");
        nombre = clavier.nextInt();
        if (nombre > 0) {
            System.out.println(nombre + " est positif.");
        } else {
            System.out.println(nombre + " est négatif ou nul.");
        }
        clavier.close();
    }
}
```

4.1 Longueur des lignes de code

Si un fichier source (`.java`) est destiné à l'impression, il ne doit pas contenir des lignes de code trop longues pour éviter qu'elles ne débordent lors de l'impression. Vous trouverez, en annexe de ce document, la méthode pour configurer *Visual Studio Code* afin de changer la longueur maximale des lignes par défaut et afficher le marqueur de la longueur définie.

4.2 Découpage des lignes

Pour couper une ligne trop longue, il faut ajouter un retour de chariot là où c'est nécessaire, de préférence après une virgule ou devant un opérateur. La deuxième ligne et les suivantes doivent être indentées d'un niveau d'indentation supplémentaire par rapport à la première ligne. Par exemple :

```
public static final byte NOTE_0 = 0, NOTE_1 = 60, NOTE_2 = 75,
    NOTE_3 = 80, NOTE_4 = 100;
message = "L'aire du rectangle = " + aire
    + "\nLe périmètre du rectangle = " + perimetre;
```

5 Déclarations

- Dans une méthode, les constantes doivent être déclarées en premier. La déclaration des variables suit celle des constantes. Par exemple :
  ```
  final int MAX = 100;
  final int MIN = 0;
  int nombre;
  ```
- On recommande de ne mettre qu'une seule déclaration par ligne, car cela facilite l'insertion de commentaires. Par exemple :
  ```
  int jour;
  int mois;
  ```
  Est préférable à :
  ```
  int jour, mois;              // À ÉVITER
  ```
- Il ne faut pas déclarer différents types de variables sur la même ligne. Par exemple :
  ```
  int foo, tableauFoo[];       // MAUVAIS!
  ```
- On déclare les tableaux de la façon suivante :
  ```
  float[] tableauBar;
  ```
  et si l'on désire l'initialiser directement :
  ```
  float[] tableauBar = { initialisation };
  ```
- Les déclarations devraient se faire uniquement au début des blocs. Par exemple :
  ```
  void maMethode() {
      int entier1 = 0;          // Début du bloc de la méthode
      if (condition) {
          int entier2 = 0;      // Début du bloc if
          ...
      }
  }
  ```
  L'exception à cette règle est l'index dans les boucles `for`, qui peut être déclaré dans l'énoncé `for`.
  ```
  for (int i = 0; i < maxBoucle; i++) {
      ...
  }
  ```

6 Utilisation des variables booléennes dans une condition

- Pour vérifier qu'une condition booléenne constituée d'une seule variable booléenne, par exemple `estPositif`, est vraie (**`true`**), utilisez cette forme :
```
if (estPositif)            // Bonne pratique
```
Évitez la forme ci-dessous :
```
if (estPositif == true)    // MAUVAISE pratique
```
- Pour vérifier qu'une condition booléenne constituée d'une seule variable booléenne, par exemple `estPositif`, est fausse (**`false`**), utilisez cette forme :
```
if (!estPositif)           // Bonne pratique
```
Plutôt que la forme suivante :
```
if (estPositif == false)   // MAUVAISE pratique
```

7 Valeurs de retour

- Une méthode qui retourne une valeur n'a qu'**un seul point de sortie** (`return`).
- La structure suivante est à **ÉVITER** :
```
if (expressionBooleenne) {
    return true;
} else {
    return false;
}
```
Elle devrait plutôt être écrite comme suit :
```
return expressionBooleenne;
```

8 Instructions

8.1 Instructions simples

Chaque ligne doit contenir au plus une instruction. Par exemple :
```
nombre += 5;            // Correct
i++;  lettre = 'C';     // Incorrect
```

8.2 Instructions composées

On compose les instructions en les plaçant dans des blocs délimités par des accolades « { … } ». Les exemples dans les sections suivantes montrent des cas d'utilisation.

- Les instructions à l'intérieur d'un bloc doivent être indentées d'un niveau supplémentaire.
- L'accolade ouvrante devrait être placée à la fin de la ligne qui commence l'instruction composée. L'accolade fermante devrait, quant à elle, être sur une nouvelle ligne avec une indentation identique à celle du début de l'instruction composée.
- Les **accolades doivent toujours être écrites**, même pour les instructions simples, lorsque celles-ci font partie d'une structure de contrôle comme une instruction **`if-else`** ou **`for`**. Cela évite d'oublier d'ajouter les accolades lorsque l'on insère de nouvelles instructions.

8.3 Instructions `if`, `if-else`, `if else-if else`

8.3.1 Instruction `if`

```
if (condition) {
    instructions;
}
```

8.3.2 Instruction `if-else`

```
if (condition) {
    instructions;
} else {
    instructions;
}
```

8.3.3 Instruction `if else-if else`

```
if (condition) {
    instructions;
} else if (condition) {
    instructions;
} else {
    instructions;
}
```

Remarque : Utilisez toujours les accolades « { … } » pour les instructions **`if`**. La mise en forme suivante provoque souvent des erreurs difficiles à détecter :

```
if (condition) foo = bar;  // À ÉVITER! Les accolades sont manquantes
```

8.4 Instruction `switch`

```
switch (condition) {
case ABC:
    instructions;
    /* passe au suivant */
case DEF:
    instructions;
    break;
case XYZ:
    instructions;
    break;
default:
    instructions;
    break;
}
```

- Il faut ajouter un commentaire là où une instruction **`break`** est manquante, comme dans l'exemple précèdent avec le commentaire *`/* passe au suivant */`*.
- Toute instruction **`switch`** devrait comporter un cas **`default`**. Le **`break`** dans ce cas est redondant, mais permet d'éviter une erreur si par la suite un autre **`case`** est ajouté.

8.5 Instruction `while`

```
while (condition) {
    instructions;
}
```

8.6 Instruction `do…while`

```
do {
    instructions;
} while (condition);
```

8.7 Instruction `for`

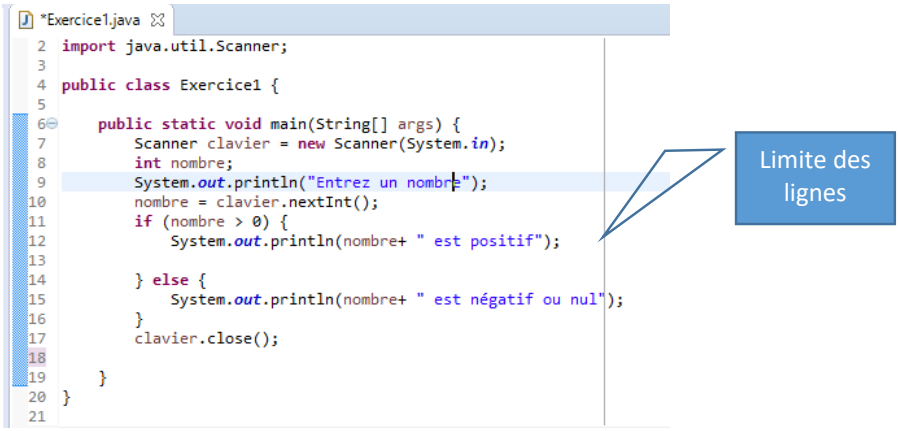
```
for (initialisation; condition; incrément) {
    instructions;
}
```

9 Références

- *Sun Microsystems. Code Conventions for the Java™ Programming Language*,
- [redacted]

10 Annexe : Mise en page du code source avec *Visual Studio Code*

- Un fichier source destiné à l'impression doit être bien formaté. Aucune ligne ne doit dépasser les limites de la page. On doit découper correctement une ligne qui est trop longue.
- Dans *Visual Studio Code*, on peut activer l'affichage de la ligne verticale indiquant la limite des lignes.



- Pour activer l'affichage des lignes, ajoutez dans `settings.json`, la ligne suivante :
  - `"editor.rulers": [80]`

11 Annexe : Structures interdites

Dans le cadre du cours, les structures suivantes sont interdites, car il s'agit généralement de mauvaises pratiques de programmation, qui doivent être corrigées.

- Nom de package, classe, variable contenant des caractères accentués.
  - → Les caractères accentués dans les noms sont interdits pour éviter des problèmes de compatibilité. Par contre, vous pouvez les utiliser dans les affichages et dans la documentation.
- Omission des `{}` à la suite d'un `if`, `else`, `while`, `for`.
  - → Les `{}` sont obligatoires, même si vous n'avez qu'une seule ligne de code à exécuter, car l'omission des `{}` peut causer des bogues difficiles à trouver.
- `{}` vides à la suite d'un `if`, `else`.
  - → Si vous avez un bloc de code vide à la suite d'un `if`, inversez votre condition. Si votre bloc vide se retrouve à la suite du `else`, effacez simplement le `else {}`.
- Plus d'un `return` dans une fonction ou dans une procédure.
  - → Une fonction ne doit contenir qu'un seul `return` sur sa dernière ligne alors qu'une procédure ne contient aucun `return`.
- Modification du compteur du `for` à l'intérieur de la boucle.
  - → Le compteur de la boucle `for` ne doit jamais être modifié à l'intérieur de la boucle, seulement par l'opération d'incrémentation à l'intérieur des `()`. Si vous souhaitez le modifier c'est que vous n'avez pas choisi le bon type de boucle.
- Utilisation du `break` et du `continue` dans une boucle.
  - → Choisissez le bon type de boucle ou ajustez votre condition afin d'inclure celle du `if` menant au `break`. Le `break` doit être utilisé exclusivement dans les instructions `switch…case`.
- Boucles infinies, par exemple `while(true)`, `for(;;)`, etc.
  - → Mettre la condition de sortie dans les `()` et non dans un `if` à l'intérieur de la boucle. La boucle `for` ne doit être contrôlée que par son compteur, par conséquent on ne doit pas mettre une condition d'arrêt composée.
- Utilisation du `System.exit()`.
  - → Modifiez la structure de vos `if…else` afin que le programme se termine à la fin du `main()`. Ainsi, toutes vos ressources seront fermées correctement.
- Utilisation de l'opérateur conditionnel (`?:`).
  - → Utilisez un `if…else` à la place.
- Appeler la méthode `main()`, par exemple pour réafficher un menu.
  - → Utilisez une boucle dans le programme principal (`main()`) afin de réafficher votre menu au lieu de faire de la récursivité.
- Type de variable `var`.
  - → L'utilisation du type `var` démontre que vous ne maîtrisez pas bien les types de variables et ce type n'existe pas en *Java 1.8*.
- Utilisation inadéquate de la conversion explicite (*cast*).
  - → La conversion explicite n'est nécessaire que dans de très rares cas, par exemple, lors de l'appel de fonctions de la libraire `Math`. Une conversion explicite démontre habituellement un choix du type de variable incorrect.